金篆信科

ZXCLOUD GoldenDB

数据库

GoldenDB产品手册_开发支持

产品版本:V6.1.03.10

法律声明

若接收金篆信科有限责任公司（以下称为“金篆信科”）的此份文档，即表示您已同意以下条款。若不同意以下条款，请停止使用本文档。

本文档版权所有金篆信科有限责任公司。保留任何未在本文档中明示授予的权利。文档中涉及金篆信科的专有信息。未经金篆信科事先书面许可，任何单位和个人不得复制、传递、分发、使用和泄漏该文档以及该文档包含的任何图片、表格、数据及其他信息。

在未经金篆信科或第三方权利人事先书面同意的情况下，阅读本文档并不表示以默示、不可反言或其他方式授予阅读者任何使用本文档中出现的任何标记的权利。

本产品符合有关环境保护和人身安全方面的设计要求，产品的存放、使用和弃置应遵照产品手册、相关合同或相关国法律、法规的要求进行。

本文档按“现状”和“仅此状态”提供。本文档中的信息随着金篆信科产品和技术的进步将不断更新，金篆信科不再通知此类信息的更新。

金篆信科有限责任公司

目录

目录 2
1. 开发支持 8
1.1. 应用开发示例 8
1.1.1. Java 语言连接示例 8
1.1.1.1. JDBC 直接连接 8
1.1.1.2. Mybatis 框架连接 9
1.1.1.3. Hibernate 框架连接 11
1.1.1.4. Springboot 框架连接 14
1.1.2. C++ 语言连接示例 16
1.1.3. Go 语言连接示例 19
1.1.4. Python 语言连接示例 20
1.1.5. GDBCI连接示例 21
1.1.6. ECGDB 嵌入式连接示例 27
1.2. 应用开发设计规范 29
1.2.1. 数据库对象命名和设计规范 29
1.2.1.1. 对象命名总体原则 29
1.2.1.2. 数据库命名规范 29
1.2.1.3. 表命名规范 29
1.2.1.4. 字段命名规范 29
1.2.1.5. 索引命名规范 29
1.2.2. 数据库对象结构规范 30
1.2.2.1. 表设计规范 30
1.2.2.2. 字段设计规范 30
1.2.2.3. 索引设计规范 30
1.2.2.4. 主键设计规范 31
1.2.2.5. 分片键设计规范 31
1.2.2.6. 分片规则设计规范 31
1.2.2.7. 分区设计规范 31
1.2.2.8. 大对象设计规范 32
1.2.3. 数据库连接规范 32
1.2.4. 数据库字符集规范 33
1.2.5. 存储模型设计规范 34
1.3. 连接数据库 35
1.3.1. 使用客户端连接 36
1.3.2. 使用驱动连接 36
1.3.2.1. Java 语言连接 36
1.3.2.2. C++ 语言连接 38
1.3.2.3. Go 语言连接 39
1.3.2.4. Python 语言连接 40
1.3.3. 使用连接池连接 41
1.3.3.1. Druid + Springboot 框架 41
1.3.3.2. Hikari + Springboot 框架 44
1.4. 创建和管理数据库对象 46
1.4.1. 关于DDL语句 46

1.4.2. 数据类型 46
1.4.2.1. 常用数据类型 46
1.4.3. 用户、密码和权限 48
1.4.3.1. 创建和管理用户 48
1.4.3.2. 设置密码 49
1.4.3.3. 设置权限 50
1.4.4. 创建和管理数据库 52
1.4.4.1. 创建、查看、修改和使用数据库 52
1.4.4.2. 删除数据库和闪回 53
1.4.5. 创建和管理表 54
1.4.5.1. 创建表 54
1.4.5.2. 修改表 58
1.4.5.3. 清空表和闪回 58
1.4.5.4. 删除表和闪回 59
1.4.5.5. 拷贝表 60
1.4.6. 创建和管理索引 61
1.4.6.1. 关于索引 61
1.4.6.2. 单节点索引 61
1.4.6.3. 分布式全局索引 62
1.4.7. 创建和管理视图 63
1.5. 数据写入 64
1.5.1. 插入数据 64
1.5.1.1. 通用插入 64
1.5.1.2. 批量插入 65
1.5.2. 更新数据 66
1.5.2.1. 通用更新 66
1.5.2.2. 批量更新 66
1.5.3. 删除数据 67
1.5.3.1. 通用删除 67
1.5.3.2. 批量删除 68
1.5.4. 替换数据 69
1.6. 数据查询 69
1.6.1. 单表查询 69
1.6.2. 条件查询 70
1.6.3. 排序查询 71
1.6.4. 分组查询 71
1.6.5. 分页查询 72
1.6.6. 多表查询 73
1.6.7. 子查询 73
1.6.8. 视图查询 74
1.6.9. 临时表查询 75
1.6.10. 高级查询 75
1.6.10.1. MPP查询 75
1.6.10.2. 流式查询 76
1.6.10.3. 闪回查询 77
1.6.10.4. 读写分离 77
1.6.10.5. 跨集群查询 78
1.7. 事务 78
1.7.1. 分布式事务 78

1.7.2. XA事务 80
1.7.3. 事务控制语句 82
1.7.3.1. 开启事务 82
1.7.3.2. 事务保存点 83
1.7.3.3. 提交事务 84
1.7.3.4. 回滚事务 85
1.7.4. 事务隔离级别 87
1.7.4.1. UR/CR 87
1.7.4.2. SW/CW 88
1.7.4.3. MVCC_RC 88
1.7.5. 锁机制 89
1.8. SQL性能调优 91
1.8.1. 执行计划 91
1.8.1.1. SQL执行计划管理 91
1.8.1.2. 执行计划查看 92
1.8.1.3. 执行计划绑定 93
1.8.2. 使用 PREPARE 94
1.8.3. 语句级并行 95
1.8.4. JOIN的优化 101
1.8.5. 子查询的优化 103
1.9. SQL实践和建议 106
1.9.1. 引言 106
1.9.1.1. 目的 106
1.9.1.2. 适用范围 106
1.9.1.3. 术语说明 106
1.9.2. SQL编写规范 107
1.9.2.1. 不建议项 107
1.9.2.2. 建议项 119
1.9.3. GoldenDB分布式数据库执行过程案例详解 121
1.9.3.1. DN 层面 EXPLAIN 说明 122
1.9.3.2. CN 层面 EXPLAIN 说明 124
1.9.3.3. 打印指定 DN/CN 的 EXPLAIN 结果 125
1.9.3.4. 典型语句举例说明 126
1.9.4. 数据导入导出规范 134
1.9.4.1. 导入导出环境检测 134
1.9.4.2. 使用命令介绍 138
1.9.4.3. 数据导入示例 146
1.9.4.4. 数据导出示例 151
1.9.4.5. 导入失败数据重试介绍 154
1.9.5. 最佳实践几点注意事项 156
1.9.5.1. 关于数据类型的选择 156
1.9.5.2. 关于隔离级别要求 157
1.9.5.3. 关于批处理设计指导 157
1.9.5.4. 关于锁超时设计 157
1.9.6. 双库运行兼容函数说明 158
1.9.6.1. GoldenDB 与 Oracle 兼容函数 158
1.9.6.2. 典型函数说明 163
1.9.7. CN 支持 gb18030 字符集 165
1.9.8. 附录 167
1.9.8.1. 保留字 167

1.9.8.2. 数据类型 172
1.9.8.3. 操作运算符 174
1.10. SQL 规范 175
1.10.1. 文档术语 176
1.10.2. 分布式SQL语法通用说明 176
1.10.3. DATA TYPES 177
1.10.3.1. GoldenDB支持数据类型列表 177
1.10.3.2. 使用说明 179
1.10.3.3. GoldenDB 支持数据类型举例 179
1.10.3.4. GEOMETRY空间数据类型 187
1.10.4. OPERATORS 213
1.10.5. FUNCTIONS 215
1.10.5.1. GoldenDB 支持函数列表 215
1.10.5.2. 函数解析模式说明 221
1.10.5.3. 复杂函数说明 221
1.10.5.4. 窗口函数 222
1.10.6. PSEUDO COLUMNS 223
1.10.6.1. SEQUENCE 223
1.10.6.2. 伪列ROWNUM 224
1.10.6.3. 伪列ROWID 226
1.10.7. DDL 226
1.10.7.1. CREATE DATABASE 226
1.10.7.2. DROP DATABASE 228
1.10.7.3. ALTER DATABASE 229
1.10.7.4. CREATE TABLE 229
1.10.7.5. DROP TABLE 249
1.10.7.6. ALTER TABLE 250
1.10.7.7. TRUNCATE TABLE 261
1.10.7.8. DROP INDEX 261
1.10.7.9. CREATE INDEX 262
1.10.7.10. COPY TABLE 264
1.10.7.11. CHECK TABLE 265
1.10.7.12. OUTLINE固定执行计划 266
1.10.7.13. 在线DDL 267
1.10.7.14. HINT固定执行计划 268
1.10.8. DML 269
1.10.8.1. INSERT 269
1.10.8.2. DELETE 273
1.10.8.3. UPDATE 276
1.10.8.4. REPLACE 280
1.10.9. DQL 283
1.10.9.1. SELECT 283
1.10.9.2. UNION 296
1.10.9.3. 通用表表达式 297
1.10.10. VIEW 299
1.10.11. TRIGGER 301
1.10.12. TRANSACTION AND LOCKING STATEMENTS 304
1.10.12.1. TRANSACTION 305
1.10.12.2. SAVEPOINT 306
1.10.13. SYNONYM 307
1.10.14. 数据库管理语句 308

1.10.14.1. 账户管理 308
1.10.14.2. SHOW 314
1.10.14.3. SET 320
1.10.14.4. DESC 321
1.10.14.5. 碎片整理 322
1.10.14.6. 系统视图 322
1.10.15. 实用语法 322
1.10.15.1. EXPLAIN 322
1.10.15.2. USE 324
1.10.15.3. 外部流水号设置 325
1.10.15.4. IGNORE_SPACE 325
1.10.15.5. PURGE 326
1.10.15.6. FLASHBACK 327
1.10.15.7. INITCAP 328
1.10.15.8. 语句加隔离级别 328
1.10.15.9. COMMENT ON 语法 330
1.10.16. USER-DEFINED FUNCTION 330
1.10.17. AP 节点(MPP)的使用 333
1.10.17.1. hint 注释控制是否走 MPP 333
1.10.17.2. 代价估算自适应配置 334
1.10.17.3. MPP的限制 334
1.10.18. 存储过程 334
1.10.18.1. 单机存储过程 334
1.10.18.2. 分布式存储过程 337
1.10.18.3. 存储过程内常用语法 341
1.10.19. XA 事务语法 347
1.10.20. Nogtid模式 349
1.10.20.1. 检测标准 349
1.10.20.2. 报错场景 349
1.10.20.3. 新增控制开关 349
1.10.21. 保留字 350
1.10.22. 函数限制 355
1.10.22.1. 随机值函数和时间函数 355
1.10.23. PACKAGE 356
1.10.24. 自定义类型 364
1.10.24.1. 全局自定义类型 364
1.10.24.2. 存储过程中的自定义类型 366
1.10.25. 正则表达式 367
1.10.25.1. 正则表达式函数语法 367
1.10.26. 透明加密 370
1.10.26.1. 表加密 370
1.10.26.2. 列加密 371
1.11. 批量协议功能 371
1.11.1. 批量协议功能总体介绍 371
1.11.2. 数据库支持批量协议说明 372
1.11.2.1. 批量 insert 不支持的场景 372
1.11.2.2. 批量 delete 不支持的场景 372
1.11.2.3. 批量 update 不支持场景 372
1.11.2.4. 批量插入和 ORACLE 行为差异补充说明 372
1.11.3. 驱动支持批量协议说明 373
1.11.3.1. OCI 驱动与 GoldenDB 数据库批量协议交互 373

1.11.3.2. ODBC 驱动与 GoldenDB 数据库批量协议交互 375
1.11.3.3. JDBC 驱动与 GoldenDB 数据库批量协议交互 376
1.11.3.4. 支持版本说明 378

1. 开发支持

本章主要介绍应用开发示例、应用开发设计、连接数据库方式、创建和管理数据库对象、数据写入、数据查询、分布式事务特性、`SQL` 性能调优、`SQL` 实践和建议等。

1.1. 应用开发示例

本节提供不同编程语言连接到 GoldenDB 代码示例。

1.1.1. Java 语言连接示例

GoldenDB 驱动完全实现 `JDBC` 标准接口，可以按照 `JDBC` 标准接口连接数据库。以下所有示例代码，都需要依赖 GoldenDB 自研驱动，需要将驱动包引入到代码依赖中。

1.1.1.1. JDBC 直接连接

- 环境配置
  推荐 `JDK1.8`。
  GoldenDB 自研驱动。

- 示例代码

```
public static void main(String[] args) {
        String url = "jdbc:goldendb://host:port/test";
        String pass = "****";
        String username = "****";
        String sql = "insert into good (id, name) values (1, 'n
ame')";
        Connection connection = null;
        Statement st = null;
        try {
            connection = DriverManager.getConnection(url, usern
ame, pass);
            st = connection.createStatement();
            st.execute(sql);
        } catch (SQLException e) {
            if(st != null){
                try {
                    st.close();
                } catch (SQLException ex) {
                    throw new RuntimeException(ex);
                }
            }
            if(connection != null){
                try {
                    connection.close();
                } catch (SQLException ex) {
```

```
                    throw new RuntimeException(ex);
                }
            }
            throw new RuntimeException(e);
        }
    }
```

核心参数说明

字段	描述
url	连接字符串，采用标准 JDBC 连接串写法。
pass	数据库密码。
username	数据库用户名。
sql	需要执行的 SQL。

1.1.1.2. Mybatis 框架连接

- 环境配置
  `JDK1.8`。
  `maven3.6` 以上。
  GoldenDB 驱动。
- 示例代码

  pom.xml

  使用 `maven` 来管理项目，其中 `pom.xml` 中引入框架依赖，`pom.xml` 文件示例如下：

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi=
"http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://
maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.test</groupId>
  <artifactId>mybatis_single3_5_0</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>
```

```
  <name>mybatis_single3_5_0</name>
  <url>http://maven.apache.org</url>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEn
coding>
  </properties>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
    <!-- https://mvnrepository.com/artifact/org.mybatis/mybatis
 -->
    <dependency>
      <groupId>org.mybatis</groupId>
      <artifactId>mybatis</artifactId>
      <version>3.5.0</version>
    </dependency>
  </dependencies>
</project>
```

mybatis-config.xml

`mybatis` 配置连接相关参数，`mybatis-config.xml` 文件示例如下：

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
        PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>
    <environments default="development">
        <environment id="development">
            <!-- JDBC transaction manager -->
            <transactionManager type="JDBC"/>
            <!-- Database pool -->
            <dataSource type="POOLED">
                <property name="driver" value="com.goldendb.jdb
c.Driver"/>
                <property name="url" value="jdbc:goldendb://hos
t:port/test"/>
                <property name="username" value="****"/>
                <property name="password" value="****"/>
            </dataSource>
        </environment>
    </environments>
    <mappers>
        <mapper resource="GoodMapper.xml"/>
    </mappers>
</configuration>
```

核心参数说明

参数	描述
driver	驱动完整类名。
url	连接字符串，采用标准 JDBC 连接串写法。
pass	数据库密码。
resource	mapper 文件地址, 文中示例 GoodMapper.xml 在 resources 目录下。

GoodMapper.xml

`MyBatis` 中，`Mapper` 是一个用于将 `SQL` 操作与 `Java` 对象进行映射的接口或 `XML` 文件，`GoodMapper.xml` 具体写法如下：

```
<?xml version="1.0" encoding="UTF8" ?>
<!DOCTYPE mapper
        PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="com.test.GoodMapper">
    <select id="listUser" resultType="com.test.Good">
        select * from user;
    </select>
</mapper>
```

核心参数说明

参数	描述
namespace	mapper 文件命名空间。
id	namespace + id 需要全局唯一。

1.1.1.3. Hibernate 框架连接

- 环境配置

  `JDK1.8`。

  `maven3.6` 以上。

  GoldenDB 驱动。

- 示例代码

  pom.xml

  使用 `maven` 来管理项目，其中 `pom.xml` 中引入框架依赖，`pom.xml` 文件示例如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 h
ttp://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>org.example</groupId>
    <artifactId>x</artifactId>
    <version>1.0-SNAPSHOT</version>
    <properties>
        <maven.compiler.source>8</maven.compiler.source>
        <maven.compiler.target>8</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sour
ceEncoding>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.hibernate</groupId>
            <artifactId>hibernate-core</artifactId>
            <version>3.6.10.Final</version>
        </dependency>
        <dependency>
            <groupId>javassist</groupId>
            <artifactId>javassist</artifactId>
            <version>3.12.1.GA</version>
        </dependency>
    </dependencies>
</project>
```

  hibernate.cfg.xml

  `hibernate` 配置连接相关参数如下，`hibernate.cfg.xml` 文件示例如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "http://www.hibernate.org/dtd/hibernate-configuration-3
.0.dtd">
```

```
<hibernate-configuration>
    <session-factory>
        <property name="hibernate.connection.driver_class">
            com.goldendb.jdbc.Driver
        </property>
          <property name="hibernate.connection.password">
              ****
          </property>
        <property name="hibernate.connection.url">
            jdbc:goldendb://host:port/test
        </property>
        <property name="hibernate.connection.username">
            ****
        </property>
        <mapping resource="Good.hbm.xml"/>
    </session-factory>
</hibernate-configuration>
```

核心参数说明

参数	描述
driver_class	驱动完整类名。
url	连接字符串，采用标准 JDBC 连接串写法。
password	数据库密码。
username	数据库用户名。
resource	hibernate 对象关系映射文件地址，文中示例 Good.hbm.xml 在 resources 目录下。

Good.hbm.xml

`hibernate` 中映射配置文件,`Good.hbm.xml` 文件示例如下:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
        "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
```

```
<hibernate-mapping>
    <class name="org.example.entity.Good" table="good2">
        <id name="id" type="java.lang.Integer">
            <column name="id" not-null="true" sql-type="NUMBER"
/>
            <generator class="native"/>
        </id>
        <property name="name" type="java.lang.String">
            <column name="name" />
        </property>
        <property name="price" type="java.lang.Double">
            <column name="price" />
        </property>
        <property name="create_time" type="java.lang.String">
            <column name="create_time" />
        </property>
    </class>
</hibernate-mapping>
```

核心参数说明

参数	描述
class 标签中 name	完整 POJO 类的类型。
class 标签中 table	数据库表名称。

1.1.1.4. Springboot 框架连接

- 环境配置
  `JDK1.8` 。
  `maven3.6` 以上。
  GoldenDB 驱动。

- 示例代码

  pom.xml

  使用 `maven` 来管理项目，其中 `pom.xml` 中引入框架依赖，`pom.xml` 文件示例如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 h
```

```
ttp://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>2.6.0</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>org.example</groupId>
    <artifactId>boot</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
            <exclusions>
                <exclusion>
                    <artifactId>spring-boot-starter-json</artif
actId>
                    <groupId>org.springframework.boot</groupId>
                </exclusion>
            </exclusions>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactI
d>
        </dependency>
    </dependencies>
</project>
```

application.yml

`Springboot` 配置连接相关参数,`application.yml` 文件示例如下:

```
spring:
  datasource:
      driver-class-name: com.goldendb.jdbc.Driver
      url: jdbc:goldendb://host:port/test
      username: ******
      password: ******
```

核心参数说明

参数	描述

driver-class-name	驱动完整类名。
url	连接字符串，采用标准 JDBC 连接串写法。
pass	数据库密码。
username	数据库用户名。

1.1.2. C++ 语言连接示例

`ODBC（Open-DataBase-Connectivity）`是最早由微软提出的开放式数据库互连技术，其基本思想是为用户提供简单、标准、透明的数据库连接的公共编程接口，搭配不同的驱动程序，用户使用同一套应用程序就可以访问不同的数据库。

- 环境配置
  GoldenDB 定制 `ODBC` 驱动。
  硬件平台：`x86_64/aarch64` 。
  操作系统：`x86_64` 架构下：`CentOS/Redhat` 系 `Linux` 发行版 7 系列，`suse12`，海光麒麟。

- 示例代码

```
#include<sql.h>
#include<sqlext.h>
#include<string.h>
#include<time.h>
#include<stdlib.h>
#include<stdio.h>

#define isSuc(result) ((result) == SQL_SUCCESS || (result) == S
QL_SUCCESS_WITH_INFO)

/** 可扩展设置 **/
char szConnectString[2560] ="DRIVER={/usr/lib64/libmyodbc5a.so}
;USER=xxx;PASSWORD=xxx;DATABASE=xx;PORT=8880;SERVER=10.10.10.10
;SAFE=1;";
char szSQLDrop[256] = "drop table if exists t1;";
char szSQLCreate[256] = "CREATE TABLE t1(id int NOT NULL primar
y key,name char(50) NOT NULL,age int NOT NULL)distributed by ha
sh(id)(g1)";
/** 输出缓存 **/
```

```
char szDriverOutput[256];
short sDriverOutputLength;

/** 用于测试的函数 **/
int main()
{
  short sret; //返回代码
  void* henv;  //环境句柄
  void* hdbc;  //连接句柄
  long mode;//提交模式
  void* hsmt; //语句句柄

  /** 申请环境句柄 **/
  sret = SQLAllocHandle(SQL_HANDLE_ENV,SQL_NULL_HANDLE,&henv);
  if(!isSuc(sret)){printf("申请环境句柄出错\n");return 0;}

  /** 设置环境属性,声明ODBC版本 **/
  sret = SQLSetEnvAttr(henv,SQL_ATTR_ODBC_VERSION,(void*)SQL_OV
_ODBC3,SQL_IS_INTEGER);
  if(!isSuc(sret)){printf("声明ODBC版本出错\n");return 0;}

  /** 申请连接句柄 **/
  sret = SQLAllocHandle(SQL_HANDLE_DBC,henv,&hdbc);
  if(!isSuc(sret)){printf("申请连接句柄出错\n");return 0;}

  /** 连接数据源 **/
  sret = SQLDriverConnect(hdbc,NULL,(SQLCHAR*)szConnectString,S
QL_NTS,(SQLCHAR *)szDriverOutput,2560,&sDriverOutputLength,SQL_
DRIVER_NOPROMPT);
  if(!isSuc(sret)){printf("连接数据源出错\n");return 0;}
  else printf("\nCreate connection success !\n\n");

  /** 分配语句句柄 **/
  sret = SQLAllocHandle(SQL_HANDLE_STMT,hdbc,&hsmt);
  if(!isSuc(sret)){printf("分配语句句柄出错\n");return 0;}

  /** 执行drop语句 **/
  printf("execute sql: %s\n\n", szSQLDrop);
  sret = SQLExecDirect(hsmt,(SQLCHAR *)szSQLDrop,256);
  if(!isSuc(sret)){printf("exec drop sql fail !\n\n");return 0;
}
  else printf("drop table t1(old) success !\n\n");

  /** 执行create语句 **/
  printf("execute sql: %s\n\n", szSQLCreate);
  sret = SQLExecDirect(hsmt,(SQLCHAR *)szSQLCreate,256);
  if(!isSuc(sret)){printf("exec create sql fail !\n\n");return
0;}
  else printf("create table t1(new) success !\n\n");

  /** 执行insert prepare语句 **/
  char szSQLPrepareInsert[256] = "insert into t1 values(?,?,?)"
;
```

```
  printf("prepare sql: %s\n\n", szSQLPrepareInsert);
  sret = SQLPrepare (hsmt,(SQLCHAR *)szSQLPrepareInsert,256);
  if(!isSuc(sret)){printf("prepare sql fail !\n\n");return 0;}
  else printf("prepare success !\n\n");

  size_t batchSize = 9;
  int param1[9] = {1,2,3,4,5,6,7,8,9};
  char param2[9][50] = {"name1","name2","name3","name4","name5"
,"name6","name7","name8","name9"};
  int param3[9] = {11,12,13,14,15,16,17,18,19};

  sret = SQLBindParameter(hsmt,1,SQL_PARAM_INPUT,SQL_C_SLONG,SQ
L_INTEGER,0,0,param1,0,NULL);
  sret = SQLBindParameter(hsmt,2,SQL_PARAM_INPUT,SQL_C_CHAR,SQL
_CHAR,5,0,param2,50,NULL);
  sret = SQLBindParameter(hsmt,3,SQL_PARAM_INPUT,SQL_C_SLONG,SQ
L_INTEGER,0,0,param3,0,NULL);
  if(!isSuc(sret))printf("bind param fail !\n\n");
  sret = SQLSetStmtAttr(hsmt, SQL_ATTR_PARAMSET_SIZE,(SQLPOINTE
R)batchSize, 0);
  if(!isSuc(sret))printf("set paramSize fail !\n\n");

  sret = SQLExecute(hsmt);
  if(!isSuc(sret))printf("exec stmt fail !\n\n");
  else printf("exec stmt success !\n\n");

  SQLLEN affRows = 0;
  sret = SQLRowCount(hsmt,&affRows);
  if(!isSuc(sret))printf("SQLRowCount stmt fail !\n\n");
        else printf("SQLRowCount stmt success !\n\n");
  printf("affect rows: %d\n\n",affRows);

  /** 执行select prepare语句 **/
  char szSQLPrepareSelect[256] = "select id,name,age from t1 wh
ere id < ?";
  printf("prepare sql: %s\n\n", szSQLPrepareSelect);
  sret = SQLPrepare (hsmt,(SQLCHAR *)szSQLPrepareSelect,256);
  if(!isSuc(sret)){printf("prepare sql fail !\n\n");return 0;}
  else printf("prepare success !\n\n");

  int select_param1 = 12;
  sret = SQLBindParameter(hsmt,1,SQL_PARAM_INPUT,SQL_C_SLONG,SQ
L_INTEGER,0,0,&select_param1,0,NULL);
  if(!isSuc(sret))printf("bind param fail !\n\n");
  sret = SQLExecute(hsmt);
  if(!isSuc(sret))printf("exec stmt fail !\n\n");
  else printf("exec stmt success !\n\n");
  sret = SQLSetStmtAttr(hsmt, SQL_ATTR_ROW_ARRAY_SIZE,(SQLPOINT
ER)1, 0);
  if(!isSuc(sret))printf("set paramSize fail !\n\n");

  int ans1 = 0;
  char ans2[50];
```

```
  int ans3 = 0;
  sret = SQLBindCol(hsmt,1,SQL_C_SLONG,&ans1,sizeof(int),NULL);
  sret = SQLBindCol(hsmt,2,SQL_C_CHAR,ans2,50,NULL);
  sret = SQLBindCol(hsmt,3,SQL_C_SLONG,&ans3,sizeof(int),NULL);
  if(!isSuc(sret))printf("define param fail !\n\n");

  do
  {
        sret = SQLFetch(hsmt);
        if(isSuc(sret))
    {
      printf("fetch success: [id:%d,name:%s,age:%d]\n",ans1,ans
2,ans3);
    }
    }while(sret == 0);

  /** 释放语句句柄 **/
  SQLFreeHandle(SQL_HANDLE_STMT,hsmt);
  /** 提交连接的事务 **/
  SQLEndTran(SQL_HANDLE_DBC,hdbc,SQL_COMMIT);
  /** 断开与数据源的连接 **/
  SQLDisconnect(hdbc);
  /** 释放连接句柄 **/
  SQLFreeHandle(SQL_HANDLE_DBC,hdbc);
  /** 释放环境句柄 **/
  SQLFreeHandle(SQL_HANDLE_ENV,henv);
  printf("The connection will be closed !\nBye !\n");
}
```

1.1.3. Go 语言连接示例

`database/sql Go` 语言标准库中用于数据库操作的接口，提供了一种通用的数据库操作方式，使得可以以统一的方式连接和操作不同的类型的数据库，而无需关心具体的数据库内部具体实现细节，GoldenDB 提供实现了该标准的 `Go` 语言驱动。

- 环境配置
  GoldenDB 定制 `Go` 驱动。

- 示例代码

```
package main

import (
  "database/sql"
  "fmt"
  _ "github.com/go-sql-driver/goldenDB"
  "log"
)
```

```
func main() {
  // 构建连接串
  dsn := "username:password@tcp(ip:port)/xxx"

  // 连接数据库
  db, err := sql.Open("mysql", dsn)
  if err != nil {
    log.Fatal(err)
  }
  defer db.Close()

  // 测试连接
  err = db.Ping()
  if err != nil {
    log.Fatal(err)
  }
  fmt.Println("Connected to the database successfully!")

  // 查询表 "good" 的第一个字段
  rows, err := db.Query("SELECT name FROM good")
  if err != nil {
    log.Fatal(err)
  }
  defer rows.Close()

  // 循环表的行
  for rows.Next() {
    var firstField string
    err := rows.Scan(&firstField)
    if err != nil {
      fmt.Println("读取行失败:", err.Error())
      return
    }
    fmt.Println("First field in table 'good':", firstField)
  }

  err = rows.Err()
  if err != nil {
    log.Fatal(err)
  }
}
```

1.1.4. Python 语言连接示例

`Python DB-API` 是 `Python` 标准库中定义的数据库接口规范，用于开发数据库连接和操作模块，GoldenDB 提供实现了这一标准的驱动，用户可以遵循相同的操作方式和语法来使用。

- 环境配置
  GoldenDB 定制 `Python` 驱动。

- 示例代码

```
# -*- coding: utf-8 -*-

import pymysql
# 连接相关参数
conn = pymysql.connect(
    host='xx.xx.xx.xx',
    port=xxxx,
    user='xxxx',
    password='xxxx',
    database='xxx',
)

# 创建游标对象
cursor = conn.cursor()

# 执行 SQL 查询语句
cursor.execute("select * from good")

# 获取查询结果
result = cursor.fetchall()
# 获取第一条参数
first_column_values = [row[0] for row in result]
print(first_column_values.pop())
# 关闭资源
cursor.close()
conn.close()
```

1.1.5. GDBCI连接示例

`GDBCI（GoldenDB Call Interface）`是与`Oracle OCI`兼容的 GoldenDB `C`语言接口调用工具，它支持开发人员使用和`ORACLE OCI`类似的接口连接使用 GoldenDB 数据库。

`OCI（Oracle Call Interface）`是`Oracle`的应用程序开发工具，通过`C`语言接口访问`Oracle`数据库，通过接口控制各类`SQL`语句的执行。它支持`SQL`所有的数据定义、数据操作、查询和事务管理等操作，支持`C`和`C++`的数据类型、调用、语法和语义。

`GDBCI`是参照`Oracle OCI`的接口标准，为 GoldenDB 的业务开发人员提供向`Oracle`兼容的相同接口。业务可以不修改代码，直接从`Oracle`切换到 GoldenDB 数据库。

- 环境配置
  硬件平台：`x86_64/aarch64`。
  操作系统：`x86_64`架构下：`CentOS/Redhat`系`Linux`发行版 7系列，`suse12`，海光麒麟。

- 示例代码

```
/*gcc ocitest.c -g -o ocitest  -lgdboci -I/usr/include/gdboci
-L/usr/lib64*/
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <malloc.h>
#include "oci.h"

void checkerr(OCIError *errhp, sword status, const char* filena
me, int line);
void query_tables();
void insert_tables();
void update_tables();
void delete_tables();

#define OCI_CHECK_RET(errhp, function) \
  checkerr(errhp, function, __FILE__, __LINE__)

OCIEnv *envhp;
OCISvcCtx *svchp;
OCIServer *srvhp;
OCISession *authp;
OCIStmt *stmthp;
OCIDescribe *dschp;
OCIError *errhp;
OCIDefine *defhp[3];
OCIBind *bidhp[4];
sb2 ind[3];

int gchpersonid;
text gchsex[2];
text gchname[10];
text gchemail[10];
char sql[256];
static text* SQL_DROP_TB   = (text*)"drop table if exists perso
n";
static text* SQL_CREATE_TB = (text*)"create table person(person
id number primary key, \
sex varchar2(256), name varchar2(256), email varchar2(256))dist
ributed by hash(personid)(g1,g2)";

int main(int argc, char *argv[]) {
    char strServerName[50];
    char strUserName[50];
    char strPassword[50];

    strcpy(strServerName, "xx.xx.xx.xx:xxxx/xxxxx");//ip:port/d
bname
    strcpy(strUserName, "xxxxxx");//username
    strcpy(strPassword, "xxxxxx");//password
```

```
    OCI_CHECK_RET(errhp, OCIInitialize(OCI_DEFAULT, NULL, NULL,
 NULL, NULL));
    OCI_CHECK_RET(errhp, OCIEnvInit(&envhp, OCI_DEFAULT, 0, 0))
;
    OCI_CHECK_RET(errhp, OCIHandleAlloc(envhp, (dvoid **)&svchp
, OCI_HTYPE_SVCCTX, 0, 0));
    OCI_CHECK_RET(errhp, OCIHandleAlloc(envhp, (dvoid **)&srvhp
, OCI_HTYPE_SERVER, 0, 0));
    OCI_CHECK_RET(errhp, OCIHandleAlloc(envhp, (dvoid **)&authp
, OCI_HTYPE_SESSION, 0, 0));
    OCI_CHECK_RET(errhp, OCIHandleAlloc(envhp, (dvoid **)&errhp
, OCI_HTYPE_ERROR, 0, 0));
    OCI_CHECK_RET(errhp, OCIHandleAlloc(envhp, (dvoid **)&dschp
, OCI_HTYPE_DESCRIBE, 0, 0));
    OCI_CHECK_RET(errhp, OCIServerAttach(srvhp, errhp, (text *)
strServerName,
                       (sb4)strlen(strServerName), OCI_DEFAULT));
    OCI_CHECK_RET(errhp, OCIAttrSet(authp, OCI_HTYPE_SESSION, (
text *)strUserName,
                (ub4)strlen(strUserName), OCI_ATTR_USERNAME, errh
p));
    OCI_CHECK_RET(errhp, OCIAttrSet(authp, OCI_HTYPE_SESSION, (
text *)strPassword,
                (ub4)strlen(strPassword), OCI_ATTR_PASSWORD, errh
p));
    OCI_CHECK_RET(errhp, OCIAttrSet((dvoid *)svchp, (ub4)OCI_HT
YPE_SVCCTX,
                (dvoid *)srvhp, (ub4)0, OCI_ATTR_SERVER, errhp));
    OCI_CHECK_RET(errhp, OCIAttrSet(svchp, OCI_HTYPE_SVCCTX, (d
void *)authp,
                0, OCI_ATTR_SESSION, errhp));
    OCI_CHECK_RET(errhp, OCISessionBegin(svchp, errhp, authp, O
CI_CRED_RDBMS, OCI_DEFAULT));
    OCI_CHECK_RET(errhp, OCIHandleAlloc(envhp, (dvoid **)&stmth
p, OCI_HTYPE_STMT, 0, 0));
    OCI_CHECK_RET(errhp, OCIStmtPrepare(stmthp, errhp, (text*)S
QL_DROP_TB,
            strlen((char *)SQL_DROP_TB), OCI_NTV_SYNTAX, OCI_DEFA
ULT));
    OCI_CHECK_RET(errhp, OCIStmtExecute(svchp, stmthp, errhp, 1
, 0, 0, 0, OCI_COMMIT_ON_SUCCESS));
    OCI_CHECK_RET(errhp, OCIStmtPrepare(stmthp, errhp, SQL_CREA
TE_TB,
            strlen((char *)SQL_CREATE_TB), OCI_NTV_SYNTAX, OCI_DE
FAULT));
    OCI_CHECK_RET(errhp, OCIStmtExecute(svchp, stmthp, errhp, 1
, 0, 0, 0, OCI_DEFAULT));

    query_tables();
    insert_tables();
    query_tables();
    update_tables();
```

```
    query_tables();
    delete_tables();
    query_tables();

    OCI_CHECK_RET(errhp, OCISessionEnd(svchp, errhp, authp, (ub
4)0));
    OCI_CHECK_RET(errhp, OCIServerDetach(srvhp, errhp, OCI_DEFA
ULT));
    OCI_CHECK_RET(errhp, OCIHandleFree((dvoid *)dschp, OCI_HTYP
E_DESCRIBE));
    OCI_CHECK_RET(errhp, OCIHandleFree((dvoid *)errhp, OCI_HTYP
E_ERROR));
    OCI_CHECK_RET(errhp, OCIHandleFree((dvoid *)authp, OCI_HTYP
E_SESSION));
    OCI_CHECK_RET(errhp, OCIHandleFree((dvoid *)svchp, OCI_HTYP
E_SVCCTX));
    OCI_CHECK_RET(errhp, OCIHandleFree((dvoid *)srvhp, OCI_HTYP
E_SERVER));
    return 0;
}

void checkerr(OCIError *errhp, sword status, const char* filena
me, int line)
{
  text errbuf[512];
  sb4 errcode = 0;
  switch (status)
  {
  case OCI_SUCCESS:
    break;
  case OCI_SUCCESS_WITH_INFO:
    (void) OCIErrorGet((dvoid *)errhp, (ub4) 1, (text *) NULL,
&errcode,
                       errbuf, (ub4) sizeof(errbuf), OCI_HTYPE_E
RROR);
    (void) printf("%s:%d Error - OCI_SUCCESS_WITH_INFO now get
is %d:%.*s\n", filename, line, errcode, 512, errbuf);
    break;
  case OCI_NEED_DATA:
    (void) printf("%s:%d Error - OCI_NEED_DATA\n", filename, li
ne);
    break;
  case OCI_NO_DATA:
    (void) printf("%s:%d Error - OCI_NODATA\n", filename, line)
;
    break;
  case OCI_ERROR:
    (void) OCIErrorGet((dvoid *)errhp, (ub4) 1, (text *) NULL,
&errcode,
                       errbuf, (ub4) sizeof(errbuf), OCI_HTYPE_E
RROR);
    (void) printf("%s:%d Error - now get is %d:%.*s\n", filenam
e, line, errcode,
```

```
            512, errbuf);
    break;
  case OCI_INVALID_HANDLE:
    (void) printf("%s:%d Error - OCI_INVALID_HANDLE\n", filenam
e, line);
    break;
  case OCI_STILL_EXECUTING:
    (void) printf("%s:%d Error - OCI_STILL_EXECUTE\n", filename
, line);
    break;
  case OCI_CONTINUE:
    (void) printf("%s:%d Error - OCI_CONTINUE\n", filename, lin
e);
    break;
  default:
    break;
  }
}

void query_tables()
{
    sword status = OCI_SUCCESS;
    memset(sql, 0, sizeof(sql));
    strcpy(sql, "select personid, name, email from person");

    OCI_CHECK_RET(errhp, OCIStmtPrepare(stmthp, errhp, (text *)
sql, strlen(sql),
                  OCI_NTV_SYNTAX, OCI_DEFAULT));
    OCI_CHECK_RET(errhp, OCIDefineByPos(stmthp, &defhp[0], errh
p, 1, &gchpersonid,
                    sizeof(gchpersonid), SQLT_INT, &ind[0], 0, 0,
 OCI_DEFAULT));
    OCI_CHECK_RET(errhp, OCIDefineByPos(stmthp, &defhp[1], errh
p, 2, (ub1 *)gchname,
                    sizeof(gchname), SQLT_STR, &ind[1], 0, 0, OCI
_DEFAULT));
    OCI_CHECK_RET(errhp, OCIDefineByPos(stmthp, &defhp[2], errh
p, 3, (ub1 *)gchemail,
                    sizeof(gchemail), SQLT_STR, &ind[2], 0, 0, OC
I_DEFAULT));
    OCI_CHECK_RET(errhp, OCIStmtExecute(svchp, stmthp, errhp, (
ub4)0, 0, NULL, NULL,
                    OCI_DEFAULT));
    printf("%-10s%-10s%-10s\n", "PERSONID", "NAME", "EMAIL");
    while ((status = OCIStmtFetch(stmthp, errhp, 1, OCI_FETCH_N
EXT, OCI_DEFAULT)) == OCI_SUCCESS) {
        printf("%-10d", gchpersonid);
        printf("%-10s", gchname);
        printf("%-10s\n", gchemail);
    }
    if (OCI_NO_DATA != status) {
        printf("error ! error ! error ! err=%d\n", status);
    } else {
```

```
        printf("finish fetch data\n\n");
    }
}

void insert_tables()
{
    memset(sql, 0, sizeof(sql));
    strcpy(sql, "insert into person values(:personid, :sex, :na
me, :email)");

    OCI_CHECK_RET(errhp, OCIStmtPrepare(stmthp, errhp, (text *)
sql, strlen(sql), OCI_NTV_SYNTAX,
                  OCI_DEFAULT));
    OCI_CHECK_RET(errhp, OCIBindByName(stmthp, &bidhp[0], errhp
, (const OraText*)":personid", 9,
    &gchpersonid, sizeof(gchpersonid), SQLT_INT, NULL, NULL, NU
LL, 0, NULL, 0));
    OCI_CHECK_RET(errhp, OCIBindByName(stmthp, &bidhp[1], errhp
, (const OraText*)":sex", 4,
    gchsex, sizeof(gchsex), SQLT_STR, NULL, NULL, NULL, 0, NULL
, 0));
    OCI_CHECK_RET(errhp, OCIBindByName(stmthp, &bidhp[2], errhp
, (const OraText*)":name", 5,
    gchname, sizeof(gchname), SQLT_STR, NULL, NULL, NULL, 0, NU
LL, 0));
    OCI_CHECK_RET(errhp, OCIBindByName(stmthp, &bidhp[3], errhp
, (const OraText*)":email", 6,
    gchemail, sizeof(gchemail), SQLT_STR, NULL, NULL, NULL, 0,
NULL, 0));

    gchpersonid = 1;
    memset(gchsex, 0, sizeof(gchsex));
    strcpy((char*)gchsex, "M");
    memset(gchname, 0, sizeof(gchname));
    strcpy((char*)gchname, "gchtest");
    memset(gchemail, 0, sizeof(gchemail));
    strcpy((char*)gchemail, "t@163.com");


    OCI_CHECK_RET(errhp, OCIStmtExecute(svchp, stmthp, errhp, (
ub4)1, (ub4)0,
          (CONST OCISnapshot *)0, (OCISnapshot *)0, (ub4)OCI_DE
FAULT));
    OCI_CHECK_RET(errhp, OCITransCommit(svchp, errhp, OCI_DEFAU
LT));
    printf("finish insert tables\n");
}

void update_tables()
{
    memset(sql, 0, sizeof(sql));
    strcpy(sql, "update person set sex='M',name='test',email='t
est@mail' WHERE personid=1");
```

```
    OCI_CHECK_RET(errhp, OCIStmtPrepare(stmthp, errhp, (text *)
sql, strlen(sql), OCI_NTV_SYNTAX,
                  OCI_DEFAULT));
    OCI_CHECK_RET(errhp, OCIStmtExecute(svchp, stmthp, errhp, (
ub4)1, (ub4)0,
          (CONST OCISnapshot *)0, (OCISnapshot *)0, (ub4)OCI_DE
FAULT));
    OCI_CHECK_RET(errhp, OCITransCommit(svchp, errhp, OCI_DEFAU
LT));
    printf("finish update tables\n");
}

void delete_tables()
{
    memset(sql, 0, sizeof(sql));
    strcpy(sql, "delete from person WHERE personid = :personid
");

    OCI_CHECK_RET(errhp, OCIStmtPrepare(stmthp, errhp, (text *)
sql, strlen(sql), OCI_NTV_SYNTAX,
                  OCI_DEFAULT));
    gchpersonid = 1;
    OCI_CHECK_RET(errhp, OCIBindByPos(stmthp, &bidhp[0], errhp,
 1, &gchpersonid,
                sizeof(gchpersonid), SQLT_INT, NULL, NULL, NULL
, 0, NULL, 0));
    OCI_CHECK_RET(errhp, OCIStmtExecute(svchp, stmthp, errhp, (
ub4)1, (ub4)0,
          (CONST OCISnapshot *)0, (OCISnapshot *)0, (ub4)OCI_DE
FAULT));
    OCI_CHECK_RET(errhp, OCITransCommit(svchp, errhp, OCI_DEFAU
LT));
    printf("finish delete tables\n");
}
```

1.1.6. ECGDB 嵌入式连接示例

`ECGDB` 是与 `Oracle Pro\*C`、`Informix ESQL` 等嵌入式预编译器兼容的 GoldenDB 预编译器，提供了不同兼容模式下的特性。

`ECGDB` 嵌入式 `SQL` 的程序由 `C` 语言编写的代码和位于特殊标记的小节中的 `SQL` 命令混合组成。

要构建该程序，源代码（`*.pc` `*.sc` `*.ec`）首先会通过嵌入式 `SQL` 预处理器 `ecgdb` 将源代码转换成一个普通 `C` 代码（`*.c`）。然后链接通过 `SQL` 执行的动态库（`libecgdb.so`）调用 GoldenDB Client 库中的函数，与 GoldenDB Server 端进行通信，执行 `SQL`，获取结果集等。

- 环境配置
  硬件平台：`x86_64/aarch64`。
  操作系统：`CentOS/Redhat`系`Linux`发行版7系列，`Kylin V10`系列。
  编译器：`GCC 4.8`。

- 示例代码

```
#include <stdio.h>
#include <stdlib.h>
#include "ecgdb_types.h"
#include "ecgdb_method.h"

int main(void)
{
    exec sql connect to tcp:goldendb://ip:port/dbname user user
name  using "password";
    if (sqlca.sqlcode)
    {
      printf ("connect error = %ld,errmsg:%s.\n", sqlca.sqlcode
,sqlca.sqlerrm.sqlerrmc);
      exit (sqlca.sqlcode);
    }

    EXEC SQL BEGIN DECLARE SECTION;
    char v1[100];
    VARCHAR v2[100];
    date v3;
    timestamp v4;
    char v5[100];
    EXEC SQL END DECLARE SECTION;

    strcpy(v1, "2022-06-11");
    strcpy(v2.arr,"2022-06-11 19:22:20");
    v2.len=strlen(v2.arr);
    EXEC SQL INSERT INTO db.tb1 (date_0, datetime_0, timestamp_
0) values(:v1, :v2, :v2);
    EXEC SQL SELECT date_0 into :v5 from db.tb1 where date_0="2
022-06-11";
    printf("v5=%s\n",v5);

    ECGDB_TYPES_date_today(&v3);
    ECGDB_TYPES_timestamp_current(&v4);
    EXEC SQL INSERT INTO db.tb1 (date_0, datetime_0, timestamp_
0) values(:v3, :v4, :v4);
    EXEC SQL SELECT datetime_0 into :v5 from db.tb1 where datet
ime_0=:v4;
    printf("v5=%s\n",v5);

    EXEC SQL disconnect;
    return 0;
}
```

1.2. 应用开发设计规范

本小章节的目的是向开发者介绍在应用开发之前应该如何进行数据库设计。

1.2.1. 数据库对象命名和设计规范

1.2.1.1. 对象命名总体原则

- 对象名做到见名知意,使用具有意义的英文单词或者英文单词缩写。
- 单词之间用下划线"_"分隔。
- 在命名时避免使用 GoldenDB 保留字。
- 不使用特殊符号，命名使用下划线、数字、字母组成。
- 禁止使用关键字单独作为对象名，避免使用关键字为前缀，如 date ， rank ， order 等。
- 数据库对象名不超过 30 个字符。

分布式数据库的系统关键字包括:

- `GTID`：全局事务的标识，分布式数据库隐藏列。
- `STORAGEDB`：提示 SQL 执行的分片。
- `DISTRIBUTED BY`：指示分布式数据库表的分发规则。
- `HASH`：指示分布式数据库表按照 `HASH` 策略进行分发。
- `RANGE`：指示分布式数据库表按照 `RANGE` 策略进行分发。
- `LIST`：指示分布式数据库表按照 `LIST` 策略进行分发。
- `DUPLICATE`：指示分布式数据库表按照 `DUPLICATE` 策略进行分发。

1.2.1.2. 数据库命名规范

- 数据库名应尽可能和所服务的业务模块名一致。不同的项目使用不同的数据库名,不建议不同的项目使用相同的数据库名。
- 数据库名必须简洁。

1.2.1.3. 表命名规范

- 采用有意义且一致的命名方式，使用清晰的表名和字段名，避免使用缩写、简写或无意义的命名，以提高代码的可读性。

1.2.1.4. 字段命名规范

- 字段类型按需使用，不要定义特别长的字段。
- 字段类型避免使用 `default NULL`，尽量使用特殊的 default 值。

1.2.1.5. 索引命名规范

- 使用统一的命名规范，并使命名标准化，取尽可能代表意义的缩写用以区分主键索引、唯一索引、普通索引。

1.2.2. 数据库对象结构规范

本部分将详细说明数据库对象（如表、视图、存储过程等）的结构规范。将介绍如何命名数据库对象、设计良好的关系模型、定义适当的字段和约束等，以确保数据库对象的一致性和高效性。

1.2.2.1. 表设计规范

- 表的字符集及字符序与库保持一致。
- 定义清晰的表结构，遵循关系模型原理，确保每个表都有合适的字段类型。
- 使用适当的字段约束，如主键、唯一约束、非空约束、默认值约束等，以保证数据的正确性和完整性。
- 为表定义合适的主键，确保唯一性和数据查询的效率。
- 设计良好的关系模型，使用外键关联表之间的关系，以确保数据的完整性和一致性。
- 根据表的数据量与数据范围设置合理的分片，以及分区设定。
- 指定注释。
- 单行数据长度不超过 1M。

1.2.2.2. 字段设计规范

- 字段的字符集与字符序与表一致。
- 字段类型按需使用，不要定义特别长的字段。
- 字段类型避免使用`default NULL`，尽量使用特殊的 default 值。
- 指定注释

1.2.2.3. 索引设计规范

- 按需使用全局唯一索引，一般需要全局唯一、写少的表、存在非分片键的字段点查速度较慢场景可以考虑使用。
- 每个业务表都必须要加主键索引，避免额外内部生成 rowid 、 _gdb_rowid 字段占用空间以及增加回表开销。
- 单表添加索引数量不建议过多，只创建必要的索引。每多一个索引，表的维护开销及 DML 性能损耗就越大，不建议单表创建 10 个以上的索引。
- 索引设计需要面向业务使用场景，将需要的字段构建索引，不需要的字段不添加。
- 在 `WHERE` 条件涉及多个字段的从而需要构建联合索引的场景，索引需要满足最左前缀原则，避免无法充分利用索引的顺序性和过滤性。
- 业务 SQL `where like '%abc'` 这种左模糊的条件，无法使用到索引，此字段无需创建索引。
- 创建联合索引的过程，字段顺序按照区分度从高到低的排列，区分度相似的字段，将占用空间较小的字段放在前面。
- 联合索引中如果包含查询较频繁的列，将该字段放在索引左侧。

- 核心 SQL 这类对性能要求比较高的场景，考虑创建覆盖索引，避免回表带来的性能损耗。
- 避免冗余的索引。
- 不在区分度低的字段上建立索引，例如某个字段'性别'只有'男'和'女'两个值，区分度太低，不建议创建索引。
- 尽量不要使用函数索引。
- 尽量避免使用外键约束。影响写操作性能和 DDL 操作检查。

1.2.2.4. 主键设计规范

- 建议定义主键。
- 主键使用常用数据类型，如整型或字符型等。

1.2.2.5. 分片键设计规范

- 分片键使用常用数据类型，如整型或字符型等。
- 优先使用表的查询字段或关联查询字段作为分片键。
- 优先使用表的主键作为分片键。
- 优先使用表的唯一索引字段作为分片键。
- 分片键尽量选择很少修改值的字段。
- 插入和修改时分片键应为确定值。

1.2.2.6. 分片规则设计规范

说明： 表的大小主要有 3 个维度 1. 表的用途(读写频率) 2. 表的数据量(行数) 3. 表的数据大小(单行平均大小)

- 小表：数据量级别在 10 万以下、单行小于 1K、读频率高，写频率低。
- 大表：数据量级别在百万以上或单行数据量大于 10K 或写频率高。

规范：

- 小表优先使用单节点存储的表。
- 大表优先使用多节点分布存储的表。
- 需要进行关联查询的相关大表，优先使用相同的分片规则和分片键。
- 去范式化原则。
- 针对多节点分布存储的表。
  - 在采用 `HASH/RANGE/LIST` 分发策略时，尽量保持各个 DN 节点上的数据量均衡。
  - `HASH` 分发：优先使用于不需要关心分发字段值的具体含义的表，可以将数据均匀的分布到各个分片上，保证各分片的数据量基本一致。
  - `RANGE` 分发：优先使用于列值或列值集合在确定范围内分布的表，例如数据按照自然月或天来分布存储。
  - `LIST` 分发：优先使用于含有一系列限定性值的列的场景，例如机构代码或国家代码字段。
  - 复制表：优先使用小表且不常修改的表。

1.2.2.7. 分区设计规范

- 考虑将数据按照时间范围、地理范围或业务逻辑进行分区，以便进行更高效的数据管理和查询优化。
- 选择适当的分区键，以确保数据在不同分区之间均衡分布，并且能够满足常见的查询需求。
- 避免选择频繁变动或者不均匀分布的列作为分区键，以避免导致分区不平衡或过度分区。
- 根据数据量和系统负载等因素，合理控制分区的数量，避免创建过多的分区导致管理复杂性增加。

1.2.2.8. 大对象设计规范

- 存储过程（`PROCEDURE`）：设计存储过程时，遵循单一职责原则，每个存储过程应只负责一个特定的任务或功能。使用参数传递数据给存储过程，避免直接引用全局变量。为存储过程添加注释，清晰地描述其功能、参数和返回值，以便其他开发人员理解和使用。
- 函数（`FUNCTION`）：与存储过程类似，函数应该遵循单一职责原则，每个函数应只执行一个特定的计算或操作。使用函数来提高查询可读性和简化复杂计算，但避免在函数中执行大量数据库访问操作，以免影响性能。
- 触发器（`TRIGGER`）：触发器用于在数据库发生特定事件时自动执行代码，因此应谨慎设计以避免不必要的复杂性和性能问题。仔细选择触发器的事件类型和触发时机，确保它们与业务逻辑和数据完整性要求一致。避免在触发器中执行过多的逻辑或大量的数据库操作，以免影响数据库性能。
- 包（`PACKAGE`）：使用包来组织和封装相关的存储过程和函数，以便更好地管理和维护代码库。在设计包时，考虑模块化和可重用性，避免包含过多的无关功能或不相关的存储过程和函数。
- 同义词（`SYNONYM`）：同义词用于简化对其他数据库对象的引用，并提供更灵活的数据库设计。使用同义词来隐藏数据库对象的实际名称和位置，以减少代码中的硬编码，从而提高代码的可维护性和可移植性。谨慎使用同义词，避免引起混淆或命名冲突，确保同义词的命名清晰且易于理解。

合理设计和使用存储过程、函数、触发器、包和同义词是数据库开发的关键，它们可以提高代码的可维护性、性能和可读性，同时帮助确保数据库的数据完整性和一致性。

1.2.3. 数据库连接规范

本章节主要介绍 GoldenDB 中进行数据库连接的规范，描述 GoldenDB 连接配置建议和注意事项。

连接准则：

- 安全性：使用安全的身份验证机制：使用强密码对数据库进行身份验证，避免使用默认的或者弱密码避免明文传输。
- 连接池管理：合理管理数据库连接池，避免每次数据库操作都创建新的连接，提高性能和资源利用率。
- 日志记录：记录数据库连接的建立、释放和异常，便于故障排查和性能优化。
- 参数化查询：使用参数化查询：避免拼接 SQL 字符串，而是使用参数化查询的方式，防止 SQL 注入攻击。

1.2.4. 数据库字符集规范

本章节主要介绍 GoldenDB 中进行字符集设置的规范，描述 GoldenDB 字符集设置建议和注意事项。

1. 字符集支持

- 可以在租户级、Database 级、表级、字段级、session 级设置字符集 `latin1`、 `gbk`、`utf8`、 `utf8mb4` 和 `gb18030` 等字符集。

2. 字符集查看

- 租户级字符集可通过 Insight 查看默认值。
- Database、表、字段级可通过 `SHOW CREATE DATABASE XX`, `SHOW CREATE TABLE XX` 查看字符集。
- 通过 SQL 命令查看默认字符集。

```
> SHOW VARIABLES LIKE '%character_set_server%';
```

3. 不同场景下字符集设置

- 设定租户级默认字符集。

```
Insight ->租户管理 -> 选中租户 ->配置管理 -> 设定`character_set_server`字符集
```

- 设置节点默认字符集。

```
> SET NAMES utf8;
```

- 创建数据库，可指定字符集。

```
> CREATE DATABASE IF NOT EXISTS XXXX CHARACTER SET utf8;
```

- 创建表，可直接指定字符集。

```
> CREATE TABLE XXXX (gdbds INT PRIMARY KEY) CHARACTER SET gbk;
```

- 字段指定字符集。

```
> CREATE TABLE XXXX (gdbds INT PRIMARY KEY, c1 CHAR(10) CHARACTER SET utf8mb4);
```

4. 不同连接方式下字符集设置

- 使用 `JDBC` 连接 GoldenDB , GBK 链路一般在 url 里修改参数设置，添加 `characterEncoding=gbk`

```
String url = "jdbc:goldendb://xx.xx.xx.xx:xxx/db?"+"user=xxxx&password=xxxx&characterEncoding=gbk";
```

- 使用客户端连接后，可通过 `SET NAMES` 指定字符集。

```
> SET NAMES utf8mb4;
```

1.2.5. 存储模型设计规范

GoldenDB 行存支持 COMPACT 和 DYNAMIC 等模式，默认是 DYNAMIC 模式，由于 COMPACT 格式和 DYNAMIC 存储上差异不大，本文以 DYNAMIC 模式为例进行行存和透明加密的分析。

首先描述行格式示意图，如下图

变长字段长度列表	NULL 值列表	记录头信息	row_id	trx_id	roll_ptr	列1值	列2值	...	列n值

行格式示意图

1. 变长字段长度列表：记录字段的真实长度，单个字段长度表示不超过 2 个字节，普通类型列值不超过 65535，该列是逆序放置。
2. NULL 值列表：记录值是 NULL 的标识，用 bit 位标识，最小 1 个字节单位，不够一个字节用 0 填充，该列是逆序放置。
3. 记录头信息：5 个字节大小，记录行的信息包括删除标识位，排序记录，下一条记录的位置等，如下图。

记录头信息5字节（40bit）							
1	1	1	1	4	13	3	16
保留	保留	删除标志	最小记录标志	组记录数量	记录相对位置	记录类型	下一条记录相对位置

4. `row_id` 隐藏列，如果没有定义主键，添加该列，占 6 个字节。
5. `trx_id` 隐藏列，事务 ID，占 6 字节。
6. `roll_ptr` 隐藏列，回滚指针，占 7 字节。

示例：

1. 第一字段固定长度表

```
> CREATE TABLE tb_a(a_1 CHAR(10), a_2 CHAR(10), a_3 CHAR(10), a_4 CHAR(10), a_5 CHAR(10), a_6 CHAR(10), a_7 CHAR(10), a_8 CHAR(10),
```

```
a_9 CHAR(10), a_10 CHAR(10), a_11 CHAR(10), a_12 CHAR(10),a_13 CHA
R(10), a_14 CHAR(10),a_15 CHAR(10), a_16 CHAR(10), a_17 CHAR(10),
a_18 CHAR(10));
> INSERT INTO tb_a(a_1) VALUES('abc');
> INSERT INTO tb_a(a_1) VALUES('def');
> INSERT INTO tb_a(a_1) VALUES('hij');
```

```
0a 03 ff fe 00 00 10 00 26 00 00 00 00 02 02 00 00 00 00 09 84 82 00 00 00 9f 01 10 61 62 63 20
20 20 20 20 20 20     'abc'
0a 03 ff fe 00 00 18 00 26 00 00 00 00 02 03 00 00 00 00 09 87 82 00 00 00 a0 01 10 64 65 66 20
20 20 20 20 20 20     'def'
0a 03 ff fe 00 00 20 ff a3 00 00 00 00 02 04 00 00 00 00 09 8e 82 00 00 00 a2 01 10 68 69 6a 20
20 20 20 20 20 20     'hij'
```

第一字段固定长度示例描述

上图中第一行是插入的 `abc` 行记录，第一个红框字段长度列表十六进制 0a 表示 10 (下面相同)，该列长度是 10，黄框 03fffe 逆序可知第一列非 NULL，其余 NULL，绿色框是头信息，蓝色框是 `row_id`,灰色框是事务ID，第二个红色框是回滚指针地址，该红框后是固定 10 个字节的数据，后面依次类推。

2. 第一字段可变长度表

```
> CREATE TABLE tb_b(a_1 VARCHAR(10), a_2 CHAR(10), a_3 CHAR(10), a
_4 CHAR(10), a_5 CHAR(10), a_6 CHAR(10), a_7 CHAR(10), a_8 CHAR(10
), a_9 CHAR(10), a_10 CHAR(10), a_11 CHAR(10), a_12 CHAR(10),a_13
CHAR(10), a_14 CHAR(10),a_15 CHAR(10), a_16 CHAR(10), a_17 CHAR(10
), a_18 CHAR(10));
> INSERT INTO tb_b(a_1) VALUES('abc');
> INSERT INTO tb_b(a_1) VALUES('def');
> INSERT INTO tb_b(a_1) VALUES('hij');
```

```
03 03 ff fe 00 00 10 00 1f 00 00 00 00 02 05 00 00 00 00 09 a7 81 00 00 00 ac 01 10 61 62 63
  'abc'
03 03 ff fe 00 00 18 00 1f 00 00 00 00 02 06 00 00 00 00 09 aa 81 00 00 00 ad 01 10 64 65 66
  'def'
03 03 ff fe 00 00 20 ff b1 00 00 00 00 02 07 00 00 00 00 09 b5 81 00 00 00 b1 01 10 68 69 6a
  'hij'
```

第一字段可变长度示例描述

上图中第一行是插入的 `abc` 行记录，第一个红框字段长度列表十六进制 03 表示 3 (下面相同)，该列长度是 3，黄框 03fffe 逆序可知第一列非 NULL，其余 NULL，绿色框是头信息，蓝色框是 `row_id`, 灰色框是事务ID，第二个红色框是回滚指针地址，该红框后是 3 个字节的数据 `abc` 的 ASCII 码，后面依次类推。

1.3. 连接数据库

主要介绍使用不同编程语言连接到 GoldenDB。

1.3.1. 使用客户端连接

GoldenDB 完全兼容 `MySQL` 客户端，在安装好的 `MySQL` 客户端环境下，直接使用如下命令连接:

```
mysql -u username -p'password' -h host -P port
```

参数说明

参数	描述
-u	用户名。
-p	密码。
-h	数据库地址。
-P	数据库端口。

1.3.2. 使用驱动连接

使用 GoldenDB 官方提供的 `JDBC` 驱动。

1.3.2.1. Java 语言连接

- 直接连接

```
public static void main(String[] args) {
        String url = "jdbc:goldendb://host:port/test";
        String pass = "****";
        String username = "****";
        String sql = "insert into good (id, name) values (1, 'name')";
        Connection connection = null;
        Statement st = null;
        try {
            connection = DriverManager.getConnection(url, username, pass);
            st = connection.createStatement();
            st.execute(sql);
        } catch (SQLException e) {
```

```
            if(st != null){
                try {
                    st.close();
                } catch (SQLException ex) {
                    throw new RuntimeException(ex);
                }
            }
            if(connection != null){
                try {
                    connection.close();
                } catch (SQLException ex) {
                    throw new RuntimeException(ex);
                }
            }
            throw new RuntimeException(e);
        }
    }
```

- 负载均衡连接

GoldenDB 提供客户端负载均衡，负载均衡策略有：随机、最小响应优先和指定列表。

```
public static void main(String[] args) {
        String url = "jdbc:goldendb:loadbalance://host1:port1,h
ost2:port2,host3:port4 ... /test";
        String pass = "****";
        String username = "****";
        String sql = "insert into good (id, name) values (1, 'n
ame')";
        Connection connection = null;
        Statement st = null;
        try {
            connection = DriverManager.getConnection(url, usern
ame, pass);
            st = connection.createStatement();
            st.execute(sql);
        } catch (SQLException e) {
            if(st != null){
                try {
                    st.close();
                } catch (SQLException ex) {
                    throw new RuntimeException(ex);
                }
            }
            if(connection != null){
                try {
                    connection.close();
                } catch (SQLException ex) {
                    throw new RuntimeException(ex);
                }
            }
```

```
            throw new RuntimeException(e);
        }
    }
```

1.3.2.2. C++ 语言连接

通过 C++ 语言写一个小程序文件连接数据库，示例如下：

```
#include<sql.h>
#include<sqlext.h>
#include<string.h>
#include<time.h>
#include<stdlib.h>
#include<stdio.h>  C++

#define isSuc(result) ((result) == SQL_SUCCESS || (result) == SQL_
SUCCESS_WITH_INFO)

/** 可扩展设置 **/
char szConnectString[2560] ="DRIVER={/usr/lib64/libmyodbc5a.so};US
ER=xxx;PASSWORD=xxx;DATABASE=xx;PORT=8880;SERVER=10.10.10.10;SAFE=
1;";
char szSQLDrop[256] = "drop table if exists t1;";
char szSQLCreate[256] = "CREATE TABLE t1(id int NOT NULL primary k
ey,name char(50) NOT NULL,age int NOT NULL)distributed by hash(id)
(g1)";
/** 输出缓存 **/
char szDriverOutput[256];
short sDriverOutputLength;

/** 用于测试的函数 **/
int main()
{
  short sret; //返回代码
  void* henv;  //环境句柄
  void* hdbc;  //连接句柄
  long mode;//提交模式
  void* hsmt; //语句句柄

  /** 申请环境句柄 **/
  sret = SQLAllocHandle(SQL_HANDLE_ENV,SQL_NULL_HANDLE,&henv);
  if(!isSuc(sret)){printf("申请环境句柄出错\n");return 0;}
  /** 设置环境属性,声明ODBC版本 **/
  sret = SQLSetEnvAttr(henv,SQL_ATTR_ODBC_VERSION,(void*)SQL_OV_OD
BC3,SQL_IS_INTEGER);
  if(!isSuc(sret)){printf("声明ODBC版本出错\n");return 0;}
  /** 申请连接句柄 **/
  sret = SQLAllocHandle(SQL_HANDLE_DBC,henv,&hdbc);
  if(!isSuc(sret)){printf("申请连接句柄出错\n");return 0;}
  /** 连接数据源 **/
  sret = SQLDriverConnect(hdbc,NULL,(SQLCHAR*)szConnectString,SQL_
```

```
NTS,(SQLCHAR *)szDriverOutput,2560,&sDriverOutputLength,SQL_DRIVER
_NOPROMPT);
  if(!isSuc(sret)){printf("连接数据源出错\n");return 0;}
  else printf("\nCreate connection success !\n\n");
  /** 分配语句句柄 **/
  sret = SQLAllocHandle(SQL_HANDLE_STMT,hdbc,&hsmt);
  if(!isSuc(sret)){printf("分配语句句柄出错\n");return 0;}
  /** 执行create语句 **/
  printf("execute sql: %s\n\n", szSQLCreate);
  sret = SQLExecDirect(hsmt,(SQLCHAR *)szSQLCreate,256);
  if(!isSuc(sret)){printf("exec create sql fail !\n\n");return 0;}
  else printf("create table t1(new) success !\n\n");

  /** 释放语句句柄 **/
  SQLFreeHandle(SQL_HANDLE_STMT,hsmt);
  /** 提交连接的事务 **/
  SQLEndTran(SQL_HANDLE_DBC,hdbc,SQL_COMMIT);
  /** 断开与数据源的连接 **/
  SQLDisconnect(hdbc);
  /** 释放连接句柄 **/
  SQLFreeHandle(SQL_HANDLE_DBC,hdbc);
  /** 释放环境句柄 **/
  SQLFreeHandle(SQL_HANDLE_ENV,henv);
  printf("The connection will be closed !\nBye !\n");
}
```

1.3.2.3. Go 语言连接

通过 `Go` 语言写一个小程序文件连接数据库，示例如下：

```
package main

import (
    "database/sql"
    "fmt"
    _ "github.com/go-sql-driver/goldenDB"
    "log"
)

func main() {
    // 构建连接串
    dsn := "username:password@tcp(ip:port)/xxx"

    // 连接数据库
    db, err := sql.Open("mysql", dsn)
    if err != nil {
        log.Fatal(err)
    }
    defer db.Close()

    // 测试连接
```

```
    err = db.Ping()
    if err != nil {
        log.Fatal(err)
    }
    fmt.Println("Connected to the database successfully!")

    // 查询表 "good" 的第一个字段
    rows, err := db.Query("SELECT name FROM good")
    if err != nil {
        log.Fatal(err)
    }
    defer rows.Close()

    // 循环表的行
    for rows.Next() {
        var firstField string
        err := rows.Scan(&firstField)
        if err != nil {
            fmt.Println("读取行失败:", err.Error())
            return
        }
        fmt.Println("First field in table 'good':", firstField)
    }

    err = rows.Err()
    if err != nil {
        log.Fatal(err)
    }
}
```

1.3.2.4. Python 语言连接

通过 `Python` 语言写一个小程序文件连接数据库，示例如下：

```
# -*- coding: utf-8 -*-

import pymysql
# 连接相关参数
conn = pymysql.connect(
    host='xx.xx.xx.xx',
    port=xxxx,
    user='xxxx',
    password='xxxx',
    database='xxx',
)

# 创建游标对象
cursor = conn.cursor()

# 执行 SQL 查询语句
cursor.execute("select * from good")
```

```
# 获取查询结果
result = cursor.fetchall()
# 获取第一条参数
first_column_values = [row[0] for row in result]
print(first_column_values.pop())
# 关闭资源
cursor.close()
conn.close()
```

1.3.3. 使用连接池连接

主要介绍不同的连接池框架。

1.3.3.1. Druid + Springboot 框架

`Druid` 是一个高性能的 `Java` 数据库连接池，它不仅提供了基本的连接池功能，还具备监控、容错、可扩展性等多种实用特性，因此在许多企业级应用中得到了广泛应用。

pom.xml

使用 `maven` 来管理项目，其中 `pom.xml` 中引入框架依赖，`pom.xml` 文件示例如下：

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="htt
p://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://mav
en.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.2.6.RELEASE</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>org.example</groupId>
  <artifactId>test-durid</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>
  <name>test-durid</name>
  <url>http://maven.apache.org</url>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncod
ing>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
```

```
    <dependency>
      <groupId>com.alibaba</groupId>
      <artifactId>druid-spring-boot-starter</artifactId>
      <version>1.1.22</version>
    </dependency>
    <dependency>
      <groupId>org.mybatis.spring.boot</groupId>
      <artifactId>mybatis-spring-boot-starter</artifactId>
      <version>2.2.2</version>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-logging</artifactId>
    </dependency>
    <dependency>
      <groupId>log4j</groupId>
      <artifactId>log4j</artifactId>
      <version>1.2.17</version>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

application.yml

`Springboot` 配置连接相关参数,`application.yml` 文件示例如下:

```
server:
  port: 8888
spring:
  # 数据源配置
  datasource:
    type: com.alibaba.druid.pool.DruidDataSource
    driver-class-name: com.goldendb.jdbc.Driver
    url: jdbc:goldendb://ip:port/wmm
    username: ****
    password: ****
    druid:
      # 初始化大小，最小，最大
```

```
      initial-size: 10
      min-idle: 10
      max-active: 100
      # 配置获取连接等待超时的时间
      max-wait: 1000
      # 配置间隔多久才进行一次检测，检测需要关闭的空闲连接，单位毫秒
      time-between-eviction-runs-millis: 60000
      # 配置一个连接在池中最小生存时间
      min-evictable-idle-time-millis: 300000
      validation-query: SELECT 1 from dual
      test-while-idle: true
      test-on-borrow: true
      test-on-return: true
logging:
  level:
    root: info
mybatis:
  mapper-locations: classpath:mapper/*Mapper.xml
```

核心参数说明

参数	描述
driver-class-name	驱动类完整名称。
initial-size	初始化时建立物理连接的个数。
min-idle	最小连接池数量。
max-active	最大连接池数量。
max-wait	获取连接时最大等待时间，单位毫秒。
time-between-eviction-runs-millis	配置间隔多久才进行一次检测，检测需要关闭的空闲连接，单位毫秒。
min-evictable-idle-time-millis	连接保持空闲而不被驱逐的最小时间
validation-query	用来检测连接是否有效的sql。
test-while-idle	申请连接时检测，如空闲时间大于 timeBetweenEvictionRunsMillis，执行 validationQuery 检测连接是否有效。

test-on-borrow	申请连接时执行 validationQuery 检测连接是否有效。
test-on-return	归还连接时执行 validationQuery 检测连接是否有效。

1.3.3.2. Hikari + Springboot 框架

`HikariCP` 是一个高性能的 `JDBC` 连接池，广泛应用于 `Java` 应用程序中。它以其简洁和高效而闻名，适应了许多高负载的企业级应用程序需求。

pom.xml

使用 `maven` 来管理项目，其中 `pom.xml` 中引入框架依赖，`pom.xml` 文件示例如下：

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="htt
p://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://mav
en.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>org.example</groupId>
  <artifactId>test-hikari</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.4.3</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <name>test-hikari</name>
  <url>http://maven.apache.org</url>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncod
ing>
    <java.version>1.8</java.version>
  </properties>
    <dependencies>
      <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
      </dependency>
      <!--加入数据源的启动依赖: springboot启动时会加载对应的自动装配类。--
>
      <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-jdbc</artifactId>
      </dependency>
      <dependency>
```

```
        <groupId>org.mybatis.spring.boot</groupId>
        <artifactId>mybatis-spring-boot-starter</artifactId>
        <version>2.2.2</version>
      </dependency>
      <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-logging</artifactId>
      </dependency>
      <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
      </dependency>
    </dependencies>
</project>
```

application.yml

`Springboot` 配置连接相关参数,`application.yml` 文件示例如下:

```
spring:
  datasource:
    driver-class-name: com.goldendb.jdbc.Driver
    username: ****
    password: ****
    url: jdbc:goldendb://ip:port/test
    hikari:
      connection-timeout: 20000
      idle-timeout: 60000
      minimum-idle: 10
      maximum-pool-size: 10
      max-lifetime: 100000
      auto-commit: true
server:
  port: 8888
logging:
  level:
    root: debug
mybatis:
  mapper-locations: classpath:mapper/*Mapper.xml
```

核心参数说明

参数	描述
driver-class-name	驱动类完整名称。
connection-timeout	获取连接等待连接池超时时间,最小值 250ms 。

idle-timeout	连接最大空闲时间，最小值 10s，可以为0，代表永远不会因为空闲被移除。
minimum-idle	最小空闲连接数。
maximum-pool-size	连接池中最大连接数。
max-lifetime	连接在连接池中最大生存时间，正在使用的连接不会检测，最小值 30s。

1.4. 创建和管理数据库对象

本小章节旨在介绍开发人员如何实现用户和数据库对象的管理，以及常用数据类型。

1.4.1. 关于DDL语句

本章节简单介绍 GoldenDB 中 DDL 语句的基本支持情况，重点包括：支持的 DDL 语句列表、DDL 在线离线支持情况。

DDL 支持的数据库对象范围

- 用户（`User`）： 创建或删除用户，并分配相应的权限。
- 数据库（`Database`）：DDL 支持创建和删除数据库实例，以及对数据库的结构进行修改。
- 表（`Table`）：DDL 语句可用于创建、修改和删除表格，包括表格的列、数据类型、约束条件等。
- 索引（`Index`）：DDL 语句可用于创建、修改和删除索引，用于加快数据检索操作的速度。
- 视图（`View`）：DDL 语句可用于创建、修改和删除视图，定义视图的查询逻辑和字段。

1.4.2. 数据类型

介绍常用的数据类型支持情况。

1.4.2.1. 常用数据类型

数值类型

GoldenDB 数值类型	描述

整数类型	INT/SMALLINT/TINYINT/MEDIUMINT/BIGINT
定点类型	DECIMAL、NUMERIC
浮点类型	FLOAT、DOUBLE
位值类型	BIT

时间和日期数据类型

GoldenDB 时间日期类型	描述
DATE	只包含日期部分 如：2023-12-28
TIMESTAMP	日期时间（考虑时区）如： 2023-12-28 10:00:00
DATETIME	日期时间（不考虑时区）如： 2023-12-28 10:00:00
TIME	时间 如：10:00:00
YEAR	年 如：2023

字符类型/大对象类型

GoldenDB 字符类型	描述
字符类型	CHAR/VARCHAR/TEXT/VARBINARY/BINARY/SET/ENUM
大对象类型	BLOB/CLOB

示例：

```
> CREATE TABLE tb_1 (
int_1 INT ,
smallint_1 SMALLINT,
tinyint_1 TINYINT ,
mediumint_1 MEDIUMINT,
bigint_1 BIGINT ,
decimal_1 DECIMAL(65,30) ,
```

```
numeric_1 NUMERIC(65,30) ,
float_1 FLOAT,
double_1 DOUBLE,
date_1 DATE,
timestamp_1 TIMESTAMP,
datetime_1 DATETIME,
time_1 TIME,
year_1 YEAR,
varchar_1 VARCHAR(15) ,
char_1 CHAR(250),
text_1 TEXT,
binary_1 BINARY(255),
varbinary_1 VARBINARY(20),
blob_1 BLOB,
clob_1 CLOB,
enum_1 ENUM('S','M','L','XL','XXL','XXXL'),
set_1 SET('ONE','TWO', 'THREE','FOUR')
);
```

1.4.3. 用户、密码和权限

介绍用户、密码及权限的支持情况。

1.4.3.1. 创建和管理用户

1. 描述

数据库管理系统中创建和维护用户账号以及管理其权限和访问控制。通过创建和管理用户，可以实现对数据库的安全访问和精细管理。

2. 语法

创建用户

```
> CREATE USER [IF NOT EXISTS]
  user auth_option
  [password_option] ...

auth_option: {
  IDENTIFIED BY 'auth_string'
}

password_option: {
  PASSWORD EXPIRE [DEFAULT | NEVER | INTERVAL N DAY]
  | PASSWORD HISTORY {DEFAULT | N}
  | PASSWORD REUSE INTERVAL {DEFAULT | N DAY}
}
```

删除用户

```
> DROP USER user
```

用户重命名

```
> RENAME USER old_user TO new_user
```

- 更详细说明详见 SQL 规范。

3. 示例

```
> CREATE USER 'jeffrey' IDENTIFIED BY 'password';
> CREATE USER 'jeffrey'@'10.0.0.0' IDENTIFIED BY 'password';
> RENAME USER jeffrey TO tony;
> DROP USER jeffrey;
```

4. 注意事项

- 建议通过 Insight 页面创建用户并进行管理。
- 定义权限：在创建用户时，需根据需求为用户分配适当的权限，以控制其对数据库资源的访问和操作。
- 密码安全：设置用户密码时应注意密码的复杂性和安全性，以免遭到破解或不当访问。
- 最小权限原则：根据安全性原则，应为用户授予最小必需的权限，以降低风险和提高数据库安全性。
- 系统管理员账号：对于创建和管理用户的操作，需要使用数据库的系统管理员账号或具备管理员权限的账号。
- 定期审查和更新：定期审查和更新用户的权限和访问控制，以适应组织变动和安全需求的变化。

1.4.3.2. 设置密码

1. 描述

数据库设置密码是指对数据库用户或管理员账户设置访问密码，以确保数据库安全性和权限控制。通过设置密码，可以防止未经授权的用户访问数据库，同时实现权限的细粒度控制。

2. 语法

```
> ALTER USER [IF EXISTS]
  user [auth_option]
  [password_option] ...

auth_option: {
  IDENTIFIED BY 'auth_string'
}
```

```
password_option: {
  PASSWORD EXPIRE [DEFAULT | NEVER | INTERVAL N DAY]
  | PASSWORD HISTORY {DEFAULT | N}
  | PASSWORD REUSE INTERVAL {DEFAULT | N DAY}
}
```

- 更详细说明详见 SQL 规范。

3. 示例

```
> ALTER USER 'jeffrey'@'10.0.0.0' IDENTIFIED BY '123!@#$%&123aA1'
```

4. 注意事项

- 密码复杂性：设置密码时应注意密码的复杂性和安全性，建议使用包括大小写字母、数字和特殊字符的复杂密码。
- 定期审查：应定期审查数据库用户的密码设置，确保密码符合安全要求，并及时更新或修正弱密码。
- 安全保存：数据库密码应当被安全地保存，不应当以明文形式存储在任何地方。

1.4.3.3. 设置权限

1. 描述

数据库设置权限是指对数据库用户、角色或对象（如表、视图等）进行授权，以控制其对数据库资源的访问和操作权限。通过设置权限，可以实现对数据库的安全访问和细粒度的权限管理。

2. 支持的权限列表

- SELECT：允许用户查询数据。
- INSERT：允许用户插入数据。
- UPDATE：允许用户更新数据。
- DELETE：允许用户删除数据。
- CREATE：允许用户创建数据库、表、视图、函数等对象。
- DROP：允许用户删除数据库、表、视图、函数等对象。
- ALTER：允许用户修改数据库、表、视图的结构。
- GRANT：允许用户授予他人权限。
- REFERENCES：允许用户创建外键关系。
- EXECUTE：允许用户执行存储过程、函数等。
- INDEX：允许用户创建索引。
- ALL ： 允许用户拥有所有权限。

3. 语法

GRANT 授权

```
> GRANT
  priv_type
  [, priv_type] ...
  ON [object_type] priv_level
  TO user
  [WITH GRANT OPTION]

object_type: {
  TABLE
  | FUNCTION
  | PROCEDURE
}

priv_level: {
  *
  | *.*
  | db_name.*
  | db_name.tbl_name
  | tbl_name
  | db_name.routine_name
}
```

REVOKE 撤销授权

```
> REVOKE
  priv_type
  [, priv_type] ...
  ON [object_type] priv_level
  FROM user
  REVOKE ALL [PRIVILEGES], GRANT OPTION
  FROM user
```

- 更详细说明详见 SQL 规范。

4. 示例

```
> GRANT ALL ON db1.* TO 'jeffrey'@'10.0.0.0';
> GRANT SELECT ON db2.invoice TO 'jeffrey'@'10.0.0.0';
> REVOKE INSERT ON *.* FROM 'jeffrey'@'localhost';
```

5. 注意事项

- 定义权限：在授予权限时，需根据需求为用户或角色分配适当的权限，以控制其对数据库资源的操作。
- 最小权限原则：根据安全性原则，应为用户或角色授予最小必需的权限，以降低风险和提高数据库安全性。

- 角色管理：在复杂的权限管理场景下，可以考虑使用角色进行权限管理，以减少重复和管理复杂度。
- 安全性：权限设置需要谨慎操作，确保不会授予过多权限或者不当权限，以避免安全风险。

1.4.4. 创建和管理数据库

介绍创建和管理数据库对象的支持情况。

1.4.4.1. 创建、查看、修改和使用数据库

1. 描述

本节主要介绍在 GoldenDB 系统中如何建立，查看，修改和使用数据库。

2. 语法

创建数据库

```
> CREATE [DATABASE | SCHEMA] [IF NOT EXISTS] db_name
  [create_specificatio]
  [database_sharding_specification]

create_specification:
  ENCRYTION = {'Y'  'N'}
  /*!80016 DEFAULT ENCRYPTION = {'Y'  'N'} */
  [DEFAULT] CHARACTER SET [=] charset_name

database_sharding_specification:
  LIST(sharding_item_list);

sharding_item_list:
  sharding_item
  | sharding_item_list, sharding_item ;

sharding_item:
  pg[num]( def_item_list ) ;

def_item_list:
  def_item
  | def_item_list, def_item ;

def_item:
  g[num]
  | g[num]<lg_db_name>
  | g[num] - g[num]
```

- 字符集支持：`latin1`、`gbk`、`utf8`、`utf8mb4` 和 `gb18030`。

- 字符序支持按照二进制值进行，即对应上述四种字符集的字符序分别为 `latin1_bin`、`gbk_bin`、`utf8_bin`、`utf8mb4_bin` 和 `gb18030_bin`。

查看数据库

```
> SHOW {DATABASES | SCHEMAS} [LIKE 'pattern' ]
> SHOW CREATE {DATABASE | SCHEMA} [IF NOT EXISTS] db_name
```

修改数据库属性

```
> ALTER {DATABASE | SCHEMA} [db_name]
    alter_specification ...

alter_specification: {
    [DEFAULT] CHARACTER SET [=] charset_nam
}
```

使用数据库

```
> USE db_name
```

- 更详细说明详见 SQL 规范。

3. 示例

```
> CREATE DATABASE IF NOT EXISTS db1 CHARACTER SET utf8;
> CREATE DATABASE encryption3 ENCRYPTION = 'Y';
> CREATE DATABASE encryption2 /*!80016 DEFAULT ENCRYPTION='Y'*/;
> CREATE DATABASE IF NOT EXISTS db3 CHARACTER SET UTF8 LIST(pg1(g1
-g4), pg2(g5-g8));
> CREATE DATABASE collate_1_20190620 COLLATE utf8_bin;
> SHOW DATABASES;
> ALTER DATABASE db1 CHARACTER SET gbk;
> USE tb1;
```

4. 注意事项

- 执行创建数据库操作需要相应的管理员或创建数据库权限。
- 应避免创建重复名称的数据库，以免造成混淆和冲突。
- 查看数据库的权限是较为普遍的，大部分用户都应该有该操作的权限。
- 使用数据库前，需要确保已经具有相应的数据库连接和操作权限。

1.4.4.2. 删除数据库和闪回

1. 描述

主要介绍如何删除系统中已存在的数据库，以及闪回技术。数据库闪回是一种用于恢复数据库到某个特定时间点的技术。

2. 语法

删除数据库

```
> DROP [DATABASE | SCHEMA] [IF EXISTS] db_name
```

库闪回恢复

```
> FLASHBACK DATABASE db_name [for recyclebin_version 'version'
] to before drop;
```

- 更详细说明详见 SQL 规范。

3. 示例

```
> DROP DATABASE db1;
```

4. 注意事项

- 删除数据库操作是不可逆的，删除后将无法恢复，因此在执行之前请务必进行确认。
- 执行删除数据库操作需要相应的管理员权限。

1.4.5. 创建和管理表

介绍创建和管理表对象的支持情况。

1.4.5.1. 创建表

1. 描述

创建表是指在数据库中定义表的结构，包括表名、各字段的数据类型、约束条件等，为数据库中的数据存储提供了框架和规范。

2. 语法

```
> CREATE [temporary] TABLE [IF NOT EXISTS] table_name
  (create_definition, ...)
  [table_options]
  [partition_options]
  [distributed_option]
  [select_statement]
  | LIKE table_name
```

```
create_definition: {
  col_name column_definition
  | [CONSTRAINT [symbol]] PRIMARY KEY [index_type] (index_col_name
,...)
  [index_option] ...
  | {INDEX|KEY} [index_name] [index_type] (index_col_name,...)
  [index_option] ...
  | [CONSTRAINT [symbol]] UNIQUE [INDEX|KEY]
  [index_name] [index_type] (index_col_name,...)
  [index_option] ...
  | GLOBAL PRIMARY KEY [index_type] (index_col_name,...)
  [index_option] ...
  | GLOBAL [UNIQUE] [INDEX|KEY]
  [index_name] [index_type] (index_col_name,...)
  [index_option] ...
}

column_definition: {
  data_type [NOT NULL | NULL] [DEFAULT default_value]
  [AUTO_INCREMENT][UNIQUE [KEY] | [PRIMARY] KEY | GLOBAL PRIMARY K
EY|
  GLOBAL UNIQUE [KEY]][COMMENT 'string']
}

table_options: {
  AUTO_INCREMENT [=] value
  | COMMENT [=] 'string'
  | COMPRESSION [=] {'ZLIB' | 'LZ4' | 'NONE'}
  | [DEFAULT] CHARACTER SET [=] charset_name
  | ENGINE [=] engine_name
  | MAX_ROWS [=] value
  | ROW_FORMAT [=] {DEFAULT|DYNAMIC|FIXED|COMPRESSED|REDUNDANT|COM
PACT}
  | [DEFAULT] CHARACTER SET [=] charset_name
  | CONNECTION [=] 'connect_string'
  | KEY_BLOCK_SIZE [=] value
}

partition_options: {
  PARTITION BY
  { [LINEAR] HASH(expr)
  | [LINEAR] KEY [ALGORITHM={1|2}] (column_list)
  | RANGE{(expr) | COLUMNS(column_list)}
  | LIST{(expr) | COLUMNS(column_list)} }
  [PARTITIONS num]
  [(partition_definition [, partition_definition] ...)]
  [PARTITIONSTORAGEDB(group_list)
  (partition_options)]
}

partition_definition: {
  PARTITION partition_name
```

```
  [VALUES
  {LESS THAN {(expr | value_list) | MAXVALUE}
  |
  IN (value_list)}]
  [COMMENT [=] 'comment_text' ]
}

distributed_option: {
  DISTRIBUTED BY
  HASH(column_list) ({groupno},…)
  | RANGE(expr) ({groupno  VALUES LESS THAN { value | MAXVALUE}},…
)
  | RANGE(expr)({groupno [value,value)},…)
  | RANGE(expr)({groupno [value,value) [value,value)...},…)
  | LIST(expr) ({groupno  VALUES IN (value_list | DEFAULT)},…)
  | DUPLICATE ({groupno},…)
}
```

- 更详细说明详见 SQL 规范。

3. 示例

```
> CREATE TABLE titles(
    title_id INT NOT NULL,
    title VARCHAR(80) NOT NULL,
    pub_id VARCHAR(4),
    PRIMARY KEY (title_id)
)DISTRIBUTED BY HASH(title_id) (g1,g2);
> CREATE TABLE test.t1 (
    year_col INT,
    col2 CHAR(5) NOT NULL PRIMARY KEY)
DISTRIBUTED BY RANGE(year_col)(
    g1 VALUES LESS THAN (1991),
    g2 VALUES LESS THAN (1995),
g3 VALUES LESS THAN MAXVALUE);
> CREATE TABLE test.t1 (
    year_col INT,
    col2 CHAR(5) NOT NULL PRIMARY KEY)
DISTRIBUTED BY LIST(year_col)(
    g1 VALUES IN (1991,1992,1993),
    g2 VALUES IN (1994,1995));
> CREATE TABLE titles(
    title_id INT NOT NULL,
    title VARCHAR(80) NOT NULL,
    pub_id VARCHAR(4) NULL,
    PRIMARY KEY (title_id)
)DISTRIBUTED BY DUPLICATE(g1);
> DELIMITER //
> CREATE TABLE `his_entrust_detail` (
`cust_no` int NOT NULL comment "客户号，用作近期查询的分发键",
`name` varchar(10) NOT NULL DEFAULT 'zhangsan',
`adress` varchar(10) NOT NULL DEFAULT 'beijing',
```

```
`id` int NOT NULL,
`entrust_date` varchar(10) not NULL, -- 日期
PRIMARY KEY (entrust_date,id)
)
partition by range columns(entrust_date)(
partition p2011 values less than ('20120101'),
partition p2012 values less than ('20130101'),
partition p2013 values less than ('20140101'),
partition p2014 values less than ('20150101'),
partition p2015 values less than ('20160101'),
partition p2016 values less than ('20170101'),
partition p2017 values less than ('20180101'),
partition p2018 values less than ('20190101'),
partition p2019 values less than ('20200101'),
partition p2020 values less than ('20210101'),
partition p2021 values less than ('20220101')
)
distributed by
case when entrust_date >= '20180101'
  then subdistributed by hash(cust_no)(g1,g2);
else subdistributed by range(entrust_date)(
g3 values less than ('20140101'),
g4 values less than ('20180101')
);
END CASE //
> DELIMITER ;
> CREATE TABLE employees3 (
    id INT NOT NULL,
    fname VARCHAR(30),
    lname VARCHAR(30),
    hired DATE NOT NULL DEFAULT '1970-01-01',
    separated DATE NOT NULL DEFAULT '9999-12-31',
    job_code INT NOT NULL,
    store_id INT NOT NULL key
)
PARTITION BY RANGE (store_id) (
    PARTITION p0 VALUES LESS THAN (6),
    PARTITION p1 VALUES LESS THAN (11),
    PARTITION p2 VALUES LESS THAN (16),
    PARTITION p3 VALUES LESS THAN (21)
)
distributed by range(id)(g1[0,100),g2[100,300),g3[300,1000));
```

4. 注意事项

- 在创建表时，需谨慎定义字段的数据类型和约束条件，以确保能满足数据存储和操作的需求。
- 考虑字段之间的关系，如主键、外键等约束条件的设置，以便维护数据的完整性和一致性。
- 遵循命名规范，为表和字段选择有意义的名称，并且要注意避免使用数据库关键字作为名称。

- 考虑表的性能需求，合理设计表结构，避免过多重复字段和不必要的字段。
- 确保有相应的权限来创建表，需要数据库管理员或具有创建表权限的用户来执行该操作。

1.4.5.2. 修改表

1. 描述

对已存在的数据库表进行结构、约束条件或其他属性的修改。通过修改表，可以添加、删除和修改表的列、约束条件、索引等，以及修改表的各种属性。

2. 语法

```
> ALTER TABLE tbl_name
  [alter_specification]
  [partition_options]  [distributed_option | distributed_force_opt
ion]
```

- 更详细说明详见 SQL 规范。

3. 示例

```
> ALTER TABLE db2.t2 ADD COLUMN c1 int, ADD COLUMN c2 int;
> ALTER TABLE db2.t2 DISTRIBUTED BY RANGE(a)(g1 VALUES LESS THAN(1
00), g2 VALUES LESS THAN MAXVALUE);
> ALTER TABLE db.t2 MODIFY a TINYINT NOT NULL;
> ALTER TABLE t1 DROP COLUMN d;
> ALTER TABLE t1 RENAME TO t1_new;
> ALTER TABLE t1 TRUNCATE PARTITION p0 UPDATE GLOBAL
INDEXES STORAGEDB G1;
```

4. 注意事项

- 在修改表时，需谨慎处理数据的完整性和一致性，确保修改不会导致数据丢失或破坏数据关系。
- 在修改表结构之前，最好先备份相关数据，以防操作失误导致数据丢失。
- 修改表的操作可能会导致表锁定，因此在生产环境中需注意操作的时机，避免对业务产生影响。
- 对于较大的表，修改表结构可能需要较长的时间来完成操作，应合理评估时间和资源成本。

1.4.5.3. 清空表和闪回

1. 描述

清空表是指删除表中的所有数据，而不删除表本身。闪回是一种用于恢复表数据到某个特定时间点或事务时刻的技术。闪回功能开启的情况下，清空数据后，可通过闪回功能恢复。

2. 语法

清空表

```
> TRUNCATE [TABLE] tbl_name ;
```

闪回恢复

```
> FLASHBACK TABLE tbl_name [for recyclebin_version 'version']
TO BEFORE TRUNCATE;
```

- 更详细说明详见 SQL 规范。

3. 示例

```
> TRUNCATE TABLE db1.t1;
> FLASHBACK table t01 TO BEFORE TRUNCATE;
> SELECT ORIG_DB_NAME,ORIG_TABLE_NAME,VERSION_ID,OPERATION FROM in
formation_schema.recyclebin;
> FLASHBACK table t01 for recyclebin_version '20231229105900_758
472_189804627'  TO BEFORE TRUNCATE;
```

4. 注意事项

- 清空表操作是不可逆的，清空后将无法恢复，请在执行之前务必进行确认。
- 清空表操作将会释放存储空间并且效率较高，适用于清除表中所有数据而保留表结构的情况。
- 执行表闪回的操作需要开启相应的功能配置。
- 表闪回操作将还原表数据到指定时间点或事务时刻，因此在执行之前需要谨慎验证和确认。
- 表闪回操作可能会受到数据库大小、日志保留策略等因素的影响，需确保系统有足够的存储空间和日志记录来支持该操作。

1.4.5.4. 删除表和闪回

1. 描述

删除表是指移除数据库中的表及其相关的数据和结构。闪回是一种在关系型数据库中恢复已删除、修改或丢失数据的技术。它允许在某个时间条件下恢复数据库中的数据。闪回功能开启后，可通过闪回恢复已删除数据。

2. 语法

删除表

```
> DROP TABLE [IF EXISTS] tbl_name
```

表闪回恢复

```
> FLASHBACK TABLE tbl_name [for recyclebin_version 'version']
to before DROP;
```

- 更详细说明详见 SQL 规范。

3. 示例

```
> DROP TABLE t01 ;
> SELECT ORIG_DB_NAME,ORIG_TABLE_NAME,VERSION_ID,OPERATION from in
formation_schema.recyclebin;
> FLASHBACK TABLE t01 for recyclebin_version '20231115154945_375
564_1581664292' to before drop;
```

4. 注意事项

- 闪回恢复功能需要事先配置再启用。
- 删除表将永久移除表及其相关数据和结构，慎重使用。
- 备份重要数据以防误操作或数据丢失是一个良好的实践。

1.4.5.5. 拷贝表

1. 描述

拷贝表是一种常见的数据库操作，可用于备份表、创建测试环境、迁移数据等场景。它可以复制表的结构以及其中的所有数据，或者只复制表的结构而不包括数据。

2. 语法

```
> CREATE TABLE <new_table> AS SELECT * FROM <original_table>;
> CREATE TABLE <new_table> LIKE <original_table>;
```

- 更详细说明详见 SQL 规范。

3. 示例

```
> CREATE TABLE TB_2 AS SELECT * FROM TB_1;
> CREATE TABLE t1 LIKE t2;
```

4. 注意事项

- 确保目标表不存在，或者使用合适的方式处理已存在的目标表。

- 包含在 `SELECT` 语句中的其他条件可用于过滤数据，例如选择特定条件下的数据进行复制。

1.4.6. 创建和管理索引

1.4.6.1. 关于索引

1. 支持的索引类型

- 单列索引：基于单个列的索引。
- 复合索引：基于多个列的组合索引。
- 唯一索引：要求索引列的值必须唯一。

2. 索引的优点和缺点

优点

- 加快数据检索速度：通过索引，数据库可以快速定位到需要的数据行，而不必进行全表扫描。
- 提高数据库的性能：对于大型数据表或经常查询的列，使用索引可以显著提高查询性能。
- 保持数据完整性：唯一索引可以确保数据一致性，避免出现重复数据。

缺点

- 占用额外存储空间：索引需要额外的存储空间，对于大型表可能会有较大的存储开销。
- 降低写操作性能：当进行插入、更新或删除操作时，数据库需要维护索引，这可能降低写操作的性能。
- 维护成本：当表发生结构变化时，索引需要重新构建和维护，增加了数据库维护的成本。

3. 索引的使用原则和注意事项

- 选择合适的索引：根据实际查询需求和数据分布选择合适的索引类型。
- 避免过度索引：不必要的索引会增加存储开销和维护成本，也可能降低写操作性能。
- 注意索引列的顺序：对于复合索引，索引列的顺序应该根据查询的使用频率进行优化。
- 定期维护索引：对于频繁更新的表，定期重建索引以保持查询性能。

1.4.6.2. 单节点索引

1. 描述

单节点索引是一种在分布式数据库中的索引类型，它在每个数据节点上分别存储索引信息。通过建立某一列或多列的索引来提高查询效率。当数据库中的数据量较小或者访问压力较小时，使用单节点索引可以满足查询需求，同时也能够提高查询性能。

2. 语法

创建索引

```
> CREATE [UNIQUE] INDEX index_name
  ON tbl_name (index_col_name,...)
  [index_type]
```

删除索引

```
> DROP INDEX index_name ON tbl_name
```

- 更详细说明详见 SQL规范。

3. 示例

```
> CREATE INDEX idx1 ON db1.t1(a);
> CREATE UNIQUE INDEX idx2 ON db1.t2(b);
> CREATE INDEX idx_hash_3 ON db1.t3(c) USING HASH;
> DROP INDEX idx1 ON t1;
```

4. 注意事项

- 单节点索引适合于数据量较小、访问压力较小的场景。
- 在分布式数据库环境下，需确保数据分布均匀，避免某些节点数据过大而导致索引性能下降。

1.4.6.3. 分布式全局索引

1. 描述

分布式全局索引是一种在分布式数据库环境下使用的索引，可以在整个分布式数据库集群中加速数据的检索操作。

2. 语法

创建全局索引

```
> CREATE GLOBAL INDEX index_name
  ON tbl_name (index_col_name,...)
> CREATE GLOBAL UNIQUE INDEX index_name
  ON tbl_name (index_col_name,...)
```

删除全局索引

```
> DROP INDEX index_name ON tbl_name
```

- 更详细说明详见 SQL规范。

3. 示例

```
> CREATE GLOBAL INDEX idx2 ON db1.t2(b);
> CREATE GLOBAL UNIQUE INDEX idx2 ON db1.t2(b);
> DROP INDEX idx1 ON t1;
```

4. 注意事项

- 在设计和创建分布式全局索引时，需要考虑整个集群的数据分布和访问模式，以确保索引的设计能够提高整个集群的查询性能。
- 由于涉及多节点的数据访问，分布式全局索引的维护和同步可能会带来额外的成本和复杂性，需要仔细权衡。
- 不支持在复制表上创建全局索引、全局唯一索引。
- 创建全局索引、全局唯一索引失败后，需使用【drop remain index 全局索引名 on 表名或库名.表名】来清理中间过程的残留（包括数据节点上的索引以及索引辅助表）。

1.4.7. 创建和管理视图

1. 描述

视图是根据 `SELECT` 查询语句的结果集生成的虚拟表，它不包含实际的数据，而是根据调用时动态生成。视图可以简化复杂查询、提供数据安全性、隐藏实际表结构，以及简化应用程序中的查询逻辑。

2. 语法

创建视图

```
> CREATE
  [OR REPLACE]
  [ALGORITHM = {UNDEFINED | MERGE | TEMPTABLE}]
  [DEFINER = { user | CURRENT_USER }]
  [SQL SECURITY { DEFINER | INVOKER }]
  VIEW view_name [(column_list)]
  AS select_statement
  [WITH CHECK OPTION]
```

修改视图

```
> ALTER
  [ALGORITHM = {UNDEFINED | MERGE | TEMPTABLE}]
  [DEFINER = { user | CURRENT_USER }]
  [SQL SECURITY { DEFINER | INVOKER }]
  VIEW view_name [(column_list)]
```

```
  AS select_statement
  [WITH CHECK OPTION]
```

删除视图

```
> DROP VIEW [IF EXISTS]
  view_name [, view_name] ...
  [RESTRICT | CASCADE]
```

- 更详细说明详见 SQL规范。

3. 示例

```
> CREATE VIEW v AS SELECT qty, price, qty*price AS value FROM
t;
> ALTER VIEW view_students_info AS SELECT id,name,age FROM tb_stud
ents_info;
> DROP VIEW view_students_info;
```

4. 注意事项

- 视图的列名和数据类型要准确匹配查询结果的列名和数据类型。
- 视图的性能受到基础查询语句的性能影响，应当考虑基础表的索引和查询性能。
- 视图的创建和使用应当符合安全和隐私规范，不应暴露不应该被访问的数据。
- 视图的更新可能受限于其基础表的约束和触发器，需要注意视图的可更新性。
- 视图可以嵌套使用，提供灵活的数据层抽象，但也要注意避免过度复杂的视图结构导致维护困难。

1.5. 数据写入

本章节旨在向开发人员简要介绍数据写入、更新、替换和删除的方法，详细使用方法及参数详见SQL 规范。

1.5.1. 插入数据

本节简单介绍基础的通用的数据插入以及批量数据插入的方法。

1.5.1.1. 通用插入

描述

- 通用插入是一种用于向数据库表中插入数据的常见操作。它允许你在一次操作中插入一行或多行数据，提供了一种便捷的方式来向表中添加数据。

语法

```
INSERT INTO tbl_name
  (col_name,...)
  {VALUES (col_values [,col_values]...),(...),... | select_stateme
nt}
  [ ON DUPLICATE KEY UPDATE
  col_name=expr [, col_name=expr] ... ]
```

示例

```
> INSERT INTO t1 (a,b,c) VALUES (1,2,3);
> INSERT INTO t1 (a,b,c) VALUES (1,2,3) ON DUPLICATE KEY UPDATE c=
c+1;
> INSERT INTO t1 (a,b,c) VALUES (1,2,3),(4,5,6) ON DUPLICATE KEY U
PDATE c=VALUES(a)+VALUES(b);
> INSERT INTO t1 (a,b,c) VALUES (1,2,3),(2,3,4)...;
```

注意事项

- 表结构匹配：确保插入的数据与表结构匹配，即确保插入的数据类型与表中对应列的数据类型相符。
- 主键冲突：避免插入主键重复的数据，因为主键必须是唯一标识符。可以使用唯一的自动生成主键或在插入前进行查询以避免重复。
- 非空约束：避免插入空值到设置为非空的列中，除非该列有默认值或可以接受空值。
- 数据完整性：遵守表中定义的约束条件，如外键约束等，以保持数据的完整性。
- 事务处理：根据需要，将插入操作放在事务中来确保数据的一致性和回滚的能力。

1.5.1.2. 批量插入

描述

- 批量插入是一种在数据库中一次性插入多行数据的操作，相比单行插入，批量插入可以提高插入效率，减少与数据库的交互次数。

语法

```
INSERT INTO tbl_name
  [PARTITION number]
  (col_name,...)
  {VALUES (col_values [,col_values]...),(...),... | select_stateme
nt}
  [ ON DUPLICATE KEY UPDATE
  col_name=expr [, col_name=expr] ... ]
```

示例

```
> INSERT INTO t1 (a, b) SELECT * FROM t2;
> INSERT INTO t1 (a, b) SELECT * FROM (SELECT c, d FROM t2 UNION S
ELECT e, f FROM t3) AS dt ON DUPLICATE KEY UPDATE b = b + c;
```

注意事项

- 批量大小：根据系统资源和性能需求，合理选择批量插入的大小。
- 数据源格式：如果从外部数据源导入数据，确保数据源的格式正确，并处理好数据类型转换、字符编码等问题。

1.5.2. 更新数据

1.5.2.1. 通用更新

描述

- 在数据库中，更新操作用于修改已经存在的数据行。通用更新是指不需要使用特殊的条件和语法格式，可以更新任意列的数据。

语法

```
UPDATE table_reference
  SET col_name1=expr [, col_name2=expr ...]
  [WHERE where_condition]
  [ORDER BY ...]
  [LIMIT row_count]
```

示例

```
> UPDATE tb1 SET col1=1 WHERE id=1;
```

注意事项

- 值类型匹配：更新操作只能更新列中的值，而不能更改列的数据类型。需要注意更新值的数据类型与列的数据类型是否匹配。
- WHERE 子句：在更新操作中，WHERE 子句非常重要，必须使用正确的 WHERE 子句来限定更新范围。否则可能会将表中所有行的数据都更新。
- 数据完整性：遵守表中定义的约束条件，如唯一约束或外键约束等，以保持数据的完整性。

1.5.2.2. 批量更新

描述

- 批量更新是指同时更新多条数据行的操作。它可以用来一次性更新满足某种条件的多个数

据行。

语法

```
UPDATE table_reference
  SET col_name1=expr [, col_name2=expr ...]
  [WHERE where_condition]
  [ORDER BY ...]
  [LIMIT row_count]
```

示例

```
> UPDATE tb SET (coi1,col2)=(SELECT id1,id2 FROM tb2 WHERE ...)
> UPDATE tb SET (a_code,b_code)
> UPDATE t1_hash SET (a_code,b_code)
= (SELECT coalesce(sum(a_code),0),max(b_code) from t1_hash WHERE E
XISTS (SELECT 1 FROM t1_range WHERE t1_hash.infom=t1_range.infom))
;
> UPDATE t1 SET a_code = (a_code +(SELECT coalesce(max(d_code),0)
FROM t2 WHERE t1.id=t2.id));
> UPDATE tb1 SET(col1,col2)=(SELECT id1,id2 FROM tb2 WHERE id=1);
```

注意事项

- 确定更新条件：在批量更新时，要确保设置恰当的条件，以精确指定要更新的数据行。否则，可能会误更新其他不符合需求的数据行。
- 大批量更新性能：如果需要执行大批量更新操作，请考虑数据库的性能和系统资源，合理控制批量大小，以避免数据- 库负载过高和事务处理时间过长。
- 数据验证：在批量更新之前，对要更新的数据进行验证和清洗，确保数据的正确性和合法性。

1.5.3. 删除数据

1.5.3.1. 通用删除

描述

- 通用删除是指从数据库表中删除满足指定条件的数据行的操作。它可以根据一定的条件删除单个或多个数据行。

语法

```
DELETE  FROM tbl_name
  [PARTITION number]
  [WHERE where_condition]
  [ORDER BY ...]
  [LIMIT row_count]
```

示例

```
> DELETE FROM t1 WHERE user = 'jcole';
> DELETE FROM t1 WHERE user = 'jcole' ORDER BY id LIMIT 2;
```

注意事项

- 确定删除条件：在执行删除操作时，要确定恰当的条件，以精确指定要删除的数据行。否则，可能会误删其他不符合需求的数据行。
- 索引注意：在删除大量数据时，需要注意关联的索引和触发器，以避免过多的索引重建和触发器的触发，造成性能问题。
- 注意外键约束：如果存在外键约束，删除数据时需先考虑和处理相关的外键约束，以避免违反完整性规则。

1.5.3.2. 批量删除

描述

- 批量删除是指一次性删除满足指定条件的多个数据行的操作。它可以根据一定的条件删除多个数据行，而不仅仅是单个数据行。

语法

```
DELETE  FROM tbl_name
  [PARTITION number]
  [WHERE where_condition]
  [ORDER BY ...]
  [LIMIT row_count]
```

示例

```
> DELETE FROM tab2 WHERE string = ((SELECT initcap(string) FROM ta
b1 WHERE id = 9));
> DELETE FROM t1 WHERE t1.num IN (SELECT tt.aa FROM (select MAX(nu
m) as aa,name FROM t1 GROUP BY name) tt) SW;
```

注意事项

- 确定删除条件：在执行删除操作时，要确定恰当的条件，以精确指定要删除的数据行。否则，可能会误删其他不符合需求的数据行。
- 索引注意：在删除大量数据时，需要注意关联的索引和触发器，以避免过多的索引重建和触发器的触发，造成性能问题。
- 注意外键约束：如果存在外键约束，删除数据时需先考虑和处理相关的外键约束，以避免违反完整性规则。
- 数据量注意：删除大量数据行可能会导致数据库操作变慢，因此需要谨慎选择删除操作的

时机。

1.5.4. 替换数据

描述

- 使用新的数据替换表中已有的数据。替换操作包括删除已有数据并插入新数据，或者使用特定的替换语句来实现。

语法

```
UPDATE table_reference
  SET col_name1=expr [, col_name2=expr ...]
  [WHERE where_condition]
  [ORDER BY ...]
  [LIMIT row_count]
```

示例

```
> UPDATE t1 SET i = -1;
> UPDATE t1_hash SET (a_code,b_code) = (SELECT coalesce(SUM(a_code
),0),MAX(b_code) FROM t1_hash WHERE EXISTS (SELECT 1 FROM t1_range
 WHERE t1_hash.infom=t1_range.infom));
> UPDATE t1 SET a_code = (a_code +(SELECT coalesce(MAX(d_code),0)
FROM t2 WHERE t1.id=t2.id));
```

注意事项

- 数据完整性：在替换数据时，需要确保新数据符合表结构定义和约束，避免违反数据完整性规则。
- 索引和性能：对于大量替换操作，需考虑索引、触发器等对性能的影响，谨慎选择操作时机和方式。

1.6. 数据查询

本章节旨在向开发人员简要介绍数据查询方法，详细使用方法及参数详见 5.10 SQL 规范章节。

1.6.1. 单表查询

描述

单表查询是在数据库中对单个表执行查询操作，从中获取所需的数据。它是最基本的数据库查询形式之一，用于检索表中的记录。

语法

```
SELECT
  expr , ...
  FROM table_references
  [WHERE where_condition]
  [PARTITION number]
  [GROUP BY col_name]
  [HAVING where_condition]
  [ORDER BY col_name [ASC | DESC], ...]
  [LIMIT [offset,] row_count ]
```

示例

```
> SELECT emp_name, salary FROM employee;
> SELECT emp_name, salary FROM employee STORAGEDB g1;
```

注意事项

- 在编写 `SELECT` 查询语句时，需要确保列名的准确性，避免拼写错误。确保 `WHERE` 子句中的条件准确反映了所需的筛选条件，避免查询结果不符合预期。

1.6.2. 条件查询

描述

条件查询是指在数据库中根据指定的条件来筛选出符合条件的数据记录，从而实现更精确的数据检索。

语法

```
SELECT
  expr , ...
  FROM table_references
  [WHERE where_condition]
  [PARTITION number]
  [GROUP BY col_name]
  [HAVING where_condition]
  [ORDER BY col_name [ASC | DESC], ...]
  [LIMIT [offset,] row_count ]
```

示例

```
> SELECT * FROM employee WHERE salary > 5000;
> SELECT * FROM employee WHERE salary > 5000 STORAGEDB g1;
```

注意事项

- 条件查询在确定查询条件时要确保条件表达式的准确性，避免漏掉符合条件的记录或获取到错误的数据。

1.6.3. 排序查询

描述

排序查询是指根据指定的列对查询结果进行排序，以便更方便地查看和分析数据。

语法

```
SELECT
  expr , ...
  FROM table_references
  [WHERE where_condition]
  [PARTITION number]
  [GROUP BY col_name]
  [HAVING where_condition]
  [ORDER BY col_name [ASC | DESC], ...]
  [LIMIT [offset,] row_count ]
```

示例

```
> SELECT * FROM employee ORDER BY salary DESC;
```

注意事项

- 当涉及到排序查询时，需要考虑排序列的数据类型和索引情况，以避免影响查询性能。
- 在对多个列进行排序时，需要考虑不同列的组合对排序结果的影响。

1.6.4. 分组查询

描述

分组查询是在数据库中将数据按照指定的列进行分组，然后对每个分组进行聚合操作，以便进行更复杂的数据统计和分析。

语法

```
SELECT
  expr , ...
  FROM table_references
  [WHERE where_condition]
```

```
  [PARTITION number]
  [GROUP BY col_name]
  [HAVING where_condition]
  [ORDER BY col_name [ASC | DESC], ...]
  [LIMIT [offset,] row_count ]
```

示例

```
> SELECT c1, COUNT(c2) c2 FROM t1 GROUP BY c1;
> SELECT name, group_concat(code separator ' ') FROM td GROUP BY n
ame;
```

注意事项

- 分组查询通常与聚合函数（如 COUNT、SUM、AVG 等）一起使用，以便对每个分组进行统计计算。

1.6.5. 分页查询

描述

分页查询是指在数据库中一次性获取大量数据时，将结果分成多个页面展示，以提高用户体验并减轻服务器负担。

语法

```
SELECT
  expr , ...
  FROM table_references
  [WHERE where_condition]
  [PARTITION number]
  [GROUP BY col_name]
  [HAVING where_condition]
  [ORDER BY col_name [ASC | DESC], ...]
  [LIMIT [offset,] row_count ]
```

示例

```
> SELECT * FROM t01 LIMIT 5;
> SELECT * FROM t01 LIMIT 0,5;
> SELECT * FROM t01 LIMIT 5,10;
```

注意事项

- 分页查询的 `offset` 表示从第几条记录开始，从 0 开始计数。
- 分页查询的 `count` 表示要返回的记录数量。

- 在分页查询时，需要注意处理结果集的总记录数，以便正确计算总页数和显示分页导航。

1.6.6. 多表查询

描述

多表查询是指在数据库中同时查询多个表，并根据表之间的关系获取相关的数据记录。

语法

```
SELECT
  expr , ...
  FROM table_references
  [WHERE where_condition]
  [PARTITION number]
  [GROUP BY col_name]
  [HAVING where_condition]
  [ORDER BY col_name [ASC | DESC], ...]
  [LIMIT [offset,] row_count ]
```

示例

```
> SELECT * FROM testjoina a, testjoinb b WHERE a.id=b.id1(+);
> SELECT t1.name, t2.salary FROM employee AS t1, info AS t2 WHERE
t1.name = t2.name;
```

注意事项

- 在多表查询中，需要确保 JOIN 子句中的连接条件是正确的，以避免获取错误的查询结果。
- 多表查询可能涉及到表之间的关联关系，如主键和外键，需要正确理解表的结构和关系。

1.6.7. 子查询

描述

子查询是指在数据库查询中嵌套一个查询语句作为另一个查询语句的一部分，用于实现更复杂的数据检索。

语法

```
SELECT
  expr , ...
  FROM table_references
  [WHERE where_condition]
  [PARTITION number]
```

```
  [GROUP BY col_name]
  [HAVING where_condition]
  [ORDER BY col_name [ASC | DESC], ...]
  [LIMIT [offset,] row_count ]
```

示例

```
> SELECT * FROM employee WHERE salary > (SELECT AVG(salary) FROM e
mployee);
> SELECT * FROM (SELECT * FROM t1 WHERE (id,id1) IN ((1,1),(2,2)))
;
> SELECT * FROM t1_hash WHERE a_code NOT IN (SELECT max(c_code) FR
OM t1_range WHERE t1_hash.id=t1_range.id) AND id=3 ORDER BY id;
```

注意事项

- 子查询的结果集应尽量控制在合理范围内，以减少查询执行时间和资源消耗。过大的结果集可能会导致性能问题。
- 子查询的性能可能较低，特别是当子查询的结果集较大时。在设计查询时需要考虑性能优化。
- 子查询嵌套层数过多可能导致查询语句难以理解和维护，建议尽量简化和优化查询逻辑。

1.6.8. 视图查询

描述

视图（View）是虚拟的表，是由一个 SQL 查询来定义的。视图包含行和列，就像一个真实的表一样，但是视图并不在数据库中以存储的数据形式存在。视图中存储的是根据 SQL 查询语句检索出的数据。视图查询允许用户以一种简化的方式访问和操作视图的数据，而无需关心底层的表结构和复杂的查询逻辑。

语法

```
SELECT
  expr , ...
  FROM table_references
  [WHERE where_condition]
```

示例

```
> SELECT employee_name, salary FROM employee_view WHERE salary > 5
000;
```

注意事项

- 视图查询的性能可能受到视图的定义、查询复杂性、底层表的性能等多个因素的影响。在设计视图和执行视图查询时，需要考虑查询的效率和性能。
- 视图查询可以用于简化复杂的查询逻辑，提高查询的可读性和可维护性。通过视图查询，可以隐藏底层表的细节，仅关注所需的数据子集。
- 对视图进行修改或删除操作时，需要注意相关表和视图之间的依赖关系，以避免不必要的影响。

1.6.9. 临时表查询

描述

临时表查询是指在数据库中对已创建的临时表进行数据检索、更新、删除等操作。临时表查询与普通表的查询类似，允许用户对临时存储的数据进行各种 SQL 操作，以满足特定的业务需求或数据处理需求。

语法

```
SELECT
  expr , ...
  FROM table_references
  [WHERE where_condition]
```

示例

```
> CREATE TEMPORARY TABLE IF NOT EXISTS tb3(a INT KEY, b INT, c CHA
R, d INT,e INT) DISTRIBUTED  BY HASH(b)(g1,g2);
> SELECT * FROM tb3;
```

注意事项

- 对临时表进行查询操作时，与普通表的操作方式基本一致，可以使用标准的 GoldenDB 查询语句和操作符进行数据检索、更新和删除。
- 当会话或连接结束时，临时表中的数据将被自动清除，不会对其他会话或连接产生影响，因此需要在会话结束前手动清理临时表中的数据。
- 在使用临时表时，应注意避免滥用临时表，避免引起不必要的混乱和数据错乱。

1.6.10. 高级查询

1.6.10.1. MPP查询

描述

MPP 是分布式 SQL 查询引擎，适用于交互式分析查询，允许你查询存储于 GoldenDB 中的 GB 到 PB 量级的数据，并可实现联邦查询。MPP 利用多个 Worker 线程可以轻松实现并行查询，

并可以通过集群进行横向扩展，增加计算节点增强查询效率。

2. 复杂查询示例

hint 注释控制是否走 MPP

```
> /*!parallel-processing=on*/SELECT COUNT(*) FROM t1;
```

3. DBLINK查询示例

```
> CREATE extended DATABASE test_dblink WITH(dbtype='goldendb',addr
ess='xx.xx.xx.xx',port=xxxx,database='db_1',username='dbproxy',pas
sword='xxxx',charset='ZHS16GBK');
> SELECT * FROM test_01@test_dblink;
> SELECT * FROM db_1.test_01@test_dblink;
```

4. 注意事项

- 目前只支持 UR 级别（读未提交）的事务隔离级别。CR 隔离级别暂时不支持。UR 由配置生效，不能以 hint 形式加在语句中。
- MPP 只能查询静态的数据。当前事务中的 DML 操作的数据，对于 MPP 都是不可见的。
- MPP 目前只支持 DQL。其中 INSERT、UPDATE 派生的 DQL 也支持。

建议：事务中暂不要使用 MPP。

1.6.10.2. 流式查询

1. 简介

- GoldenDB 流式查询是一种能够针对大型结果集进行分页处理的技术，它不需要等待整个结果集的完成，就可以将部分结果返回给客户端并开始处理。从而提供更快速的响应和更高的用户体验。

语法

SQL 语句添加 hint

```
/*+ SET_VAR(use_flex_stream_fetch = ON) */
```

示例

```
> SELECT /*+ SET_VAR(use_flex_stream_fetch = ON) */ tb1.id , tb2.l
ocation FROM tb1 JOIN tb2 ON tb1.id = tb2.id ORDER BY tb1.id, tb2.
location;
```

注意事项

- 需要二次计算的查询语句，不支持支持流式查询。

1.6.10.3. 闪回查询

描述

闪回查询是指查询历史数据的能力。

语法

```
SELECT
  expr , ...
  FROM table_references
  AS OF TIMESTAME expr...
  [WHERE where_condition]
```

示例

```
> SELECT salary FROM employees
   AS OF TIMESTAMP (SYSTIMESTAMP - INTERVAL '1' MINUTE)
   WHERE last_name = 'ABC';
```

注意事项

- 执行闪回查询期间，不能锁表。
- undo 空间需要有归档和回收空间的功能。

1.6.10.4. 读写分离

描述

读写分离是指用户的读操作和写操作可以根据实际需要分发到主 DB 或不同备 DB 上执行，一方面可以缓解主 DB 的压力，另一方面提高备 DB 的利用率。

语法

```
1 | 在 Insight 页面上，租户管理->实例[cluster]->服务端口->编辑 子页面可以配置服务端口的读写分离规则。
2 | SELECT 后面加 readslave。
```

示例

```
> SELECT * FROM t1 readslave;
```

4. 注意事项

- 读语句的隔离级别为 UR。

1.6.10.5. 跨集群查询

描述

跨集群查询是指在分布式数据库系统中跨多个独立数据库集群执行查询操作。这种查询方式可用于跨多个数据中心或地理位置的数据库集群之间进行数据聚合和分析。使用跨机器查询，需先部署 `SuperProxy`，登录 `SuperProxy` 所在的主机。查询语法和普通查询语法一致。

示例

```
> SELECT * FROM tb;
```

注意事项

- 跨集群查询通常需要配置适当的安全措施，以确保数据的保密性和完整性。
- 在进行跨集群查询时，应该考虑到数据一致性和同步的问题，特别是在分布式环境中更为重要。

1.7. 事务

本章主要介绍分布式事务、`XA` 事务、事务控制语句以及事务隔离级别等与事务相关的使用方法及注意事项。

1.7.1. 分布式事务

1. 简介

事务是数据库管理系统执行过程中的一个逻辑单位，由一个有限的数据库操作序列构成。它是数据库最小的工作单元，是不可以再分的，它可能包含了一个或者一系列的 `DML` 语句，包括 `INSERT、 DELETE、UPDATE`。

事务的四大特性：ACID。

- 原子性(`Atomicity`)：事务中的所有操作要么全部执行成功，要么全部失败回滚，没有中间状态。
- 一致性(`Consistent`)：事务开始前和结束后，数据库的完整性约束没有被破坏。
- 隔离性(`Isolation`)：并发执行的事务之间是相互隔离的，一个事务的执行不应影响其他事务的执行。
- 持久性(`Durability`)：一旦事务提交，其结果应该是永久性的，即使系统发生故障，事务的结果也不应丢失。

分布式事务就是指事务的参与者、支持事务的服务器、资源服务器以及事务管理器分别位于不同的分布式系统的不同节点之上。分布式事务遵循 CAP 原则，又称 CAP 定理，指的是在一个分布式系统中一致性(`Consistency`)、可用性(`Availability`)、分区容错性(`Partition tolerance`)三者不可兼容。

- 一致性：在分布式系统中的所有数据备份，在同一时刻是否同样的值，等同于所有节点访问同一份最新的副本。
- 可用性：在集群中一部分节点故障后，集群整体是否还能响应客户端的读写请求，对数据的更新具备高可用性。
- 分区容错性：以实际效果而言，分区相当于对通信的时限要求，系统如果不能在时限内达成数据一致性，就意味着发生了分区的情况，必须就当前操作在一致性与可用性之间做出选择。

GoldenDB 分布式事务的核心思想是引入全局事务管理器(GTM：Global Transaction Manager)，记录当前所有正在执行的全局事务标识(`GTID`)及其状态,作为分布式事务的全局标识，通过全局事务机制实现全局事务的隔离性，并协助实现全局事务的原子性。

GoldenDB 分布式事务是借鉴单机事务实现，与单机事务的区别在于 GoldenDB 会将所有的表中增加 `GTID` 隐含列，该列在分布式事务中存放该全局事务 `ID` (每个分布式写事务会都得到一个全局唯一且递增的整形值)，在执行完所有 `SQL` 后，所有 DN 节点统一进行 `COMMIT`。如果有 DN 节点 `COMMIT` 提交失败，所有节点的 DBAgent 通过解析 `binlog` 中是否含有该异常事务的全局事务 `ID` 来构造反向回滚语句，并向 DN 下发该反向回滚语句，进行事务回滚，从而保证分布式事务的数据原子性。故 GoldenDB 分布式事务是具有 `ACID` 特性的。

2. 使用说明

要在 GoldenDB 中显示开启事务，以下是普通事务支持的语句和简要描述：

事务语句	描述
START TRANSACTION [CONSISTENCY_OPTION][SAMEDB]/BEGIN	开启一个事务。
COMMIT	提交事务。
ROLLBACK TO SAVEPOINT identifier	回滚事务。
SAVEPOINT	事务保存点。

3. 示例

```
> START TRANSACTION;
> INSERT INTO test(id,name) VALUES (1,'Alex');
> COMMIT;
```

4. 注意事项

- 如果一致性级别设置在事务和SQL语句中互相冲突时，以SQL语句中一致性级别为准。
- 不支持 `SET TRANSACTION` 语法，如：`SET TRANSACTION READ ONLY`。
- 在所有存储程序（存储过程和函数、 触发器等）中，解析器将 `BEGIN [WORK]` 视为 `BEGIN ... END` 块，若要在此上下文中开始事务，请使用 `START TRANSACTION` 代替。

1.7.2. XA事务

1. 简介

最早的分布式事务模型是 `X/Open` 国际联盟提出的 `X/Open Distributed Transaction Processing（DTP）` 模型，简称 `XA` 协议。`DTP` 模型中包含一个全局事务管理器 `（TM，Transaction Manager）` 和多个资源管理器 `（RM，Resource Manager）`。

全局事务管理器负责管理全局事务状态与参与的资源，协同资源一起提交或回滚；资源管理器则负责具体的资源操作。`XA` 协议描述了 `TM` 与 `RM` 之间的接口，允许多个资源在同一分布式事务中访问，基于 `DTP` 模型的分布式事务流程大致如下：

- 应用程序（`AP，Application`）向 `TM` 申请开始一个全局事务。
- 针对要操作的 `RM`，`AP` 会先向 `TM` 注册（`TM` 负责记录 `AP` 操作过哪些 `RM`，即分支事务），`TM` 通过 `XA` 接口函数通知相应 RM 开启分布式事务的子事务，接着 AP 就可以对该 RM 管理的资源进行操作。
- 当 `AP` 对所有 `RM` 操作完毕后，`AP` 根据执行情况通知 `TM` 提交或回滚该全局事务，`TM` 通过 `XA` 接口函数通知各 RM 完成操作。`TM` 会先要求各个 `RM` 做预提交，所有 `RM` 返回成功后，再要求各 `RM` 做正式提交，`XA` 协议要求，一旦 `RM` 预提交成功，则后续的正式提交也必须能成功；如果任意一个 `RM` 预提交失败，则 `TM` 通知各 `RM` 回滚。
- 所有 `RM` 提交或回滚完成后，全局事务结束。

GoldenDB 的 `XA` 实现使 GoldenDB 服务器能够充当资源管理器，用于处理全局 `XA` 事务,连接到 GoldenDB 服务器的客户端程序充当事务管理器。当全局事务进入到提交阶段后，事务管理器采用两阶段提交协议保证该全局事务的原子性：

- 准备阶段：事务管理器向所有参与者发送准备请求，并等待它们的响应；参与者在收到准备请求后，将事务的状态（准备就绪或不准备）通知事务管理器。
- 提交阶段：如果所有参与者都准备就绪，事务管理器向所有参与者发送提交请求，参与者在收到提交请求后，执行事务的最终提交，并通知事务管理器；如果任何一个参与者在准

备阶段表示不准备或在提交阶段发生错误，事务管理器会向所有参与者发送回滚请求。

`XA` 协议中没有描述如何实现分布式事务的隔离性，但是 `XA` 协议要求 `DTP` 模型中的每个 `RM` 都要实现本地事务，也就是说，基于 `XA` 协议实现的分布式事务的隔离性是由每个 `RM` 本地事务的隔离性来保证的，当一个分布式事务的所有子事务都是隔离的，那么这个分布式事务天然的就实现了隔离性。

2. 使用说明

要在 GoldenDB 中执行 `XA` 事务，以下是 `XA` 事务支持的语句和简要描述：

XA 事务语句	描述
XA {START/BEGIN} xid	开启一个 XA 事务。
XA END xid	标志一个 XA 事务结束。
XA PREPARE xid	将一个 XA 事务预提交。
XA COMMIT xid [ONE PHASE]	XA COMMIT 将一个 XA 事务提交，XA COMMIT xid ONE PHASE 在 XA END 后执行， 将 XA 事务以一阶段协议提交。
XA ROLLBACK xid	将一个已提交的 XA 事务回滚。
XA RECOVER [CONVERT XID]	将一个预提交的 `XA` 事务回滚。

说明: xid 是一个 XA 事务标识符，其值由客户端或 GoldenDB 服务器生成，包含三个部分：xid: gtrid [,bqual [, formatid]]。其中，gtrid 是一个全球事务标识符，bqual 是一个分支限定符，formatID 为数字标识 gtrid 和 bqual 值使用的格式。

3. 示例

- 开启 `XA` 事务，使用 `XA PREPARE` 提交

```
> XA START 'xatest';
> INSERT INTO mytable (i) VALUES(10);
> XA END 'xatest';
> XA PREPARE 'xatest';
> XA COMMIT 'xatest';
```

- 开启 `XA` 事务，使用一阶段协议提交

```
> XA START 'xatest';
> INSERT INTO mytable (i) VALUES(10);
> XA END 'xatest';
> XA COMMIT 'xatest' ONE PHASE;
```

4. 注意事项

- 不能在普通事务中开启 `XA` 事务
- 不能在 `XA` 事务中开启普通事务
- 部分语句类型不能在 `XA` 事务中执行，如 `DDL` 语句
- 不支持 `xid` 内容为空，如 `XA START ""`
- GoldenDB 语法中，`data+formatID` 相同认为是同一个 `xid`,其中 `data` 包含 `gtrid` 与 `bqual` 。例如 `"aa","bb"`, 与 `"aab","b"`,认为是同一个 `xid`。

1.7.3. 事务控制语句

本节介绍了几种事务控制的语句，主要包括：事务开启、事务保存点、事务提交、事务回滚。

1.7.3.1. 开启事务

1. 简介

标志事务的开始，在 `COMMIT` 或 `ROLLBACK` 之前的所有操作将视为一个一个逻辑单位，要么全部执行成功，要么全部执行失败回滚。

2. 语法格式和说明

GoldenDB 中开启事务的语句和简要描述：

事务开启语句	描述
START TRANSACTION [CONSISTENCY_OPTION][SAMEDB] /BEGIN/XA {START/BEGIN} xid	开启事务。

3. 示例

- 执行 `BEGIN` 语句，开启普通事务。

```
> BEGIN;
> INSERT INTO test(id,name) VALUES (1,'Alex');
```

```
> COMMIT;
```

- 执行 `START TRANSACTION` 语句，开启普通事务。

```
> START TRANSACTION;
> INSERT INTO test(id,name) VALUES (1,'Alex');
> COMMIT;
```

- 执行 `XA START xid` 或 `XA BEGIN xid` 语句，开启 `XA` 事务。

```
> XA START 'A-1-1'; | XA BEGIN 'A-1-1';
> INSERT INTO test(id,name) VALUES (1,'Alex');
> XA END 'A-1-1';
> XA PREPARE 'A-1-1';
> XA COMMIT 'A-1-1';
```

4. 注意事项

- 不支持 `xid` 内容为空，如 `XA START ''`。
- GoldenDB 语法中，`data+formatID` 相同认为是同一个 `xid`,其中 `data` 包含 `gtrid` 与 `bqual` 。例如：`xid 'aa','bb',7` 与 `xid 'aab','b',7` 为同一个 `xid`。

1.7.3.2. 事务保存点

1. 简介

事务过程中标记一个"保存点"，事务可以选择回滚到这个保存点。

2. 语法格式和说明

GoldenDB 中标记、回滚，释放"保存点"的语句和简要描述：

事务保存点语句	描述
SAVEPOINT identifier	声明一个事务保存点。
ROLLBACK TO SAVEPOINT identifier	回滚事务状态到指定的事务保存点。
RELEASE SAVEPOINT identifier	释放指定的事务保存点。

说明: identifier表示事务保存点的名称。

3. 示例

```
> CREATE TABLE test(id INT PRIMARY KEY, name CHAR(10)) DISTRIBUTED
 BY HASH(id)(G1,G2,G3,G4);
> START TRANSACTION;
> INSERT INTO test(id,name) VALUES (1,'Alex');
> SAVEPOINT A;
> INSERT INTO test(id,name) VALUES (2,'Bob');
> SAVEPOINT B;
> INSERT INTO test(id,name) VALUES (3,'Candy');
> SAVEPOINT C;
> INSERT INTO test(id,name) VALUES(4,'David');
> RELEASE SAVEPOINT B;
> ROLLBACK TO A;
> INSERT INTO test(id,name) VALUES(5,'Taylor');
> COMMIT;
> SELECT * FROM test;
+----+--------+
| ID | NAME   |
+----+--------+
|  1 | Alex   |
|  5 | Taylor |
+----+--------+
```

4. 注意事项

- `RELEASE SAVEPOINT identifier` 执行成功后，不能再回滚到 `identifier` 及晚于 `identifier` 创建时刻的事务保存点。

1.7.3.3. 提交事务

1. 简介

将事务的所有操作永久性地保存到数据库，清除保存点并释放事务所获取的所有锁。

2. 语法格式和说明

默认情况下，GoldenDB 数据库在启用自动提交模式的情况下运行，即不显示开启事务时，每个语句的执行是原子的，执行成功后无法撤销更改，执行失败时会自动回滚。隐式禁用单个事务的自动提交模式即显示开启事务，执行 `START TRANSACTION` 开启事务，并通过 `COMMIT` 语句提交事务。若要显示禁用自动提交模式，请使用 `SET autocommit=0`，并在语句执行完后通过执行 `COMMIT` 语句提交修改。

GoldenDB 中提交事务的语句和简要描述：

事务回滚语句	描述
COMMIT	提交事务。

3. 示例

- 自动提交

```
INSERT INTO test(id,name) VALUES (1,'Alex');
```

- 禁用单个事务的自动提交

```
> START TRANSACTION;
> INSERT INTO test(id,name) VALUES (1,'Alex');
> COMMIT;
```

- 禁用自动提交模式

```
> SET autocommit=0;
> INSERT INTO test(id,name) VALUES (1,'Alex');
> COMMIT;
```

4. 注意事项

- `autocommit` 是一个会话变量，并且必须为每个会话设置，不设置时默认为 0 。

1.7.3.4. 回滚事务

1. 简介

将事务内的所有修改进行撤销，或撤销到某一个事务保存点。

2. 语法格式和使用说明

GoldenDB 中回滚事务的语句和简要描述：

事务回滚语句	描述
ROLLBACK	回滚整个事务，所有修改会被丢弃，清除保存点，释放事务获取的锁。

ROLLBACK TO SAVEPOINT identifier	回滚事务状态到指定保存点，保存点之后的修改被丢弃，清除指定保存点之后的保存点，释放指定保存点之后事务获取的锁。

说明: identifier表示事务保存点的名称。

3. 示例

- 回滚所有修改

```
> CREATE TABLE test(id INT PRIMARY KEY, name CHAR(10)) DISTRIB
UTED BY HASH(id)(G1,G2,G3,G4);
> INSERT INTO test(id,name) VALUES (1,'Alex');
> SELECT * FROM test;
+----+------+
| ID | NAME |
+----+------+
|  1 | Alex |
+----+------+
> START TRANSACTION;
> INSERT INTO test(id,name) VALUES (2,'Bob');
> SELECT * FROM test;
+----+------+
| ID | NAME |
+----+------+
|  1 | Alex |
|  2 | Bob  |
+----+------+
> ROLLBACK;
> SELECT * FROM test;
+----+------+
| ID | NAME |
+----+------+
|  1 | Alex |
+----+------+
```

- 回滚到指定事务保存点

```
> CREATE TABLE test(id INT PRIMARY KEY, name CHAR(10)) DISTRIB
UTED BY HASH(id)(G1,G2,G3,G4);
> START TRANSACTION;
> INSERT INTO test(id,name) VALUES (1,'Alex');
> SAVEPOINT A;
> INSERT INTO test(id,name) VALUES (2,'Bob');
> SAVEPOINT B;
> INSERT INTO test(id,name) VALUES (3,'Candy');
> SAVEPOINT C;
```

```
> INSERT INTO test(id,name) VALUES(4,'David');
> RELEASE SAVEPOINT B;
> SELECT * FROM test;
+----+-------+
| ID | NAME  |
+----+-------+
|  1 | Alex  |
|  2 | Bob   |
|  3 | Candy |
|  4 | David |
+----+-------+
> ROLLBACK TO A;
> SELECT * FROM test;
+----+------+
| ID | NAME |
+----+------+
|  1 | Alex |
+----+------+
> INSERT INTO test(id,name) VALUES(5,'Taylor');
> COMMIT;
> SELECT * FROM test;
+----+--------+
| ID | NAME   |
+----+--------+
|  1 | Alex   |
|  5 | Taylor |
+----+--------+
```

4. 注意事项

- 无法回滚非事务性表。

1.7.4. 事务隔离级别

本节介绍几种隔离级别，主要包括：`UR/CR`、`SW/CW`、`MVCC_RC`。

1.7.4.1. UR/CR

1. 简介

控制不同会话间数据的查询逻辑，脏读(`UR`)、强一致性读(`CR`)。

2. 使用说明

- UR：查询时，读取的数据中允许包含脏数据，或者用于查询时不存在其他事务修改的情况。
- CR：查询时，读取的数据是已提交的。

3. 示例

```
> SELECT * FROM test /*M!gdb_isolation_grade=ur*/;
> SELECT * FROM test /*M!gdb_isolation_grade=cr*/;
```

4. 注意事项

- `UR`，`CR` 不区分大小写，默认为强一致性读(`CR`)。

1.7.4.2. SW/CW

1. 简介

控制不同会话间数据的修改逻辑，单节点写(`SW`)、强一致性写(`CW`)。

2. 使用说明

- SW：不考虑分布式事务下的对部分已提交事务的数据的修改。适用于修改的数据不会在两个及以上事务中同时修改或涉及本数据的所有事务都只会修改单条记录。
- CW：需要考虑分布式事务下的对部分已提交事务的数据的修改。

3. 示例

```
> INSERT INTO test(id,name) VALUES (1,'Alex') SW;
> INSERT INTO test(id,name) VALUES (1,'Alex') CW;
> INSERT INTO test(id,name) VALUES (1,'Alex') /*M!gdb_isolation_gr
ade=sw*/;
> INSERT INTO test(id,name) VALUES (1,'Alex') /*M!gdb_isolation_gr
ade=cw*/;
```

4. 注意事项

- `SW`，`CW` 不区分大小写，默认为强一致性读(`CW`)。

1.7.4.3. MVCC_RC

1. 简介

单机 `MySQL` 实现了并发多版本控制，即 `MVCC`，解决了脏读、幻读、不可重复读等问题。GoldenDB 分布式数据库，在集群隔离级别为 `CR` 下，通过活跃 `GTID` 的校验机制解决了分布式脏读的问题，但对于聚合函数查询，计算节点无法进行活跃 `GTID` 校验，会导致聚合函数查询存在数据不一致的问题。以一笔转账过程做示例，账户表 `account`，有两个账户 `A` 和 `B`，初始金额均为 100：

```
> START TRANSACTION;
> UPDATE account SET acnt=acnt-50 WHERE id=A;
> UPDATE account SET acnt=acnt+50 WHERE id=B;
> COMMIT;
```

现考虑如下场景：账户 `A` 数据在 `G1` 分片，账户 `B` 数据在 `G2` 分片，分布式事务在两分片 `COMMIT` 无法完全达到同时，假设 `G1` 分片的 `COMMIT` 先结束，`G2` 分片的 `COMMIT` 后结束，如果在此间隙进行 `SELECT SUM(acnt)`，`G1` 的 `COMMIT` 已经完成，数据可见，导致查询结果是 150，数据不一致。 为解决 GoldenDB 分布式场景下，聚合函数查询可能出现数据不一致的问题，引入分布式多版本控制方案，即集群读隔离级别新增 `MVCC_RC`。

2. 使用说明

通过 Insight 页面统一配置，动态生效，租户管理->实例->实例信息->隔离级别配置。

3. 注意事项

- 如果集群读隔离为 `MVCC_RC` 时，如果计算节点写隔离级别设置为 `SW`，不保证 `MVCC` 一致性。
- 分布式 `MVCC` 与 `GTID` 强相关，需要更新数据的同时，记录对应的 `GTID` 列也要更新到当前事务申请的 GTID，由于 SW 下的 DELETE 语句没有更新记录 `GTID` 的操作，故读在 `MVCC_RC` 的隔离级别下，写隔离级别是 `CW`，语句带 `SW` 的 `hint` 会报错。
- 分布式 `MVCC` 依赖集群的活跃 `GTID` 列表，若 CN 配置 `Active_GTID_Flag=0` 时，分布式 `MVCC` 无法使用。
- 分布式 `MVCC` 依赖集群的活跃 `GTID` 列表，若 CN 配置 `force_read_split=1`，即强制读写分离，强制读写分离下，查询语句不需要活跃 `GTID` 列表，并且会下发至备机，分布式 `MVCC` 无法使用。
- 目前分布式 `MVCC` 只针对聚合函数查询，普通语句查询按 `CR` 隔离级别处理。
- 需关注长时间不释放的 `GTID` 的影响，如果有活跃 `GTID` 长时间不释放，会导致数据节点历史版本得不到及时 `purge`，`undo` 空间越撑越大；在此背景下，一但该 `GTID` 释放，会导致短时间内大量历史版本 `purge`，影响数据节点性能。
- 优先级：语句级，计算节点隔离级别，集群级隔离级别。

1.7.5. 锁机制

GoldenDB 数据库支持多种数据锁机制，且为行级锁。支持死锁侦测功能，参见 统计监控 中 诊断工具 下的 死锁检测。

GoldenDB 数据库的锁机制是为了解决并发事务带来的问题而诞生的，主要是确保数据库中，多条工作线程并行执行时的数据安全性。

锁是计算机协调多个进程或线程并发访问某一资源的机制。在数据库中，除传统的计算资源（CPU、RAM、I/O）的争用以外，数据也是一种供许多用户共享的资源。如何保证数据并发访问的一致性、有效性是所有数据库必须解决的一个问题，锁冲突也是影响数据库并发访问性能的一个重要因素。从这个角度来说，锁对数据库而言显得尤其重要，也更加复杂。

锁机制与索引机制类似，都是由存储引擎负责实现的，这也就意味着不同的存储引擎，支持的

锁也并不同，这里是指不同的引擎实现的锁粒度不同，GoldenDB 的锁体系有以下几种：

- 以锁粒度的维度划分
  - 全局锁：锁定数据库中的所有表。加上全局锁之后，整个数据库只能允许读，不允许做任何写操作。
  - 表级锁：每次操作锁住整张表。主要分为三类：
    - 表锁：分为表共享读锁 read lock、表独占写锁 write lock。
    - 元数据锁（meta data lock，MDL）：基于表的元数据加锁，加锁后整张表不允许其他事务操作。这里的元数据可以简单理解为一张表的表结构。
    - 意向锁：分为意向共享锁、意向排他锁，这个是为了支持多粒度的锁，兼容行锁、表锁而设计的，使得表锁不用检查每行数据是否加锁，使用意向锁来减少表锁的检查。
  - 行级锁：每次操作锁住对应的行数据。主要分为三类：
    - 记录锁 / Record 锁：也就是行锁，一条记录和一行数据是同一个意思。防止其他事务对此行进行 `UPDATE` 和 `DELETE`，在 RC、RR 隔离级别下都支持
    - 间隙锁 / Gap 锁：锁定索引记录间隙（不含该记录），确保索引记录间隙不变，防止其他事务在这个间隙进行 `INSERT`，产生幻读。在 RR 隔离级别下都支持
    - 临键锁 / Next-Key 锁：间隙锁的升级版，同时具备记录锁+间隙锁的功能，在 RR 隔离级别下支持
- 以互斥性的角度划分
  - 共享锁 / S锁：不同事务之间不会相互排斥、可以同时获取的锁
  - 排他锁 / X锁：不同事务之间会相互排斥、同时只能允许一个事务获取的锁
  - 共享排他锁 / SX锁：解决 SMO 带来的问题
- 以操作类型的维度划分
  - 读锁：查询数据时使用的锁
  - 写锁：执行插入、删除、修改、DDL 语句时使用的锁以加锁方式的维度划分
  - 显示锁：编写 SQL 语句时，手动指定加锁的粒度
  - 隐式锁：执行 SQL 语句时，根据隔离级别自动为 SQL 操作加锁
- 以思想的维度划分

- 乐观锁：每次执行前认为自己会成功，因此先尝试执行，失败时再获取锁
- 悲观锁：每次执行前都认为自己无法成功，因此会先获取锁，然后再执行

GoldenDB 支持行锁死锁与分布式死锁及 MDL 死锁的检测与自动解锁功能。针对分布式死锁，可以配置 `online_ddl_detect_switch=1` 将功能打开。支持该功能的相关参数，如重试次数、判定间隔、重试次数等的个性化配置。

GoldenDB 在 CM 上增加全局死锁检测开关及定时周期（单位为秒），支持动态生效；当开关打开时，能在死锁发生时的下个定时周期内检测到分布式死锁；当检测到全局死锁时能够准确检测到分布式死锁，CM 能正确判断回滚代价较小的事务，执行回滚操作解除死锁，另外在各组件异常场景下设定异常检测周期值时返回默认值，并能正常检测全局死锁，CN 可根据死锁前更新的语句数量和更新记录数，判断代价较小的事务并执行相应的回滚，解除死锁。

1.8. SQL性能调优

本章节描述如何优化 GoldenDB 应用中 SQL 语句的性能。

数据库应用性能，从数据库应用设计的时候就要考虑。

SQL 语句的性能优化，无法从根本上弥补设计上引入的性能问题。

1.8.1. 执行计划

1.8.1.1. SQL执行计划管理

描述

GoldenDB 支持图形化绑定/解绑 `Outline` 信息，并支持在线修改执行计划。

执行路径： Insight -> DBA -> DN，DBA 需要使用有写权限的用户。GoldenDB 已经支持 `Outline` 语法，语法可参考 SQL规范 中 OUTLINE 固定执行计划 下的语法介绍 。

由 Insight -> DBA 新增内部消息流。 DBA 执行 SQL 语句。上层输入的是 SQL，由 dba 去查询 SQL 的 DIGEST，去组装语法。

`outline_name` 命名规则： 以 `SQL DIGEST` 命名。

支持命令行绑定执行计划：

绑定执行计划可以将 `SELECT` 语句的执行计划写入到系统表 `execution_plan`，再次执行同类型的 `SELECT` 语句，即可复用相应的执行计划。对于系统表 `execution_plan` 中的执行计划，可以进行更新、删除、启用和禁用的操作（需要相应的系统表权限）。

语法

- 打开固定执行计划功能：

```
SET GLOBAL persist_execution_plan = on;
```

- 固定执行计划，INSERT 执行计划到系统表 `execution_plan`（需要系统表的 `INSERT` 权限），如果已存在，报错处理。

```
EXPLAIN FOR persist SELECT stmt;
```

- 强制固定执行计划，同样 `INSERT` 执行计划到系统表 `execution_plan`（需要系统表的 `INSERT` 权限），如果已存在，进行 `UPDATE` 操作，否则仍进行 `INSERT` 操作。

```
EXPLAIN FOR force persist SELECT stmt;
```

- 启用已固定的执行计划，从系统表 `execution_plan` 中 `UPDATE` 相应的执行计划的 `ENABLED` 字段为 `YES`（需要系统表的 `UPDATE` 权限），如果不存在，报错处理。

```
EXPLAIN FOR enable persist SELECT stmt;
```

- 禁用已固定的执行计划，从系统表 `execution_plan` 中 `UPDATE` 相应的执行计划的 `ENABLED` 字段为 `NO`（需要系统表的 `UPDATE` 权限），如果不存在，报错处理。

```
EXPLAIN FOR disable persist SELECT stmt;
```

- 删除已固定的执行计划，从系统表 `execution_plan` 中 `DELETE` 相应的执行计划（需要系统表的 `DELETE` 权限），如果不存在，报错处理。

```
EXPLAIN FOR DELETE persist SELECT stmt;
```

1.8.1.2. 执行计划查看

描述

GoldenDB 可以查看数据查询，插入，更新，删除的执行计划。通过查看执行计划，您可以了解查询的瓶颈和潜在的性能优化机会。

语法

```
EXPLAIN [db|proxy] explainable_stmt [storagedb]
```

```
explainable_stmt: {
    SELECT statement
    | DELETE statement
    | INSERT statement
    | UPDATE statement
}
```

示例

```
> EXPLAIN PROXY SELECT * FROM t1 WHERE id=1 or id=11 ORDER B
Y id;
> EXPLAIN SELECT * FROM t1 WHERE id=1 or id=11 ORDER BY id;
> EXPLAIN DB SELECT * FROM t1 STORAGEDB g1;
> EXPLAIN SELECT * FROM t1 WHERE id=1 or id=11 ORDER BY id;
> EXPLAIN INSERT INTO t1(id) VALUES(15);
```

注意事项

1. 支持仅输出计算节点的分布式执行计划。此时需使用 PROXY 选项，即 `EXPLAIN PROXY explainable_stmt`。
2. 支持仅输出 DN 的执行计划。此时需使用 DN 选项，并指明要查看的存储节点。
3. 支持将分布式执行计划和 DN 执行计划结合输出。
4. 执行计划提供了关于查询执行的信息，但并不一定意味着执行计划本身会解决性能问题。需要综合其他优化技术和经验来改进查询性能。

1.8.1.3. 执行计划绑定

描述

当查询的执行计划会因为统计信息的变化、优化器版本的更新、系统配置的更改等原因而产生变化时，可以使用 `Outline` 固定执行计划绑定来确保查询使用预期的执行计划，从而稳定查询性能。

语法

```
CREATE [OR REPLACE] [public_option] OUTLINE outline_name
  ON select /*HINT*/ expr , ...
  FROM table_references
  [WHERE where_condition]

DROP OUTLINE [IF EXISTS] outline
```

参数解释

字段	描述

public_option	与大纲访问权限相关的可选项 PUBLIC：表示该查询大纲的访问权限是公共的，这意味着在数据库中，多个用户或会话可以访问和使用这个查询大纲 PRIVATE：表示该查询大纲的访问权限是私有的。这意味着只有特定的用户或角色能够访问和使用这个查询大纲，其他用户无法使用。
HINT	支持绑定的HINT列表和说明 BKA/NO_BKA: 影响BKA批量键访问表连接算法处理 JOIN_INDEX/NO_JOIN_INDEX: 对任何访问方法使用或忽略指定的一个或多个索引 GROUP_INDEX/NO_GROUP_INDEX：在 GROUP BY 操作中使用或忽略指定的一个或多个索引进行索引扫描 ORDER_INDEX/NO_ORDER_INDEX: 使用或忽略指定的一个或多个索引对行进行排序 INDEX/NO_INDEX: 充当 JOIN_INDEX、GROUP_INDEX 和 ORDER_INDEX 的组合，或作为 NO_JOIN_INDEX、NO_GROUP_INDEX 和 NO_ORDER_INDEX 的组合等。

示例

```
> CREATE OUTLINE ot1 ON select /*+ INDEX(t1 idx)*/ * from t1 w
here id=114528;
> CREATE OR REPLACE OUTLINE ot2 ON select /*+ JOIN_PREFIX(t1)*/ *
from t1 left join t2 on t1.a=t2.a;
> DROP OUTLINE ot1;
```

注意事项

固定执行计划不支持跨节点的查询，请配合 `explain` 使用。

1.8.2. 使用 PREPARE

描述

PREPARE 是用于将 SQL 查询预先编译为准备好的查询的命令。它允许应用程序在需要执行相同查询多次时节省时间，因为预编译的查询只需要解析和优化一次，然后可以重复执行多次。

语法

集中式语法格式：

```
1 | PREPARE stmt_name FROM preparable_stmt
2 | EXECUTE stmt_name [USING @var_name [, @var_name] ...]
3 | {DEALLOCATE | DROP} PREPARE stmt_name
```

分布式数据库通过 `JDBC` 执行，链接方法参见应用开发示例

示例

集中式：

```
> PREPARE stmt1 FROM 'CREATE TABLE IF NOT EXISTS t1 (a INT, b CHAR
(20) )';
> EXECUTE stmt1;
> DROP PREPARE stmt1;
```

分布式：

```
  PreparedStatement p_stmt = conn.prepareStatement("INSERT INTO
t1(a,b)VALUES(?,?)");
  for(int i=1;i<=1000;i++){
      p_stmt.setInt(1,i);
      p_stmt.setString(2,"test"+i);
      p_stmt.addBatch();
      if(i % 50 == 0){
          p_stmt.executeBatch();
      }
  }
  p_stmt.executeBatch();
```

注意事项

1. 使用 PREPARE 时应注意绑定变量的使用，以避免 SQL 注入等安全风险。
2. 在分布式场景下，PREPARE 期间执行影响表结构变更的ddl操作后，会返回 `Prepared statement needs to be re-prepared` 错误码给 `JDBC` 侧，此时需要业务侧重新执行适配最新表结构的 PREPARE 操作。

1.8.3. 语句级并行

描述

语句级并行是一种并行处理技术，用于提高操作的执行性能。它将一个操作分解为多个子任务，并同时在多个执行线程中并行执行这些子任务，从而加速整体操作的完成时间。

- `UPDATE/DELETE`并发

  语法

  - 目前支持两种 `UPDATE/DELETE` 并发，基于表分区的 `UPDATE/DELETE` 并发（默认）和 基于表直方图的 `UPDATE/DELETE` 并发。
  - `UPDATE/DELETE` 并发 HINT：`/*+ parallel(n [, mod])*/`。 其中，`n` 为期望

并发数；`mod` 为并发模式，包含 `PARTITION`（默认）、`HISTOGRAM` 两种，不写默认为 `PARTITION`。

- 实际的并发数和 `UPDATE/DELETE` 并发的执行效果因受待更新表的结构(包括表定义、分区数量或直方图定义等)、数据分布、系统资源 等客观因素影响，与 `UPDATE/DELETE` 并发 HINT 指定值可能存在一定偏差，属于正常现象。
- 推荐使用基于表分区的 `UPDATE/DELETE` 并发功能。

示例

```
> UPDATE /*+parallel(10,HISTOGRAM)*/ t1 SET id6 = 3;
> DELETE /*+parallel(10,PARTITION)*/ FROM t1;
```

- 该功能对应有 CN 配置项 `small_table_rows_num`，用于配置小表数据量的上限值。配置项位于 `proxy.ini` 文件 [GENERAL] 段中，默认值为 0，表示不开启该优化。自动判断大小表需要用到表总行数，需要提前执行语句 `ANALYZE HISTOGRAM;` 用于获取当前时刻所有表的总行数；如果想使用表结果集的行数，则需要提前在关注的字段上额外建立直方图。

注意事项

1. `UPDATE/DELETE` 并发仅支持 `UPDATE/DELETE` 单表。
2. `UPDATE/DELETE` 并发 HINT 需紧跟 `UPDATE/DELETE` 关键字。
3. 必须开启半一致性读。
4. `UPDATE/DELETE` 并发语句不含子查询、`ORDER BY`、`LIMIT` 等。
5. 待更新表上未建有触发器。
6. 使用 `UPDATE/DELETE` 并发功能，需开启自动提交，或单语句的事务（带 `UPDATE/DELETE` 并发 HINT 的一条语句只能单条执行，不能和其他 SQL 放在一个事务里）。
7. 语句更新数据量过大，或并发数过多时，会长时间占用大量系统资源，这可能会导致诸如 DN 节点断链等异常。
8. 基于表分区的 `UPDATE/DELETE` 并发，要求表上建有分区（不同分片不同分区表不支持 `UPDATE/DELETE` 并发），且对于 `UPDATE` 语句，待更新列不能为分区键和分发键。
9. 基于直方图信息的 `UPDATE/DELETE` 并发，要求表上建有直方图，且对于 `UPDATE` 语句，待更新列不能完全覆盖建有直方图的列。
10. CN 节点 必须打开 `open_dml_parallel` 配置。

- `INSERT SELECT` 并行

语法

- 基于直方图的 INSERT SELECT 并行

- 原理：依据 HINT 指定的拆分字段的直方图信息，对全表数据进行切分，并将切分后的n个数据切片对应的过滤条件补充到 WHERE 条件中，生成 n 条并行语句，并行执

行

- HINT: /+*parallel([dbname].[tabnane].[field],n[,histogram])* /。其中，dbname.tabnane.field 指定并发拆分字段（注意库名、表名、字段名中的'.'不可省略），n 指定并发数

示例

```
> CREATE TABLE if NOT EXISTS CA_BILL_EXPORT_392_20230913 (
  `bill_no` decimal(15,0) NOT NULL,
  `cust_id` decimal(15,0) DEFAULT NULL,
  `acct_id` decimal(15,0) DEFAULT NULL,
  `default_acct_id` decimal(15,0) DEFAULT NULL,
  `resource_id` decimal(15,0) DEFAULT NULL,
  `phone_id` varchar(64) DEFAULT NULL,
  `region_code` decimal(14,0) DEFAULT NULL,
  `county_code` decimal(14,0) DEFAULT NULL,
  `main_offer_id` decimal(15,0) DEFAULT NULL,
  `user_property` decimal(14,0) DEFAULT NULL,
  `pay_type` decimal(14,0) DEFAULT NULL,
  `brand` decimal(16,0) DEFAULT NULL,
  `user_type` decimal(14,0) DEFAULT NULL,
  `create_date` datetime DEFAULT NULL,
  `bill_month` decimal(6,0) DEFAULT NULL,
  `begin_date` datetime DEFAULT NULL,
  `end_date` datetime DEFAULT NULL,
  `item_code` decimal(9,0) DEFAULT NULL,
  `primal_fee` decimal(12,0) DEFAULT NULL,
  `discount_fee` decimal(12,0) DEFAULT NULL,
  `cdr_discount` decimal(12,0) DEFAULT NULL,
  `adj_fee` decimal(12,0) DEFAULT NULL,
  `tax_rate` decimal(14,0) DEFAULT NULL,
  `tax_fee` decimal(15,0) DEFAULT NULL,
  `tax_measure_id` decimal(9,0) NOT NULL,
  `tax_off_fee` decimal(12,0) NOT NULL,
  `write_off_fee` decimal(12,0) DEFAULT NULL,
  `pay_method` decimal(14,0) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=gbk COLLATE=gbk_bin
distributed by duplicate(g1,g2,g3);

> DELIMITER //;
BEGIN
  for i in 5000..20000 loop
    insert into CA_BILL_EXPORT_392_20230913 values(i,i,i,i,i,'a
a'||i,i,i,i,i,i,i,i,'2023-06-28 10:26:29',i,'2023-06-28 10:26:2
9','2023-06-28 10:26:29',i,i,i,i,i,i,i,i,i,i,i);
    end loop;
    end //
DELIMITER ;//

> ANALYZE table CA_BILL_EXPORT_392_20230913  update histogram o
```

```
n cust_id,bill_no with 10 buckets;
> ANALYZE histogram;
> SELECT * from _gdb_sysdb._gdb_table_histogram_info where tabl
e_schema = 'DB1' and table_name = 'CA_BILL_EXPORT_392_20230913'
 ;

> CREATE TABLE ACCP_USERBILL_TMP_F PARALLEL 4 NOLOGGING AS SELE
CT RESOURCE_ID serv_id,CUST_ID,ACCT_ID,DEFAULT_ACCT_ID DEF_ACCT
,ITEM_CODE bill_item,PRIMAL_FEE,0 - DISCOUNT_FEE BILL_DISCOUNT,
CDR_DISCOUNT,ADJ_FEE,PAY_METHOD PAY_FLAG,to_char(BEGIN_DATE, 'y
yyymmdd') user_cycle_begin,to_char(END_DATE, 'yyyymmdd') user_c
ycle_end,to_char(BEGIN_DATE, 'yyyymmdd') acct_cycle_begin,to_ch
ar(END_DATE, 'yyyymmdd') acct_cycle_end  from  CA_BILL_EXPORT_3
92_20230913 distributed by hash(serv_id)(g2,g3);

> INSERT  /*+parallel(db1.CA_BILL_EXPORT_392_20230913.bill_no,1
0)*/ INTO ACCP_USERBILL_TMP_F SELECT RESOURCE_ID serv_id,CUST_I
D,ACCT_ID,DEFAULT_ACCT_ID DEF_ACCT,ITEM_CODE bill_item,PRIMAL_F
EE,0 - DISCOUNT_FEE BILL_DISCOUNT,CDR_DISCOUNT,ADJ_FEE,PAY_METH
OD PAY_FLAG,to_char(BEGIN_DATE, 'yyyymmdd') user_cycle_begin,to
_char(END_DATE, 'yyyymmdd') user_cycle_end,to_char(BEGIN_DATE,
'yyyymmdd') acct_cycle_begin,to_char(END_DATE, 'yyyymmdd') acct
_cycle_end  from  CA_BILL_EXPORT_392_20230913 /*M!gdb_isolation
_grade=ur*/;
```

注意事项

1. 不支持 union、minus。
2. 不支持 insert all。
3. 不支持 on duplicate key update。
4. 不支持与其他并行 HINT 混用。
5. 不支持 start with、connect by。
6. 不支持 limit、rownum。
7. 不支持 自定义函数。
8. 不支持 distinct。
9. 不支持 WITH CUBE, WITH ROLLUP, ROLLUP()。
10. 不支持触发器。
11. 拆分字段为 select 最外层实表字段。
12. 拆分字段上建有直方图。

13. 语句含窗口函数时，窗口函数需 partition by 拆分字段。
14. 语句含聚合函数时，必须含 group by 子句。
15. 语句含 group by 子句时，必须 group by拆分字段。
16. CN 节点 必须打开 `open_dml_parallel` 配置。
17. 使用 `INSERT SELECT` 并发功能，需开启自动提交，或事务中不再使用并发语句中的表。
18. 拆分字段库名、表名、字段名指定不全时，拆分字段选择规则（满足 HINT 指定信息的前提下）：
    - 优先选择数值类型字段。
    - 对于数值类型字段，选择直方图信息中，distinct 数最高的字段 （bucket 数高，重复度低的字段 更易被选中）。
    - 不存在数值类型字段时，选择在某一分片数据量最大的字段。
    - HINT 未指定库名时，默认为 use 库。

其他说明

1. 建议打开 DN 节点 打开 `disable_full_table_scan_in_delete` 和 `disable_full_table_scan_in_select` 配置，减少发生锁冲突的几率。
2. 语句更新数据量过大，或并发数过多时，会长时间占用大量系统资源，这可能会导致诸如DN节点断链等异常。
3. 并行是基于全表数据进行拆分，尽量避免选取 WHERE 条件中存在过滤条件的字段作为拆分字段，以防部分拆分语句失效（并行拆分补充的过滤条件与 WHERE 中原有过滤条件不重叠（或重叠范围较小），失去拆分意义）。
4. `INSERT SELECT` 基于直方图并发语句，可以通过 explain 查看 extra 信息中是否含有 parallel(histogram):X，确认语句本身是否满足并行限制（虽然这不意味着语句执行一定可以基于直方图并发）。
5. 拆分字段库名、表名、字段名指定不全时，拆分字段自动选择发生在使用限制第 13-15 条判断之前，无法保证自动选择出的拆分字段满足使用限制第 13-15 条。
6. 拆分字段为数值类型时，按全表数据的直方图信息进行全表数据切分；拆分字段为字符类型时，按数据量最大分片上的直返图信息进行全表数据切分。
7. 直方图 bucket 数建议在并发数两倍以上，较低可能会影响实际并发数。
8. 及时更新直方图信息，一定程度上避免语句拆分的不合理。

- `SELECT` 并行

语法

- 基于 MVCC 的 SELECT 并行。
- 原理：leader 线程依据树结构，对全表数据进行拆分，分发给多个 worker 线程查询，由 leader 线程汇总输出到客户端。
- HINT 控制：/+*PARALLEL(N)* /。其中 N 指定并发数。

- 开关控制：force_parallel_execute 配置项控制打开或关闭 SELECT 并行功能，默认 OFF 关闭。parallel_default_dop 配置项控制使用并发数，默认为 4 并发。parallel_cost_threshold 配置项控制 SELECT 并行的代价阈值，默认为 1000。

示例

```
hint控制：
> SELECT /*+PARALLEL(32) */ * FROM t1;

开关控制：
> SET force_parallel_execute = on;
> SET parallel_cost_threshold = 0;
> SELECT * FROM t1;
```

注意事项

1. 不支持 TRIGGER/存储过程/系统表/串行化隔离级别/临时表/非 INNODB 表。
2. 不支持 PREPARE。
3. 不支持 UNION/INTERSECT/EXCEPT/MINUS 等集合操作。
4. 不支持窗口函数。
5. 不支持 VIEW。
6. 不支持最简单的 SELECT DISTINCT xx FROM / SELECT COUNT(xx) FROM。
7. 不支持表涉及 BLOB /JSON/GEOMETRY 的字段类型。
8. 不支持部分函数 MATCH 函数/@变量或者:变量来访问临时变量/自定义函数/XML 函数。
9. 不支持部分聚合函数 COUNT DISTINCT/SUM DISTINCT/AVG DISTINCT/GROUP_CONCAT/JSON_AGG/自定义的聚合函数。
10. 子查询的 SELECT 不支持并行。
11. 支持依赖 depend_subquery 的子查询，如标量子查询，或者是依赖外表的子查询。
12. 不支持 SEQUENCE。
13. 不支持 ROWNUM。
14. 不支持 SEMI-JOIN 的 物化查询。

其他说明

1. 全表扫无需使用并行查询，性能不会提升。
2. 语句更新数据量过大，或并发数过多时，会长时间占用大量系统资源，这可能会导致诸如DN节点断链等异常。
3. 并行是基于全表数据进行拆分，尽量避免选取 WHERE 条件中存在过滤条件的字段作为拆分字段，以防部分拆分语句失效（并行拆分补充的过滤条件与 WHERE 中原有过滤条件不重叠（或重叠范围较小），失去拆分意义）。
4. 可以通过 EXPLAIN 的 EXTRA 字段查看是否成功使用 SELECT 并行，且 EXPLAIN 中显示并发数才是实际并发数。
5. 上述不支持项以计划执行为准，如：ROWNUM 正在部分情况下会被优化为 LIMIT，则不受限制。

1.8.4. JOIN的优化

描述

小表驱动大表是利用小表结果集，减少大表的捞取数据量，只有同时具有大表和小表时，才有可能走小表驱动大表优化。因此需要判断语句中是否同时存在大表和小表，并区分各个表是大表还是小表用于后续判定。

语法

- 大小表的判断标准：

  小表和大表的区分不是根据表的原始数据来判断的，而是根据条件过滤后的数据来判断。

- 已知大表、小表分别是哪个时，对于简单的两表 `JOIN` 语句，可以使用 HINT 进行优化，提高查询效率。该 HINT 为：`/*+ leading(small_table_name) use_nl(big_table_name) */`。

示例

```
  SELECT /*+ leading(tb1) use_nl(tb2) */ tb1.colour, tb2.location
FROM
  tb1 LEFT JOIN tb2 ON tb1.id=tb2.id AND tb1.location=tb2.location
;
```

- 该 HINT 支持两表做简单的 `INNER JOIN`、`CROSS JOIN`、逗号 `JOIN`、`LEFT JOIN`、`RIGHT JOIN` 或者 (+) 构成外联结的场景。
- 该功能对应有 CN 配置项 `small_table_rows_num`，用于配置小表数据量的上限值。配置项位于 `proxy.ini` 文件 [GENERAL] 段中，默认值为 0，表示不开启该优化。自动判断大小表需要用到表总行数，需要提前执行语句 `ANALYZE HISTOGRAM;` 用于获取当前时刻所有表的总行数；如果想使用表结果集的行数，则需要提前在关注的字段上额外建立直方图。

判断是否可以走小表驱动大表优化：

```
> SELECT COUNT(1) CNT FROM (

(SELECT C.ORDER_ID, A.CREATE_DATE FROM

A, B, C

WHERE

A.ORDER_ID = B.ORDER_ID AND B.ORDER_ID = C.OBJ_INST_ID

AND A.CREATE_DATE >= TRUNC(SYSDATE, 'DD') -1
```

```
AND A.CREATE_DATE < SYSDATE - 2 / 24 / 60

AND A.OP_CODE = 'PARTNERTOPBOSSDELAYORDER'

AND B.ORDER_STATUS IN (90, 91, 93, 99)

AND B.ORDER_BUSI_TYPE = 10002) TT

JOIN OMBROADBAND.PM_ORDER_FLOW_EXEC D ON

TT.ORDER_ID = D.ORDER_ID

AND TRUNC(TT.CREATE_DATE, 'MI') = TRUNC(D.CREATE_DATE, 'MI')

AND D.FLOW_NODE_CODE = 'AMS_DELAY_HANDLE'

AND D.EXEC_ID IS NULL);
```

1. 检查语句类型是否支持

存在两个 JOIN 语句， 外层TT LEFT JOIN D 含有子查询，不支持； SELECT A,B,C 为多表逗号 JOIN, 且不含子查询;

检查 ABCD 各表是否含全局索引，这里假设不含;

SELECT A, B, C 语句中不含非本层 SELECT 的字段;

判定 SELECT A, B, C 语句类型支持。

2. 判断是否同时有大表和小表

语句不含指定大小表的 HINT, 需要利用配置项判断大小表，检查配置项 `small_table_rows_num` 的值，这里假设该值为一非零值;

执行 `SELECT count(*) from A; SELECT count(*) from B; SELECT count(*) from C;` 使用该值与 `small_table_rows_num` 对比，大于该值为大表，小于等于该值为小表；如果同时存在大小表，则无需利用 where 和直方图，直接进行第三步判断；这里假设查询行数均大于 `small_table_rows_num`;

对 where 语句进行识别，语句中不含有 or，是支持的 where 语句；提取表达式 `B.ORDER_STATUS IN (90, 91, 93, 99)`, `B.ORDER_BUSI_TYPE = 10002`, `D.FLOW_NODE_CODE = 'AMS_DELAY_HANDLE'`，检查字段 B.ORDER_STATUS, B.ORDER_BUSI_TYPE, D.FLOW_NODE_CODE 的类型是否是支持的类型，这里假设分别为 INT 类型、INT 类型、VARCHAR 类型；执行语句 `SELECT count(*) from B where B.ORDER_STATUS IN (90, 91, 93, 99) ;`, `SELECT count(*) from B where B.ORDER_BUSI_TYPE = 10002;`, `SELECT count(*) from D where`

`D.FLOW_NODE_CODE = 'AMS_DELAY_HANDLE';`; 假定第一条语句返回行数大于 `small_table_rows_num`，第二条语句返回行数小于 `small_table_rows_num`，第三条语句返回行数大于 `small_table_rows_num`，则在字段 B.ORDER_BUSI_TYPE 上建立直方图后，B 表可以识别为小表; 此时大表为 A、C，小表为 B，语句中同时存在大表和小表；

建立直方图并获取总行数和直方图信息：

ANALYZE TABLE B UPDATE HISTOGRAM ON ORDER_BUSI_TYPE WITH 10 BUCKETS;

ANALYZE HISTOGRAM;

3. 判断大表是否能够优化

使用 explain proxy 查询 SELECT A,B,C... 语句的执行计划，可以看到大表 A 有 on expr `A.ORDER_ID = B.ORDER_ID`,即 A 有关联的小表 B; 大表 B 有on expr `B.ORDER_ID = C.OBJ_INST_ID`, 即 A 有关联的小表 C;

A, B, C 的 JOIN 运算顺序为先进行 JOIN 运算 “A,B”, 然后使用该运算结果集与 C 进行 JOIN 运算；A, B 参与同一 JOIN 运算, 小表 B 可以对大表 A 进行优化; 大表 C 在小表 B 运算完成后进行 JOIN 运算, 小表 B 可以对大表 C 进行优化。

4. 结论

经上述逐步判断，上述语句可以走小表驱动大表优化。具体为使用小表 B 对大表 A、C 进行优化。如果要实现该优化，需要执行相应语句获取各表总行数并在 B.ORDER_BUSI_TYPE 字段上建立直方图。

注意事项

1. 获取的表行数是近似值，并不与表中实际数据量完全一致。
2. 支持 `SELECT`、`INSERT SELECT` 语句，不支持其他类型的语句。
3. 支持 `FROM` 子查询，不支持其他类型的子查询。
4. 不支持语句中有跨层关联字段。
5. 如果语句采用了其他优化手段，可能不再走该优化。
6. 具体优化效果依表内数据和具体语句而定。
7. 根据表数据量优化。

1.8.5. 子查询的优化

示例

- `EXISTS` 子查询

`EXISTS` 结果集数据量比较少的场景下，改写成 IN 优化效果比较明显。

语句如下：

```
> SELECT t1.id FROM t1
WHERE EXISTS (SELECT id FROM t2 WHERE t1.id = t2.id and  t2.id1 =2
);
```

语句可以加优化的 HINT：

```
> SELECT t1.id FROM t1 /*+set_optimize_sqlmod=exists_to_in*/
WHERE EXISTS (SELECT id FROM t2 WHERE t1.id = t2.id and  t2.id1 =2
);
```

加 HINT 后转换为：

```
> SELECT t1.id FROM t1
WHERE t1.id IN (SELECT id FROM t2 WHERE t2.id1 =2);
```

注意事项

1. `EXIST` 后面的子查询中必须要关联字段等值条件。
2. 在 1 的基础上可以有子查询层的表字段与常量的等值条件、非等值条件和in条件。
3. `EXIST` 可以多次出现，比如 EXISTS (SELECT SQL) AND/OR EXISTS(SELECT SQL) AND (EXISTS (SELECT SQL) AND/OR EXISTS (SELECT SQL))。
4. `EXIST` 后跟的子查询如果不符合上述条件就不优化。
5. `EXIST` 的子查询可以是单表查询也可以是多表连接查询。
6. `EXIST` 外层的查询可以是单表查询也可以是多表连接查询。
7. `EXIST` 子查询 WHERE 中的条件，必须是子查询中表的字段，不能是外表的字段。`Select 1 from t1 where exists (select 1 from t2 where t1.id=t2.id and t1.id=1` 这种就不优化。
8. `EXIST` 子查询中出现 Union、Minus、Intersect 语句不做优化。
9. `EXIST` 转 `IN` 优化在 `UPDATE`、`DELETE` 语句中同样适用。

- `NOT EXISTS` 子查询

`NOT EXISTS` 优化为 `LEFT JOIN` 连接查询，主要是利用 CN `LEFT JOIN` 的计算流程处理 `NOT EXIST`， 提升性能。

语句如下:

```
> SELECT t1.id FROM t1
WHERE NOT EXISTS (SELECT id FROM t2
WHERE t1.id = t2.id and  t2.id1 =2);
```

语句可以加优化的 HINT：

```
> SELECT t1.id FROM t1 /*+set_optimize_sqlmod=noexists_to_leftjoin
*/
WHERE NOT EXISTS (SELECT id FROM t2 WHERE t1.id = t2.id and t2.id1
 =2);
```

加 HINT 后转换为：

```
> SELECT t1.id FROM t1 LEFT JOIN (SELECT id as __gtid_col1 FROM t2
)
_gtid_alias ON (t1.id = _gtid_alias.__gtid_col1 AND t2.id1 = 2)
WHERE _gtid_alias .__gtid_col1 IS NULL;
```

注意事项

1. `NOT EXIST` 的子查询部分符合exists 需要优化为in的条件才需要优化。
2. `NOT EXIST` 只能有一个符合优化条件，多个符合的话就不优化。
3. `NOT EXIST` 后面的子查询字段只关注第一个表的主键字段 +on 条件涉及的字段。
4. `NOT EXIST` 优化是把后面的整个子查询作为虚表，后面的 on 条件的等值字段都用 `_gtid_alias.__gtid_col1` 这样的格式。
5. `NOT EXIST` 条件改造为了 `字段 is null`，这里字段是子查询第一个表的第一个主键字段的别名。
6. `NOT EXIST` 的子查询可以是单表查询也可以是多表连接查询。
7. `NOT EXIST` 外层的查询可以是单表查询也可以是多表连接查询。
8. `NOT EXIST` 子查询 WHERE 中的条件，必须是子查询中表的字段，不能是外表的字段。`Select 1 from t1 where not exists (select 1 from t2 where t1.id=t2.id and t1.id=1)` 这种就不优化。
9. Union、Minus、Intersect语句不做优化，类似于这种 `select 1 from t1 where not exists (select 1 from t2 where t1.id=t2.id union select 1 from t2 where t1.id=t2.id)`。
10. `NOT EXIST` 转 `LEFT JOIN` 优化在 `UPDATE`、`DELETE` 语句中同样适用。

- `IN` 子查询

针对包含单个 `IN` 子查询的语句，目前的处理方法是设置一个阈值，如果 `IN` 子查询的结果集行数大于阈值，整个子查询都会在 CN 层进行计算，如果结果集行数小于阈值，则 CN 层将子查询下压到 DN 进行。在大数据量的场景下，这种处理方法显然会使 CN 层执行效率降低，且无法发挥 DN 计算性能高的的优势。

从 CN 角度看，子查询优化的主要目标一般是尽量使语句下压到 DN 执行，尽可能少的捞取数据到 CN 层进行计算，`IN` 批量下压优化可以有效实现这个目标。

语句如下:

```
> SELECT * FROM t1 WHERE a IN (SELECT a FROM t2);
```

语句可以加优化的 HINT：

```
> /*+set_optimize_sqlmod=batch_convert*/ SELECT * FROM t1 WHERE a
IN (SELECT a FROM t2);
```

加 HINT 后转换如下，其中 `n` 是 `proxy.ini` 中 `subwhere_result_line` 设置的值，表示把子查询结果集等分下压到 DN：

```
SELECT * FROM t1 WHERE a IN (常量1, 常量2 ... 常量n); SELECT * FROM t1 WHERE a IN (常量n+1, 常量n+2 ... 常量2n); SELECT * FROM t1 WHERE a IN (常量2n+1, 常量2n+2 ... 常量3n);
```

`IN` 批量下压优化在 `UPDATE`、`DELETE` 语句中同样适用。

1.9. SQL实践和建议

本章主要介绍 SQL 编写规范、数据导入导出规范以及双库运行兼容函数说明等。

1.9.1. 引言

1.9.1.1. 目的

为了保持应用程序编写风格的一致性和连贯性，正确合理利用分布式数据库的编程特性，提高应用程序的执行效率，增强应用程序的可读性和可维护性，特制订此开发规范。

1.9.1.2. 适用范围

基于分布式数据库的应用系统开发人员，运维人员。

分布式数据库适合于数据一致性要求高、短事务、高并发的 OLTP 的应用场景，不适用于 OLAP 分析型应用场景。

1.9.1.3. 术语说明

本规范文档涉及术语以及说明参见下表。

表 1-1 术语说明表

术语	英文	说明
脏读	UR	查询时，读取的数据中允许包含脏数据，或者用于查询时不存在其他事务修改的情况。

强一致性读	CR	查询时，读取的数据是已提交的。
单节点写	SW	不考虑分布式事务下的对部分已提交事务的数据的修改。 下面两种情况使用 SW： • 修改的数据不会在两个及以上事务中同时修改。 • 涉及本数据的所有事务都只会修改单条记录。
强一致性写	CW	需要考虑分布式事务下的对部分已提交事务的数据的修改。

1.9.2. SQL编写规范

1.9.2.1. 不建议项

不建议无索引的条件查询

- **不建议原因**

无索引的条件查询需要走全表扫描，当表数据量很大时，无索引查询导致高开销的数据库磁盘IO。

除非该表是系统配置表，数据量很小，记录行数小于 1 万行。

- **优化方法**

增加基于条件的索引。

- **原 SQL 示例**

表 `ACCOUNT` 的 `FNAL_OPEN_DOOR_DATE` 没有建立索引

```
> SELECT NAME FROM ACCOUNT WHERE FNAL_OPEN_DOOR_DATE =’20200421’;
```

- **优化 SQL 示例**

给表 `ACCOUNT` 的 `FNAL_OPEN_DOOR_DATE` 建立索引

```
> ALTER TABLE ACCOUNT ADD INDEX idx_account_1 (FNAL_OPEN_DOOR_DATE
);
> SELECT NAME FROM ACCOUNT WHERE FNAL_OPEN_DOOR_DATE =’20200421’;
```

不建议 SELECT *

- **不建议原因**

SELECT *会返回整行记录所有字段，如果该字段在应用中未被使用，无谓增加网络间流量。

当表新增字段时，原有大量交易不使用新增字段，但会增加网络负担。

- **优化方法**

将本次应用需要使用的字段全部列出，即使当前每个字段均使用到。

- **原 SQL 示例**

```
> SELECT * FROM ACCOUNT WHERE ACCOUNT_ID='A001';
```

- **优化 SQL 示例**

```
> SELECT ACCOUNT_ID, ACCOUNT_NAME, ..., REGISTER_DATE
  FROM ACCOUNT
  WHERE ACCOUNT_ID='A001';
```

不建议三层或以上的嵌套子查询

- **不建议原因**

子查询出现在 `FROM` 子句或者 `WHERE` 子句中，查询语句不能出现三次 `SELECT`。如在主语句、`FROM` 子句和 `WHERE` 子句中同时出现 `SELECT`。同理，写语句不能出现两次 `SELECT`。

子查询需要将数据从数据节点获取后，在计算节点进一步处理。计算远离数据，内部传输流量大，计算节点处理负荷重、中间结果积压，容易成为处理瓶颈，阻塞其他连接处理。此外，复杂 SQL 不利于数据库性能横向扩展。

建议不出现子查询，存在子查询的语句需阐明合理原因(如子查询结果集不大于200)。

- **优化方法**

将子查询分拆成多个单独查询，通过应用集群和分布式数据库的高并发处理优势，解决复杂 SQL 问题。

- **IN 嵌套原 SQL 示例**

```
示例1:
> SELECT A,B,C FROM T1 WHERE B IN (
  SELECT B FROM T2 WHERE C IN(
  SELECT C FROM T3 WHERE D ='d1000'));
示例2:
> SELECT ... FROM t1
  WHERE col1 IN (
```

```
  SELECT col2 FROM t2 WHERE ...
  );
```

- **优化 SQL 示例**

```
示例1:
> SELECT C FROM T3 WHERE D='d0001';
> SELECT B FROM T2 WHERE C ='c1000';
> SELECT A FROM T1 WHERE B IN('b0001','b0002','b0003');
示例2:
> SELECT col2 FROM t2 WHERE ...
  SELECT ... FROM t1 WHERE col1 IN(...)
#应用先执行子查询 "SELECT col2 FROM t2 WHERE ...得到列表(赋给变量 list1
); 再执行主查询（主查询改为"SELECT ... FROM t1 WHERE col1 IN (?)", list
1 作为查询参数传入主查询）。
```

- **EXIST嵌套原 SQL 示例**

```
示例1:
> SELECT ... FROM t1 WHERE EXISTS (SELECT 1 FROM t2 WHERE t2.col_n
 &lt; t1.col_m) ...
```

- **优化 SQL 示例**

```
拆分为:
语句A: SELECT ... FROM t1 WHERE ...
语句B: SELECT 1 FROM t2 WHERE t2.col_n < ?
#应用首先执行语句 A 得到结果集 RestultSet1，然后对于 ResultSet1 中每条记录
，循环执行语句 B，将查询结果为空的记录从 ResultSet1 中剔除掉，得到最后的结果
集。
```

不建议三表或以上的表关联

- **不建议原因**

多表关联(JOIN)需要将每个表从数据节点获取后，在计算节点进一步处理。计算远离数据，计算节点处理负荷重，容易积压中间结果集数据，成为处理瓶颈，阻塞其他连接处理。此外，复杂 SQL 不利于数据库性能横向扩展。

建议不出现表关联，存在表关联的语句需符合如下任一条件：

其中一个表是复制表其中一个表记录行小于 200、两表的分片规则相同且含有分片键的等值条件。

- **优化方法**

将多表分拆成多个单独查询，通过应用集群和分布式数据库的高并发能力，解决复杂 SQL 问

题。

- **原 SQL 示例**

```
> SELECT ... FROM t1
  INNER JOIN t2 ON ...
  INNER JOIN t3 ON ...
  INNER JOIN t4 ON ...
  WHERE ...
```

- **优化 SQL 示例**

```
拆分成语句A:
> SELECT ... FROM t1 INNER JOIN t2 ON ...
和语句B:
> SELECT ... FROM t3 WHERE ...
语句C:
> SELECT ... FROM t4 WHERE ...
```

不建议两个大表关联

- **不建议原因**

超过 100 万条记录的表算作大表。

两个大表关联(`JOIN`)时，需要将每个大表的数据从数据节点获取后，在计算节点进一步处理。当缺少关联条件判断或关联性不强时，需要将所有大表数据都提取到计算节点，网络容易成为瓶颈，计算节点积压大量中间数据，无法响应其他连接请求。

除非两表分片键相同且有分片键等值的查询条件，此时 `JOIN` 可以下沉到多个数据分片上进行本地计算。

- **优化方法**

通过合理设计，将大表 `JOIN` 下沉到数据节点，避免计算节点阻塞通道。如将两表设计为相同分片规则，且明确查询条件是两表基于相同分片键的等值查询。

- **原SQL示例**

```
> SELECT T1.COL1,T2.COL2 FROM T1 JOIN T2 ON T1.A=T2.A; |
```

- **优化SQL示例**

```
> CREATE TABLE t1(COL1,...) DISTRIBUTED BY HASH(COL1)(g1,g2);
> CREATE TABLE t2(COL1,...) DISTRIBUTED BY HASH(COL1)(g1,g2);
#两个表的分发规则相同;
> SELECT T1.COL1,T2.COL2
```

```
  FROM T1 JOIN T2 ON T1.A=T2.A
  WHERE T1.COL1=T2.COL1;
```

不建议左关联

- **不建议原因**

左关联(`LEFT JOIN`)需要将两个表的数据从数据节点获取后，在计算节点进一步处理。由于关联条件松散，需要将数据都提取到计算节点，网络容易成为瓶颈，计算节点积压大量中间数据，无法响应其他连接请求。

同理，不建议右关联。

- **优化方法**

先查询主表(T1)，然后根据主表条件再查询从表(T2)。

应用层先执行T1表查询得到结果集 `ResultSet1`，然后对于 `ResultSet1` 中每条记录，循环执行查询 T2 表，最后拼装成一个结果集。

如贷款详情和还款详情需要关联并同时显示时，则先查贷款记录，根据贷款记录的信息再查询还款记录。有多次贷款需要分页显示时，只有显示到该分页时，才下发查询此页关联的还款记录。

- **原 SQL 示例**

```
> SELECT t1.col1, t2.col2
  FROM t1
  LEFT JOIN t2 ON t2.id = t1.id
  WHERE ...
```

- **优化 SQL 示例**

```
先查询 T1 表：
> SELECT t1.id, t1.col1 FROM t1 WHERE ...
在查询T2表：
> SELECT t2.col2 FROM t2 WHERE t2.id = ? AND ...
```

不建议大结果集

- **不建议原因**

大结果集查询容易对磁盘 IO、网络造成冲击，影响高并发事务交易，阻塞其他连接请求。

- **优化方法**

根据业务场景在应用层控制最大结果集,如分页列表最多显示 2000 行。

分批多次查询，每次只查询指定记录行数。

- **原 SQL 示例**

```
SELECT ORGN_NUM, REGISTER_DATE FROM orbsta;
```

- **优化 SQL 示例**

```
#增加条件，按更细粒度范围，分成多次查询
> SELECT ORGN_NUM, REGISTER_DATE FROM orbsta
  WHERE REGISTER_DATE > '2020-1-19' AND
  REGISTER_DATE <= '2020-1-20';
> SELECT ORGN_NUM,,,REGISTER_DATE FROM orbsta WHERE
  REGISTER_DATE > '2020-1-20' AND REGISTER_DATE <= '2020-1-21';
```

不建议大事务

- **不建议原因**

大数据量写事务产生大事务日志，跨机房数据同步时容易造成网络延迟。大量记录行加写锁，无法被其他写事务更新，影响高并发事务处理。大事务更新数据量规模大、处理时间长，事务回滚成本高。

- **优化方法**

分批次更新，每次只更新小规模数据。

避免在联机交易高峰时执行大事务。

- **原 SQL 示例**

```
#T1 在 DP001 与 DP100 间的记录行超过百万级别
> UPDATE t1 SET t1.STATUS = 'ACTIVE'
  WHERE DEPT_NO  >= 'DP001' AND DEPT_NO  <= 'DP100';
```

- **优化 SQL 示例**

```
#拆分成多个 UPDATE 事务，每个事务执行一个 DP001 的状态变更
FOR(...)
{
UPDATE t1 SET t1.STATUS = 'ACTIVE'
WHERE DEPT_NO= '?';//每次传入个部门编号。
}
```

不建议大字段

- **不建议原因**

单字段不超过 16M、单行不超过 64M。

联机交易数据库基于小数量、高并发、短时延事务模型进行设计，大记录行处理时容易对磁盘、网络形成瓶颈。

- **优化方法**

使用非联机交易数据库系统存储大记录行。

不与有高并发要求的应用系统共用数据库。

- **原 SQL 示例**

```
> INSERT INTO t1(id,blob_max) VALUES(1,'fasdfdsfdsafdsafadsfdsfsad
fasdfsddfasdfdsfdsfdsafasdfdsfdsfdsfsdfdsfdsfdsfdsfdsfsdaffadsfads
fsadfsdfdsfadsfasdfasddddddddddfdsafdsafdsafaddddasfdsfdsfsdfsdfsd
afsdaffdsff...');
#blob_max值大于 16M
```

- **优化 SQL 示例**

无

不建议 LIMIT 分页

- **不建议原因**

`LIMIT(n,m)` 分页查询，每页查询返回的顺序并不相同，导致页间数据重复或缺失。此问题在单机数据库中存在，在分布式数据库中更加明显。其次，数据库每次都从首行开始扫描，直到过滤到第n行时才命中记录行，页数越靠后的查询响应延迟越大，数据库开销越高。

- **优化方法**

通过按照主键或者索引进行排序控制分页顺序，并通过主键或索引控制每页起始行。如为表建立主键(PK)，查询时按主键排序，第一页时取 `LIMIT 100`,第二页时加上这个主键条件"PK>上一页最大值 `LIMIT 100`"，每页依次递归。

控制结果总行数(如每页 100 条，最大 2000 条)，在应用层缓存多余页面数据。

- **原 SQL 示例**

```
> SELECT A,B,C, REGDATE FROM HISTORY
  WHERE A = 'a0001' LIMIT(20,40);
```

- **优化 SQL 示例**

```
方法一：
> SELECT A,B,C ,REGDATE FROM HISTORY
  WHERE A='a0001' AND PK1 > 12020
  ORDER BY PK1 LIMIT 100 ;
```

```
方法二：
> SELECT A,B,C ,REGDATE FROM HISTORY
  WHERE A='a0001' LIMIT 2000;
```

不建议在UPDATE语句中同时更新两张或以上表

- **不建议原因**

SQL 语义更清晰。

- **优化方法**

单语句拆分成多条语句。

- **原 SQL 示例**

```
> UPDATE student s , class c SET s.class_name='test00',c.stu_name=
'test00' WHERE s.class_id = c.id
```

- **优化 SQL 示例**

```
> UPDATE student SET student.class_name='test00' WHERE student.cla
ss_id=1;
> UPDATE class SET class.stu_name='test00' WHERE class.id=1;
```

不建议 UNION 和 UNION ALL

- **不建议原因**

`UNION` 和 `UNION ALL` 需要将中间结果集从每个数据分片获取后，在计算节点进一步处理(特别是 `UNION` 需要去重处理)。数据在计算节点停留时间变长，计算节点内存和计算负荷加重，容易成为处理瓶颈，阻塞其他连接处理。

- **优化方法**

拆分成多个子查询，将每个子查询的返回结果取到内存中合并。

- **原 SQL 示例**

```
> SELECT ACCOUNT_ID, ACCOUNT_NAME, ...
  FROM ACCOUNT1
  WHERE REGISTER_DATE  > '2020-10-20' AND DEPT='DP01'
```

```
  UNION
  SELECT ACCOUNT_ID, ACCOUNT_NAME, ...
  FROM ACCOUNT2
  WHERE REGISTER_DATE  > '2020-10-20' AND DEPT='DP02'
  UNION
  SELECT ACCOUNT_ID, ACCOUNT_NAME, ...
  FROM ACCOUNT3
  WHERE REGISTER_DATE  > '2020-10-20' AND DEPT='DP03';
```

- **优化 SQL 示例**

```
#修改成三条单独的 SQL 语句，应用层将三个结果集获取后进行去重。
> SELECT ACCOUNT_ID, ACCOUNT_NAME, ...
  FROM ACCOUNT1
  WHERE REGISTER_DATE  > '2020-10-20' AND DEPT='DP01';
  SELECT ACCOUNT_ID, ACCOUNT_NAME, ...
  FROM ACCOUNT2
  WHERE REGISTER_DATE  > '2020-10-20' AND DEPT='DP02';
  SELECT ACCOUNT_ID, ACCOUNT_NAME, ...
  FROM ACCOUNT3
  WHERE REGISTER_DATE  > '2020-10-20' AND DEPT='DP03';
```

不建议元素过多的IN列表

- **不建议原因**

`IN` 从句列表来自应用动态传参，长度可变，有的长度会达到上百个。`IN` 列表长导致应用传递给数据库的单 SQL 语句过长。数据库对长列表 `IN` 字句优化程度没有等值查询高。

不建议IN列表长度超过 50。

- **优化方法**

不采用动态生成的 `IN` 列表，对可能超过 50 个元素的 `IN` 列表改为在应用层做 `FOR` 循环来实现。

- **原 SQL 示例**

```
> SELECT ... FROM t1 WHERE col1 in (#{list1})
#变量list1运行时可能含有超过 50 个元素
```

- **优化 SQL 示例**

```
FOR(...)
{ ...
SELECT ... FROM t1 WHERE col1 = ?
...
}
```

```
应用代码对 list1 中每个元素循环执行 SELECT 语句，最后合并结果集
```

不建议大表使用复制表

- **不建议原因**

一个表的设计容量大于 100 万行，该表称为大表。不建议大表采用复制表模式。

复制表在每个分片上均有相同的全量数据，适合用于被其他表进行本地关联查询。如果表的规模较大，并没有达到数据分散存储的目的，无法通过增加机器数量扩展分布数据库的存储和计算性能。

- **优化方法**

随着业务增长记录行数会增加的表，应避免采用复制表模式。

大表应采用哈希、范围或列表等分片模式，将数据打散到多个分片。

- **原 SQL 示例**

```
用户表使用复制表：
> CREATE TABLE USER(USER_ID, REGISTER_DEPT....)
  DISTRIBUTED BY DUPLICATE(g1,g2,g3);
用户借贷记录表，根据机构号使用哈希分片：
> CREATE TABLE PAYMENT_HIST(USER_ID, REGISTER_DEPT,....)
  DISTRIBUTED BY HASH(REGISTER_DEPT)(g1,g2,g3);
```

- **优化 SQL 示例**

```
用户表随着业务运行数据量增大，不应采用复制表。
用户表、借贷记录采用相同的分片模式
> CREATE TABLE USER(USER_ID, REGISTER_DEPT....)
  DISTRIBUTED BY HASH(REGISTER_DEPT)(g1,g2,g3);
> CREATE TABLE PAYMENT_HIST(USER_ID, REGISTER_DEPT,....)
  DISTRIBUTED BY HASH(REGISTER_DEPT)(g1,g2,g3);
```

不建议高频变更表使用复制表

- **不建议原因**

对复制表的变更，如插入、更新和删除操作，需要在每个分片都执行成功，整个变更事务才能成功。如果分片数量较大，则更新效率低下。只允许日常更新较少的表使用复制表，不建议经常更新的表使用复制表模式。

- **优化方法**

如果表数据量较少，可以采用单节点表。

如果表的规模较大，采用哈希、范围或者列表等模式，将数据打散到各个分片上。

- **原SQL示例**

```
汇率表采用复制表，但每分钟需要更新一次
> CREATE TABLE EXCHANGE_RATE(CURRENCY,RATE,....)
  DISTRIBUTED BY DUPLICATE(g1,g2,g3);
每次更新需要在多个分片上同时执行
> UPDATE EXCHANGE_RATE SET RATE=... WHERE CURRENCY='CNY';
```

- **优化SQL示例**

```
汇率表改用单节点表（只使用g1一个分片）
> CREATE TABLE EXCHANGE_RATE(CRRENCY,RATE,....)
  DISTRIBUTED BY DUPLICATE(g1);
> UPDATE EXCHANGE_RATE SET RATE=... WHERE CURRENCY='CNY';
```

不建议对分发字段进行修改

- **不建议原因**

修改分片字段引起数据在数据分片间迁移，可以理解为在新数据分片进行INSERT，在原数据分片进行DELETE。数据库同步工具接入每个数据分片并行同步数据时，无法识别 `DELETE` 和 `INSERT` 先后顺序，导致同步到下游时出现数据丢失。

- **优化方法**

根据具体业务场景，选择合适的字段作为分片键。

额外增加技术分片键方式，控制技术分片键不修改。

- **原 SQL 示例**

```
> CREATE TABLE PAYMENT(DEPT,....)
  DISTRIBUTED BY HASH(DEPT)(g1,g2,g3);
> UPDATE PAYMENT SET DEPT= 'DP001' WHERE DEPT= 'DP002';
```

- **优化 SQL 示例**

```
增加技术分片键，基于技术分片键进行分片。
> CREATE TABLE PAYMENT(PKDEPT ..., DEPT,....)
  DISTRIBUTED BY HASH(PKDEPT)(g1,g2,g3);
> UPDATE PAYMENT SET DEPT= 'DP001' WHERE DEPT= 'DP002';
```

不建议分布式存储过程、视图、触发器、临时表功能

- **不建议原因**

不建议使用上述数据库高级对象。

- **优化方法**

应用减少使用高级数据库对象，对数据库和中间件具有更好的兼容性。

不建议选择时间戳字段作为主键

- **不建议原因**

使用时间戳作为主键可能导致主键冲突。

- **优化方法**

禁用。

- **原 SQL 示例**

```
> CREATE TABLE `gipacp` (
  `BANK_NUM` char(3) NOT NULL COMMENT '银行号',
  `BSIN_SCEN_FLG` char(6) NOT NULL COMMENT '业务场景标识',
  `BSIN_SCEN_FLG_CMNT` char(122) NOT NULL COMMENT
  '业务场景标识说明',
  `id` int(11) NOT NULL COMMENT '序号',
  `ACCT_INSI` char(5) NOT NULL COMMENT '账务机构',
  `CCY` char(3) NOT NULL COMMENT '币种',
  `ACCT_TYP` char(1) NOT NULL COMMENT '账户类别',
  `TRAN_ACTNUM` char(22) NOT NULL COMMENT '交易账号',
  `DBIT_CRDT_FLG` char(1) NOT NULL COMMENT '借贷标识',
  `CRAT_TMTP` datetime(6) NOT NULL COMMENT '建立时间戳',
  `LAST_TMTP` datetime(6) NOT NULL COMMENT '最后时间戳',
  PRIMARY KEY (`CRAT_TMTP`),
  INDEX `I_GIPACP_01` (`BSIN_SCEN_FLG`,`BANK_NUM`)
  ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_bin
  DISTRIBUTED BY DUPLICATE(g1);
```

- **优化 SQL 示例**

```
> CREATE TABLE `gipacp` (
  `BANK_NUM` char(3) NOT NULL COMMENT '银行号',
  `BSIN_SCEN_FLG` char(6) NOT NULL COMMENT '业务场景标识',
  `BSIN_SCEN_FLG_CMNT` char(122) NOT NULL COMMENT
  '业务场景标识说明',
  `id` int(11) NOT NULL COMMENT '序号',
  `ACCT_INSI` char(5) NOT NULL COMMENT '账务机构',
  `CCY` char(3) NOT NULL COMMENT '币种',
  `ACCT_TYP` char(1) NOT NULL COMMENT '账户类别',
  `TRAN_ACTNUM` char(22) NOT NULL COMMENT '交易账号',
  `DBIT_CRDT_FLG` char(1) NOT NULL COMMENT '借贷标识',
  `CRAT_TMTP` datetime(6) NOT NULL COMMENT '建立时间戳',
```

```
  `LAST_TMTP` datetime(6) NOT NULL COMMENT '最后时间戳',
  PRIMARY KEY (`BSIN_SCEN_FLG`,`id`,`BANK_NUM`),
  INDEX `I_GIPACP_01` (`BSIN_SCEN_FLG`,`BANK_NUM`)
  ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_bin
  DISTRIBUTED BY DUPLICATE(g1);
```

不建议同一集群创建不同字符集的库或表

- **不建议原因**

不能统一字符集配置，使用不当容易出现乱码。

- **优化方法**

禁用。

不建议数据库名长度超过30B

GoldenDB 数据库名长度不超过 64B；`Oracle` 数据库名长度不超过 30B。统一两个数据库名长度要求为：不建议数据库名长度超过 30B。

不建议表名长度超过 64B

如题。

不建议字段名长度超过 64B

如题。

不建议连接实例用户名超过 128(最长支持 128、超过截断)

如题。

UPDATE 和 DELETE 不建议使用 LIMIT

如题。

不建议在 UPDATE 的 SET 子句中出现 AND 和 OR

如题。

1.9.2.2. 建议项

关于索引使用规范

总要求：索引使用的目的是加速查询的速度，对于经常查询的字段、业务编号字段、外键字段，建议创建索引（例：客户编号、产品编号、合同编号等）。索引不是越多越好，索引会降低数据更新、插入的速度，因为在数据变动时，索引也会进行调整和更新，消耗系统资源，同

时索引也要消耗系统磁盘空间，数据量很小的表（如：参数表，配置表等）建议不要创建索引。

具体细则如下：

A.关于索引命名规范。索引命名参考如下：idx_表名_序号(递增序号)。
例：`idx_cust_corporat_1`。

B.不要在区分度低的字段上建立索引。比如全表 10000 条数据，某个字段 `DISTINCT` 值共 5 个，建立索引没有任何意义。

C.关于唯一索引创建原则。如果某一列的值能确认是唯一的，可以建立唯一索引，查询速度会更快，一定要确定数据能保持唯一，不然后期数据出现重复，入数会失败。

D.关于分组、排序的索引创建原则。如果某个字段经常需要用来 `GROUP BY` 或 `ORDER BY` 等，考虑建索引，分组、排序速度会提高。

E.关于联合索引创建原则。区分度高的字段放前面，比如 `index_name(a,b,c,d)`，哪个字段的区分度高，往前放。

F.联合索引使用原则。联合索引应遵循"最左前缀原则"，比如某个联合索引 `index_name(a,b,c)`，如果查询使用 `a= AND b= AND c=` 或者 `c= AND b=` 或者 `a=` 是可以使用索引的（具体 a= 和 b= 谁写前面没关系，系统会自动优化），如果只用到 `b`、`c` 或者只用到 `c` 作为条件是无法使用 `index_name` 这个索引。

G.如索引列使用函数时不能使用索引。索引列使用函数参与运算则不能使用索引，比如 `a INT` 上有索引，`WHERE a=2-1` 可以使用索引，如果写为 `WHERE a+1=2` 就不能使用索引。

H.关于LIKE通配符索引的使用原则。 如果以 % 开头，也不能使用索引。比如 `a CHAR(5)` 有索引，`WHERE a LIKE 'name%' ?` 可以使用索引，`WHERE a LIKE '%name%'` 不能使用索引。

I.索引使用时应避免隐式的数据类型转换。计算等式两侧类型要一致，比如 `a CHAR(5)` 有索引，使用 a=5 不能使用索引，需要改为 `a='5'`。

J.在关联字段、查询条件需优先考虑建立索引。分析某个 SQL 语句是否使用到了索引，请使用 `EXPLAIN SQL` 语句进行分析。

K.以上原则必须结合 `EXPLAIN` 执行计划分析的结果。

关于生产表的分区要求

总要求：对于任何交易系统生产表，凡总量大小超过 10G，记录数超过 500W 以上，原则上必须考虑设计分区。

如若单分区大小超过 10G，记录数超过 500W 以上，则可依据业务实际需求，考虑设计子分区。具体参考原则如下：

1. 对于数据量比较大的表,根据表数据的属性进行分区,以得到较好的性能。
2. 如果表按某些字段进行增长,则采用按字段值范围进行范围分区。
3. 如果表按某个字段的几个关键值进行分布,则采用列表分区。
4. 对于静态表,则采用 `HASH` 分区或列表分区。
5. 对于流水表，需要定期清理大量数据的表，建议使用表分区。如果使用分片后，表数据量仍然很大，可以使用表分区，表分区的分区规则，建议和表分片的分片规则一致。根据需要保留的数据周期，定期进行分区创建/卸载或者翻台。
6. 分区键字段必须包含在表主键/唯一索引中。根据数据增量的大小，可以选择日期或者月份做分区键。
7. 每个表分区都是一个单独的文件，表分区过多会导致数据库需要打开过多的文件，占用系统的文件句柄数。在设计表分区时，需要考虑分区数目的限制，不能超过系统用户句柄数限制。

SQL 中 WHERE 条件建议带分发键，如果无法直接带分发键，建议在分发键和约束条件之间建立分片映射

建议以逻辑删除数据代替物理删除，不建议直接物理删除数据

等价 JOIN 代替 WHERE 子查询，除非明确知道 WHERE 子查询结果集少于 20 条

对于使用 LIMIT 的分页查询，建议应用对 ORDER BY 所涉及的列确保唯一性

WHERE 子句中，建议以AND ...OR...代替多字段 IN，提高执行效率

如多字段 `WHERE (a,b,c) IN ((1,2,3),(2,3,4))` 替换为：`WHERE （a=1 AND b=2 AND c=3）OR（a=2 AND b=3 AND c=4）`。

建议应用程序使用 JDBC 提供的预处理相关函数

建议 UPDATE 和 DELETE 尽量依据主键或唯一索引

1.9.3. GoldenDB分布式数据库执行过程案例详解

GoldenDB 的 `EXPLAIN` 结果主要包括两个部分：DN 层面的 `EXPLAIN` 结果及CN层面的 `EXPLAIN` 结果。二者按照计划树的执行顺序由上至下打印，通过标识符id进行区分。CN 层及 DN 层的 `EXPLAIN` 说明分别如下：

1.9.3.1. DN 层面 EXPLAIN 说明

- **id（从1开始）**

id 为识别符即 `SELECT` 的查询序号；当id相同时，执行顺序由上至下；id不同时，如有子查询 id 序号递增，值越大优先执行。与 CN 层面 id 的区别为：DN 层面id 从 1 开始，CN 层面则由 10001 开始。

- **select_type**

表示查询的类型，常见 `select_type` 类型包括：`SIMPLE`、`PRIMARY`、`SUBQUERY`、`DERIVED`、`UNION`、`UNION RESULT`。

1. `SIMPLE`:不含子查询和 `UNION` 的简单查询语句。
2. `PRIMARY`:包含子查询的查询语句（`UNION` 或带子查询等语句）中，最外层查询（主语句）被标记为 `PRIMARY`。
3. `SUBQUERY`：子查询中的第一个查询（其结果不依赖于外部查询）。
4. `DERIVED`：`FROM` 子句的查询被标记为 `DERIVED`，主节点递归执行这些子查询，并将其结果存放于临时表中。
5. `UNION`：`UNION` 后的查询被标记为 `UNION`。当 `UNION` 包含于 `FROM` 子句的查询中时，外层语句将被标记为 `DERIVED`。
6. `UNION RESULT`：`UNION` 表获取的结果。

- **table**

显示当前语句所查询的表。

- **partitions**

当存在分区表时，将显示当前查询所匹配到的分区。非分区表情况下，值为 NULL。

- **type**

主要指查询过程的类型，常见的type类型包括：`all`、`index`、`range`、`ref`、`eq_ref`、`const`、`null`。`type` 值的不同将影响语句的执行时间。

1. `all`：遍历全表查询匹配的行。
2. `index`：遍历索引查询匹配的行。

3. `range`：语句中的查询条件为对索引的范围约束时，定义为 `range`，将通过索引扫描给定范围的行，非索引键时为 `all`。

4. `ref`：非唯一索引扫描。多表连接中通过某些列或常量查询索引列上的值，从而查询到匹配的行，并显示关联字段于 `ref` 中。

5. `eq_ref`：唯一索引扫描。多表连接中使用唯一索引或主键作为关联条件，通过某些列或常量查询其值，并显示关联字段于 `ref` 中。

6. `const`：表示通过索引一次即能查询的值。当对查询部分进行优化将查询条件转化为一个常量时，定义为 `const`。

7. `null`：语句经优化后，执行时甚至不用访问表或索引，即为 `null`。

- **possible_keys**

表示查询过程中可以利用的索引。查询涉及的字段若存在索引，则该索引将被列出，但不一定在查询过程中实际被使用，表示该查询可利用的索引。如果为NULL则没有可用的索引字段。

- **key**

查询过程实际使用的索引键。如果为NULL，则没有使用索引，包括可能没有建立索引或索引失效。

- **key_len**

表示索引中使用的字节数，可通过该列计算查询中使用的索引的长度。key_len显示的值为索引字段的最大可能长度，并非实际使用长度，即 `key_len` 是根据表定义计算而得，不是通过表内检索得来的。

- **ref**

表示表的连接匹配条件，即显示哪些字段或者常量被用于查找索引列的值。若为常数等值查询ref值为const；若条件使用了表达式或函数，`ref` 可能显示为 `func`；如果是连接查询 `ref` 会显示驱动表的关联字段。

- **rows**

各分片查询的表行数，根据表统计信息及索引选用情况，大致估算出找到所需的记录所需要读取的行数，越少查询的时间越短。

- **filtered**

查询筛选率，即按表条件筛选的行所占百分比。

- **extra**

以上各项中未包含的一些重要信息，包括语句下发的分片信息，处理级别以及表的分发属性等。常见语句释义如下：

`Using where`:表明查询语句中含有 `WHERE` 条件过滤。

`Using temporary`：在对查询结果排序时，使用临时表来存储结果集，常见于排序和分组查询 `GROUP BY`、`ORDER BY` 中。

`Using index`：表示查询过程中使用了索引，如果同时出现 `Using where` 表明索引被用来进行索引键的值查找。

`Using filesort`：当查询过程中包含 `ORDER BY` 操作，且无法利用索引完成的排序操作。

`Using join buffer`：表明使用了连接缓存。

`Impossible where`：表明 `WHERE` 语句会导致没有符合条件的行。

`No tables used`：查询语句中使用系统表 `DUAL` 表或不涉及任何表的语句。

1.9.3.2. CN 层面 EXPLAIN 说明

- **id（从 10001 开始）**

对执行计划节点进行标记，从根节点开始记为 10001；子节点 id 依次加 1；兄弟节点 id 相同。执行顺序则由大到小，兄弟节点并行执行，子节点执行完毕再执行父节点，直至根节点执行完毕。

- **select_type**

表示 CN 层面生成的执行计划节点，主要包括 `InsertSQLNode`、`UpdateNode`、`DeleteNode`、`SQLNode`、`MSQLNode`、`JoinNode`、`UnionNode`。

- **table（从 1 开始）**

对各节点进行编号，从根节点开始记为 1；遍历查询各子节点table值依次增加，叶子节点间 table 值相邻。

- **partitions**

在 CN 层处理的分区表，将显示当前查询所匹配到的分区。

- **type**

暂未定义。

- **possible_keys**

该节点拼接将下发至 DN 的语句。

- **key**

显示下发对应的集群信息及 DN 信息。`JoinNode`、`UnionNode` 及含有子节点的 `MSQLNode` 输出为"null"。

- **key_len**

暂未定义。

- **ref**

说明当前节点的父节点（Parent）、子节点（Child）和兄弟节点（Next），有则输出对应的table值，无则输出 "NULL"。

- **rows**

暂未定义。

- **filtered**

暂未定义。

- **extra**

其他信息，不同类型node所包含的可能信息有：

`JoinNode`:读写隔离级别、`SORT_MERGE`(`JOIN` 流程)、`USING GROUP`(`GROUP BY` 流程)、`USING SORT` （`ORDER BY` 流程）、`USING WHERE` （带 `WHERE` 条件）。

`MSQLNode`:读写隔离级别、SELECT FOR `UPDATE` （CW 条件下 `UPDATE/DELETE` 流程将先进行 `SELECT` 流程，以确保表中 GTID 不存在当前活跃事务列表中。）、`LOCK IN SHARE MODE` （锁信息）、表信息（库表名及分发属性）。

`SQLNode`:读写隔离级别、`SELECT FOR UPDATE`、`LOCK IN SHARE MODE`、`USING SORT`、`USING WHERE`、`USING GROUP`、表信息。

`UnionNode`:读写隔离级别、`USING SORT`。

`InsertSQLNode`、`UpdateNode`、`DeleteNode`：读写隔离级别信息、GTID信息（申请GTID时打印该字段）、表信息，`SELECT FOR UPDATE`，`USING SORT`。

1.9.3.3. 打印指定 DN/CN 的 EXPLAIN 结果

1. 指定 DN

`EXPLAIN` 后加 `DB`，并在语句末尾通过 `STORAGEDB` 指定 DN 打印 `EXPLAIN` 结果。

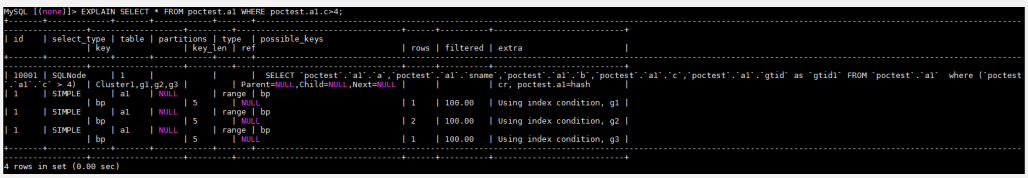
例：EXPLAIN DB SELECT * FROM poctest.a1 WHERE poctest.a1.c>4 STORAGEDB g1;

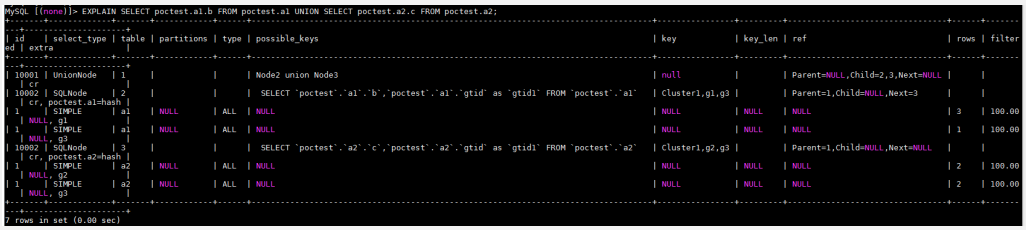
2. 指定 CN

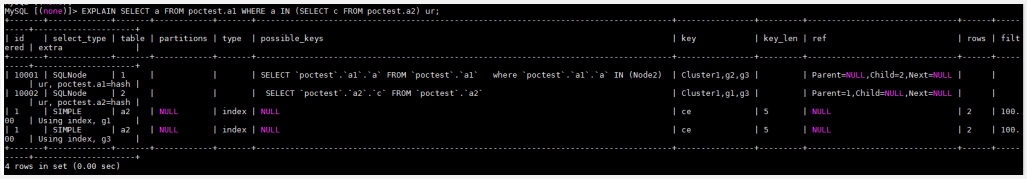
通过 `EXPLAIN` 后加 `PROXY` 指定对应的 CN 打印 `EXPLAIN` 结果。

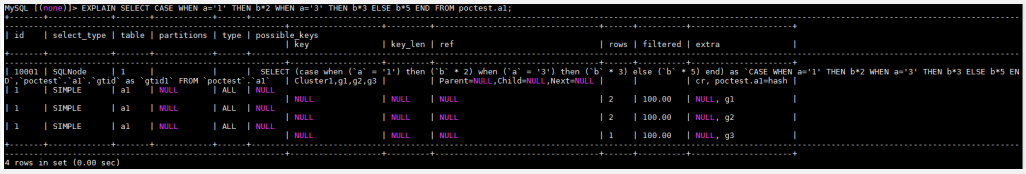
例：EXPLAIN PROXY SELECT * FROM poctest.a1 WHERE poctest.a1.c>4;ne

1.9.3.4. 典型语句举例说明

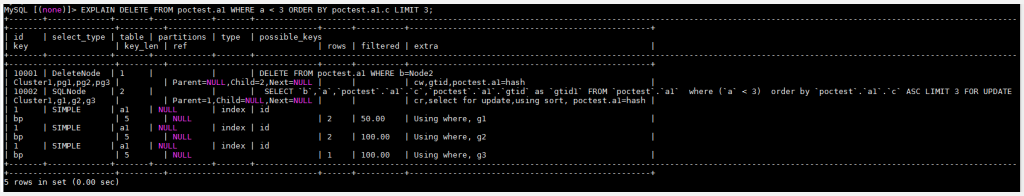
语句	SELECT * FROM poctest.a1 WHERE poctest.a1.c > 4;
预置条件	create database if not exists poctest; create table if not exists poctest.a1 (a int, sname char (10) not null, b int primary key, c int, index id (a), unique index bp (c)) distributed by hash (a)(g1,g2,g3); insert into poctest.a1 (a, sname, b, c) values (1, 'a', 102, 104), (2, 'b',2, 5), (3, 'c', 106, 2), (4, 'd', 3, 3); explain select * from poctest.a1 where poctest.a1.c>4;
EXPLAIN结果	
EXPLAIN说明	执行流程1：id为10001，CN层面的执行流程，该流程下生成了一个简单的SQLNode节点，possible_keys为CN拼接下发至DN的语句，key表明语句将下发至cluster1的g1、g2、g3分片执行，ref表明该语句为单一节点无父子兄弟节点，extra则说明了该流程为cr，表a1为哈希表。 执行流程2：id为1，各分片同时对a1表执行select_type为SIMPLE的查询，type为range表明SQL语句为指定条件范围查询，DN上查询可供使用及实际使用的索引为唯一索引'bp'，extra表明当前行所说明的分片及查询途径。

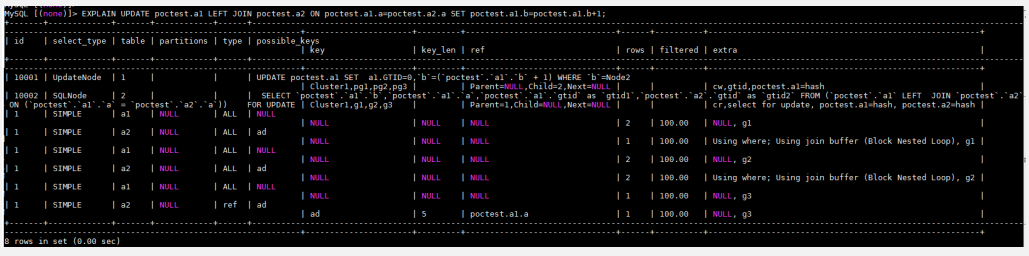
语句	SELECT poctest.a1.b FROM poctest.a1 UNION SELECT poctest.a2.c FROM poctest.a2;
预置条件	create table if not exists poctest.a1 (a int, sname char (10) not null, b int primary key, c int, index id (a), unique index bp (c)) distributed by hash (a)(g1,g3); insert into poctest.a1 (a, sname, b, c) values (1, 'a', 102, 104), (2, 'b',2, 5), (3, 'c', 106, 2), (4, 'd', 3, 3); create table if not exists poctest.a2 (a int, sname char (10) not null primary key, b int, c int null, index ad (a), unique index ce (c)) distributed by hash (a)(g2,g3); insert into poctest.a2 (a, sname, b, c) values (8, 'a', 102, 104), (2, 'b',9, 7), (9, 'c', 106, 2), (4, 'd', 10, 3); explain select poctest.a1.b from poctest.a1 union select poctest.a2.c from poctest.a2;
explain结果	
explain说明	执行流程1：两个id同为10002表明该流程为CN层面的并行流程，该流程下生成了一个简单的SQLNode节点，ref表明该流程有一个标记为1的父节点和一个兄弟节点。 执行流程2：id为1表明为DN层流程，各分片执行select_type为SIMPLE的查询，查询所涉及的表（table）由接收到的从CN端拼接发送的SQL语句所决定。查询方式为遍历全表的查询方式。 执行流程3：id为10001，CN层面流程。该流程生成了一个UnionNode节点，possible_key表明该节点将标记为2,3的节点进行UNION，ref表明该节点有两个子节点。
语句	SELECT a FROM poctest.a1 WHERE a IN (SELECT c FROM poctest.a2) ur;

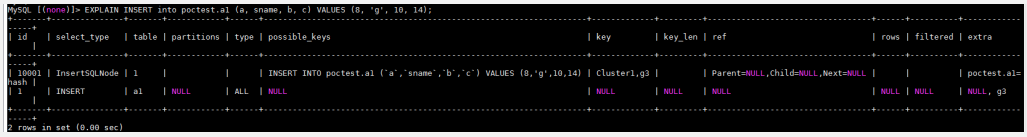
预置条件	create table if not exists poctest.a1 (a int, sname char (10) not null, b int primary key, c int, index id (a), unique index bp (c)) distributed by hash (a)(g2,g3); insert into poctest.a1 (a, sname, b, c) values (1, 'a', 102, 104), (2, 'b',2, 5), (3, 'c', 106, 2), (4, 'd', 3, 3); create table if not exists poctest.a2 (a int, sname char (10) not null primary key, b int, c int null, index ad (a), unique index ce (c)) distributed by hash (a)(g1,g3); insert into poctest.a2 (a, sname, b, c) values (8, 'a', 102, 104), (2, 'b',9, 7), (9, 'c', 106, 2), (4, 'd', 10, 3); explain select a from poctest.a1 where a in (select c from poctest.a2) ur;
explain结果	
explain说明	执行流程1：id为10002，CN层面流程，该流程下生成了一个简单的SQLNode节点，下发至DN的拼接语句为[SELECT poctest.a2.c FROM poctest.a2]，该流程有一个标记为1的父节点，语句下发分片为g1、g3。该流程为ur，语句未生成gtid列a2为哈希表。 执行流程2：id为1，DN层面流程，分片g1/g2执行select_type为SIMPLE的查询，type为index表明通过唯一索引'ce'查询匹配行。 执行流程3：id为10001，CN层面流程。该流程生成SQLNode节点，该节点的子节点为2，无父兄节点。该流程为依赖于子节点的查询结果，直接在CN层进行查询并返回结果。
语句	SELECT CASE WHEN a='1' THEN b*2 WHEN a='3' THEN b*3 ELSE b*5 END FROM poctest.a1;
预置条件	create table if not exists poctest.a1 (a int, sname char (10) not null, b int primary key, c int, index id (a), unique index bp (c)) distributed by hash (a)(g1,g2,g3); insert into poctest.a1 (a, sname, b, c) values (1, 'a', 102, 104), (2, 'b',2, 3), (3, 'c', 106, 2), (4, 'd', 3, 3); explain select case when a='1' then b*2 when a='3' then b*3 else b*5 end from poctest.a1;

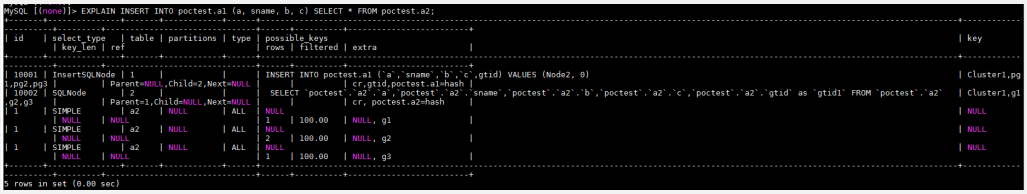
explain结果	
explain说明	执行流程1：id为10001，CN层面流程，该流程生成SQLNode节点。语句将下发至所有节点。 执行流程2：id为1，DN层面流程，各分片分别执行select_type为SIMPLE的查询，查询形式为遍历全表的逐行查询。
语句	SELECT poctest.a1.b,poctest.a2.c,poctest.a1.c FROM poctest.a1 JOIN poctest.a2 ON poctest.a1.a=poctest.a2.a;
预置条件	create table if not exists poctest.a1 (a int, sname char (10) not null, b int primary key, c int, index id (a), unique index bp (c)) distributed by hash (a)(g2,g3); insert into poctest.a1 (a, sname, b, c) values (1, ‘a’, 102, 104), (2, ‘b’,2, 5), (3, ‘c’, 106, 2), (4, ‘d’, 3, 3); create table if not exists poctest.a2 (a int, sname char (10) not null primary key, b int, c int null, index ad (a), unique index ce (c)) distributed by hash (a)(g1,g3); insert into poctest.a2 (a, sname, b, c) values (8, ‘a’, 102, 104), (2, ‘b’,9, 7), (9, ‘c’, 106, 2), (4, ‘d’, 10, 3); explain select poctest.a1.b,poctest.a2.c,poctest.a1.c from poctest.a1 join poctest.a2 on poctest.a1.a=poctest.a2.a;

explain结果	
explain说明	执行流程1：CN层面执行id为10002的两个并行流程，该流程下同时生成一个SQLNode节点分别标记为2、3，ref表明该流程有一个标记为1的父节点和一个兄弟节点。extra显示该流程执行过程中对语句进行了排序处理，下发的语句中可以发现查询的字段将按照‘a’升序进行排序。 执行流程2：id为1为DN层面流程，各分片执行select_type为simple的查询，通过查询所有行返回结果。 执行流程3：id为10001，CN层面流程。该流程生成了一个JoinNode节点，possible_key详细说明了执行计划，包括JOIN类型、JOIN条件以及各计划节点涉及的表、排序键、所需查询的字段等。
语句	DELETE FROM poctest.a1 WHERE a < 3 ORDER BY poctest.a1.c LIMIT 3;
预置条件	create table if not exists poctest.a1 (a int, sname char (10) not null, b int primary key, c int, index id (a), unique index bp (c)) distributed by hash (a)(g1,g2,g3); insert into poctest.a1 (a, sname, b, c) values (1, ‘a’, 102, 104), (2, ‘b’,2, 5), (3, ‘c’, 106, 2), (4, ‘d’, 3, 3); explain delete from poctest.a1 where a < 3 order by poctest.a1.c limit 3;

explain结果	
explain说明	执行流程1：id为10002，CN层面流程，生成SQLNode节点标记为2。有一个标记为1的父节点。extra表明该流程为SELECT FOR UPDATE，且使用排序（poctest.a1.c升序排列）。 执行流程2：id为1，DN层流程；执行select_type为simple的查询；查询为使用索引的查询；各分片可能的索引键为‘id’，实际索引为‘bp’；g1分片的查询筛选率为50%，即表示储存于g1分片的表，符合条件范围的行数仅占总行数的50%，只需对这部分进行查询；extra表示查询语句所下发的分片及使用了WHERE条件。 执行流程3：id为10001，CN层面的DELETE流程。该流程生成了一个DeleteNode节点，其子节点为SQLNode。
语句	UPDATE poctest.a1 SET poctest.a1.b = poctest.a1.b+1 WHERE poctest.a1.a > 2;
预置条件	create table if not exists poctest.a1 (a int, sname char (10) not null, b int primary key, c int, index id (a), unique index bp (c)) distributed by hash (a)(g1,g2,g3); insert into poctest.a1 (a, sname, b, c) values (1, ‘a’, 102, 104), (2, ‘b’,2, 5), (3, ‘c’, 106, 2), (4, ‘d’, 3, 3); explain update poctest.a1 set poctest.a1.b = poctest.a1.b+1 where poctest.a1.a>2;
explain结果	

explain说明	执行流程1：id为10002，CN层面流程，生成SQLNode节点标记为2。父节点为UpdateNode标记1。extra表明该流程为SELECT FOR UPDATE。 执行流程2：id为1，DN层面流程；执行select_type为SIMPLE的查询；查询形式为范围查询；各分片可能及实际使用的索引键均为'id'。执行语句为[SELECT poctest.a1.b, poctest.a1.a, poctest.a1.gtid as gtid1 FROM poctest.a1 WHERE poctest.a1.a > 2;] 执行流程3：id为10001，CN层面的UPDATE流程。该流程生成了一个UpdateNode节点，子节点为SQLNode。执行[UPDATE poctest.a1 SET poctest.a1.GTID=0, poctest.a1.b = (poctest.a1.b + 1) WHERE b = Node2],其中b=Node2为依赖标记为2子节点返回的结果。
语句	UPDATE poctest.a1 LEFT JOIN poctest.a2 ON poctest.a1.a=poctest.a2.a SET poctest.a1.b=poctest.a1.b+1;
预置条件	create table if not exists poctest.a1 (a int, sname char (10) not null, b int primary key, c int, index id (a), unique index bp (c)) distributed by hash (a)(g1,g2,g3); insert into poctest.a1 (a, sname, b, c) values (1, 'a', 102, 104), (2, 'b',2, 5), (3, 'c', 106, 2), (4, 'd', 3, 3); create table if not exists poctest.a2 (a int, sname char (10) not null primary key, b int, c int null, index ad (a), unique index ce (c)) distributed by hash (a)(g1,g2,g3); insert into poctest.a2 (a, sname, b, c) values (8, 'a', 102, 104), (2, 'b',9, 7), (9, 'c', 106, 2), (4, 'd', 10, 3); explain update poctest.a1 left join poctest.a2 on poctest.a1.a=poctest.a2.a set poctest.a1.b=poctest.a1.b+1;
explain结果	

explain说明	执行流程1：id为10002，CN层面流程，生成SQLNode节点标记为2。父节点为UpdateNode标记1。extra表明该流程为SELECT FOR UPDATE。 执行流程2：id为1，DN层面流程；分别对a1、a2表执行select_type为SIMPLE的查询；查询形式为范围查询；g3分片上对a2表采用非唯一索引扫描通过索引‘ad’即字段a（ref：poctest.a1.a显示）寻找匹配值的行，其余各分片对两表均通过逐行查询返回查询结果。extra显示g1、g2分片a2表采用了条件查询同时用到了JOIN缓冲存储区，存储条件查询的结果。 执行流程3：id为10001，CN层面的UPDATE流程。该流程生成了一个UpdateNode节点，子节点为SQLNode。执行[UPDATE poctest.a1 SET a1.GTID = 0, poctest.a1.b = (poctest.a1.b + 1) WHERE b = Node2],其中b = Node2为依赖标记为2子节点返回的结果。
语句	INSERT INTO poctest.a1 (a, sname, b, c) VALUES (8, ‘g’, 10, 14);
预置条件	create table if not exists poctest.a1 (a int, sname char (10) not null, b int primary key, c int, index id (a), unique index bp (c)) distributed by hash (a)(g1,g2,g3); insert into poctest.a1 (a, sname, b, c) values (1, ‘a’, 102, 104), (2, ‘b’,2, 5), (3, ‘c’, 106, 2), (4, ‘d’, 3, 3); explain insert into poctest.a1 (a, sname, b, c) values (8, ‘g’, 10, 14);
explain结果	
explain说明	执行流程1：id为10001，CN层面的INSERT流程。生成InsertSQLNode节点，根据哈希算法其插入值落于g3分片，因此key显示只需在g3分片执行INSERT流程。 执行流程2：id为1，DN层面流程；执行select_type显示为INSERT流程；插入形式为全部插入。
语句	INSERT INTO poctest.a1 (a, sname, b, c) SELECT * FROM poctest.a2;

预置条件	create table if not exists poctest.a1 (a int, sname char (10) not null, b int primary key, c int, index id (a), unique index bp (c)) distributed by hash (a)(g1,g2,g3); insert into poctest.a1 (a, sname, b, c) values (1, 'a', 102, 104), (2, 'b',2, 3), (3, 'c', 106, 2), (4, 'd', 3, 3); create table if not exists poctest.a2 (a int, sname char (10) not null primary key, b int, c int null, index ad (a), unique index ce (c)) distributed by hash (a)(g1,g2,g3); insert into poctest.a2 (a, sname, b, c) values (8, 'a', 102, 104), (2, 'b',9, 3), (9, 'c', 106, 2), (4, 'd', 10, 3); explain insert into poctest.a1 (a, sname, b, c) select * from poctest.a2;
explain结果	
explain说明	执行流程1：id为10002，CN层面流程，生成SQLNode节点标记为2。父节点为InsertSQLNode标记1。 执行流程2：id为1，DN层面流程；执行select_type为SIMPLE的查询；查询形式为遍历全表的逐行查询。执行语句为[SELECT poctest.a2.a, poctest.a2.sname, poctest.a2.b, poctest.a2.c, poctest.a2.gtid as gtid1 FROM poctest.a2;] 执行流程3：id为10001，CN层面的INSERT流程。生成InsertSQLNode节点，执行语句为[INSERT INTO poctest.a1 (a, sname, b, c, gtid) VALUES (Node2, 0);],其中Node2位依赖子节点的查询结果。

1.9.4. 数据导入导出规范

支持分布式数据库批量数据导入；应用场景如：异构数据库之间的数据迁移，例如将 `Oracle` 数据库数据导出为数据文件，可直接使用该工具导入到 GoldenDB 数据库中。

支持分布式数据库批量数据导出；应用场景除了异构数据库之间的数据迁移，同时应用为数据逻辑备份，应用数据分析、统计、处理等。

1.9.4.1. 导入导出环境检测

系统环境检测

1. 使用 LoadServer 数据导入导出功能，需保证服务器数据目录所在磁盘空间足够大，建议

至少保证磁盘剩余空间是数据文件大小的两倍以上。

2. 使用之前需要确认 LoadServer 所在服务器 FTP 服务开启，使用如下命令检测：

```
[manager@dbmgr02 ~]$ service vsftpd status

Redirecting to /bin/systemctl status vsftpd.service

● vsftpd.service - Vsftpd ftp daemon

Loaded: loaded (/usr/lib/systemd/system/vsftpd.service; disabled; vendor preset: disabled)

Active: active (running) since 一 2020-09-14 17:29:49 CST; 3 days ago

Process: 28092 ExecStart=/usr/sbin/vsftpd /etc/vsftpd/vsftpd.conf (code=exited, status=0/SUCCES
S)

Main PID: 28093 (vsftpd)

CGroup: /system.slice/vsftpd.service

└─28093 /usr/sbin/vsftpd /etc/vsftpd/vsftpd.conf

9月 14 17:29:49 local164 systemd[1]: Starting Vsftpd ftp daemon...

9月 14 17:29:49 local164 systemd[1]: Started Vsftpd ftp daemon.
```

命令结果显示 FTP 服务处于开启状态。

运行环境检测

LoadServer 执行导入导出任务时，需要与 MetadataServer，ClusterManager 模块进行通信，必须保证以上模块运行正常。可在以上模块安装用户下执行如下命令检测模块运行状态。该命令显示，以上三个模块运行正常。

```
[manager@dbmgr02 ~]$ dbstate

[metadataserver]The metadataserver process is running, PID:6263

[proxymanager]The proxymanager process is running, PID:6384

[clustermanager]The clustermanager process is running, PID:6589

[ommagent]The ommagent process is running, PID:6881
```

在 MetadataServer 安装用户执行如下命令检测 LoadServer 与其通信状态。如下执行命令显示 MetadataServer 与 LoadServer 通信正常。

```
[manager@dbmgr02 zkjtest]$ dbtool -mds -linkstate

Send message to module[MetaDataServer,10.47.182.248:6406] localport[5600] pid[1583] successful
ly!

The response message: RSP Code[0].{0:success; 1:provisional response; other: fail.}

[12-31 16:21:47:270]Successful response:

--------------------------------------------------------------------------------

link state|Local client logicport |Server listening EntityName USER DomainName

--------------------------------------------------------------------------------

--------------------------------------------------------------------------------

as clientnum=0

--------------------------------------------------------------------------------

link state|Local listening |client logicport EntityName USER

--------------------------------------------------------------------------------

LINKON | 10.47.182.248:6406 | 10.47.182.248:14824 6001 ProxyManager zxmanager

LINKON | 10.47.182.248:6406 | 10.47.182.248:16828 6011 ClusterManager zxmanager

LINKON | 10.47.182.248:6406 | 10.47.182.243:36278 6231 LoadServer zxload

LINKON | 10.47.182.248:6406 | 10.47.182.244:18925 6021 GTM zxgtm

LINKON | 127.0.0.1:6406 | 127.0.0.1:6473 6471 OMMAgent zxmanager

LINKON | 10.47.182.248:6406 | 10.47.182.248:5600 5600 ComTool zxmanager

--------------------------------------------------------------------------------

LocalEntityName MetaDataServer

LocalAddr [10.47.182.248:6406]

LocalAddr [127.0.0.1:6406]

success
```

在 ClusterManager 安装用户执行如下命令检测 LoadServer 与其通信状态。如下执行命令显示 ClusterManager 与 LoadServer 通信正常（绿色文字显示）。

```
[manager@dbmgr02 zkjtest]$ dbtool -cm -linkstate

Send message to module[ClusterManager,10.47.182.248:6016] localport[5600] pid[11744] successfully!

The response message: RSP Code[0].{0:success; 1:provisional response; other: fail.}

[12-31 16:22:54:300]Successful response:

--------------------------------------------------------------------------------

link state|Local client logicport |Server listening EntityName USER DomainName

--------------------------------------------------------------------------------

LINKON | 10.47.182.248:16828 6011 | 10.47.182.248:6406 MetaDataServer zxmanager

LINKON | 10.47.182.248:18542 6011 | 10.47.182.248:6006 ProxyManager zxmanager

LINKON | 10.47.182.248:24686 6011 | 10.47.182.245:6026 GTM zxgtm

--------------------------------------------------------------------------------

as clientnum=3

--------------------------------------------------------------------------------

link state|Local listening |client logicport EntityName USER

--------------------------------------------------------------------------------

LINKON | 10.47.182.248:6016 | 10.47.182.243:33970 6231 LoadServer zxload

LINKON | 10.47.182.248:6016 | 10.47.182.245:28758 6101 DBAgent zxdb1

LINKON | 127.0.0.1:6016 | 127.0.0.1:6473 6471 OMMAgent zxmanager

LINKON | 10.47.182.248:6016 | 10.47.182.248:5600 5600 ComTool zxmanager

--------------------------------------------------------------------------------

LocalEntityName ClusterManager

LocalAddr [10.47.182.248:6016]

LocalAddr [127.0.0.1:6016]

success
```

数据目录介绍

1. 该数据目录一般位于用户主目录下的 loaddata （具体依赖配置文件

`loadserver.ini`），主要用来存放 LoadServer 工作工程中产生的数据文件。

2. 主要结构目录如下：

backup 目录：导入任务结束，导入数据文件保存目录，是否保存文件依赖配置文件选项。

in 目录：导入数据文件存放目录，该目录中子目录名称为集群号，第一次使用需创建，并且需要将数据文件移到至该目录中。

out 目录：导出数据文件存放目录。

errordata 目录：导入失败中间数据文件存放目录。

split/merge 目录：导入/导出临时数据文件存放目录。

data 目录：存放导入/导出创建的数据库的默认字符集编码和字符集排序规则。

insql 目录：存放导入转 SQL 功能的 SQL 语句目录。

1.9.4.2. 使用命令介绍

命令选项介绍

数据导入导出命令由 LoadServer 服务器所在用户的 dbtool 客户端发起。命令介绍如下：

```
dbtool -loadserver [OPTION]
```

其中OPTION：

选项名称	dbtool -h content	注释
-l[ink]s[tate]	show ModuleName link info	查看模块链路信息
-l[oad-]c[onfig]	dynamic reload config	动态生效配置参数
-V	print middle working message of LoadServer	显示详细步骤
-type	in/out	命令类型：in-导入；out-导出
-load-retry	load retry	导入重试

-convert-to-insert	convert load in failure file to SQL file	导入失败文件转sql文件
-skip-ok-check	it doesn't work, just for keep the interface compatible	不起实际作用，仅保持业务接口兼容
-clusterid	cluster id	导入导出功能所在集群号
-errorignore	the max error data numbers we can ignore when import data	导入过程容忍最大错误数据条数，超过该值，导入任务失败
-backup	backup loadin datafile to the backup directory	备份导入文件到备份目录
-errordata	merge error data and failure loadin data to a file(absolute path)	导入过程中失败数据、错误数据存放文件
-endstate	Loadout : finally state when export a table, value: -1: use the value in the config file, 0: uploaded and merge, 1: uploaded, 2: saved on DN Loadin:	结束阶段标志： 导出： 0-导出文件在out目录，并合并为一个文件 1-导出文件在out目录，不合并 2-导出文件存放在DBAgent端 导入： 不填时默认导入全流程 split-导入在拆分阶段后停止
-splitpath		导入在拆分阶段停止时指定拆分文件路径
-sql	SQL Command	SQL命令，具体参考2.2.1小节。

-host	host ip(or domain name) and port	LoadServer服务器所在IP(域名)及端口 例：192.168.1.16:6036 dns.manager.com:6036
-loadin-support-null	whether loadserver support “,,” as NULL value; 0:not support 1: support	是否支持0长度字符串为NULL，0为不支持，1为支持
-set-unique-column=	assign the unique column when do a loadin job, attention: when this option occurs, you should list all the loadin columns in the -sql option except this one;	导入时指定文件内唯一列，注意：当使用该option时，需要列出除指定文件内唯一列之外的所有导入列
-user=	load data check authentication	集群鉴权用户名
-password=	load data check authentication	集群鉴权密码
-q[uery] -w[ork]l[oad]	query loadserver’s workload	查询导入导出工作负载情况
-tablename	load retry table name or table name for convert to SQL file	重试命令中指定的表名 或者导入失败文件转sql文件命令中指定的表名
-importdate=	load retry date[YYYY-MM-DD] or date[YYYY-MM-DD] for convert to SQL file	重试命令指定重试数据日期 或者导入失败文件转sql文件命令中指定的数据日期
-importdir=	load in fail directory for convert to SQL file	导入失败文件转sql文件命令中指定的失败文件目录
-exportdir=	SQL file directory	导入失败文件转sql文件命令中指定的sql文件目录

-batchcommit=	how many lines in a batch do we commit when make a ConvertSQL operation;0:all SQL in a batch, default:5000	导入失败文件转sql文件时一次提交行数，0:一次提交所有行，默认5000行
-linkoffrebuild	link can be rebuilded with this parameter if linkoff happened	Loadserver与Comtool断链重连参数
-filenum=	specify the number of loadout files	Endstate=3时，指定导出文件数目
-fileline=	specify the number of lines per loadout file	Endstate=3时，指定导出文件最大行数

批量脚本命令选项介绍

批量导入导出命令由 LoadServer 服务器所在用户的batchload.py脚本发起。命令介绍如下：

Usage:切换到 loadserver 所在用户下，`python bin/batchload.py \[OPTION]`

其中OPTION：

选项名称	Python batchload.py –help	注释
-h, –help	show this help message and exit	帮助信息
-u USER, –user USER	Username to use when connecting to CN.	CN用户名
-p PASSWORD, –password PASSWORD	Password to use when connecting to CN.	CN用户密码
-H HOST, –host HOST	Host ip to use when Connecting to CN.	CN 用户IP连接地址
-P PORT, –port PORT	Port number to use when connecting to CN (default: 0).	CN 连接端口
–databases [DATABASES [DATABASES ...]]	Databases list to load.	批量导入导出库名列表

–do-tables [DO_TABLES [DO_TABLES ...]]	Tables list to load.	批量导入导出指定表名列表
–ignore-tables [IGNORE_TABLES [IGNORE_TABLES ...]]	Tables list not to load.	批量导入导出忽略表名列表
–clusterid [CLUSTERID]	Cluster ID (default: 0).	集群ID
–clustermode [CLUSTERMODE]	Cluster mode (default: multiple).	集群模式，single or multiple
–type [TYPE]	Batch load type, value is out or in.	批量导数类型，导出 or 导入
–parallel [PARALLEL]	Parallel degree, range in [1,20] (default: 1).	批量导出并发度
–fields-terminated-by [FIELDS_TERMINATED_BY]	Fields in the output file are terminated by the given string (default: ,).	列分隔符
–fields-enclosed-by [FIELDS_ENCLOSED_BY]	Fields in the output file are enclosed by the given character.	列包裹符
–fields-optionally-enclosed-by [FIELDS_OPTIONALLY_ENCLOSED_BY]	Fields in the output file are optionally enclosed by the given character.	可选列包裹符
–fields-escaped-by [FIELDS_ESCAPED_BY]	Fields in the output file are escaped by the given character.	转义字符
–lines-terminated-by [LINES_TERMINATED_BY]	Lines in the output file are terminated by the given string (default:).	行分隔符
–authuser [AUTHUSER]	Authority username for loadserver.	集群鉴权用户名
–authpasswd [AUTHPASSWD]	Authority password for loadserver.	集群鉴权密码
–targetdir [TARGETDIR]	Target directory.	导入导出根目录

–suffix [SUFFIX]	Suffix name for load file.	导入导出文件后缀名
–encrypt-key [ENCRYPT_KEY]	Encrypt key for zip file.	Zip压缩加密解密密码
–charset [CHARSET]	Character set for loadserver.	导入导出字符集类型
–endstate [ENDSTATE]	Loadout file final state, 0: uploaded and merge, 1: uploaded, 2: saved on DN, 3: specify the number of loadout files or the number of lines per loadout file.	结束阶段标志： 导出： 0-导出文件在out目录，并合并为一个文件 1-导出文件在out目录，不合并 2-导出文件存放在DBAgent端 导入： 不填时默认导入全流程
–filenum [FILENUM]	Split File number, use it with para endstate (default: 0).	endstate=3时，指定导出文件数目
–fileline [FILELINE]	File Lines per loadout file, use it with para endstate (default: 0).	endstate=3时，指定导出文件最大行数
–errorignore [ERRORIGNORE]	Max error data numbers we can ignore when import data (default: 0).	导入最大忽视错误的行数
–retry	Retry batch loadout (default: False).	导出重试
-C, –compress	Compress file with zip for loadout file (default: False).	是否支持导入导出压缩

–backup	Backup loadin file (default: False).	是否备份导入文件
–no-data	Not load data (default: False).	是否导入导出数据文件
–no-create-info	Not load create info (default: False).	是否导入导出DDL文件
–loadin-support-null	loadserver support “,,” as NULL value (default:False).	是否支持0长度字符串为NULL，0为不支持，1为支持

SQL 语法介绍

1. 导入命令中 SQL 语法如下：

```
LOAD DATA [LOW_PRIORITY | CONCURRENT] [LOCAL] INFILE 'file_name'
  [REPLACE | IGNORE]
  INTO TABLE tbl_name
  [PARTITION (partition_name,...)]
  [CHARACTER SET charset_name]
  [{FIELDS | COLUMNS}
    [TERMINATED BY 'string']
    [[OPTIONALLY] ENCLOSED BY 'char']
    [ESCAPED BY 'char']
  ]
  [LINES
    [STARTING BY 'string']
    [TERMINATED BY 'string']
  ]
  [IGNORE number {LINES | ROWS}]
  [(col_name_or_user_var,...)]
  [SET col_name = expr,...]
  [WHEN when_clause]
  [TRAILING NULLCOLS]
```

参数解释：

参数	描述
LOW_PRIORITY	使用该选项，数据库等待导入涉及表没有操作时才进行导入，该选项仅对使用表锁存储引擎有效（MYISAM、MERGE、MEMORY等）。

CONCURRENT	当表满足并发插入数据时，Load data 语句可与INSERT语句并行执行。
LOCAL	使用该选项指明数据库读取导入数据文件。若未指定，则默认数据文件在数据库所在服务器。
file_name	导入数据文件名
REPLACE	当数据导入存在主键冲突或唯一索引冲突，替换表中冲突数据。
IGNORE	当数据导入存在主键冲突或唯一索引冲突，忽略该行导入数据。
tbl_name	数据要导入的库表。
PARTITION	表所在分区。
FIELDS [TERMINATED BY '*string*']	行数据列值以 '*string*' 结尾分割。
FIELDS [[OPTIONALLY] ENCLOSED BY '*char*']	指定OPTIONALLY，非数据列值使用 '*char*' 字符引用，不指定则列数据值全部被 '*char*' 引用。
FIELDS [ESCAPED BY '*char*']	指定数据转义字符 '*char*' 。
LINES [STARTING BY '*string*']	指定行数据以 '*string*' 开始。
LINES [TERMINATED BY '*string*']	指定行数据以 '*string*' 结尾。（注意：LoadServer 限制该 '*string*' 长度小于等于64个字节）。
IGNORE	忽略数据文件开始行数或列数目。
[(*col_name_or_user_var,...*)]	指定列导入，若不指定则默认指定全部列导入。（LoadServer数据导入必须指定列）。
[SET *col_name = expr,...*]	SET语句，用于对数据列值进行需要的转换操作。

WHEN	条件导入语法，支持表达式运算符：+ 、- 、* 、/ 、=、>=、=<、> < 、<>、SUBSTR()、CONCAT()、IN、BETWEEN、LIKE函数。
[TRAILING NULLCOLS]	数据文件缺少的最后面的列，导入时用NULL补充。

2. 导出命令中 SQL 语法如下：

```
SELECT (col_name[,...]) FROM tbl_name INTO  OUTFILE 'file_name'
  [WHERE where_clause]
  [{FIELDS | COLUMNS}
    [TERMINATED BY 'string']
    [[OPTIONALLY] ENCLOSED BY 'char']
    [ESCAPED BY 'char']
  ]
  [LINES
    [STARTING BY 'string']
    [TERMINATED BY 'string']
  ]
```

3. 参数解释：

参数	描述
(col_name[,...])	指定数据列导出，部分或全部，不支持*代表全部列。
tbl_name	导出数据的库表。
file_name	导出数据存放文件，存放在out目录。
WHERE	WHERE字句与标准SQL语句相同。
FIELDS、LINES	使用与导入语句相同。

1.9.4.3. 数据导入示例

数据导入全部字段

1. 使用 `dbtool` 客户端发起导入数据命令，导入表全字段如下：

数据导入命令示例：

```
dbtool  -loadserver  -type="in"  -clusterid=6 -sql="load data infi
```

```
le  'loadout.data' into table load_manual.
t0101_in fields terminated by ',' optionally enclosed by '\"' line
s terminated by '\n' ;"   -user="userA" -password="passA"
```

2. 导入数据文件如下示：数据文件格式为：列分隔符‘，’，行结束符‘\n’，字符串包含符‘“”’。

```
17,"fanchuan",2016,6,2,217
15,"haokai",2013,5,3,216
19,"haokai",2018,5,1,318
4,"jinjiang",1998,2,1,218
```

3. 数据导入表数据如下：

SELECT * FROM load_manual.t0101_in;

id	hotelname	year	month	day	rent
17	fanchuan	2016	6	2	217
15	haokai	2013	5	3	216
19	haokai	2018	5	1	318
4	jinjiang	1998	2	1	218

4. 数据导入成功客户端返回命令如下：

```
The response message: RSP Code[0].{0:success; 1:provisional respon
se; other: fail.}
Successful response:
[05-12 13:03:32:95 ]  command to  FILE_PROCESS_END
 Load data code:0    Load data message:deal with the datafile succ
ess!
 file split result :success
split file num:1     success file num:1 failure file num:0

~success~
```

5. 限制：

- 导入数据文件每行必须包含命令指定行结束符，若最后一行不包含行结束符，则之前数据全部导入成功，但最行一行数据丢失。
- 导入数据中含有 blob(test) 字段，则导入命令有所变化（形式上与导入部分字段类似）：

```
dbtool  -loadserver -V -type="in"   -clusterid=1   -sql="load data
 infile 'loadout.data' into table
 load_manual.t0101_in fields terminated by ',' optionally enclosed
 by '\"' lines terminated by '\n' (id,hotelname,year ,month ,day ,
 rent,@b) set test=unhex(@b);"   -user="userA" -password="passA"
```

数据导入部分字段

1. 使用 `dbtool` 客户端发起导入数据命令，导入表部分字段如下：

数据导入命令示例：

```
dbtool  -loadserver  -type="in"  -clusterid=6  -skip-ok-check  -sq
l="load data infile  'loadout.data' into
 table load_manual.t0101_in fields terminated by ',' optionally en
closed by '\"' lines terminated by '\n' ( id,hotelname,year) ;"
 -user="userA"  -password="passA"
```

2. 导入数据文件如下示：数据文件格式为：列分隔符‘，’，行结束符 > ‘\n’，字符串包含符‘“”’。

```
17,"fanchuan",2016,6,2,217
15,"haokai",2013,5,3,216
19,"haokai",2018,5,1,318
4,"jinjiang",1998,2,1,218
```

3. 数据导入表数据如下：

mysql> SELECT * FROM load_manual.t0101_in;

id	hotelname	year	month	day	rent
17	fanchuan	2016	NULL	NULL	NULL
15	haokai	2013	NULL	NULL	NULL
19	haokai	2018	NULL	NULL	NULL
4	jinjiang	1998	NULL	NULL	NULL

4. 数据导入成功客户端返回命令如下：

```
The response message: RSP Code[0].{0:success; 1:provisional respon
```

```
se;
other: fail.}
Successful response:
[05-12 13:03:32:95 ] command to FILE_PROCESS_END
Load data code:0 Load data message:deal with the datafile
success!
file split result :success
split file num:1 success file num:1 failure file num:0
~success~
```

数据导入 SET 子句

1. 使用 `dbtool` 客户端发起导入数据命令，导入表数据使用 `SET` 子句如下：

数据导入命令示例：

```
dbtool -loadserver -type="in" -clusterid=6 -skip-ok-check -s
ql="load data infile 'loadout.data' into
 table load_manual.t0101_in fields terminated by ',' optionally en
closed by '\"' lines terminated by '\n'( id,@var,year) set hotelna
me=concat(@var, '|') ;" -user="userA" -password="passA"
```

2. 导入数据文件如下示：数据文件格式为：列分隔符 ‘，’，行结束符 ‘\n’，字符串包含符’”“’。

```
17,"fanchuan",2016,6,2,217
15,"haokai",2013,5,3,216
19,"haokai",2018,5,1,318
4,"jinjiang",1998,2,1,218
```

3. 数据导入表数据如下：

mysql> SELECT * FROM load_manual.t0101_in;

id	hotelname	year	month	day	rent
17	fanchuan		2016	NULL	NULL
15	haokai		2013	NULL	NULL
19	haokai		2018	NULL	NULL
4	jinjiang		1998	NULL	NULL

4. 数据导入成功客户端返回命令如下：

```
The response message: RSP Code[0].{0:success; 1:provisional respon
se;
other: fail.}
Successful response:
[05-12 13:03:32:95 ] command to FILE_PROCESS_END
Load data code:0 Load data message:deal with the datafile
success!
file split result :success
split file num:1 success file num:1 failure file num:0
~success~
```

5.9.4.3.4. 数据导入 WHEN 子句

1. 使用 `dbtool` 客户端发起导入数据命令，导入表数据使用 `WHEN` 子句如下：

数据导入命令示例：

```
dbtool -loadserver -type="in" -clusterid=6 -skip-ok-check -sql="lo
ad data infile 'loadout.data' into table
  load_manual.t0101_in fields terminated by ',' optionally enclose
d by '"' lines terminated by 'n' ( id,
  hotelname,year) when year>2013 ;" -user="userA" -password="passA
"
```

2. 导入数据文件如下示：数据文件格式为：列分隔符 ‘，’，行结束符 ‘\n’，字符串包含符‘“”’。

```
17,"fanchuan",2016,6,2,217
15,"haokai",2013,5,3,216
19,"haokai",2018,5,1,318
4,"jinjiang",1998,2,1,218
```

3. 数据导入表数据如下：

mysql> SELECT * FROM load_manual.t0101_in;

id	hotelname	year	month	day	rent
17	fanchuan	2016	NULL	NULL	NULL
19	haokai	2018	NULL	NULL	NULL

4. 数据导入成功客户端返回命令如下：

```
The response message: RSP Code[0].{0:success; 1:provisional respon
se;
```

```
other: fail.}
Successful response:
[05-12 13:03:32:95 ] command to FILE_PROCESS_END
Load data code:0 Load data message:deal with the datafile
success!
file split result :success
split file num:1 success file num:1 failure file num:0
~success~
```

1.9.4.4. 数据导出示例

数据导出全部字段

1. 使用 `dbtool` 客户端发起导出数据命令：

```
dbtool -loadserver -type="out" -clusterid=2  -sql="select id,hotel
name,year ,month ,day , rent from
  load_manual.normal into outfile 'normal.data' fields terminated
by ',' lines terminated by '\n';"  -user="userA"  -password="passA
"
```

2. 表数据如下：

mysql> SELECT * FROM load_manual.t0101_in;

id	hotelname	year	month	day	rent
17	fanchuan	2016	6	2	217
15	haokai	2013	5	3	216
19	haokai	2018	5	1	318
4	jinjiang	1998	2	1	218

3. 导出数据文件如下示：数据文件格式为：列分隔符 ‘，’，行结束符 ‘\n’，字符串包含符‘”’。

```
17,"fanchuan",2016,6,2,217
15,"haokai",2013,5,3,216
19,"haokai",2018,5,1,318
4,"jinjiang",1998,2,1,218
```

4. 数据导出成功客户端返回命令如下：

```
The response message: RSP Code[0].{0:success; 1:provisional respon
se;
other: fail.}
Successful response:
[05-12 13:03:31:52 ] command to FILE-PROCESSING-END
Load data code:0 Load data message:file processing success
File Export Location:
10.47.169.195 /mnt/raid/loaddata/out/6/loadout.data
~success~
```

数据导出部分字段

1. 使用 `dbtool` 客户端发起导出数据命令：

```
dbtool -loadserver -type="out"  -clusterid=2  -sql="select id, hot
elname,year from load_manual.normal into
  outfile 'normal.data' fields terminated by ',' lines terminated
by '\n';"     -user="userA"  -password="passA"
```

2. 表数据如下：

mysql> SELECT * FROM load_manual.t0101_in;

id	hotelname	year	month	day	rent
17	fanchuan	2016	6	2	217
15	haokai	2013	5	3	216
19	haokai	2018	5	1	318
4	jinjiang	1998	2	1	218

3. 导出数据文件如下示：数据文件格式为：列分隔符 ‘，’，行结束符 ‘\n’，字符串包含符‘“”’。

```
17,"fanchuan",2016
15,"haokai",2013
19,"haokai",2018
4,"jinjiang",1998
```

4. 数据导出成功客户端返回命令如下：

```
The response message: RSP Code[0].{0:success; 1:provisional respon
```

```
se;
other: fail.}
Successful response:
[05-12 13:03:31:52 ] command to FILE-PROCESSING-END
Load data code:0 Load data message:file processing success
File Export Location:
10.47.169.195 /mnt/raid/loaddata/out/6/loadout.data
~success~
```

数据导出 WHERE 字句

1. 使用 `dbtool` 客户端发起导出数据命令：

```
dbtool -loadserver -type="out"  -clusterid=2  -sql="select  id,hot
elname,year ,month ,day , rent from load_manual.normal where id<10
 into outfile 'normal.data'  fields terminated by ',' lines termin
ated by '\n';"   -user="userA"  -password="passA"
```

2. 表数据如下：

mysql> SELECT * FROM load_manual.t0101_in;

id	hotelname	year	month	day	rent
17	fanchuan	2016	6	2	217
15	haokai	2013	5	3	216
19	haokai	2018	5	1	318
4	jinjiang	1998	2	1	218

3. 导出数据文件如下示：数据文件格式为：列分隔符 ‘，’，行结束符 ‘\n’，字符串包含符‘“”’。

```
4,"jinjiang",1998,2,1,218
```

4. 数据导出成功客户端返回命令如下：

```
The response message: RSP Code[0].{0:success; 1:provisional respon
se;
other: fail.}
Successful response:
[05-12 13:03:31:52 ] command to FILE-PROCESSING-END
Load data code:0 Load data message:file processing success
```

```
File Export Location:
10.47.169.195 /mnt/raid/loaddata/out/6/loadout.data
~success~
```

1.9.4.5. 导入失败数据重试介绍

导入失败数据重试

当客户端处于数据导入过程中，发生异常导致数据导入中断或失败时，在 loadserver 服务端失败文件目录或导入临时文件目录存放本次导入失败的文件，同时 `dbtool` 客户端打印本次数据文件成功拆分的原始文件行数。用户可以选择将已经拆分成功但导入失败的文件重新导入到数据库中，导入重试命令会将失败目录下和分裂文件临时目录中，该库表的所有失败文件查询并导入。

1. 导入失败数据重试命令：

```
dbtool -loadserver -load-retry -clusterid=6 -tablename=load_manual
.normal -user="userA" -password="passA"
```

2. loadserver服务器查询到失败文件所在目录如下：

```
Intermediate response:
Find the sql file;
FilePath:[/mnt/raid/loaddata/errordata/82017051214255543],
FileName:[/mnt/raid/loaddata/errordata/82017051214255543/loadout.d
ata.sql]
Intermediate response:
[05-12 14:49:20:85 ] event: LOAD_READ_FAILED_FILE
result code:0 result message:read failed file success
```

3. 最终数据导入成功客户端返回如下：

```
The response message: RSP Code[0].{0:success; 1:provisional respon
se;
other: fail.}
Successful response:
[05-12 14:49:31:18 ]event: LOAD_DELETE_FILE_END
result code:0
[FileTotalCount]:1
[FileSuccessCount]: 1
[FileFailureCount]: 0
result message:delete file end
~success~
```

导入失败数据指定日期重试

1. 导入失败数据重试命令：

```
dbtool -loadserver -load-retry  -clusterid=6 -importdate='2017-07-
07' -tablename= load_manual.normal
   -user="userA"  -password="passA"
```

2. loadserver 服务器查询到失败文件所在目录如下：

```
Intermediate response:
Find the sql file;
FilePath:[/home/dbagentd/loaddata/split/2017-07-07/load_test_manua
l.normal/20170707190611266.3],
FileName:[/home/dbagentd/loaddata/split/2017-0
7-07/load_manual.normal/20170707190611266.3/normal.data.sql]Interm
ediate
response:
Find the sql file;
FilePath:[/home/dbagentd/loaddata/errordata/2017-07-07/load_test_m
anual.normal/20170707190611266.3],
FileName:[/home/dbagentd/loaddata/errordat
a/2017-07-07/load_manual.normal/20170707190611266.3/normal.data.sq
l]Intermediate
response:
[08-15 19:43:30:72 ] event: LOAD_READ_FAILED_FILE
result code:0 result message:read failed file success
```

3. 最终数据导入成功客户端返回如下：

```
The response message: RSP Code[0].{0:success; 1:provisional respon
se;
other: fail.}
Successful response:
[08-15 19:43:31:21 ]event: LOAD_DELETE_FILE_END
result code:0
[FileTotalCount]:2
[FileSuccessCount]: 2
[FileFailureCount]: 0
result message:delete file end
~success~
```

导入重试支持指定路径

1.导入重试支持指定路径命令参数如下：

```
dbtool -loadserver -load-retry -clusterid=2 -splitpath = "/home/ld
s/split"
   -user="userA"  -password="passA"
```

1）-load-retry：指定任务类型为重试；

2）-clusterid：重试任务所在集群号；

3）-splitpsth：导入重试文件存放的绝对路径；-tablename 与 -splitpsth 必须且仅能指定其中一个。

2.导入重试支持指定路径运行结果如下：

```
Send handShake to
module[LoadServer,zxload,10.47.157.174:10.47.157.174:6281] Comtool
ID[
560044:a8:42:03:a4:720200218162854913] localport[5600] pid[3502]
successfully!
HANDSHARK SUCCESS!
Send message to other module successfully!
The response message: RSP Code[0].{0:success; 1:provisional respon
se;
other: fail.}
[02-18 16:28:55:070]Successful response:
[02-18 16:28:55:070]event: LOAD_DELETE_FILE_END
result code:0
[FileTotalCount]:4
[FileSuccessCount]: 4
[FileFailureCount]: 0
result message:delete file end
~success~
```

上面执行结果各个字段含义如下：

1）FileTotalCount：重试任务总文件个数；

2）FileSuccessCount：重试任务总文件成功个数；

3）FileFailureCount：重试任务总文件失败个数。

1.9.5. 最佳实践几点注意事项

1.9.5.1. 关于数据类型的选择

字段的数据类型影响了 SQL 语句的编写形式，进而影响了数据库优化器的执行路径，因此必须遵循如下原则：

- 同一个业务概念对应的数据类型的定义必须完全一致，包括类型、长度和精确度。比如，一个系统中的客户号、账号可能出现在多个数据表中，但这些定义必须完全相同。
- 定长数据类型的效率比变长的效率要高，因此选择数据类型时要优先考虑：INT/BIGINT、CHAR。比如对于"币种"字段确定为 3 位字符型，则定义为 CHAR(3)。

- 数据类型的选择必须与实际存储的数据完全一致，如：时间戳字段的类型选择 TIMESTAMP，日期字段的类型选择 DATE，金额字段选择 DECIMAL，数字字段选择 INT/NUMBER。

1.9.5.2. 关于隔离级别要求

写操作使用 CW 隔离级别，读操作默认使用CR隔离级别，如果业务场景允许脏读，则推荐使用 UR，可以提升处理性能。比如原语句为 `SELECT id FROM t1`,可以改成 `SELECT id FROM t1 UR`。

语句类型	隔离级别	隔离级别说明	适用场景
读语句	UR(uncommitsted read)	未提交读，不判断分布式读写冲突	适用于允许脏读或者不存在读写冲突的业务场景
	CR(consistency read)	强一致性读，先查询活跃GTID，后查询数据，严格保证返回结果处于分布式事务已提交状态，不存在脏读可能	适用于分布式事务中不允许脏读场景
写语句	CW(consistency write)	强一致性写，需要判断分布式写冲突，不允许多个事务同时写相同数据	适用于存在分布式写冲突，但不允许多个事务同时写相同数据场景

1.9.5.3. 关于批处理设计指导

1. 为充分利用分布式数据库的特性，建议按分片进行批处理，通过 `SHOW DISTRIBUTION FROM table` 获取表分片信息后，再通过多个线程使用 `STORAGEDB` 语法针对多个分片并发处理。
2. 批处理涉及大数据量查询，建议通过游标方式先把数据的主键查询到本地，在本地确定数据分块的上下边界后，通过使用主键范围作为 `WHERE` 条件分批次查询数据。

1.9.5.4. 关于锁超时设计

锁超时建议调整为 10~15 秒，防止分布式死锁等待时间过长。

1.9.6. 双库运行兼容函数说明

为充分保障生产系统健壮性，可以指定双库运行方案。经分析，双库兼容主要难点在于底层函数不同，因此首先需要找出双库底层兼容函数。

1.9.6.1. GoldenDB 与 Oracle 兼容函数

类别	功能	GoldenDB函数	GoldenDB与Oracle兼容函数
类型转换函数	数据类型转换	CONVERT	CAST
数字函数	进一位取整	CEIL(3.0002)	CEIL(3.0002)
	舍掉小数部分	FLOOR(5.33)	FLOOR(5.33)
	四舍五入（保留Y位小数）	ROUND(4.456,2)	ROUND(4.456,2)
	将当前数据取绝对值	ABS(-3.6)	ABS(-3.6)
	幂运算	POW(x,y)	POW(x,y)
	返回x/y的模（余数）	MOD(x,y)	MOD(x,y)
	返回字符串s的字符数	CHAR_LENGTH(s)	LENGTH
	返回字符串 str 最左边的那个字符的 ASCII 码值。如果 str 是一个空字符串，那么返回值为 0。如果 str 是一个 NULL，返回值也是 NULL.	ASCII(str)	ASCII(str)
	CHAR() 以整数类型解释参数，返回这个整数所代表的 ASCII 码值给出的字符组成的字符串。NULL 值将被忽略.	CHAR(N)	CHAR(N)

字符串函数	在字符串 str 中所有出现的字符串 from_str 均被 to_str 替换，然后返回这个字符串.	REPLACE(str,from_str,to_str)	REPLACE(str,from_str,to_str)
	查找字符串s1在s中的位置,返回首次出现的位置的索引值	LOCATE	INSTR(s,s1)
	获取字符串s中的第n个位置开始长度为len的字符串	SUBSTRING(s,n,len)	SUBSTR(s,n,len)
	从字符串s中获取s1的开始位置	LOCATE(s1,s)	SUBSTR(s1,instr(s1,s))
	用s2代替s中的s1	REPLACE(s,s1,s2)	REPLACE(s,s1,s2)
	返回字符串s1, 其左边用s2填补到len字符长度	LPAD(s1,len,s2)	LPAD(s1,len,s2)
	转换为大写	UPPER(iv_user_id)	UPPER(iv_user_id)
	转换为小写	LOWER(iv_user_id)	LOWER(iv_user_id)
	返回字符串s从左边开始的n个字符	LEFT(s, n)	SUBSTR(s,1,n)
	返回字符串s从右边开始的n个字符	RIGHT(s, n)	SUBSTR(s,-n)
	返回字符串s1, 其右边被字符串s2填补至len字符长度	RPAD(s1, len, s2)	RPAD(s1, len, s2)
	将字符串s反序	REVERSE(s)	REVERSE(s)
	返回当前日期	CURDATE()	TO_DATE(TO_CHAR(sysdate,'yyyymmdd'))

日期和时间函数	返回当前时间	CURTIME()	TO_DATE(TO_CHAR(sysdate,'hh24:mi:ss'))
	返回当前日期和时间	NOW()	SYSDATE
	返回当前日期和时间	SYSDATE()	SYSDATE
	字符串转日期	STR_TO_DATE	TO_DATE
	日期增减	DATE_ADD	ADD_MONTHS
	获取日期变量的年/月	YEAR(dt), MONTH(dt)	TO_CHAR
	日期转字符串-并格式化	DATE_FORMAT/ TIME_FORMAT	TO_CHAR
	日期间隔	DATEDIFF	日期相减
聚合函数	返回指定列的平均值	AVG(col)	AVG(col)
	返回指定列中非NULL值的个数	COUNT(col)	COUNT(col)
	返回指定列的最小值	MIN(col)	MIN(col)
	返回指定列的最大值	MAX(col)	MAX(col)
	返回指定列的所有值之和	SUM(col)	SUM(col)
控制流函数	如果testN是真，则返回resultN，否则返回default	CASE WHEN[test1] THEN [result1]...ELSE [default]	CASE WHEN[test1] THEN [result1]...ELSE [default]
	如果arg1不是空，返回arg1，否则返回arg2	IFNULL(ARG1,ARG2)	NVL(ARG1,ARG2)
	如果test是真，返回t；否则返回f	IF(test,t,f)	CASE test THEN t ELSE f END

	如果arg1=arg2返回NULL；否则返回arg1	NULLIF(arg1,arg2)	NULLIF(arg1,arg2)
	显示数据从n开始到m	LIMIT n,m	ROWNUM > =n AND ROWNUM <= m
注释	注释	//，#	/*注释内容*/
分组	分组	GROUP BY（部分维度列可以不出现在GROUP BY子句）	GROUP BY后面加上全部维度列
判断函数	恒满足	WHERE TRUE	WHERE 1=1
单行函数	只返回一行记录	SELECT x 不带FROM	SELECT x FROM dual
SQL语法	GROUP BY子句中使用列别名	可以在GROUP BY 使用列的别名 SELECT id as a,COUNT(1)c FROM tab GROUP BY a	不在GROUP BY 使用列的别名，使用完整列
	列别名加单引号	支持	不支持列别名加单引号
	别名	MySQL查询时FROM表名后面可以加上AS，不会报错	Oracle查询时FROM表名后面，不能加上AS 不然会报错
符号	单引号，双引号	单引号，双引号都可以	不支持双引号
SQL函数	自增长列	自增长列	尽量不用，否则需要在Oracle创建序列并且INSERT显示插入增长列

特性	分片VS分区:表示SQL语句执行所涉及到的DN节点，当指定了STORAGEDB时，CN以指定的STORAGEDB为准，而不再根据表的定义来判断 示例：SELECT * FROM gzyh.a5 storagerdb g1；只返回分片1的数据	分片STORAGEDB	通过WHERE控制数据范围
	从该条语句开始，后续语句涉及下发的DN节点均和该语句保持一致，该提示可以提升数据库的处理性能。比如同一个会话中，前面设置访问分片一后，本会话的后面的查询也只访问分片一。	分片SAMEDB	尽量不使用
逻辑判断	if then判断	IF iv_weekly_day = 'MON'THEN SET ii_weekly_day = 'MON'; ELSEIF iv_weekly_day = 'TUE' THEN SET ii_weekly_day = 'TUE'; END IF;	把IF转换为CASE WHEN 语句
其他	多行合并	GROUP_CONCAT	无公共函数，可以考虑在Oracle创建函数
	随机函数	RAND	无公共函数，可以考虑在Oracle创建函数
	以UNIX时间戳的形式返回当前时间	UNIX_TIMESTAMP()	无公共函数，可以修改为UNIX_TIMESTAMP(sysdate)，可以考虑在Oracle创建函数
	把UNIX时间戳转化为普通时间格式返回	FROM_UNIXTIME()	无公共函数，可以修改为FROM_UNIXTIME(sysdate)，可以考虑在Oracle创建函数

1.9.6.2. 典型函数说明

下面对一些SQL编码中比较突出的兼容性语法进行描述

- **限制返回行数/分页**

为了兼容 GoldenDB 和 `Oracle`，避免使用 `LIMIT` 函数，使用如下任意方式编码：

1、提交到 CN，并使用 `ROWNUM` 进行返回记录数限制。

- **限制访问分片/分区**

为了兼容 GoldenDB 和 `Oracle`，避免使用分片关键字 `STORAGEDB`，对于需要控制访问数据分片/分区范围的，通过WHERE条件控制所访问的分片/分区。

- **分组聚合GROUP BY**

为了兼容 GoldenDB 和 `Oracle`，避免使用 `MySQL` 特有的非标准 `SQL92` 的 `GROUP BY`，即部分维度列不出现于 `GROUP BY` 中，使用如下任意方式编码：

1、提交到 CN，并使用 `GROUP BY` 及 `row_number` 联合进行分组、返回行数限制。

另外，避免在 `GROUP BY` 子句中使用列的别名，而应该写出实际列或者完整列逻辑代码。

- **SQL 代码注释**

为了兼容 GoldenDB 和 `Oracle`，注释关键字不能使用"//"，"#"，需要使用 GoldenDB 和 `Oracle` 都支持的"/*注释内容*/"这种方式。如：

```
> SELECT t.dt,/*日期*/ count(1)/*记录数*/ FROM tabname t;
```

- **恒等于/条件拼接 1=1**

为了兼容 GoldenDB 和 `Oracle`，"恒等于"条件不能使用 `WHERE TRUE`，需要使用 GoldenDB 和 `Oracle` 都支持的 `WHERE 1=1` 这种方式。

- **空值转换**

为了兼容 GoldenDB 和 `Oracle`，避免使用 `IFNULL` 函数，使用如下任意方式编码：

1、提交到 CN，并使用 `nvl` 进行空值转换。

- **日期格式化**

为了兼容 GoldenDB 和 `Oracle`，避免使用 `DATE_FORMAT` 函数，使用如下任意方式编码：

1、提交到 CN，并使用 `TO_CHAR` 进行日期格式化为字符串。

数据类型显示转换

为了兼容 GoldenDB 和 `Oracle`，避免使用 `CONVERT` 函数，因为 `MySQL` 的 `CONVERT` 函数代表数据类型转换，而 `Oracle` 的 `CONVERT` 代表字符集转换，含义不一样，使用如下方式编码：

使用 GoldenDB 和 `Oracle` 都支持并且含义一样的 `cast` 函数。例如

```
> SELECT CAST(val AS NUMBER(10,4)) as val FROM tabname;
```

- **截取字符串**

为了兼容 GoldenDB 和 `Oracle`，避免使用 `SUBSTRING`、`SUBSTRING_INDEX`、`LOCATE`、`LEFT`、`RIGHT` 函数，使用如下任意方式编码：

1. 提交到 CN，并使用 `SUBSTR`、`INSTR` 进行字符串截取。

- **单行函数DUAL**

为了兼容 GoldenDB 和 `Oracle`，避免使用没有 `FROM` 的 SQL 语句，只需查单行也需要加上 `FROM DUAL`。

- **当前时间**

为了兼容 GoldenDB 和 `Oracle`，避免使用 `NOW()` 函数，使用如下任意方式编码：

1. 提交到 CN，并使用 `SYSDATE` 获取当前时间。

- **单双引号**

为了兼容 GoldenDB 和 `Oracle`，列值避免使用双引号”，而应该使用单引号。

- **表的别名**

为了兼容 GoldenDB 和 `Oracle`，表与表的别名之间避免使用AS关键字。

- **列的别名**

为了兼容 GoldenDB 和 `Oracle`，列的别名避免使用单引号。

- **IF语句**

为了兼容 GoldenDB 和 `Oracle`，逻辑判断语句避免使用 `IF`，用 `CASE WHEN` 替代。

- **取整**

为了兼容 GoldenDB 和 `Oracle`，数字向上取整避免使用 `CEILING`，使用如下任意方式编码：

1. 提交到 CN，并使用 `CEIL` 进行数字向上取整。

字符串长度

为了兼容 GoldenDB 和 `Oracle`，避免使用 `CHAR_LENGTH`，使用如下任意方式编码：

1. 提交到 CN，并使用 `length` 进行获取字符串长度。

- **时间间隔**

为了兼容 GoldenDB 和 `Oracle`，避免使用 `DATEDIFF`，使用如下任意方式编码：

1. 提交到 CN，并使用两个时间相减获取时间间隔。

- **自增长列**

为了兼容 GoldenDB 和 `Oracle`，慎重使用自增长列，因为 `Oracle` 没有自增长列，需要通过序列配合 `INSERT` 进行实现，使用如下任意方式编码：

1. 增加连接库类型的全局参数，`Oracle` 和 GoldenDB 分别拼接 SQL，`Oracle` 语句使用 `INSERT` 和 `SEQUENCE` 实现。

- **在Oracle定义对应函数**

以下函数通过 CN 和 DN 都难以实现的，需要在 `Oracle` 创建对应函数实现。

1）目前发现已有使用的函数是

多行合并：`GROUP_CONCAT`

2）较为常见的函数：

区间内随机数：`RAND`

日期增加：`DATE_ADD`

1.9.7. CN 支持 gb18030 字符集

- CN 支持系统字符集配置为 `gb18030`

  配置文件为 `proxy.ini`，配置段为[GENERAL]，配置项为 `character_set_server`；

配置时：

```
[GENERAL]
character_set_server=gb18030
```

- 配置项作用

1. 客户端链接CN时，不设置链路字符集时，链路的以下字符集项采用配置的字符集：

```
character_set_client gb18030
character_set_connection gb18030
character_set_results gb18030
```

2. 创建库时，库的字符集不指定，其默认字符集使用配置的字符集。 目前 `gb18030` 字符集的字符序只支持 `gb18030_bin`；

- CN 支持设置链路字符集为 `gb18030`

  执行以下操作设置链路字符集：

```
> SET NAMES gb18030;
```

执行后，可通过以下操作查看链路信息：

```
> SHOW VARIABLES LIKE \"char%\";
```

设置完成链路字符集，其 `character_set_client` 为 `gb18030`，客户端发送给 CN 的操作字符串需要以此字符集编码，CN 以此字符集解析字符串。

其 `character_set_results` 为 `gb18030`，客户端接收到的结果集，其字符字段也以此字符集解析。

- CN 设置为 `gb18030` 字符集时

  表和字符字段的字符集应同为 `gb18030` 字符集；同时，链路字符集同为 `gb18030` 字符集时，性能最高，功能最完善。建议整个分布式系统都使用同一种字符集操作。

- `gb18030` 字符集的影响

  `gb18030` 字符集采用变长多字节编码，每个字符可以由 1 个、2 个或 4 个字节组成；注意受字节长度限制的操作。如表主键长度受字节长度限制，在 `gbk` 字符集下，主键字段字符个数要大于 `gb18030` 字符集的字符数。

1.9.8. 附录

1.9.8.1. 保留字

保留字	保留字	保留字
ACCESSIBLE	ADD	ALL
ALTER	ANALYZE	AND
AS	ASC	ASENSITIVE
BEFORE	BETWEEN	BIGINT
BINARY	BLOB	BOTH
BY	CALL	CASCADE
CASE	CHANGE	CHAR
CHARACTER	CHECK	COLLATE
COLUMN	CONDITION	CONSTRAINT
CONTINUE	CONVERT	CREATE
CROSS	CURRENT_DATE	CURRENT_TIME
CURRENT_TIMESTAMP	CURRENT_USER	CURSOR
DATABASE	DATABASES	DAY_HOUR
DAY_MICROSECOND	DAY_MINUTE	DAY_SECOND
DEC	DECIMAL	DECLARE
DEFAULT	DELAYED	DELETE

DESC	DESCRIBE	DETERMINISTIC
DISTINCT	DISTINCTROW	DIV
DOUBLE	DROP	DUAL
EACH	ELSE	ELSEIF
ENCLOSED	ESCAPED	EXISTS
EXIT	EXPLAIN	FALSE
FETCH	FLOAT	FLOAT4
FLOAT8	FOR	FORCE
FOREIGN	FROM	FULLTEXT
GET	GRANT	GROUP
HAVING	HIGH_PRIORITY	HOUR_MICROSECOND
HOUR_MINUTE	HOUR_SECOND	IF
IGNORE	IN	INDEX
INFILE	INNER	INOUT
INSENSITIVE	INSERT	INT
INT1	INT2	INT3
INT4	INT8	INTEGER
INTERVAL	INTO	IO_AFTER_GTIDS
IO_BEFORE_GTIDS	IS	ITERATE

JOIN	KEY	KEYS
KILL	LEADING	LEAVE
LEFT	LIKE	LIMIT
LINEAR	LINES	LOAD
LOCALTIME	LOCALTIMESTAMP	LOCK
LONG	LONGBLOB	LONGTEXT
LOOP	LOW_PRIORITY	MASTER_BIND
MASTER_SSL_VERIFY_SERVER_CERT	MATCH	MAXVALUE
MEDIUMBLOB	MEDIUMINT	MEDIUMTEXT
MIDDLEINT	MINUTE_MICROSECOND	MINUTE_SECOND
MOD	MODIFIES	NATURAL
NOT	NO_WRITE_TO_BINLOG	NULL
NUMERIC	ON	OPTIMIZE
OPTION	OPTIONALLY	OR
ORDER	OUT	OUTER
OUTFILE	PARTITION	PRECISION
PRIMARY	PROCEDURE	PURGE
RANGE	READ	READS

READ_WRITE	REAL	REFERENCES
REGEXP	RELEASE	RENAME
REPEAT	REPLACE	REQUIRE
RESIGNAL	RESTRICT	RETURN
REVOKE	RIGHT	RLIKE
SCHEMA	SCHEMAS	SECOND_MICROSECOND
SELECT	SENSITIVE	SEPARATOR
SET	SHOW	SIGNAL
SMALLINT	SPATIAL	SPECIFIC
SQL	SQLEXCEPTION	SQLSTATE
SQLWARNING	SQL_BIG_RESULT	SQL_CALC_FOUND_ROWS
SQL_SMALL_RES ULT	SSL	STARTING
STRAIGHT_JOIN	TABLE	TERMINATED
THEN	TINYBLOB	TINYINT
TINYTEXT	TO	TRAILING
TRIGGER	TRUE	UNDO
UNION	UNIQUE	UNLOCK
UNSIGNED	UPDATE	USAGE

USE	USING	UTC_DATE
UTC_TIME	UTC_TIMESTAMP	VALUES
VARBINARY	VARCHAR	VARCHARACTER
VARYING	WHEN	WHERE
WHILE	WITH	WRITE
XOR	YEAR_MONTH	ZEROFILL
PARTITION	PROXY	DISTRIBUTED
MULTI_STEP_QUERY	UR	SEMI
CR	SW	CW
CHGDISKEY	STORAGEDB	SYSDATE
DECODE	ROWNUM	SEQUENCES
INCREMENT	NOMAXVALUE	MINVALUE
NOMINVALUE	CYCLE	NOCYCLE
NOCACHE	NOORDER	READMASTER
READSLAVE	READBALANCE	CURRVAL
NEXTVAL	GROUPS	NULLCOLS
SYNONYM	PUBLIC	SAMEDB
VARCHAR2	CLOB	PLS_INTEGER
SUBDISTRIBUTED	PARSE_VCOL_EXPR	PAGE_CHECKSUM

RETURNING	CURRENT_ROLE	DISTRIBUTION
OVER	KEEPALIVE	NOGTID
WRITE_SAMEDB	REDISTRIBUTING	RULE

1.9.8.2. 数据类型

GoldenDB数据类型	说明
INT	4字节整数，SIGNED：-2147483648 – 2147483647 UNSIGNED：0 - 4294967295
SMALLINT	2字节整数，SIGNED：-32768 – 32767 UNSIGNED：0 - 65535
BIGINT	8字节整数，SIGNED：-9223372036854775808 – 9223372036854775807 UNSIGNED：0 - 18446744073709551615
TINYINT	1字节整数，SIGNED：-128 – 127 UNSIGNED：0 - 255
MEDIUMINT	1字节整数，SIGNED：-128 – 127 UNSIGNED：0 - 255
INTEGER	4字节整数，SIGNED：-2147483648 – 2147483647 UNSIGNED：0 - 4294967295
FLOAT(5)	4字节单精度浮点数
DOUBLE	8字节双精度浮点数
DECIMAL(n,s)	数值型，n为数值总长度，范围1-65，s为小数点后长度，范围0-30，s必须不大于n
NUMERIC(n,s)	同DECIMAL
NUMBER(n,s)	同DECIMAL
DEC(n,s)	同DECIMAL

DATE	日期，范围：1000-01-01 – 9999-12-31
TIMESTAMP	UTC日期和时间，范围：1970-01-01 00:00:01.000000 – 2038-01-19 03:14:07.999999
DATETIME	日期和时间，范围：1000-01-01 00:00:01.000000 – 9999-12-31 23:59:59.999999
TIME	时间，范围是'-838:59:59'到'838:59:59'
YEAR	两位或四位格式的年。默认是四位格式。在四位格式中，允许的值是1901到2155和0000。在两位格式中，允许的值是70到69，表示从1970年到2069年
VARCHAR(10)	变长字符型,最多65535个字符
CHAR(10)	定长字符型,最多255个字符
BLOB	最大长度为65535(216–1)字节的BLOB列
LONGBLOB	二进制形式的极大文本数据，0-4294967295字节
TINYBLOB	不超过 255 个字符的二进制字符串
MEDIUMBLOB	二进制形式的中等长度文本数据，0-16777215字节
CLOB	二进制形式的长文本数据，0-65535字节
BINARY	类似CHAR的二进制存储，特点是插入定长不足补0
VARBINARY	类似VARCHAR的变长二进制存储，特点是定长不补0
LONGTEXT	极大文本数据，0-4294967295字节
TINYTEXT	短文本字符串，0-255字节
MEDIUMTEXT	中等长度文本数据，0-16777215字节

ENUM	枚举，最大可达65535个不同的枚举值
TEXT	可变长度，0-65535字节
SET	多选字符串数据类型，最多可以包含64项元素
VARCHAR2	支持 VARCHAR2(num) 、VARCHAR2(num BYTE) 、VARCHAR2(num CHAR) 三种用法；前两种默认转换为 varbinary，第三种转为 varchar

1.9.8.3. 操作运算符

运算符	说明
-	减号
-	负号
=	等于操作
% MOD	取余
<=>	等于操作
>	大于
>=	大于等于
!=	不等于
<>	不等于
*	乘
/	除
||,OR	或

+	加
<=	小于等于
<	小于
AND, &&	与
BETWEEN ... AND ...	范围判断
DIV	除
IS	是判断
IS NOT	非判断
IS NOT NULL	判断不为null
IS NULL	判断null
LIKE	LIKE函数
NOT BETWEEN ... AND ...	范围判断
NOT LIKE	LIKE函数
NOT,!	非
XOR	异或
||	连接符

1.10. SQL 规范

本章介绍分布式数据库支持的语法、分布式下的应用限制、分布式下的最佳应用方式，用于帮助业务开发人员了解如何基于分布式数据库编写 SQL 开发业务程序。

1.10.1. 文档术语

本规范文档涉及术语以及说明参见下表。

术语说明表

术语	英文	说明
脏读	UR	查询时，读取的数据中允许包含 脏数据，或者用于查询时不存在其他事务修改的情况。
强一致性读	CR	查询时，读取的数据是已提交的。
单节点写	SW	不考虑分布式事务下的对部分已提交事务的数据的修改 下面两种情况使用SW： 修改的数据不会在两个及以上事务中同时修改； 涉及本数据的所有事务都只会修改单条记录。
强一致性写	CW	需要考虑分布式事务下的对部分已提交事务的数据的修改。

1.10.2. 分布式SQL语法通用说明

- **CONSISTENCY_OPTION**

  数据一致性级别：脏读(UR)、强一致性读(CR)、单节点写(SW)、强一致性写(CW)；默认值为强一致性读(CR)和强一致性写(CW)。

- **STORAGEDB**

  表示 SQL 语句执行所涉及到的 DN 节点，当指定了 `STORAGEDB` 时，计算节点以指定的 `STORAGEDB` 为准，而不再根据表的定义来判断。

- **SAMEDB**

  该提示可以出现在 `START TRANSACTION` 或事务中的任一条SQL的末尾。

  含义为从该条语句开始，后续语句涉及下发的 DN 节点均和该语句保持一致，该提示可以提升数据库的处理性能。

- **table_name说明**

  本规范中 `table_name` 可以为 `db_name.table_name`，从 Insight 界面发起的 DDL 操作要求必须携带库名。

1.10.3. DATA TYPES

1.10.3.1. GoldenDB支持数据类型列表

GoldenDB 支持的数据类型列表

GoldenDB 数据类型	说明
INT	4字节整数，SIGNED：-2147483648 – 2147483647 UNSIGNED：0 - 4294967295
SMALLINT	2字节整数，SIGNED：-32768 – 32767 UNSIGNED：0 -65535
BIGINT	8字节整数，SIGNED：-9223372036854775808 – 9223372036854775807 UNSIGNED：0 - 18446744073709551615
TINYINT	1字节整数，SIGNED：-128 – 127 UNSIGNED：0 - 255
MEDIUMINT	3字节整数，SIGNED：-8388608 – 8388607 UNSIGNED：0 - 16777215
INTEGER	4字节整数，SIGNED：-2147483648 – 2147483647 UNSIGNED：0 - 4294967295
FLOAT(5)	4字节单精度浮点数
DOUBLE	8字节双精度浮点数
DECIMAL(n,s)	数值型，n为数值总长度，范围1-65，小数点前最大长度为总长度 - 小数点后最大长度，s为小数点后长度，范围0-30，s必须不大于n
NUMERIC(n,s)	同DECIMAL
NUMBER(n,s)	同DECIMAL
DEC(n,s)	同DECIMAL
DATE	日期，范围：1000-01-01 – 9999-12-31

TIMESTAMP	UTC日期和时间，范围：1970-01-01 00:00:01.000000 – 2038-01-19 03:14:07.999999
DATETIME	日期和时间，范围：兼容0000-00-00 00:00:01.000000 – 9999-12-31 23:59:59.999999
TIME	时间，范围是'-838:59:59'到'838:59:59'
YEAR	两位或四位格式的年。默认是四位格式。在四位格式中，允许的值是1901到2155和0000。在两位格式中，允许的值是70到69，表示从1970年到2069年
VARCHAR(10)	变长字符型,最多65535个字符
CHAR(10)	定长字符型,最多255个字符
BLOB	最大长度为65,535(2^{16}–1)字节的BLOB列
LONGBLOB	二进制形式的极大文本数据，0-4294967295字节
TINYBLOB	不超过 255 个字符的二进制字符串
MEDIUMBLOB	二进制形式的中等长度文本数据，0-16777215字节
CLOB	二进制形式的长文本数据，0-65535字节
BINARY	类似CHAR的二进制存储，特点是插入定长不足补0
VARBINARY	类似VARCHAR的变长二进制存储，特点是定长不补0
LONGTEXT	极大文本数据，0-4294967295字节
TINYTEXT	短文本字符串，0-255字节
MEDIUMTEXT	中等长度文本数据，0-16777215字节
ENUM	枚举，最大可达65535个不同的枚举值

TEXT	可变长度，0-65535字节
SET	多选字符串数据类型，最多可以包含64项元素
VARCHAR2	支持VARCHAR2(num) 、VARCHAR2(num BYTE) 、VARCHAR2(num CHAR)三种用法，前两种用法等价；存储过程参数类型为VARCHAR2且未指定长度时，默认长度为4000字节
JSON	-
GEOMETRY	空间数据类型，包括 POINT、LINESTRING、POLYGON 三种最基础的类型以及由他们扩展的 MULTIPOINT、MULTILINESTRING、MULTIPOLYGON 三种集合类型和可以代表任意 GEOMETRY 类型的集合 GEOMETRYCOLLECTION（其中 GEOMETRY 是一种虚类型，可以代表任意基础类型）

1.10.3.2. 使用说明

不支持 BINARY、VARBINARY、BLOB、TEXT 类型字段的关联、运算等，计算节点只支持透传。 不建议使用 FLOAT、DOUBLE 类型进行比较操作，操作不限于 `GROUP BY`、`DISTINCT`、`ON` 等值条件、`WHERE` 过滤。

1.10.3.3. GoldenDB 支持数据类型举例

非常规类型举例说明。

CLOB、BLOB大对象插入

- 创建库表：

```
> DROP DATABASE IF EXISTS poc_test;
> CREATE DATABASE IF NOT EXISTS poc_test;
> CREATE TABLE poc_test.t1 (a INT PRIMARY KEY ,b BLOB,c CLOB)
DISTRIBUTED BY DUPLICATE(g1,g2);
```

- 插入数据

```
> INSERT INTO poc_test.t1 VALUES (1,LOAD_FILE(
  '/home/zxdb/bin/binlog_apply_rollback.py'),LOAD_FILE(
  '/home/zxdb/bin/binlog_apply_rollback.py'));
```

- 导出图片

```
> SELECT b FROM poc_test.t1 INTO dumpfile '/home/zxdb';
```

JSON

建表

- 建表语句：

```
> DROP DATABASE IF EXISTS poc_test;
> CREATE DATABASE IF NOT EXISTS poc_test;
> CREATE TABLE poc_test.json_tab
(
id INT PRIMARY KEY,
json_info JSON COMMENT
'json数据
',
json_id INT GENERATED ALWAYS AS (json_info -
>
'
$.id
') COMMENT

'json数据的虚拟字段
',
json_name VARCHAR(5) GENERATED ALWAYS AS (json_info -
>
'
$.name
'),
INDEX json_info_id_idx (json_id)
)DISTRIBUTED BY HASH(id) (g1,g2);
```

- 建表带 JSON 数据类型说明：
  1. 支持 JSON 虚拟列。
  2. 支持 JSON 虚拟列做索引。
  3. `ALTER TABLE` 支持增加、修改、删除 JSON 类型字段。
  4. `ALTER TABLE` 支持增加、修改、删除 JSON 虚拟列。
  5. JSON 类型字段不能做分发键、主键、[全局]索引。
  6. JSON 虚拟列不能做分发键。

插入数据

插入数据语句：

```
> INSERT INTO poc_test.json_tab(id,json_info) VALUES(1,'"abc"');
> INSERT INTO poc_test.json_tab(id,json_info) VALUES(2,'123');
> INSERT INTO poc_test.json_tab(id,json_info) VALUES(3,'{"id": 1,
"name": "张三"}');
> INSERT INTO poc_test.json_tab(id,json_info) VALUES(4,'[1, 2, 3]'
);
```

```
> INSERT INTO poc_test.json_tab(id,json_info) VALUES(5,'{"id": 1,
"name": "张三", "age": 18, "sister": [{"name":"张大姐", "age": 30},
{"name": "张二姐", "age": 20}]}');
> INSERT INTO poc_test.json_tab(id,json_info) VALUES(6,JSON_OBJECT
('id', 2, 'name', '李四', 'age', 18, 'sister',
JSON_ARRAY(JSON_OBJECT('name', '李大姐', 'age', 28), JSON_OBJECT('n
ame', '李二姐', 'age', 25'))));
```

计算节点支持的 JSON 函数列表

计算节点支持的 JSON 函数列表

JSON函数	说明
JSON_ARRAY	创建 JSON 数组
JSON_ARRAY_APPEND	添加数据到指定 JSON 数组结尾
JSON_ARRAY_INSERT	往 JSON 数组指定位置插入数据
JSON_CONTAINS	JSON 文档是否在路径中包含特定对象
JSON_CONTAINS_PATH	JSON 文档是否在路径中包含任何数据
JSON_DEPTH	JSON 文档的最大深度
JSON_EXTRACT，->	返回 JSON 文档指定路径的数据
JSON_INSERT	将数据插入 JSON 文档，添加不存在的值但不替换已存在的值
JSON_KEYS	返回有 JSON 对象的键名构成的数组
JSON_LENGTH	JSON 文档的元素数量
JSON_MERGE	合并 JSON 文档，保留重复的键
JSON_MERGE_PATCH	合并 JSON 文档，替换重复键的值
JSON_MERGE_PRESERVE	同 JSON_MERGE

JSON_OBJECT	创建 JSON 对象
JSON_QUOTE	引用 JSON 文档
JSON_REMOVE	从 JSON 文档中删除数据
JSON_REPLACE	替换 JSON 文档中的值
JSON_SET	将数据插入 JSON 文档，添加不存在的值并替换已存在的值
JSON_SEARCH	返回指定字符串在 JSON 文件中的路径
JSON_TYPE	JSON 值类型
JSON_UNQUOTE	取消引用 JSON 值
JSON_VALID	JSON 值是否有效

支持的JSON函数示例

- JSON_ARRAY([val[, val] ...])

```
> SELECT JSON_ARRAY(1, "abc", NULL, TRUE, CURTIME());
+---------------------------------------------+
| JSON_ARRAY(1, "abc", NULL, TRUE, CURTIME()) |
+---------------------------------------------+
| [1, "abc", null, true, "20:29:16.000000"] |
+---------------------------------------------+
```

- JSON_ARRAY_APPEND(json_doc, path, val[, path, val] ...)

```
> SELECT JSON_ARRAY_APPEND('["a", ["b", "c"], "d"]','$[1]', 1);
+-----------------------------------------------------------+
| JSON_ARRAY_APPEND('["a", ["b", "c"], "d"]','$[1]', 1) |
+-----------------------------------------------------------+
| ["a", ["b", "c", 1], "d"] |
+-----------------------------------------------------------+
```

- JSON_ARRAY_INSERT(json_doc, path, val[, path, val] ...)

```
> SELECT JSON_ARRAY_INSERT('["a", {"b": [1, 2]}, [3, 4]]','$[1]',
'x');
```

```
+-----------------------------------------------------------------+
| JSON_ARRAY_INSERT('["a", {"b": [1, 2]}, [3, 4]]','$[1]', 'x') |
+-----------------------------------------------------------------+
| ["a", "x", {"b": [1, 2]}, [3, 4]] |
+-----------------------------------------------------------------+
```

- JSON_CONTAINS(target, candidate[, path])

```
> SELECT JSON_CONTAINS('{"a": 1, "b": 2, "c": {"d": 4}}','1', '$.a
');
+-----------------------------------------------------------------+
| JSON_CONTAINS('{"a": 1, "b": 2, "c": {"d": 4}}', '1','$.a') |
+-----------------------------------------------------------------+
| 1 |
+-----------------------------------------------------------------+
```

- JSON_CONTAINS_PATH(json_doc, one_or_all, path[, path] ...)

```
> SELECT JSON_CONTAINS_PATH('{"a": 1, "b": 2, "c": {"d":
4}}', 'one', '$.a', '$.e');
+-----------------------------------------------------------------
-----------+
| JSON_CONTAINS_PATH('{"a": 1, "b": 2, "c": {"d": 4}}','one', '$.a
', '$.e') |
+-----------------------------------------------------------------
-----------+
| 1 |
+-----------------------------------------------------------------
-----------+
```

- JSON_DEPTH(json_doc)

```
> SELECT JSON_DEPTH('[10, {"a": 20}]');
+-----------------------------+
| JSON_DEPTH('[10, {"a": 20}]') |
+-----------------------------+
| 3 |
+-----------------------------++
```

- JSON_EXTRACT(json_doc, path[, path] ...)

```
> SELECT JSON_EXTRACT('[10, 20, [30, 40]]', '$[1]');
+-------------------------------------------+
| JSON_EXTRACT('[10, 20, [30, 40]]', '$[1]') |
+-------------------------------------------+
| 20 |
+-------------------------------------------+
```

- JSON_INSERT(json_doc, path, val[, path, val] ...)

```
> SELECT JSON_INSERT('{ "a": 1, "b": [2, 3]}', '$.a', 10,'$.c', '[
true, false]');
+-----------------------------------------------------------------
---------+
| JSON_INSERT('{ "a": 1, "b": [2, 3]}', '$.a', 10,'$.c', '[true, f
alse]') |
+-----------------------------------------------------------------
---------+
| {"a": 1, "b": [2, 3], "c": "[true, false]"} |
+-----------------------------------------------------------------
---------+
```

- JSON_KEYS(json_doc[, path])

```
> SELECT JSON_KEYS('{"a": 1, "b": {"c": 30}}', '$.b');
+-------------------------------------------+
| JSON_KEYS('{"a": 1, "b": {"c": 30}}', '$.b') |
+-------------------------------------------+
| ["c"] |
+-------------------------------------------+
```

- JSON_LENGTH(json_doc[, path])

```
> SELECT JSON_LENGTH('{"a": 1, "b": {"c": 30}}', '$.b');
+--------------------------------------------+
| JSON_LENGTH('{"a": 1, "b": {"c": 30}}', '$.b') |
+--------------------------------------------+
| 1 |
+--------------------------------------------+
```

- JSON_MERGE(json_doc, json_doc[, json_doc] ...)

```
> SELECT JSON_MERGE('[1, 2]', '[true, false]');
+---------------------------------------+
| JSON_MERGE('[1, 2]', '[true, false]') |
+---------------------------------------+
| [1, 2, true, false] |
+---------------------------------------+
```

- JSON_MERGE_PATCH(json_doc, json_doc[, json_doc] ...)

```
> SELECT JSON_MERGE_PATCH('{ "a": 1, "b":2 }','{ "a": 3,"c":4 }');
+-----------------------------------------------------------+
| JSON_MERGE_PATCH('{ "a": 1, "b":2 }','{ "a": 3, "c":4
}') |
+-----------------------------------------------------------+
```

```
| {"a": 3, "b": 2, "c": 4} |
+--------------------------------------------------------+
```

- JSON_MERGE_PRESERVE(json_doc, json_doc[, json_doc] ...)

```
> SELECT JSON_MERGE_PRESERVE('{ "a": 1, "b": 2 }', '{ "a": 3,"c":
4 }');
+-----------------------------------------------------------------
+
| JSON_MERGE_PRESERVE('{ "a": 1, "b": 2 }', '{ "a": 3,"c": 4 }') |
+-----------------------------------------------------------------
+
| {"a": [1, 3], "b": 2, "c": 4} |
+-----------------------------------------------------------------
+
```

- JSON_OBJECT([key, val[, key, val] ...])

```
> SELECT JSON_OBJECT('id', 87, 'name', 'Tom');
+-------------------------------------+
| JSON_OBJECT('id', 87, 'name', 'Tom') |
+-------------------------------------+
| {"id": 87, "name": "Tom"} |
+-------------------------------------+
```

- JSON_QUOTE(string)

```
> SELECT JSON_QUOTE('[1, 2, 3]');
+-------------------------+
| JSON_QUOTE('[1, 2, 3]') |
+-------------------------+
| "[1, 2, 3]" |
+-------------------------+
```

- JSON_REMOVE(json_doc, path[, path] ...)

```
> SELECT JSON_REMOVE('["a", ["b", "c"], "d"]','$[1]');
+-----------------------------------------------+
| JSON_REMOVE('["a", ["b", "c"], "d"]', '$[1]')
|
+-----------------------------------------------+
| ["a", "d"] |
+-----------------------------------------------+
```

- JSON_REPLACE(json_doc, path, val[, path, val] ...)

```
> SELECT JSON_REPLACE('{ "a": 1, "b": [2, 3]}', '$.a', 10,'$.c', '
```

```
[true, false]');
+-----------------------------------------------------------------
---------+
| JSON_REPLACE('{ "a": 1, "b": [2, 3]}', '$.a', 10,'$.c', '[true,
false]') |
+-----------------------------------------------------------------
---------+
| {"a": 10, "b": [2, 3]} |
+-----------------------------------------------------------------
---------+
```

- JSON_SET(json_doc, path, val[, path, val] ...)

```
> SELECT JSON_SET('{ "a": 1, "b": [2, 3]}', '$.a', 10,'$.c', '[tru
e, false]');
+-----------------------------------------------------------------
------+
| JSON_SET('{ "a": 1, "b": [2, 3]}', '$.a', 10, '$.c','[true, fals
e]') |
+-----------------------------------------------------------------
------+
| {"a": 10, "b": [2, 3], "c": "[true, false]"} |
+-----------------------------------------------------------------
------+
```

- JSON_SEARCH(json_doc, one_or_all, search_str[, escape_char[, path] ...])

```
> SELECT JSON_SEARCH('["abc", [{"k": "10"}, "def"], {"x":"abc"}, {
"y":"bcd"}]', 'all', '%a%') AS path;
+--------------------+
| path |
+--------------------+
| ["$[0]", "$[2].x"] |
+--------------------+
```

- JSON_TYPE(json_val)

```
> SELECT JSON_TYPE(JSON_EXTRACT('{"a": [10, true]}','$.a[0]'));
+-----------------------------------------------------------+
| JSON_TYPE(JSON_EXTRACT('{"a": [10, true]}', '$.a[0]')) |
+-----------------------------------------------------------+
| INTEGER |
+-----------------------------------------------------------+
```

- JSON_UNQUOTE(json_val)

```
> SELECT JSON_UNQUOTE('"abc"');
+----------------------+
```

```
| JSON_UNQUOTE('"abc"') |
+-----------------------+
| abc |
+-----------------------+
```

- JSON_VALID(val)

```
> SELECT JSON_VALID('{"a": 1}');
+------------------------+
| JSON_VALID('{"a": 1}') |
+------------------------+
| 1 |
+------------------------+
```

JSON函数数据类型说明

1. 计算节点计算 `JSON_OBJECT` 函数时，不支持按键名进行键-值对自动排序。

2. 计算节点计算 `JSON_TYPE` 函数时，目前只能识别 OBJECT、ARRAY、STRING、DOUBLE、INTEGER、BOOLEAN、NULL 这些 JSON 值类型。

3. 计算节点不能解析用于表示 JSON 数组子集范围的 `to` 参数和 `last` 参数（示例：'$[last-3 to last-1]'）。

4. 计算节点聚合函数目前有 `COUNT`、`COUNT DISTINCT`、`SUM`、`SUM DISTINCT`、`MAX`、`MIN`、`AVG`、`AVG DISTINCT` 支持 JSON 数据类型，`GROUP_CONCAT` 函数支持带 JSON 数据类型的复制表。 此外，`MAX` 和 `MIN` 支持 DOUBLE、INTEGER 等 JSON 值类型，其他 JSON 值类型按字符串处理。

5. 计算节点 `GROUP BY` 和 `ORDER BY` 目前支持 STRING、DOUBLE、INTEGER、BOOLEAN、NULL 等 JSON 值类型。但数据节点和计算节点均不支持OBJECT、ARRAY 等 JSON 值类型进行 `GROUP BY` 或 `ORDER BY` 操作，只能得到一个近似结果。

1.10.3.4. GEOMETRY空间数据类型

Geometry空间数据类型介绍

GoldenDB 的 Geometry 是一种空间数据类型，表示地理空间数据。GoldenDB 的 Geometry 类型遵循 OpenGIS 标准，可以存储点、线段、多边形等空间数据对象。其中

Point: 表示二维空间中的一个点。

WKT 格式示例：

```
Point(1 1)
```

LineString: 表示点的序列连接成的一条线，线段不能自相交。

WKT 格式示例：

```
LineString(0 0, 1 1, 2 2)
```

Polygon: 表示封闭的多边形，由一个封闭的外边界和零个或多个封闭的内边界（洞）组成，内外边界也叫多边形的内外环，注意内环不能为点，内环也不能在外环的外部。

WKT 格式示例：

```
Polygon((0 0, 0 1, 1 1, 1 0, 0 0)) 、Polygon((0 0, 10 0, 10 10, 0 10, 0 0), (3 3, 3 7, 7 7, 7 3, 3 3))
```

MultiPoint: 表示多个点的集合。

WKT 格式示例：

```
MultiPoint(1 1, 2 2)
```

MultiLineString: 表示多条线的集合，线段元素之间可以相交，但线段本身不能自相交。

WKT 格式示例：

```
MultiLineString((0 0, 1 1), (2 2, 3 3))
```

MultiPolygon: 表示多个多边形的集合，多个多边形之间可以接触但是不能有重叠。

WKT 格式示例：

```
MultiPolygon(((0 0, 0 1, 1 1, 1 0, 0 0)))
```

GeometryCollection: 表示多种几何对象的集合，多种几何对象之间要有相同的坐标系。

WKT 格式示例：

```
GeometryCollection(Point(1 1), LineString(0 0, 1 1))
```

Geometry: 表示一种虚类型，可以代表任意基础类型。

空间参考系统SRS 、空间参考标识符SRID以及空间数据格式

空间数据的空间参考系统 (Spatial Reference System，SRS) 是基于坐标的地理位置系统。 空间参考系统一般包括如下类型： - 投影 SRS。投影 SRS 是地球在平面上的投影，也就是平面地

图。该平面上的坐标系是笛卡尔坐标系，使用长度单位（米、英尺等），而不是经度和纬度。- 地理 SRS。地理 SRS 是一种非投影 SRS，它以任何角度单位表示椭圆体上的经纬度（或纬度-经度）坐标。

空间参考标识符 SRID： 每个空间数据对象都对应一个 SRID，该值标识了当前对象属于哪个空间参考系 (SRS)。目前 GoldenDB 支持地理和投影两类共 5152 个空间参考系，GoldenDB 也支持用户自定义空间参考系，可以通过查询表 information_schema.st_spatial_reference_systems 获取系统中所有空间参考系的详细定义。GoldenDB中比较常用空间参考系的有 SRID 0 以及 SRID 4326，SRID 0是一个无限大的平面笛卡尔坐标系，如果一个空间数据对象没有定义 SRID，则默认使用 SRID 0；SRID 4326 是 GPS 系统使用的地理空间参考系。在 GoldenDB 中对空间数据对象进行计算时各个空间数据对象必须在同一坐标系中，不在同一参考系的话会报错，如果需要对不同坐标系中的对象计算可以使用 GoldenDB 提供的坐标系转换函数在不同的空间坐标系中进行转换。

GoldenDB数据库支持两种标准的空间数据格式用于查询中的空间数据对象: - Well-Know Text Format（WKT） - Well-Know Binary Format（WKB） WKT 即文本格式，在用户操作空间数据类型的数据时可以使用直观的文本进行插入或查询，GoldenDB 支持 OpenGIS 定义的语法来写 WKT 数据，WKT 既可以作为数据格式，也可以用于 SRS 的定义，示例如下

```
POINT(15 20)
LINESTRING(0 0, 10 10, 20 25, 50 60)
POLYGON((0 0,10 0,10 10,0 10,0 0),(5 5,7 5,7 7,5 7, 5 5))
MULTIPOINT(0 0, 20 20, 60 60)
MULTILINESTRING((10 10, 20 20), (15 15, 30 15))
MULTIPOLYGON(((0 0,10 0,10 10,0 10,0 0)),((5 5,7 5,7 7,5 7, 5 5)))
GEOMETRYCOLLECTION(POINT(1 -1), POINT(10 10), POINT(30 30), LINEST
RING(15 15, 20 20))
```

WKB 是基于 OpenGIS 规范开发的，使用二进制数据来保存信息，支持 7 种类型（Point、Linestring、Polygon、Multipoint、Multilinestring、Multipolygon 和 Geometrycollection）以及对应格式定义，示例如下

```
0101000000000000000000F03F000000000000F0BF
```

表示平面坐标系下的点 (1,-1)，其中第一个字节标识了字节序，0是大端，1是小端；紧接着的四个字节表示数据类型，目前 GoldenDB 只支持上述七种数据类型（不包括 GEOMETRY），因此只使用了 1 到 7 来分别标识七种数据；因为点只有 x 坐标和 y 坐标，所以紧接着的 16 字节中前 8 字节是 x 坐标，后 8 字节是 y 坐标。 在 WKB 格式中，前五个字节是所有类型的 GIS 数据都具有的，而后续具体表示数据的内容就因类型而异了，复杂的数据类型有复杂的表达方式，具体可以参考 OpenGIS 标准。用户可以通过 ST_GeomFromWKB 等函数来将 WKB 格式的 GIS 数据转换为内部格式插入到系统中，在进行查询时可以使用 ST_AsWKB 等函数将内部数据转换成 WKB 结果格式。

建表

- 建带有空间列表的语句：

```
> CREATE TABLE spatial_data (
    id INT AUTO_INCREMENT PRIMARY KEY,
    location GEOMETRY NOT NULL SRID 4326,
    name VARCHAR(50)
);
```

- 空间数据类型使用限制：

1. 分布式场景下空间列不支持作为分区键。
2. 不支持在分区表中使用空间列。
3. 创建空间列时不指定 SRID 则使用 0 作为默认 SRID，表示无限平坦的笛卡尔平面，以及没有规定特定的物理单位也没有地理参考。
4. 空间列设置为 NOT NULL 时才能创建空间索引（SPATIAL 关键字指定）。
5. 空间索引默认使用 MBR 加速查询，空间列上可以创建空间索引，但是普通列上无法创建空间索引。
6. 对没有设置 SRID 的空间列创建空间索引，那么空间索引会失效。
7. 暂时仅支持二维数据。

插入数据

```
> INSERT INTO spatial_data (location)
VALUES (ST_GeomFromText('POINT(1 1)',4326));
> INSERT INTO spatial_data (location)
VALUES (ST_GeomFromText('POINT(0 1)', 4326));
> INSERT INTO spatial_data (location)
VALUES (ST_GeomFromText('LINESTRING(1 1,2 2,3 3,4 4,5.3 5.1,6 6,7
7,8 8)', 4326));
> INSERT INTO spatial_data (location)
VALUES (ST_GeomFromText('POLYGON((0 0,0 10,10 10,10 0,0 0))', 4326
));
> INSERT INTO spatial_data (location)
VALUES (ST_GeomFromText('POLYGON((3 3,3 7,7 7,7 3,3 3),(0 0,0 10,1
0 10,10 0,0 0))', 4326));
> INSERT INTO spatial_data (location)
VALUES (ST_GeomFromText('MULTIPOINT((1 1),(2 2),(3 3),(4 4),(5.3 5
.1),(6 6),(7 7),(8 8))', 4326));
> INSERT INTO spatial_data (location)
VALUES (ST_GeomFromText('MULTILINESTRING((0 0,1 1,2 2),(3 3,4 4,5
5))', 4326));
> INSERT INTO spatial_data (location)
VALUES (ST_GeomFromText('MULTIPOLYGON(((0 0,10 0,10 10,0 10,0 0),(
2 2,3 2,3 3,2 3,2 2),(7 7,8 7,8 8,7 8,7 7)),((20 20,30 20,30 30,20
 30,20 20),(22 22,23 22,23 23,22 23,22 22),(27 27,28 27,28 28,27 2
8,27 27)))', 4326));
> INSERT INTO spatial_data (location)
VALUES (ST_GeomFromText('geometrycollection(MULTILINESTRING((0 0,1
 1,2 2),(3 3,4 4,5 5),(6 6,7 7,8 8,9 9,10 10)), MULTIPOLYGON(((0 0
,10 0,10 10,0 10,0 0),(2 2,3 2,3 3,2 3,2 2),(7 7,8 7,8 8,7 8,7 7))
```

```
,((20 20,30 20,30 30,20 30,20 20),(22 22,23 22,23 23,22 23,22 22),
(27 27,28 27,28 28,27 28,27 27))))', 4326));
```

在空间列上创建空间索引

```
> ALTER TABLE spatial_data ADD SPATIAL INDEX location_index(locati
on);
```

常用GEOMETRY相关函数及其使用示例

- ST_GeomFromText(wkt [, srid [, options]]): 从 WKT 创建空间数据对象

```
> SELECT ST_GeomFromText('POINT(1 1)');
+-----------------------------------------------------------------
+
| ST_GeomFromText('POINT(1 1)')
|
+-----------------------------------------------------------------
+
| 0x000000000101000000000000000000F03F000000000000F03F
|
+-----------------------------------------------------------------
+
```

- ST_GeomFromWKB(wkb [, srid [, options]]): 从 WKB 创建空间数据对象

```
> SELECT ST_GeomFromWKB(0x0101000000000000000000F03F0000000000
00F03F);
+-----------------------------------------------------------------
-----------------------------------------------------------------+
| ST_GeomFromWKB(0x0101000000000000000000F03F000000000000F03F)
                                                                 |
+-----------------------------------------------------------------
-----------------------------------------------------------------+
| 0x000000000101000000000000000000F03F000000000000F03F
                                                                 |
+-----------------------------------------------------------------
-----------------------------------------------------------------
+
```

- ST_SRID(g [, srid]): 返回空间数据对象的空间参考系统标识符（SRID）

```
> SELECT ST_SRID(ST_GeomFromText('POINT(1 1)'));
+---------------------------------------+
| ST_SRID(ST_GeomFromText('POINT(1 1)')) |
+---------------------------------------+
|                                     0 |
+---------------------------------------+
```

- ST_IsEmpty(g): 检查空间数据对象是否为空，返回为 1 代表 true，为 0 代表 false

```
> SELECT ST_IsEmpty(ST_GeomFromText('GEOMETRYCOLLECTION()'));
+-----------------------------------------------------+
| ST_IsEmpty(ST_GeomFromText('GEOMETRYCOLLECTION()')) |
+-----------------------------------------------------+
|                                                   1 |
+-----------------------------------------------------+
```

- ST_IsSimple(g): 检查空间数据对象是否简单（不自相交），返回 1 代表 true，0 代表 false

```
> SELECT ST_IsSimple(ST_GeomFromText('LINESTRING(0 0, 1 1)'));
+-------------------------------------------------------+
| ST_IsSimple(ST_GeomFromText('LINESTRING(0 0, 1 1)')) |
+-------------------------------------------------------+
|                                                     1 |
+-------------------------------------------------------+
```

- ST_Contains(g1,g2): 检查空间数据对象 g1 是否包含另一个空间数据对象 g2，返回 1 代表 true，0 代表 false

```
> SELECT ST_Contains(
    ->     ST_GeomFromText('POLYGON((0 0, 0 2, 2 2, 2 0, 0 0))'
),
    ->     ST_GeomFromText('POINT(1 1)')
    -> );
+-----------------------------------------------------------------
-----------------------------------------------+
| ST_Contains(
    ST_GeomFromText('POLYGON((0 0, 0 2, 2 2, 2 0, 0 0))'),
    ST_GeomFromText('POINT(1 1)')
) |
+-----------------------------------------------------------------
-----------------------------------------------+
|
                                                 1 |
+-----------------------------------------------------------------
-----------------------------------------------+
```

- ST_Buffer(g, d [, strategy1 [, strategy2 [, strategy3]]]): 创建空间数据对象的缓冲区，单位采用空间数据类型自身坐标系的单位，可以添加 ST_Buffer_Strategy 函数指定策略

```
> SELECT ST_Buffer(ST_GeomFromText('point(1 1)'),10);
+-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------------------------------
```

```
+------------------------+
| ST_Buffer(ST_GeomFromText('point(1 1)'),10)
                                                
```

```
61437A24400208C225739B0740EB99FAE19E9D2540700000000000F03F00000
00000002640541E0897CC6DEEBFF099FAE19E9D254091535A505B9D06C05098
7F61437A2440EA021A080A3912C00C25A9D91FA12240B20A1F00C64818C0668
50F0063242040044A52B33F421DC008031A080A391A4049987F61437A20C0EA
292DA8AD4E1340ED99FAE19E9D21C0DC07C225739B0740000000000000022C02
20000000000F03FEF99FAE19E9D21C0EA1E0897CC6DEEBF4C987F61437A20C0
B4535A505B9D06C00C4A52B33F421DC0FA021A080A3912C0BF0A1F00C64818C
0C00A1F00C64818C0F8021A080A3912C00E4A52B33F421DC0BA535A505B9D06
C04C987F61437A20C0141F0897CC6DEEBFEE99FAE19E9D21C0030000000000F
03F00000000000022C0C807C225739B0740EE99FAE19E9D21C0DE292DA8AD4E
13404C987F61437A20C0FB021A080A391A400C4A52B33F421DC060850F00632
42040BF0A1F00C64818C00625A9D91FA12240FA021A080A3912C04C987F6143
7A2440BC535A505B9D06C0EE99FAE19E9D2540181F0897CC6DEEBF000000000
0002640000000000000F03F |
+-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------+
```

- ST_AsGML(g): 将空间数据对象转换为 GML 类型

```
> SELECT ST_AsGML(ST_GeomFromText('POINT(1 1)'));
+-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------+
| ST_AsGML(ST_GeomFromText('POINT(1 1)'))

                        |
+-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------+
| <gml:Point srsName="SDO:0" xmlns:gml="http://www.opengis.net/
gml"><gml:coordinates decimal="." cs="," ts=" ">1.0,1.0 </gml:c
oordinates></gml:Point> |
+-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------+
```

- ST_AsPointsJson(g): 提取空间数据对象的所有顶点坐标并返回为 json 数组格式

```
> SELECT ST_AsPointsJson(ST_GeomFromText('LINESTRING(0 0, 1 1,
 2 2)'));
+-----------------------------------------------------------------
-+
| ST_AsPointsJson(ST_GeomFromText('LINESTRING(0 0, 1 1, 2 2)'))
 |
+-----------------------------------------------------------------
-+
| [[0.0,0.0],[1.0,1.0],[2.0,2.0]]
 |
+-----------------------------------------------------------------
-+
```

- ST_AsText(g [, options]): 将空间数据对象转化为 WKT 格式

```
> SELECT ST_AsText(ST_GeomFromText('POINT(1 1)'));
+-----------------------------------------+
| ST_AsText(ST_GeomFromText('POINT(1 1)')) |
+-----------------------------------------+
| POINT(1 1)                               |
+-----------------------------------------+
```

- ST_AsBinary(g [, options]): 将空间数据对象转换为 WKB 二进制格式

```
> SELECT ST_AsBinary(ST_GeomFromText('POINT(1 1)'));
+-----------------------------------------------------------------
-------------------------+
| ST_AsBinary(ST_GeomFromText('POINT(1 1)'))
                          |
+-----------------------------------------------------------------
-------------------------+
| 0x0101000000000000000000F03F000000000000F03F
                          |
+-----------------------------------------------------------------
-------------------------+
```

- ST_Within(g1,g2): 判断第一个空间数据对象是否被第二个空间数据对象包含，返回 1 代表 true，0 代表 false

```
> SELECT ST_Within(
    ->     ST_GeomFromText('POINT(1 1)'),
    ->     ST_GeomFromText('POLYGON((0 0, 0 2, 2 2, 2 0, 0 0))'
)
    -> );
+-----------------------------------------------------------------
-----------------------------------------+
```

```
| ST_Within(
    ST_GeomFromText('POINT(1 1)'),
    ST_GeomFromText('POLYGON((0 0, 0 2, 2 2, 2 0, 0 0))')
) |
+-----------------------------------------------------------------
-----------------------------------------+
|
                                                  1 |
+-----------------------------------------------------------------
-----------------------------------------+
```

- ST_Touches(g1,g2): 检查两个空间数据对象是否仅在边界处相接触，返回 1 代表 true，0 代表 false

```
> SELECT ST_Touches(
    ->   ST_GeomFromText('POINT(1 1)'),
    ->   ST_GeomFromText('LINESTRING(1 1, 2 2)')
    -> );
+-----------------------------------------------------------------
---------------------------+
| ST_Touches(
  ST_GeomFromText('POINT(1 1)'),
  ST_GeomFromText('LINESTRING(1 1, 2 2)')
) |
+-----------------------------------------------------------------
---------------------------+
|
                           1 |
+-----------------------------------------------------------------
---------------------------+
```

- ST_Overlaps(g1,g2): 检查两个空间数据对象是否部分重叠且不能是完全包含或者完全分离，返回 1 代表 true，0 代表 false

```
> SELECT ST_Overlaps(
    ->   ST_GeomFromText('POLYGON((0 0, 5 0, 5 5, 0 5, 0 0))'),
    ->   ST_GeomFromText('POLYGON((3 3, 7 3, 7 7, 3 7, 3 3))')
    -> );
+-----------------------------------------------------------------
-----------------------------------------------------------------
----+
| ST_Overlaps(
  ST_GeomFromText('POLYGON((0 0, 5 0, 5 5, 0 5, 0 0))'),
  ST_GeomFromText('POLYGON((3 3, 7 3, 7 7, 3 7, 3 3))')
) |
+-----------------------------------------------------------------
-----------------------------------------------------------------
----+
|
```

```
  1 |
+-----------------------------------------------------------------
-----------------------------------------------------------------
----+
```

- ST_Intersects(g1,g2): 判断两个空间数据对象是否相交，返回 1 代表 true，0 代表 false

```
> SELECT ST_Intersects(
    ->     ST_GeomFromText('POINT(1 1)'),
    ->     ST_GeomFromText('POLYGON((0 0, 0 2, 2 2, 2 0, 0 0))'
)
    -> );
+-----------------------------------------------------------------
-----------------------------------------------+
| ST_Intersects(
    ST_GeomFromText('POINT(1 1)'),
    ST_GeomFromText('POLYGON((0 0, 0 2, 2 2, 2 0, 0 0))')
) |
+-----------------------------------------------------------------
-----------------------------------------------+
|
                                                 1 |
+-----------------------------------------------------------------
-----------------------------------------------+
```

- ST_NumPoints(ls): 返回线段空间数据对象中顶点的数量

```
> SELECT ST_NumPoints(ST_GeomFromText('LINESTRING(0 0, 1 1, 2
2)'));
+-------------------------------------------------------------+
| ST_NumPoints(ST_GeomFromText('LINESTRING(0 0, 1 1, 2 2)')) |
+-------------------------------------------------------------+
|                                                           3 |
+-------------------------------------------------------------+
```

- ST_Length(ls [, unit]): 计算线段、多条线段空间数据对象的长度

```
> SELECT ST_Length(ST_GeomFromText('LINESTRING(0 0, 1 1, 2 2)'
));
+-------------------------------------------------------------+
| ST_Length(ST_GeomFromText('LINESTRING(0 0, 1 1, 2 2)')) |
+-------------------------------------------------------------+
|                                  2.8284271247461903 |
+-------------------------------------------------------------+
```

- ST_Area({poly mpoly}): 计算多边形空间数据对象的面积

```
> SELECT ST_Area(ST_GeomFromText('POLYGON((0 0, 0 5, 5 5, 5 0,
```

```
 0 0))'));
+-----------------------------------------------------------------
--+
| ST_Area(ST_GeomFromText('POLYGON((0 0, 0 5, 5 5, 5 0, 0 0))')
) |
+-----------------------------------------------------------------
--+
|                                                                2
5 |
+-----------------------------------------------------------------
--+
```

- ST_Centroid({poly mpoly}): 返回空间数据对象的中心点

```
> SELECT ST_AsText(ST_Centroid(ST_GeomFromText('POLYGON((0 0,
0 4, 4 4, 4 0, 0 0))')));
+-----------------------------------------------------------------
----------------+
| ST_AsText(ST_Centroid(ST_GeomFromText('POLYGON((0 0, 0 4, 4 4
, 4 0, 0 0))'))) |
+-----------------------------------------------------------------
----------------+
| POINT(2 2)
                 |
+-----------------------------------------------------------------
----------------+
```

- ST_Union({poly mpoly}): 返回两个空间数据对象的并集

```
> SELECT ST_AsText(ST_Union(ST_GeomFromText('POINT(0 0)'), ST_
GeomFromText('POINT(1 1)')));
+-----------------------------------------------------------------
--------------------+
| ST_AsText(ST_Union(ST_GeomFromText('POINT(0 0)'), ST_GeomFrom
Text('POINT(1 1)'))) |
+-----------------------------------------------------------------
--------------------+
| MULTIPOINT((0 0),(1 1))
                     |
+-----------------------------------------------------------------
--------------------+
```

- ST_Difference(g1,g2): 返回两个空间数据对象的差集

```
> SELECT ST_AsText(ST_Difference(ST_GeomFromText('POLYGON((0 0
, 0 2, 2 2, 2 0, 0 0))'), ST_GeomFromText('POLYGON((1 1, 1 3, 3
 3, 3 1, 1 1))')));
+-----------------------------------------------------------------
-----------------------------------------------------------------
```

```
-----------+
| ST_AsText(ST_Difference(ST_GeomFromText('POLYGON((0 0, 0 2, 2
 2, 2 0, 0 0))'), ST_GeomFromText('POLYGON((1 1, 1 3, 3 3, 3 1,
 1 1))'))) |
+-----------------------------------------------------------------
-----------------------------------------------------------------
-----------+
| POLYGON((1 2,0 2,0 0,2 0,2 1,1 1,1 2))

           |
+-----------------------------------------------------------------
-----------------------------------------------------------------
-----------+
```

- GeometryCollection(wkt [, srid [, options]]): 创建空间数据集合

```
> SELECT ST_AsText(GeometryCollection(Point(1, 1), LineString(
Point(0, 0), Point(1, 1))));
+-----------------------------------------------------------------
--------------------+
| ST_AsText(GeometryCollection(Point(1, 1), LineString(Point(0,
 0), Point(1, 1)))) |
+-----------------------------------------------------------------
--------------------+
| GEOMETRYCOLLECTION(POINT(1 1),LINESTRING(0 0,1 1))
                    |
+-----------------------------------------------------------------
--------------------+
```

- ST_NumGeometries(gc): 返回空间数据集合中空间数据对象的数量

```
> SELECT ST_NumGeometries(ST_GeomFromText('GEOMETRYCOLLECTION(
POINT(1 1), POINT(2 2))'));
+-----------------------------------------------------------------
-------------------+
| ST_NumGeometries(ST_GeomFromText('GEOMETRYCOLLECTION(POINT(1
1), POINT(2 2))')) |
+-----------------------------------------------------------------
-------------------+
|
                  2 |
+-----------------------------------------------------------------
-------------------+
```

- ST_Dimension(g): 返回空间数据对象的维度（0 = 点，1 = 线，2 = 面）

```
> SELECT ST_Dimension(ST_GeomFromText('POLYGON((0 0, 0 5, 5 5,
 5 0, 0 0))'));
+-----------------------------------------------------------------
```

```
-------+
| ST_Dimension(ST_GeomFromText('POLYGON((0 0, 0 5, 5 5, 5 0, 0
0))')) |
+-------------------------------------------------------------
-------+
|
     2 |
+-------------------------------------------------------------
-------+
```

- MBRContains(g1,g2): 判断一个空间数据对象的 MBR (最小外接矩形)是否包含另一个空间数据对象的 MBR (最小外接矩形)，返回 1 代表true，0 代表 false

```
> SELECT MBRContains(ST_GeomFromText('POLYGON((0 0, 0 5, 5 5,
5 0, 0 0))'), ST_GeomFromText('POINT(1 1)'));
+-------------------------------------------------------------
-----------------------------------+
| MBRContains(ST_GeomFromText('POLYGON((0 0, 0 5, 5 5, 5 0, 0 0
))'), ST_GeomFromText('POINT(1 1)')) |
+-------------------------------------------------------------
-----------------------------------+
|
                                  1 |
+-------------------------------------------------------------
-----------------------------------+
```

- MBRCoveredBy(g1,g2): 判断一个空间数据对象的 MBR 是否被另一个对象的 MBR 覆盖，返回 1 代表 true，0 代表 false

```
> SELECT MBRCoveredBy(ST_GeomFromText('POINT(1 1)'), ST_GeomFr
omText('POLYGON((0 0, 0 2, 2 2, 2 0, 0 0))')
);
+-------------------------------------------------------------
---------------------------------------+
| MBRCoveredBy(ST_GeomFromText('POINT(1 1)'), ST_GeomFromText('
POLYGON((0 0, 0 2, 2 2, 2 0, 0 0))') ) |
+-------------------------------------------------------------
---------------------------------------+
|
                                    1 |
+-------------------------------------------------------------
---------------------------------------+
```

- MBRCovers(g1,g2): 判断一个空间数据对象的 MBR 是否覆盖另一个对象的 MBR

```
> SELECT MBRCovers(ST_GeomFromText('POLYGON((0 0, 0 2, 2 2, 2
0, 0 0))'),ST_GeomFromText('POINT(1 1)') );  +----------------
-------------------------------------------------------------
-----------------+
```

```
| MBRCovers(ST_GeomFromText('POLYGON((0 0, 0 2, 2 2, 2 0, 0 0))
'),ST_GeomFromText('POINT(1 1)') ) |
+-----------------------------------------------------------------
---------------------------------+
|
                                  1 |
+-----------------------------------------------------------------
---------------------------------+
```

- MBRDisjoint(g1,g2): 判断两个空间数据对象的 MBR 是否互不相交

```
> SELECT MBRDisjoint(ST_GeomFromText('POLYGON((0 0, 0 1, 1 1,
1 0, 0 0))'),ST_GeomFromText('POLYGON((2 2, 2 3, 3 3, 3 2, 2 2)
)') );
+-----------------------------------------------------------------
-----------------------------------------------------------+
| MBRDisjoint(ST_GeomFromText('POLYGON((0 0, 0 1, 1 1, 1 0, 0 0
))'),ST_GeomFromText('POLYGON((2 2, 2 3, 3 3, 3 2, 2 2))') ) |
+-----------------------------------------------------------------
-----------------------------------------------------------+
|
                                                          1 |
+-----------------------------------------------------------------
-----------------------------------------------------------+
```

- MBREquals(g1,g2): 判断两个空间数据对象的 MBR 是否相等

```
> SELECT MBREquals(ST_GeomFromText('POLYGON((0 0, 0 1, 1 1, 1
0, 0 0))'),ST_GeomFromText('POLYGON((0 0, 0 1, 1 1, 1 0, 0 0))'
) );
+-----------------------------------------------------------------
-----------------------------------------------------------+
| MBREquals(ST_GeomFromText('POLYGON((0 0, 0 1, 1 1, 1 0, 0 0))
'),ST_GeomFromText('POLYGON((0 0, 0 1, 1 1, 1 0, 0 0))') ) |
+-----------------------------------------------------------------
-----------------------------------------------------------+
|
                                                        1 |
+-----------------------------------------------------------------
-----------------------------------------------------------+
```

- MBRIntersects(g1,g2): 判断两个空间数据对象的 MBR 是否相交

```
> SELECT MBRIntersects(ST_GeomFromText('POLYGON((0 0, 0 2, 2 2
, 2 0, 0 0))'),ST_GeomFromText('LINESTRING(1 1, 3 3)')
    -> );
+-----------------------------------------------------------------
-----------------------------------------------+
| MBRIntersects(ST_GeomFromText('POLYGON((0 0, 0 2, 2 2, 2 0, 0
```

```
 0))'),ST_GeomFromText('LINESTRING(1 1, 3 3)')
) |
+-----------------------------------------------------------------
-----------------------------------------------+
|
                                                              1 |
+-----------------------------------------------------------------
-----------------------------------------------+
```

- MBROverlaps(g1,g2): 判断两个空间数据对象的 MBR 是否部分重叠

```
> SELECT MBROverlaps(ST_GeomFromText('POLYGON((0 0, 0 2, 2 2,
2 0, 0 0))'),ST_GeomFromText('POLYGON((1 1, 1 3, 3 3, 3 1, 1 1)
)') );
+-----------------------------------------------------------------
-------------------------------------------------------------+
| MBROverlaps(ST_GeomFromText('POLYGON((0 0, 0 2, 2 2, 2 0, 0 0
))'),ST_GeomFromText('POLYGON((1 1, 1 3, 3 3, 3 1, 1 1))') ) |
+-----------------------------------------------------------------
-------------------------------------------------------------+
|
                                                           1 |
+-----------------------------------------------------------------
-------------------------------------------------------------+
```

- MBRTouches(g1,g2): 判断两个空间数据对象的 MBR 是否接触

```
> SELECT MBRTouches(ST_GeomFromText('POLYGON((0 0, 0 1, 1 1, 1
 0, 0 0))'),ST_GeomFromText('POLYGON((1 0, 1 1, 2 1, 2 0, 1 0))
')
    -> );
+-----------------------------------------------------------------
-------------------------------------------------------------+
| MBRTouches(ST_GeomFromText('POLYGON((0 0, 0 1, 1 1, 1 0, 0 0)
)'),ST_GeomFromText('POLYGON((1 0, 1 1, 2 1, 2 0, 1 0))')
) |
+-----------------------------------------------------------------
-------------------------------------------------------------+
|
                                                             1 |
+-----------------------------------------------------------------
-------------------------------------------------------------+
```

- MBRWithin(g1,g2): 判断一个空间数据对象的 MBR 是否完全位于另一个对象的 MBR 内

```
> SELECT MBRWithin(ST_GeomFromText('POLYGON((1 1, 1 2, 2 2, 2
1, 1 1))'),ST_GeomFromText('POLYGON((0 0, 0 3, 3 3, 3 0, 0 0))'
)
    -> );
```

```
+-------------------------------------------------------------------
-------------------------------------------------------------+
| MBRWithin(ST_GeomFromText('POLYGON((1 1, 1 2, 2 2, 2 1, 1 1))
'),ST_GeomFromText('POLYGON((0 0, 0 3, 3 3, 3 0, 0 0))')
) |
+-------------------------------------------------------------------
-------------------------------------------------------------+
|
                                                            1 |
+-------------------------------------------------------------------
-------------------------------------------------------------+
```

- ST_Latitude(p [, new_latitude_val]): 一个参数时返回 Point 空间数据类型的纬度值，两个参数时设置一个点的纬度值

```
> SELECT ST_Latitude(ST_GeomFromText('POINT(10 20)',4326));
+-----------------------------------------------------+
| ST_Latitude(ST_GeomFromText('POINT(10 20)',4326)) |
+-----------------------------------------------------+
|                                                  10 |
+-----------------------------------------------------+
```

- ST_Longitude(p [, new_longitude_val]): 一个参数时返回 Point 空间数据类型的经度值，两个参数时色湖之一个点的经度值

```
> SELECT ST_Longitude(ST_GeomFromText('POINT(10 20)',4326));
+-----------------------------------------------------+
| ST_Longitude(ST_GeomFromText('POINT(10 20)',4326)) |
+-----------------------------------------------------+
|                                                  20 |
+-----------------------------------------------------+
```

- ST_LineInterpolatePoint(ls,fractional_distance): 返回线段上指定百分比位置的点，0.5 代表 50%

```
> SELECT ST_AsText(ST_LineInterpolatePoint(ST_GeomFromText('LI
NESTRING(0 0, 10 10)'), 0.5));
+-------------------------------------------------------------------
----------------------+
| ST_AsText(ST_LineInterpolatePoint(ST_GeomFromText('LINESTRING
(0 0, 10 10)'), 0.5)) |
+-------------------------------------------------------------------
----------------------+
| POINT(5 5)
                      |
+-------------------------------------------------------------------
----------------------+
```

- ST_LineInterpolatePoints(ls,fractional_distance): 返回线段上按百分比间隔的多个点

```
> SELECT ST_AsText(ST_LineInterpolatePoints(ST_GeomFromText('L
INESTRING(0 0, 10 10)'), 0.2));
+-----------------------------------------------------------------
----------------------+
| ST_AsText(ST_LineInterpolatePoints(ST_GeomFromText('LINESTRIN
G(0 0, 10 10)'), 0.2)) |
+-----------------------------------------------------------------
----------------------+
| MULTIPOINT((2 2),(4 4),(6 6),(8 8),(10 10))
                       |
+-----------------------------------------------------------------
----------------------+
```

- ST_PointAtDistance(ls,distance): 返回线段上指定距离的点

```
> SELECT ST_AsText(ST_PointAtDistance(ST_GeomFromText('LINESTR
ING(0 0, 10 10)'), 5));
+-----------------------------------------------------------------
---------------+
| ST_AsText(ST_PointAtDistance(ST_GeomFromText('LINESTRING(0 0,
 10 10)'), 5)) |
+-----------------------------------------------------------------
---------------+
| POINT(3.5355339059327373 3.5355339059327373)
               |
+-----------------------------------------------------------------
---------------+
```

- ST_NumInteriorRings(poly): 返回多边形的内环数量

```
> SELECT ST_NumInteriorRings(ST_GeomFromText('POLYGON((0 0, 10
 0, 10 10, 0 10, 0 0), (2 2, 8 2, 8 8, 2 8, 2 2))'));
+-----------------------------------------------------------------
-------------------------------------------+
| ST_NumInteriorRings(ST_GeomFromText('POLYGON((0 0, 10 0, 10 1
0, 0 10, 0 0), (2 2, 8 2, 8 8, 2 8, 2 2))')) |
+-----------------------------------------------------------------
-------------------------------------------+
|
                                           1 |
+-----------------------------------------------------------------
-------------------------------------------+
```

- ST_Validate(g): 检查空间数据是否有效

```
> SELECT ST_AsText(ST_Validate(ST_GeomFromText('POINT(1 1)')))
;
```

```
+-----------------------------------------------------+
| ST_AsText(ST_Validate(ST_GeomFromText('POINT(1 1)'))) |
+-----------------------------------------------------+
| POINT(1 1)                                          |
+-----------------------------------------------------+
```

- ST_StartPoint(ls): 返回线段类型的起始点

```
> SELECT ST_StartPoint(ST_GeomFromText('LINESTRING(0 0, 10 10,
 20 20)'));
+-----------------------------------------------------------------
-----------------------------------------------------------------
-----+
| ST_StartPoint(ST_GeomFromText('LINESTRING(0 0, 10 10, 20 20)'
))
     |
+-----------------------------------------------------------------
-----------------------------------------------------------------
-----+
| 0x000000000101000000000000000000000000000000000000
     |
+-----------------------------------------------------------------
-----------------------------------------------------------------
-----+
```

- ST_SwapXY(g): 返回其 X 和 Y 坐标值交换后的输入空间数据对象

```
> SELECT ST_AsText(ST_SwapXY(ST_GeomFromText('POINT(10 20)')))
;
+-----------------------------------------------------+
| ST_AsText(ST_SwapXY(ST_GeomFromText('POINT(10 20)'))) |
+-----------------------------------------------------+
| POINT(20 10)                                        |
+-----------------------------------------------------+
```

- ST_SymDifference(g1,g2): 返回两个空间数据对象的对称差集（即它们各自独占的部分，而不包括交集）

```
> SELECT ST_AsText(ST_SymDifference(ST_GeomFromText('POLYGON((
0 0, 10 0, 10 10, 0 10, 0 0))'),ST_GeomFromText('POLYGON((5 5,
15 5, 15 15, 5 15, 5 5))')));
+-----------------------------------------------------------------
-----------------------------------------------------------------
--------------------+
| ST_AsText(ST_SymDifference(ST_GeomFromText('POLYGON((0 0, 10
0, 10 10, 0 10, 0 0))'),ST_GeomFromText('POLYGON((5 5, 15 5, 15
 15, 5 15, 5 5))'))) |
+-----------------------------------------------------------------
```

```
-----------------------------------------------------------------
--------------------+
| MULTIPOLYGON(((10 5,5 5,5 10,0 10,0 0,10 0,10 5)),((10 5,15 5
,15 15,5 15,5 10,10 10,10 5)))
                     |
+-----------------------------------------------------------------
-----------------------------------------------------------------
--------------------+
```

- ST_Transform(g,target_srid): 将空间数据对象的坐标从一个空间参考系（SRID）转换到另一个

```
> SELECT ST_Transform(ST_GeomFromText('POINT(10 20)',4326),381
9);
+-----------------------------------------------------------------
-----------------------------------------------------+
| ST_Transform(ST_GeomFromText('POINT(10 20)',4326),3819)
                                                     |
+-----------------------------------------------------------------
-----------------------------------------------------+
| 0xEB0E00000101000000B41840FC2E0034403D45619E6EFD2340
                                                     |
+-----------------------------------------------------------------
-----------------------------------------------------+
```

- ST_X(p [, new_x_val]): 一个参数时返回 Point 空间数据对象的 X 坐标值，两个参数时设置 Point 空间数据对象的 X 坐标值

```
> SELECT ST_X(ST_GeomFromText('POINT(10 20)'));
+---------------------------------------+
| ST_X(ST_GeomFromText('POINT(10 20)')) |
+---------------------------------------+
|                                    10 |
+---------------------------------------+
```

- ST_Y(p [, new_x_val]): 一个参数时返回 Point 空间数据对象的 Y 坐标值，两个参数时设置 Point 空间数据对象的 Y 坐标值

```
> SELECT ST_Y(ST_GeomFromText('POINT(10 20)'));
+---------------------------------------+
| ST_Y(ST_GeomFromText('POINT(10 20)')) |
+---------------------------------------+
|                                    20 |
+---------------------------------------+
```

- ST_AsGeoJSON(g [, max_dec_digits [, options]]): 将空间数据以 GeoJSON 格式的字符串表示(该函数暂时分布式不支持，集中式支持)

```
> SELECT ST_AsGeoJSON(ST_GeomFromText('POINT(1 2)'));
+-------------------------------------------+
| ST_AsGeoJSON(ST_GeomFromText('POINT(1 2)')) |
+-------------------------------------------+
| {"type": "Point", "coordinates": [1.0, 2.0]} |
+-------------------------------------------+
```

- ST_Buffer_Strategy(strategy [, points_per_circle]): 生成 ST_Buffer 需要的缓冲策略，包括缓冲区形状 point_circle、point_square，缓冲区端点可用的策略 end_round、end_flat、join_round、join_miter 以及设置构成缓冲区的顶点数量

```
> SELECT ST_Buffer(ST_GeomFromText('POINT(1 1)'),2,ST_Buffer_S
trategy('point_circle', 5));
+-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------+
| st_buffer(st_geomfromtext('POINT(1 1)'),2,ST_Buffer_Strategy(
'point_circle', 5))

                                          |
+-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------+
| 0x000000000103000000010000000600000000000000000008400000000000
000F03FA6F4979B77E3F93F802AA2098737074052E92F37EFC6E3BF2EAD3A82
916701404EE92F37EFC6E3BFF8D2AA231879C6BFA8F4979B77E3F93FFEA9882
61CDEECBF0000000000000840000000000000F03F |
+-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------------------------------
-----------------------------------------+
```

- ST_ConvexHull(g): 获取输入空间数据对象的凸包

```
> SELECT ST_AsText(ST_ConvexHull(ST_GeomFromText('MULTIPOINT((
1 1), (2 2), (3 3), (4 1))')));
+-----------------------------------------------------------------
----------------------+
| ST_AsText(ST_ConvexHull(ST_GeomFromText('MULTIPOINT((1 1), (2
 2), (3 3), (4 1))'))) |
+-----------------------------------------------------------------
----------------------+
| POLYGON((1 1,4 1,3 3,1 1))
                       |
+-----------------------------------------------------------------
----------------------+
```

- ST_Crosses(g1,g2): 判断两个空间数据对象是否有交差关系

```
> SELECT ST_Crosses(
    ->     ST_GeomFromText('LINESTRING(0 0, 1 1)'),
    ->     ST_GeomFromText('LINESTRING(2 2, 3 3)')
    -> );
+-----------------------------------------------------------------
----------------------------------------+
| ST_Crosses(
    ST_GeomFromText('LINESTRING(0 0, 1 1)'),
    ST_GeomFromText('LINESTRING(2 2, 3 3)')
) |
+-----------------------------------------------------------------
----------------------------------------+
|
                                        0 |
+-----------------------------------------------------------------
----------------------------------------+
```

- ST_FrechetDistance(g1,g2): 计算两个空间数据对象之间的 FrechetDistance

```
> SELECT ST_FrechetDistance(
    ->     ST_GeomFromText('LINESTRING(0 0, 1 1, 2 2)'),
    ->     ST_GeomFromText('LINESTRING(0 0, 1 1.1, 2 2)')
    -> );
+-----------------------------------------------------------------
----------------------------------------------------------------+
| ST_FrechetDistance(
    ST_GeomFromText('LINESTRING(0 0, 1 1, 2 2)'),
    ST_GeomFromText('LINESTRING(0 0, 1 1.1, 2 2)')
) |
+-----------------------------------------------------------------
----------------------------------------------------------------+
|
                                    0.10000000000000009 |
+-----------------------------------------------------------------
----------------------------------------------------------------
+
```

- ST_Envelope(g): 计算空间数据对象的最小边界矩形

```
> SELECT ST_AsText(ST_Envelope(ST_GeomFromText('POLYGON((1 1,
2 3, 3 1, 1 1))')));
+-----------------------------------------------------------------
------------+
| ST_AsText(ST_Envelope(ST_GeomFromText('POLYGON((1 1, 2 3, 3 1
, 1 1))'))) |
+-----------------------------------------------------------------
------------+
| POLYGON((1 1,3 1,3 3,1 3,1 1))
```

```
            |
+-------------------------------------------------------------------
------------+
```

- ST_Distance_Sphere(g1, g2 [, radius]): 计算地球上两个空间数据类型（适用于点 point 和多点 multipoint 对象）之间的最小球面距离

```
> SELECT ST_Distance_Sphere(
    ->     ST_GeomFromText('POINT(116.4074 39.9042)'), -- 北京
    ->     ST_GeomFromText('POINT(121.4737 31.2304)')  -- 上海
    -> );
+-------------------------------------------------------------------
-----------------------------------------------------------+
| ST_Distance_Sphere(
    ST_GeomFromText('POINT(116.4074 39.9042)'),
    ST_GeomFromText('POINT(121.4737 31.2304)')
) |
+-------------------------------------------------------------------
-----------------------------------------------------------+
|
                                          1067307.8255586782 |
+-------------------------------------------------------------------
-----------------------------------------------------------+
```

- ST_Equals(g1,g2): 计算两个空间数据类型是否相等

```
> SELECT ST_Equals(
    ->     ST_GeomFromText('POINT(1 1)'),
    ->     ST_GeomFromText('POINT(1 1)')
    -> );
+-------------------------------------------------------------------
---------------------+
| ST_Equals(
    ST_GeomFromText('POINT(1 1)'),
    ST_GeomFromText('POINT(1 1)')
) |
+-------------------------------------------------------------------
---------------------+
|
                   1 |
+-------------------------------------------------------------------
---------------------+
```

- ST_GeoHash(longitude, latitude, max_length) / ST_GeoHash(point, max_length): 用于将点转换为 GeoHash 格式的字符串（需要指定hash值长度）

```
> SELECT ST_GeoHash(ST_GeomFromText('POINT(116 39)'),10);
+-----------------------------------------------+
| ST_GeoHash(ST_GeomFromText('POINT(116 39)'),10) |
```

```
+-----------------------------------------------+
| wwfmzesx7y                                    |
+-----------------------------------------------+
```

- ST_EndPoint(ls): 提取线段的终点

```
> SELECT ST_AsText(ST_EndPoint(ST_GeomFromText('LINESTRING(0 0
, 1 1, 2 2)')));
+-----------------------------------------------------------------
--------+
| ST_AsText(ST_EndPoint(ST_GeomFromText('LINESTRING(0 0, 1 1, 2
 2)'))) |
+-----------------------------------------------------------------
--------+
| POINT(2 2)
        |
+-----------------------------------------------------------------
--------+
```

- ST_ExteriorRing(poly): 用于返回多边形的外环

```
> SELECT ST_AsText(ST_ExteriorRing(ST_GeomFromText('POLYGON((0
 0, 0 2, 2 2, 2 0, 0 0))')));
+-----------------------------------------------------------------
--------------------+
| ST_AsText(ST_ExteriorRing(ST_GeomFromText('POLYGON((0 0, 0 2,
 2 2, 2 0, 0 0))'))) |
+-----------------------------------------------------------------
--------------------+
| LINESTRING(0 0,0 2,2 2,2 0,0 0)
                      |
+-----------------------------------------------------------------
--------------------+
```

- ST_GeometryN(gc,N): 用于从空间数据集合（如 GEOMETRYCOLLECTION、MULTIPOINT、MULTILINESTRING 或 MULTIPOLYGON）中提取指定的第 N 个空间数据对象

```
> SELECT ST_AsText(ST_GeometryN(ST_GeomFromText('MULTIPOINT(1
1, 2 2, 3 3)'), 2));
+-----------------------------------------------------------------
------------+
| ST_AsText(ST_GeometryN(ST_GeomFromText('MULTIPOINT(1 1, 2 2,
3 3)'), 2)) |
+-----------------------------------------------------------------
------------+
| POINT(2 2)
            |
+-----------------------------------------------------------------
```

```
------------+
```

- ST_GeometryType(g): 返回输入空间数据对象的类型

```
> SELECT ST_GeometryType(ST_GeomFromText('POINT(1 1)'));
+-----------------------------------------------+
| ST_GeometryType(ST_GeomFromText('POINT(1 1)')) |
+-----------------------------------------------+
| POINT                                         |
+-----------------------------------------------+
```

- ST_HausdorffDistance(g1,g2 [, unit]): 用于计算两空间数据对象之间的 Hausdorff 距离

```
> SELECT ST_HausdorffDistance(
    ->     ST_GeomFromText('LINESTRING(0 0, 1 1, 2 2)'),
    ->     ST_GeomFromText('LINESTRING(0 0, 2 1, 3 3)')
    -> );
+-------------------------------------------------------------------
-------------------------------------------------------------------+
| ST_HausdorffDistance(
    ST_GeomFromText('LINESTRING(0 0, 1 1, 2 2)'),
    ST_GeomFromText('LINESTRING(0 0, 2 1, 3 3)')
) |
+-------------------------------------------------------------------
-------------------------------------------------------------------+
|
                                                                  1 |
+-------------------------------------------------------------------
-------------------------------------------------------------------
+
```

- ST_InteriorRingN(poly,N): 用于从多边形（POLYGON）中提取第 N 个内环（Interior Ring）

```
> SELECT ST_AsText(ST_InteriorRingN(ST_GeomFromText(
    ->   'POLYGON((0 0, 5 0, 5 5, 0 5, 0 0), (1 1, 4 1, 4 4, 1
4, 1 1))'
    -> ), 1));
+-------------------------------------------------------------------
-----------------------------------------------------+
| ST_AsText(ST_InteriorRingN(ST_GeomFromText(
  'POLYGON((0 0, 5 0, 5 5, 0 5, 0 0), (1 1, 4 1, 4 4, 1 4, 1 1)
)'
), 1)) |
+-------------------------------------------------------------------
-----------------------------------------------------+
| LINESTRING(1 1,4 1,4 4,1 4,1 1)
                                                                |
+-------------------------------------------------------------------
```

```
-----------------------------------------------------+
```

- ST_IsClosed({ls mls}): 判断线段或者多线段空间数据对象是否是闭合的

```
> SELECT ST_IsClosed(ST_GeomFromText('LINESTRING(0 0, 1 1, 2 2
, 0 0)'));
+-----------------------------------------------------------
--+
| ST_IsClosed(ST_GeomFromText('LINESTRING(0 0, 1 1, 2 2, 0 0)')
) |
+-----------------------------------------------------------
--+
|
1 |
+-----------------------------------------------------------
--+
```

- ST_LatFromGeoHash(geohash_str): 用于从 GeoHash 编码中提取纬度值

```
> SELECT
    ->     ST_LatFromGeoHash('wx4g0'),
    ->     ST_LatFromGeoHash('wx4g09'),
    ->     ST_LatFromGeoHash('wx4g09z3');
+----------------------------+----------------------------+---
----------------------------+
| ST_LatFromGeoHash('wx4g0') | ST_LatFromGeoHash('wx4g09') | ST
_LatFromGeoHash('wx4g09z3') |
+----------------------------+----------------------------+---
----------------------------+
|                      39.92 |                      39.91 |
                   39.9122 |
+----------------------------+----------------------------+---
----------------------------+
```

- ST_LongFromGeoHash(geohash_str): 用于从 GeoHash 编码中提取经度值

```
> SELECT
    ->     ST_LongFromGeoHash('wx4g0'),
    ->     ST_LongFromGeoHash('wx4g09'),
    ->     ST_LongFromGeoHash('wx4g09z3');
+-----------------------------+-----------------------------+-
-----------------------------+
| ST_LongFromGeoHash('wx4g0') | ST_LongFromGeoHash('wx4g09') |
ST_LongFromGeoHash('wx4g09z3') |
+-----------------------------+-----------------------------+-
-----------------------------+
|                       116.4 |                       116.4 |
                    116.3993 |
+-----------------------------+-----------------------------+-
```

```
------------------------------+
```

- ST_MakeEnvelope(pt1, pt2): 基于两个点 pt1 和 pt2 创建 MBR。它返回的结果可以是点（POINT）、线段（LINESTRING）或多边形（POLYGON），具体取决于这两个点的关系

```
> SET @pt1 = ST_GeomFromText('POINT(0 0)');
Query OK, 0 rows affected (0.00 sec)

> SET @pt2 = ST_GeomFromText('POINT(0 1)');
Query OK, 0 rows affected (0.00 sec)

> SELECT ST_AsText(ST_MakeEnvelope(@pt1, @pt2));
+---------------------------------------+
| ST_AsText(ST_MakeEnvelope(@pt1, @pt2)) |
+---------------------------------------+
| LINESTRING(0 0,0 1)                   |
+---------------------------------------+
```

1.10.4. OPERATORS

操作符

运算符	说明
-	减号
-	负号
=	等于操作
% MOD	取余
<=>	等于操作
>	大于
>=	大于等于
!=	不等于
<>	不等于

*	乘
/	除
||,OR	或
+	加
<=	小于等于
<	小于
AND, &&	与
BETWEEN ... AND ...	范围判断
DIV	除
IS	是判断
IS NOT	非判断
IS NOT NULL	判断不为null
IS NULL	判断null
LIKE	LIKE函数
NOT BETWEEN ... AND ...	范围判断
NOT LIKE	LIKE函数
NOT,!	非
XOR	异或
||	连接符

1.10.5. FUNCTIONS

1.10.5.1. GoldenDB 支持函数列表

函数说明表

函数名	说明
ABS	绝对值
AVG	平均值
CASE WHEN	CASE操作
CAST	类型转换
CEIL/CEILING	最小整数值
CHAR	数值转字符
CHAR_LENGTH	字符长度
COALESCE	返回第一个非空的参数值
CONCAT	返回结果为连接参数产生的字符串
CONVERT	类型转换
COUNT	行总数
COUNT(DISTINCT)	不同值的总数
CURDATE	获取日期
CURRENT_DATE	同CURDATE
CURRENT_TIMESTAMP	同NOW

CURTIME	当前时间
DATE	取日期部分值
DATE_ADD	增加时间
DATE_FORMAT	格式日期
DATE_SUB	减时间
DATEDIFF	两日期值相减
DAY	同DAYOFMONTH
DAYNAME	返回星期名词
DAYOFMONTH	返回一月的第几天
DAYOFWEEK	返回星期序号
DAYOFYEAR	返回一年的第几天
FLOOR	返回小于等于入参值的最大整数
HEXTORAW	十六进制字符串转换为RAW
HOUR	返回time 对应的小时数
IN	IN函数
INSTR	子串的位置
LAST_DAY	月最后一天
LCASE	同LOWER
LENGTH	字符串长度

LOWER	转为小写字符
LTRIM	去除空格
MAX	最大值
MIN	最小值
MINUTE	分钟
MONTH	月
NOT IN	IN函数
NOW	获取当前时间
RAND	随机值
REPLACE	字符串
ROUND	返回参数X, 其值接近于最近似的整数
RTRIM	去除空格
STR_TO_DATE	字符串转为日期
STRCMP	字符串比较
SUBSTR	取子字符串
SUM	求和
TIME	时间函数
TIMEDIFF	时间相减
TIMESTAMPADD	日期相加

TIMESTAMPDIFF	日期相减
TO_DAYS	转换成天
TO_SECONDS	转换成秒
TRIM	去除空格
UPPER	转为大写字符
WEEK	范围星期值
YEAR	获取年
GROUP_CONCAT	分组连接
FROM_UNIXTIME	时间格式转换
LEFT	字符串左截取
RIGHT	字符串右截取
LPAD	左补字符
RPAD	右补字符
TO_DATE	日期转换函数
SYSDATE	获取系统当前时间
INITCAP	将字符串中各单词首字母转大写，其他字母转小写
NVL	获取非空串
DECODE	类似CASE WHEN
||	连接符

ROWNUM	序列
TO_TIMESTAMP	日期转换函数
TO_CHAR	字符串转换函数
CHR(X)	返回在数据库字符集中与X拥有等价数值的字符
TRUNC(x[,y])	算截尾到y位小数的x值.y缺省为0,结果变为一个整数值。如果y是一个负数,那么就截尾到小数点左边对应的位上
NEXT_DAY(SYSDATE,X)	从当前日期开始得到到未来星期数的日期
INSTR（string1,string2[a,b])	得到在string1中包含string2的位置
TO_NUMBER()	将CHAR或VARCHAR2类型的string转换为一个number类型的数值
SYSTIMESTAMP	函数返回本机数据库上当前系统日期和时间(包括微秒和时区)
ADD_MONTHS(X, Y)	计算在时间x之上机上Y个月后的时间值，要是Y的值为负数的话就是在这个时间点之前的时间值（这个时间-Y个月）
LENGTH(str)	在设置Oracle模式时为获取字符串的字符数；在设置为MySQL模式时为获取字符串的字节数
LENGTHB(str)	Oracle模式和MySQL模式下均为获取字符串的字节数
FETCH FIRST N ROW[S][ONLY]	获取前n行, n的上限为ulonglong(18446744073709551615)
ROW_NUMBER() OVER()	分析函数，会在数据表生成一个排序列
UUID()	用于生成uuid，即通用唯一标识符
REPLACE(UUID(),'-','')	默认生成的uuid含有'-'，使用该函数可以替换掉uuid中的'-'
SUBSTRB(c1 ,n1,n2)	从字符串中指定的开始位置，取得指定字节数的字符串

LNNVL(EXPR)	如果expr是空值,则lnnvl函数会返回1,否则会返回0
NLSSORT(EXPR)	二进制排序
ROW_NUMBER() over(order by)	分组排序
nvl2(exp1,exp2,exp3)	根据参数1是否为null返回参数2或参数3的值
months_between(X,Y)	用于计算X和Y之间几个月
RAWTOHEX(exp)	将RAW类数值exp转换为一个相应的十六进制表示的字符串
to_clob()	字符串和字符大对象之间的转换
ORA_HASH(FIELD,NUM)	会把FIELD的值进行hash运算,然后放到NUM个桶里
INSTRB(C1,C2,s,e)	查找C2在C1指定范围中的位置
wm_concat()	行列转换函数,该函数可以把列值以‘,’分割开来
USEREVN(opt)	返回当前用户环境的信息 opt可以是: SESSIONID,LANGUAGE,instance
sys_context()	获取当前环境信息
ntile(3) over (order by)	分析函数，排序分级
rowidtochar()	rowid类型转为char
chartorowid()	char类型转为rowid
LAG/LEAD(args) over()	分组内偏移位置 前/后 n 行的值
MIN/MAX(args) over()	分组内最小/大 值
SUM/AVG(args) over()	分组内总和/平均值

DENSE_RANK() over()	分组结果集中排名，相同的值会获得相同的排名，而且排名不会有间隔
COUNT(args) over()	分组范围内行数的计数
RATIO_TO_REPORT(args) over()	分组范围内计算该分组中每一行的比例
GROUP_CONCAT(args) over()	分组分组内字符串拼接，可指定分隔符
WM_CONCAT(args) over()	分组分组内字符串拼接
LISTAGG() whin group()	多行字符串拼接

1.10.5.2. 函数解析模式说明

分布式数据库中涉及到的函数的解析模式分为两种，通过计算节点的配置文件(`proxy.ini`)中的配置项 `parse_mode` 进行设置，其值为 1（默认）时表示 `MySQL` 模式，值为 2 时表示 `Oracle` 模式，当涉及到的函数在 `MySQL` 和 `Oracle` 之间有二义性时需要根据需要设置函数解析模式。以下几个 `Oracle` 函数与 `MySQL` 具有二义性，需要开启 `Oracle` 模式，其余函数均为 `MySQL` 模式：

- `DECODE()` 函数：在设置 `Oracle` 模式时为根据条件设置输出的结果，类似于 `SELECT ... CASE ... WHEN ...`；在设置 `MySQL` 模式时为密文解密，与`ENCODE()` 为互逆。
- `LENGTH()` 函数：在设置 `Oracle` 模式时为获取字符串的字符数；在设置为 `MySQL` 模式时为获取字符串的字节数。
- `LENGTHB()` 函数：在设置 `Oracle` 模式时支持在多字节字符集中对 COLB 和 NCOLB 数据使用, `Oracle` 不支持,两者有差异。
- ||运算符：在设置 `Oracle` 模式时表示字符串拼接，等价于 `MySQL` 的 `CONCAT()` 函数；在设置 `MySQL` 模式时表示逻辑关系"或"。
- 日期减法：在计算两个日期相减且第一个参数或者两个都是日期相关函数时需要设置为 `Oracle` 模式，否则会将日期字符串转换为数字计算减法。涉及到的日期函数包括 `SYSDATE()`、`STR_TO_DATE()`、`TO_DATE()`、`LAST_DAY()`。

1.10.5.3. 复杂函数说明

- `CAST/CONVERT` 类型转换函数支持 SIGNED/UNSIGNED/DATE/TIME/DATETIME/DECIMAL 类型。
- `NEXT_DAY` 函数，第二个参数支持数字，目前不支持对字段操作，第一个参数为纯字段

的报错。

- `TO_CHAR` 函数，支持数值型和日期型转化为字符型，其中日期型转化为字符型时第一个参数可以是时间字符串（时间字符串语法规则与 `MySQL` 中的 `DATE_FORMAT` 函数相同）。数值型 FM 格式只支持 `FM09`。
- `TRUNC` 函数，处理特殊值及非法值时，处理结果与 `Oracle` 存在出入。
- `TO_DATE` 函数，目前支持输入一个或者两个参数，日期字符串格式显示键附录A.2，当前日期的输出支持按照年月日时分秒的 24 小时制输出，计算日期的减法时要在`parse_mode=2` 模式下执行。
- `INITCAP` 函数，目前支持输入一个参数，参数类型支持常量、字段、函数。支持输入参数个数不为1时报错。
- `ROW_NUMBER() OVER()` 与 `DENSE_RANK() OVER()` 窗口函数，目前支持 `OVER([PARTITION BY] ORDER BY)`，可以出现的位置包括 `FIELD LIST`、`FROM` 子查询、`WHERE` 子查询。不支持 `OVER` 中带有参数的窗口函数与 `GROUP BY`、其他聚合函数、`ORDER BY` 操作共用。

1.10.5.4. 窗口函数

窗口函数是一种在关系数据库中执行计算的高级功能。它能够对查询结果进行分组、排序和聚合，并在每个分组内按照指定的顺序处理数据。窗口函数提供了一种灵活且强大的方法来处理各种复杂的分析和聚合操作，提供了更多的计算和分析能力。

示例

```
> SELECT senior,class_1,st_id,name,math,lag(math,2,100) OVER(ORDER
 BY senior,class_1) AS lag_1 FROM student_decimal_hash;
> SELECT senior,class_1,st_id,name,math,min(math) OVER(ORDER BY se
nior,class_1) AS min FROM student_decimal_hash;
> SELECT student_info_hash.senior,student_info_hash.class_1,studen
t_score_hash.st_id,student_info_hash.name,student_score_hash.MATH,
avg(student_score_hash.MATH) OVER(PARTITION BY student_info_hash.s
enior  ORDER BY student_info_hash.class_1 ) AS avg_hash_4
FROM student_info_hash,student_score_hash WHERE student_score_hash
.st_id=student_info_hash.st_id ;
> SELECT *, rank() OVER (ORDER BY score desc) AS ranking, dense_ra
nk() OVER (ORDER BY score desc) AS dese_rank, row_number() OVER (O
RDER BY score desc) AS row_num FROM test_window_func;
> SELECT senior,class_1,st_id,name,math,count(math) OVER(ORDER BY
1) AS min FROM student_decimal_hash;
> SELECT test_id,c1,sum_blocks,TRUNC((ratio_to_report(sum_blocks)
OVER())*100,2) pcnt FROM(
> SELECT a.st_id AS test_id,decode(a.st_id,1,'ENGLISH','MATH') c1,
sum(a.math) sum_blocks FROM student_decimal_hash a,student_decimal
_hash b WHERE a.st_id < 40 and a.st_id=b.st_id GROUP BY a.senior G
ROUP BY decode(a.st_id,1,'a.senior','a.class_1'));
> SELECT senior,class_1,st_id,name,math,group_concat(CASE WHEN sen
ior in (1) THEN math ELSE class_1 end SEPARATOR '|') OVER(PARTITIO
N BY senior) AS group_concat FROM student_decimal_hash;
> SELECT senior,class_1,st_id,name,math,wm_concat(math) OVER(PARTI
```

```
TION BY senior) AS group_concat FROM student_decimal_hash;
```

使用说明

- 目前 `OVER` 参数仅支持 `OVER([PARTITION BY] [ORDER BY])`，可以出现的位置包括 `FIELD LIST`、`FROM` 子查询、`WHERE` 子查询。
- 支持多个窗口函数，示例如上 4。
- 支持的窗口函数列表请检索 5.1 章节
- `LISTAGG` 函数拼接字符串的长度与 `group_concat_max_len` 这个参数强相关,超过这个长度字符串则会被截断。在 `Oracle` 中, `LISTAGG` 函数拼接长度大于 4096 字节则会报错。

使用限制

- 不支持窗口函数与 `GROUP BY` ，聚合函数一起使用。
- 不支持窗口函数直接做 `WHERE` 条件使用。
- `lag default` 参数不支持为包含field、查询语句，可以为int, string, decimal, 日期，计算表达式等。
- 函数使用限制与单独使用限制雷同，eg: `group_concat` 的列长度限制，`max/min/avg/sum` 计算类型等。
- `LISTAGG` 函数不支持与 `OVER` 一起使用,即不支持 `LISTAGG() WITHIN GROUP() OVER()`。

1.10.6. PSEUDO COLUMNS

1.10.6.1. SEQUENCE

兼容Oracle SEQUENCE语法。

语法

```
CREATE SEQUENCE schema.sequence
[INCREMENT BY num]
[START WITH num]
[MAXVALUE num | NOMAXVALUE]
[MINVALUE num | NOMINVALUE]
[CYCLE | NOCYCLE]
[CACHE | NOCACHE]

ALTER SEQUENCE schema.sequence
INCREMENT BY num
RESTART WITH num | RESTART

DROP SEQUENCE schema.sequence
```

参数解释

参数	描述
INCREMENT BY	序列变化的步进，负值表示递减，默认值为 1。
START WITH	指定序列的初始值，默认值为 1。
MAXVALUE	序列可生成的最大值，默认不限制最大值，即 NOMAXVALUE。
MINVALUE	序列可生成的最小值，默认不限制最小值，即 NOMINVALUE。
CYCLE/NOCYCLE	序列可生成的最小值，默认不限制最小值，即 NOMINVALUE。
CACHE	表示缓存序列的个数，数据库异常终止可能会导致序列中断不连续的情况，默认值为 100。如果不使用缓存，可设置 NOCACHE。

示例

```
> CREATE SEQUENCE schema.seq_abc INCREMENT BY 1 START WITH 10 MAXV
ALUE
  9999999 NOCYCLE NOCACHE;
> ALTER SEQUENCE schema.seq_abc INCREMENT BY 5;
> DROP SEQUENCE schema.seq_abc;
```

使用说明

- `SEQUENCE` 默认 CACHE 为 100、最大值(int64 最大值)、最小值 1。
- `SEQUENCE` 负的最小值应该比理论 int64 最小值大 1。
- `SEQUENCE` 前缀含义非用户概念，类似 `DATABASE` 概念。
- `SEQUENCE` 支持 `ORDER|NOORDER` 属性。
- `SEQUENCE` 修改步长时会导致当前 CACHE 丢失。
- `SEQUENCE` 的 DDL 操作需要指定库名。

限制说明

由于 `SEQUENCE` 的值是在 SQL 语句执行之前向 GTM 申请，语句执行失败的情形下申请的 `SEQUENCE` 值不能够回收，所以表中的数据会有不连续的情形。

1.10.6.2. 伪列ROWNUM

`Oracle` 用 `ROWNUM` 伪列来标示各记录的行号，多用于 `SELECT` 查询和 `WHERE` 条件过滤。

语法

```
SELECT ROWNUM FROM tbl_name WHERE ROWNUM < N;
```

参数解释

参数	描述
ROWNUM	可以限制返回的行数。如语法所示，返回 tbl_name 表中的 N-1 条数据。

示例

- 例子1：

```
> SELECT ROWNUM, t1.* FROM t1 WHERE ROWNUM < 3; #单独字段，无别名
> SELECT ROWNUM AS rn, t1.* FROM t1 WHERE ROWNUM < 3;
#单独字段，有别名
```

- 例子2：

```
> SELECT (ROWNUM+1)*2 AS rn, t1.* FROM t1 WHERE ROWNUM < 3; #
表达式计算
> SELECT COUNT(ROWNUM), MIN(ROWNUM),MAX(ROWNUM) FROM t1; # 聚合函数
> SELECT MAX(ROWNUM)+1 AS rn FROM t1 GROUP BY type;
#聚合函数+表达式计算
```

- 例子3：

```
> SELECT ROWNUM AS rn, t1.id FROM t1 UNION ALL SELECT ROWNUM, t1.i
d FROM
  t1 ; # union
```

限制说明

- `SELECT` 语句中，在表中不存在索引字段且 `ORDER BY` 普通字段的情形下，分布式数据库结果集与 `Oracle` 结果集相同，都是对结果集先进行编号后进行排序，但当多行数据的排序字段值相同时结果集与 `Oracle` 的会有不同。
- `SELECT` 语句中，在表中不存在普通索引且 `ORDER BY` 主键降序排列时，分布式数据库结果集与 `Oracle` 结果集不同，分布式数据库先对结果集进行编号然后按照主键降序排列；`Oracle` 先按照主键降序排列然后再对结果集进行编号。
- `SELECT` 语句中，在表中存在联合索引且 `ORDER BY` 普通字段的情形下，分布式数据库结果集与 `Oracle` 数据库结果集不同，分布式数据库先按照 `ORDER BY` 字段进行排列，然后对结果集进行编号；`Oracle` 数据库先对结果集进行编号然后对结果集进行排序。
- `SELECT` 语句中，在表中存在索引且 `ORDER BY` 主键的情形下，分布式数据库结果集与 `Oracle` 数据库结果集不同，分布式数据库先对结果集进行编号然后进行排序；`Oracle`

数据库先对结果集按照主键进行排序然后进行编号。

1.10.6.3. 伪列ROWID

`ROWID` 是 `Oracle` 中的一个重要的概念。采用 `base64` 编码用于定位数据库中一条记录的一个相对唯一地址值。

语法

```
SELECT ROWID FROM tbl_name WHERE EXPR;
```

参数解释

参数	描述
ROWID	ROWID 值唯一标识数据库中的一行, 是访问单行的最快方式。

示例

```
> SELECT ROWID FROM t1 WHERE ROWID='ACAwEAAAA=========';
   #查询单条记录
```

限制说明

- `ROWID` 加反引号``表示表中普通列
- 伪列 `ROWID` 不能作为分发键使用
- 不支持插入或更新 `ROWID` 伪列

1.10.7. DDL

1.10.7.1. CREATE DATABASE

描述

创建数据库，并可以指定数据库的默认属性。

语法

```
CREATE {DATABASE SCHEMA} [IF NOT EXISTS] db_name
[create_specification]
[database_sharding_specification]

create_specification:
```

```
[DEFAULT] CHARACTER SET [=] charset_name
| ENCRYPTION [=] {'Y' | 'N'}
| /*!80016 DEFAULT ENCRYPTION [=] {'Y' | 'N'}*/

database_sharding_specification:
LIST(sharding_item_list);

sharding_item_list:
sharding_item
| sharding_item_list, sharding_item ;

sharding_item:
pg[num]( def_item_list ) ;

def_item_list:
def_item
| def_item_list, def_item ;

def_item:
g[num]
| g[num]<lg_db_name>
| g[num] - g[num]
```

参数解释

参数	描述
charset_name	字符集支持：latin1、gbk、utf8、utf8mb4 和 gb18030。
ENCRYPTION	支持库加密，如：c database encryption3 ENCRYPTION = 'Y'; 或 create database encryption2 */!80016 DEFAULT ENCRYPTION='Y'/*;

示例

- 一般建库语句

```
> CREATE DATABASE IF NOT EXISTS db1 CHARACTER SET utf8;
```

- 分库建库语句

写法1：不显式指定物理库名，系统自动生成，下面语句生成的物理库名为db1_1 - db1_8

```
> CREATE DATABASE IF NOT EXISTS db1 CHARACTER SET UTF8
LIST(pg1(g1,g2,g3,g4), pg2(g5,g6,g7,g8));
```

写法2：显式指定物理库名

```
> CREATE DATABASE IF NOT EXISTS db2 CHARACTER SET UTF8
LIST(pg1(g1<my_db1>,g2<my_db2>,g3<my_db3>,g4<my_db4>),
pg2(g5<my_dbb5>,g6<my_db6>,g7<my_db7>,g8<my_db8>));
```

写法3：写法1的简写

```
CREATE DATABASE IF NOT EXISTS db3 CHARACTER SET UTF8
LIST(pg1(g1-g4), pg2(g5-g8));
```

使用说明

对于带有分库属性的库禁止以下操作

- 建表不允许 多级分片表/全局索引。
- 表闪回。
- `COPY TABLE`。
- 视图。
- 存储过程。
- 自定义函数。
- 同义词。

1.10.7.2. DROP DATABASE

描述

删除数据库。

语法

```
DROP {DATABASE SCHEMA} [IF EXISTS] db_name
```

参数解释

参数	描述
DATABASE SCHEMA	这部分表明要删除的对象是数据库或模式。
IF EXISTS	这是一个可选的关键字。如果指定了“IF EXISTS”，当要删除的数据库或模式不存在时，不会产生错误，只是给出一个提示信息。如果没有指定，而要删除的对象不存在，可能会导致错误。
db_name	指定要删除的数据库或模式的名称。

示例

```
> DROP DATABASE IF EXISTS db1;
```

1.10.7.3. ALTER DATABASE

描述

修改数据库属性。

语法

```
ALTER {DATABASE SCHEMA} [db_name]
alter_specification ...

alter_specification:
[DEFAULT] CHARACTER SET [=] charset_name
```

示例

```
> ALTER DATABASE db1 CHARACTER SET gbk;
```

1.10.7.4. CREATE TABLE

描述

在数据库中创建新表。

语法

```
CREATE TABLE [IF NOT EXISTS] table_name
(create_definition, ...)
[table_options]
[partition_options]
[distributed_option]
[select_statement]

CREATE TABLE [IF NOT EXISTS] table_name
{LIKE schema.table_name_1 | AS SELECT select_statement}

create_definition:
col_name column_definition
| [CONSTRAINT [symbol]] PRIMARY KEY [index_type]
(index_col_name,...)
[index_option] ...
| {INDEXKEY} [index_name] [index_type] (index_col_name,...)
[index_option] ...
```

```
| [CONSTRAINT [symbol]] UNIQUE [INDEXKEY]
[index_name] [index_type] (index_col_name,...)
[index_option] ...
| GLOBAL PRIMARY KEY [index_type] (index_col_name,...)
[index_option] ...
| GLOBAL [UNIQUE] [INDEXKEY]
[index_name] [index_type] (index_col_name,...)
[index_option] ...

column_definition:
data_type [NOT NULL NULL] [DEFAULT default_value]
[AUTO_INCREMENT][UNIQUE [KEY] [PRIMARY] KEY GLOBAL
PRIMARY KEY
GLOBAL UNIQUE [KEY][COMMENT 'string']

data_type:
| TINYINT[(length)] [UNSIGNED]
| SMALLINT[(length)] [UNSIGNED]
| MEDIUMINT[(length)] [UNSIGNED]
| INT[(length)] [UNSIGNED]
| INTEGER[(length)] [UNSIGNED]
| BIGINT[(length)] [UNSIGNED]
| DOUBLE[(length,decimals)] [UNSIGNED]
| FLOAT[(length,decimals)] [UNSIGNED]
| DECIMAL[(length[,decimals])] [UNSIGNED]
| NUMERIC[(length[,decimals])] [UNSIGNED]
| DATE
| TIME[(FSP)]
| TIMESTAMP[(FSP)]
| DATETIME[(FSP)]
| YEAR
| CHAR[(length)]
| VARCHAR(length)
| BINARY[(length)]
| VARBINARY(length)
| BLOB
| TEXT [BINARY]
| ENUM(value1,value2,value3,...)
| SET(value1,value2,value3,...)

index_col_name:
col_name [(length)]

index_type:
USING {BTREE | HASH}

index_option:
| COMMENT 'string'

table_options:
AUTO_INCREMENT [=] value
| COMMENT [=] 'string'
| COMPRESSION = {'ZLIB' | 'LZ4' | 'NONE'}
```

```
| [DEFAULT] CHARACTER SET [=] charset_name
| ENGINE [=] engine_name
| MAX_ROWS [=] value
| ROW_FORMAT [=]
{DEFAULT | DYNAMIC | FIXED | COMPRESSED | REDUNDANT | COMPACT}
| [DEFAULT] CHARACTER SET [=] charset_name
| CONNECTION [=] 'connect_string'
| KEY_BLOCK_SIZE [=] value

partition_options:
PARTITION BY
{ [LINEAR] HASH(expr)
| [LINEAR] KEY [ALGORITHM={12}] (column_list)
| RANGE{(expr) COLUMNS(column_list)}
| LIST{(expr) COLUMNS(column_list)} }
[PARTITIONS num]
[(partition_definition [, partition_definition] ...)]
[PARTITIONSTORAGEDB(group_list)
(partition_options)]

partition_definition:
PARTITION partition_name
[VALUES
{LESS THAN {(expr value_list) MAXVALUE}
|
IN (value_list)}]
[COMMENT [=] 'comment_text' ]

distributed_option:
DISTRIBUTED BY
HASH(column_list) ({groupno},...)
| RANGE(expr) ({groupno VALUES LESS THAN { value MAXVALUE}},...)
| RANGE(expr)({groupno [value,value)},...)
| RANGE(expr)({groupno [value,value) [value,value)...},...)
| LIST(expr) ({groupno VALUES IN (value_list DEFAULT)},...)
| DUPLICATE ({groupno},...)
```

- 说明

GoldenDB 在建表时支持加 hint 按照表为粒度设置允许分片键为空，hint 格式是 `/*+ support_hash_null*/`，对分发键和分发策略的前置要求或者限制是：分发键的类型目前支持varchar、char、decimal，分发策略目前支持hash。

参数解释

参数	描述
create_definition	这里定义了表的结构，包括列的名称、数据类型、约束等。例如：column1 INT PRIMARY KEY, column2 VARCHAR(255) NOT NULL 。

table_options	这些是与表相关的各种选项，例如表的存储引擎（如 InnoDB、MyISAM 等）、字符集（如 UTF8、GBK 等）、自增主键的起始值等。
partition_options	如果您需要对表进行分区存储，在这里可以指定分区的规则和方式。例如按照某个列的值范围或哈希值进行分区。
distributed_option	分布式表的分发选项定义。
FSP	表示日期时间精度；即秒的小数点后的位数。这个 FSP 值（如果有）必须在 0 到 6 之间。值为 0 表示没有小数部分。如果省略，默认精度为 0。

注意事项

分发选项新增range表创表分发范围带区间形式，目前仅支持左闭右开（g1[0,100 ））形式,该语法不影响原有创表语法规则，不支持和原有分发规则混用。

- 单节点存储：表数据只存储于一个数据节点上，且分发策略为复制类型。

  设置方法：

```
DISTRIBUTED BY DUPLICATE(groupno);
```

- 多节点分布存储：表数据是水平分布在不同数据节点上，数据分发策略包括：`HASH` 分发、`RANGE` 分发、`LIST` 分发等。

  设置方式为在 `distributed_option` 中设置分发字段和分发策略，设置存储的数据节点安全组。

  数据节点安全组的个数可以设置一个或多个。

- 复制表：表数据是全量存储在各个数据节点上。

  设置方式为在 `distributed_option` 中设置 `DUPLICATE`，设置要全量存储的数据节点安全组。

  数据节点安全组的个数可以设置一个或多个。

数据分发策略原则

- `HASH` 分发

  适用于将数据均匀的分布到预先定义的各个数据节点上，保证各数据节点的数据数量大致一致，一般情况下不需要关心分发字段的值的具体含义。

比较适合场景：该表上的操作基本都是等值操作。

- `RANGE` 分发

适用于指定一个给定的列值或列值集合应该保存在哪个数据节点上，常用于时间字段上，比如数据按照自然月或天来分布存储。

- `LIST` 分发

适用于含有一系列限定性值的列的场景，常用于机构代码或国家代码字段上。

- 复制表

适用于小表且不常修改的表，或者 `JOIN` 和子查询中使用的表。主要目的是减少节点间网络数据的传输，以提高查询的性能。

- 针对多节点分布存储的表

在采用 `HASH/RANGE/LIST` 分发策略时，尽量保持各个数据节点上的数据量均衡，均衡的数据分布在多节点并行计算时会有更高的性能表现。

- 主文件类的表

优先考虑用 `HASH` 分发策略，分发键使用业务主键，对于联机和批量同时存在的情况优先保障联机场景。

- 其他场景优先考虑使用 `RANGE/LIST` 分发策略。

分发选项说明

分发选项说明表

分发选项	说明
HASH	HASH分发
RANGE	范围分发
LIST	列表分发
DUPLICATE	复制表
GroupNo	存储安全组定义

不同分区不同分片表

计算节点通过关键字 `PARTITIONSTORAGEDB` 可以在不同的分片上指定不同的分区规则。

```
CREATE TABLE ps_test.range(
int_1 int not null,
int_2 int,
date_1 date default '2021-01-01',
time_1 time,
datetime_1 datetime,
timestamp_1 timestamp,
year_1 year,
json_1 json,
primary KEY(date_1, int_1, int_2)
)
PARTITION BY RANGE (TO_DAYS(date_1))
(
PARTITION p202101 values LESS THAN (TO_DAYS('2021-01-01')),
PARTITION p202102 values LESS THAN (TO_DAYS('2021-02-01')),
PARTITION p202103 values LESS THAN (TO_DAYS('2021-03-01')),
PARTITION p202104 values LESS THAN (TO_DAYS('2021-04-01')),
PARTITION p202105 values LESS THAN (TO_DAYS('2021-05-01')),
PARTITION p202106 values LESS THAN (TO_DAYS('2021-06-01')),
PARTITION p202107 values LESS THAN (TO_DAYS('2021-07-01')),
PARTITION p202108 values LESS THAN (TO_DAYS('2021-08-01')),
PARTITION p202109 values LESS THAN (TO_DAYS('2021-09-01')),
PARTITION p202110 values LESS THAN (TO_DAYS('2021-10-01')),
PARTITION p202111 values LESS THAN (TO_DAYS('2021-11-01')),
PARTITION p202112 values LESS THAN (TO_DAYS('2021-12-01')),
PARTITION other values LESS THAN maxvalue)
PARTITIONSTORAGEDB(g1,g2)
(PARTITION BY RANGE(int_1)
(
PARTITION p0 values LESS THAN (100),
PARTITION p1 values LESS THAN (200),
PARTITION p2 values LESS THAN (300),
PARTITION p3 values LESS THAN MAXVALUE))
PARTITIONSTORAGEDB(g3)
(PARTITION BY RANGE(int_2)
(
PARTITION p0 values LESS THAN (10000),
PARTITION p1 values LESS THAN (20000),
PARTITION p2 values LESS THAN (30000),
PARTITION p3 values LESS THAN MAXVALUE))
DISTRIBUTED BY HASH(int_1)(g1,g2,g3,g4);
```

对于表 `ps_test.range`

通过 `PARTITIONSTORAGEDB(g1,g2)`，指定g1, g2上的分片规则为

```
PARTITION BY RANGE (`int_1`)
(PARTITION p0 VALUES LESS THAN (100) ENGINE = InnoDB,
PARTITION p1 VALUES LESS THAN (200) ENGINE = InnoDB,
PARTITION p2 VALUES LESS THAN (300) ENGINE = InnoDB,
PARTITION p3 VALUES LESS THAN MAXVALUE ENGINE = InnoDB)
```

g3 上的分发规则

```
PARTITION BY RANGE (`int_2`)
(PARTITION p0 VALUES LESS THAN (10000) ENGINE = InnoDB,
PARTITION p1 VALUES LESS THAN (20000) ENGINE = InnoDB,
PARTITION p2 VALUES LESS THAN (30000) ENGINE = InnoDB,
PARTITION p3 VALUES LESS THAN MAXVALUE ENGINE = InnoDB)
```

g4未通过 `PARTITIONSTORAGEDB` 指定分发策略，使用默认分发策略

```
PARTITION BY RANGE (to_days(`date_1`))
(PARTITION p202101 VALUES LESS THAN (738156) ENGINE = InnoDB,
PARTITION p202102 VALUES LESS THAN (738187) ENGINE = InnoDB,
PARTITION p202103 VALUES LESS THAN (738215) ENGINE = InnoDB,
PARTITION p202104 VALUES LESS THAN (738246) ENGINE = InnoDB,
PARTITION p202105 VALUES LESS THAN (738276) ENGINE = InnoDB,
PARTITION p202106 VALUES LESS THAN (738307) ENGINE = InnoDB,
PARTITION p202107 VALUES LESS THAN (738337) ENGINE = InnoDB,
PARTITION p202108 VALUES LESS THAN (738368) ENGINE = InnoDB,
PARTITION p202109 VALUES LESS THAN (738399) ENGINE = InnoDB,
PARTITION p202110 VALUES LESS THAN (738429) ENGINE = InnoDB,
PARTITION p202111 VALUES LESS THAN (738460) ENGINE = InnoDB,
PARTITION p202112 VALUES LESS THAN (738490) ENGINE = InnoDB,
PARTITION other VALUES LESS THAN MAXVALUE ENGINE = InnoDB)
```

对于不同分区不同分片表，`SHOW CREATE TABLE` 并不展示表的 `PARTITION` 定义，展示提示信息，需要通过 `SHOW CREATE TABLE` 加 `STORAGEDB` 展示具体 db 的分区定义。

```
> SHOW CREATE TABLE ps_test.range;
> Create Table: CREATE TABLE `range` (
`int_1` INT(11) NOT NULL,
`int_2` INT(11) NOT NULL DEFAULT '0',
`date_1` date NOT NULL DEFAULT '2021-01-01',
`time_1` time DEFAULT NULL,
`datetime_1` datetime DEFAULT NULL,
`timestamp_1` timestamp NULL DEFAULT NULL,
`year_1` year(4) DEFAULT NULL,
`json_1` json DEFAULT NULL,
PRIMARY KEY (`date_1`,`int_1`,`int_2`)
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_bin
DISTRIBUTED BY HASH(`int_1`)(g1,g2,g3,g4);
THIS TABLE HAS DIFFERENT PARTITION IN DIFFERENT GROUP, PLEASE USE
STORAGEDB
```

想要知道具体 group 的分区定义，通过语句

`SHOW CREATE TABLE ps_test.range STORAGEDB G1;` 具体进行查看。

多级分片表（该功能为企业版特性）

```
DISTRIBUTED BY
CASE case_value
WHEN when_value THEN statement_list
[WHEN when_value THEN statement_list] ...
[ELSE statement_list]
END CASE
```

或者

```
DISTRIBUTED BY
CASE
WHEN case_value > when_value THEN statement_list
[WHEN case_value < when_value THEN statement_list] ...
[ELSE statement_list]
END CASE
```

- case_value:
  - 字段名；
  - 简单表达式(+、-、*、/)；
  - `TIME`、 `DATE`、 `HOUR`、 `MINUTE`、 `DAY`、 `MONTH`、 `YEAR` 和 `SUBSTR`(`SUBSTRING`) 函数；
- when_value:
- value 值；
- statement_list:
  - `CASE` 分支， `SUBDISTRIBUTED BY`；
  - group 值；
  - 多组分发策略；
  - `SUBDISTRIBUTED BY` 支持 `RANGE` 带区间形式（仅支持左闭右开）；

多级分片表最多可支持到五级。

使用 DEFAULT 值的 LIST 表

对于 `LIST` 分发策略，如果无法穷举分发值，可以通过使用DEFAULT，来将不能穷举的值，分发到指定的 `GROUP` 上。

```
CREATE TABLE tb1(id INT KEY) DISTRIBUTED BY LIST(id)(G1 VALUES
  IN(100,200,300), G2 VALUES IN(400,500), G3 VALUES IN(DEFAULT));
```

对于上述语句，分发键不在穷举值 `[100,200,300,400,500]` 之外的值，全部都落在 `DEFAULT` 的 G3 中。

对于 `ALTER TABLE` 修改表分发策略，涉及到 `DEFAULT` 分区的场景，支持和不支持的典型场景说明如下：

支持的场景:

- 支持新增 `DEFAULT` 分区

```
> CREATE TABLE titles(
id INT KEY
)
DISTRIBUTED BY LIST(id)(g1 values IN(100,200,300), g2 values
IN(400,500));
> ALTER TABLE titles
DISTRIBUTED BY LIST(id)(g1 values IN(100,200,300), g2 values
IN(400,500), g3 values IN(DEFAULT));
```

- 支持对非 `DEFAULT` 分区做扩展（暂时不考虑数据问题，由业务侧自己保证）

```
> CREATE TABLE titles(
id INT KEY
)
DISTRIBUTED BY LIST(id)(g1 values in(100,200,300), g2 values
in(400,500), g3 values in(DEFAULT));
> ALTER TABLE titles
DISTRIBUTED BY LIST(id)(g1 values in(100,200,300), g2 values
in(400,500,600), g3 values in(DEFAULT));
```

不支持的场景：

- 不支持非扩展的修改分片界限值

```
> CREATE TABLE titles(
id INT KEY
)
DISTRIBUTED BY LIST(id)(g1 values IN(100,200,300), g2 values
IN(400,500), g3 values IN(DEFAULT));
> ALTER TABLE titles
DISTRIBUTED BY LIST(id)(g1 values IN(100,200), g2 values
```

```
IN(300,400,500), g3 values IN(DEFAULT));
```

单个分片下支持多个区间段的RANGE表

```
RANGE(expr) ({groupno VALUES LESS THAN { value MAXVALUE}},...)
| RANGE(expr)({groupno [value,value)},...)
| RANGE(expr)({groupno [value,value) [value,value)...},...)
```

对于 `RANGE` 分发策略，既支持 `VALUES LESS THAN` 的形式，也支持指定区间范围的形式（即[value,value)，左闭右开区间），并且支持在单个分片上指定多个区间段，各个分片上区间段数量可以不相同，各区间段不要连续且递增。例如：

```
CREATE TABLE tb1(id INT KEY) DISTRIBUTED BY RANGE(id)
(
G1[100,200)[300,400),
G2[200,300) [400,500),
G3[50,100)
);
```

对于指定区间范围的 `RANGE` 表建表语句，存在以下限制条件：

- 指定区间范围的形式与 `VALUES LESS THAN` 的形式不能混用。
- 各区间段的范围互不重叠。
- 每个区间段里，右边界值须大于左边界值。
- 对于多级分片表，指定区间范围的RANGE形式仅支持在叶子节点使用（即 `SUBDISTRIBUTED BY`）。
- 区间段数量不超过分片数量最大值限制，对于多级分片表则是每一层叶子节点(`SUBDISTRIBUTED BY`)区间段数量不超过分片数量最大值。

自定义HASH算法

对于 `HASH` 分发模式，支持在分发策略后方添加参数来使用其它 `HASH` 算法计算分发节点，示例如下：

```
CREATE TABLE tb1
(
id INT NOT NULL PRIMARY KEY,
cno VARCHAR(20)
)
DISTRIBUTED BY HASH(cno)(g1,g2,g3,g4)
ALGORIT
HM='plugin_name/algorithm_name',PARAM='oper=lpad,length=15,data="0
"';
```

其中，`plugin_name` 为算法插件名，即动态库.so文件的名称（不包含 lib 和后缀，

如：`libxxx.so` 的名称为 xxx），`algorithm_name` 表示函数的名称。`PARAM` 参数用于在计算前对分发键值做处理，目前支持 `lpad` 操作，示例中的含义为：如果分发键值长度不足 15，则用 “0” 对其进行左填充，使用处理后的值计算 `HASH` 值。如无 `PARAM` 参数，使用原分发键值计算。

该语法目前限制条件：

- 不支持多级分片表
- `ALGORITHM` 中不允许有空格。
- `PARAM` 中 `oper` 目前仅支持 `lpad`。
- `PARAM` 中 `oper`、`length`、`data` 使用小写。
- `PARAM` 中 `length` 长度限制 <=10000。

使用说明：

1. 算法编写

- 需严格按照约定格式，参考 `murmur.h` 和 `murmur.cc` 格式。

2. 编译

- 使用 gcc/g++ 4.8.5 版本编译。
- 编译环境(架构系统)必须与 GoldenDB 安装环境保持一致。
- 动态库编译参数示例为 `g++ -shared -fPIC -fsigned-char -o libhermes.so murmur.cc`，其中 `libhermes.so` 为编译生成的动态库文件，`hermes` 为算法算法插件名，可自定义。

3. 加载动态库

- 在 GoldenDB 的 Insight 界面加载动态库 so 文件，或手动替换，即将编译好的算法动态库 .so 文件放在 CN 用户的 lib 目录下，并确认或修改文件用户和属组与 bin 目录的可执行文件 dbproxy 一致。>
- 使用前需检查环境，如果已成功加载了非法的动态库 so 文件，则加载新的合法的动态库 so文件需要重启 CN 才能正常生效。

无主键表

[GENERAL]段 配置项 `support_without_primarykey = 1`，即可支持创建无主键表。

使用限制：

- 无主键表创建分区时，请确保 DN 上配置项 `remove_partition_key_limitation = ON`。

不支持的场景：

- 无主键表成功建表后，不支持新增主键

- 不支持 `REPLACE INTO` 语法。
- 不支持 `CREATE SELECT` 语法。
- 不支持通过分发策略配置文件设定分发策略。

CREATE AS SELECT

不支持的场景：

SELECT single table

- `SELECT` 子句不支持带 `FOR UPDATE`。
- `SELECT items` 不支持 gtid* 字段。
- 不支持创建全局索引和自增列。
- CR 场景不支持 `REPLACE` 或 `IGNORE`。
- 对于无 `CREATE list` 的 `CREATE SELECT` 语句 `SELECT` 的 `item list` 必须包含所有的主键列。
- `CREATE AS/LIKE` 不支持全局临时表，全局临时表不支持"已提交事务回滚"，需要人工处理。

Select Multi Tables

- 不支持带有 `CREATE list`。
- 必须指定分发信息。

注意项

- 新表会忽略源表索引(普通索引和全局索引)以及自增列(`AUTO_INCREMENT`)属性。

CREATE AS 多表

CREATE AS 多表的分发规则

语句本身指定分发规则：

- 语句本身指定分发规则，使用语句指定的分发规则

配置项 `default_distribute_method`：

[GENERAL]段 配置项 `default_distribute_method`

- `default_distribute_method = 0`，报错
- `default_distribute_method = 1`，`HASH` 分发策略
  - 没有配置[GENERAL]段 配置项 `default_distribute_keys`，分发规则为 `HASH(_gdb_r_rowid)`。
  - 配置[GENERAL]段 配置项 `default_distribute_keys`，遍历指定的对应库下可能的分发键，将第一个出现在新表字段列表中的字段作为分发键，若配置分发键均未出现在新表字段列表中，则默认以第一个字段作为分发键。

- `default_distribute_method = 2`，`DUPLICATE` 分发策略

列加密、表加密

- 支持列加密选项 `ENCRYPTION='AES'` 或 `ENCRYPTION='SM4'`，仅支持数据类型为 varchar 和 varbinary,其他数据类型报错。如：`CREATE TABLE t1 (a varchar(1000) ENCRYPTION='AES', b varbinary(1000) ENCRYPTION='SM4')`。
- 支持表加密选项 `ENCRYPTION='Y'` 或 `ENCRYPTION='N'`。如：`CREATE TABLE user6(id int, first_name varchar(10) not null, primary key (id)) ENCRYPTION='Y' DISTRIBUTED BY HASH(id)(g1,g2)`。

建表示例

- `HASH` 表

```
> CREATE TABLE titles(
title_id INT NOT NULL,
title VARCHAR(80) NOT NULL,
pub_id VARCHAR(4),
PRIMARY KEY (title_id)
)
DISTRIBUTED BY HASH(title_id) (g1,g2);
> CREATE TABLE t1(
id INT KEY,
f DATETIME
)
DISTRIBUTED BY HASH(f)(g1,g2,g3);
```

- `RANGE` 表

```
> CREATE TABLE test.t1 (
year_col INT,
col2 CHAR(5) NOT NULL PRIMARY KEY)
DISTRIBUTED BY RANGE(year_col)(
g1 VALUES LESS THAN (1991),
g2 VALUES LESS THAN (1995),
g3 VALUES LESS THAN MAXVALUE);
CREATE TABLE t1(
id INT KEY,
f DATETIME)
DISTRIBUTED BY RANGE(MONTH(f))(g1 VALUES LESS THAN(5), g2 VALUES L
ESS
THAN(12));
> CREATE TABLE test.t1 (
year_col INT,
col2 CHAR(5) NOT NULL PRIMARY KEY)
```

```
DISTRIBUTED BY RANGE(year_col)(
g1[0,1970) [2000,2010) [1990,1995),
g2[1970,1990),
g3[2010,2020) [2020,2030));
```

- `LIST` 表

```
> CREATE TABLE test.t1 (
year_col INT,
col2 CHAR(5) NOT NULL PRIMARY KEY)
DISTRIBUTED BY LIST(year_col)(
g1 VALUES IN (1991,1992,1993),
g2 VALUES IN (1994,1995));
```

- `DUPLICATE` 表

```
> CREATE TABLE IF NOT EXISTS test.dup_t1(
int_1 INT NOT NULL PRIMARY KEY,
set_1 SET('One','Two','Three','Four') NOT NULL,
set_2 SET('A','B','C','D','E','F','G'),
enum_1 ENUM('S', 'M', 'L', 'XL', 'XXL', 'XXXL') NOT NULL,
enum_2 ENUM('F','M') NOT NULL
) DISTRIBUTED BY DUPLICATE(g1,g2);
```

- 单节点表

```
> CREATE TABLE titles(
title_id INT NOT NULL,
title VARCHAR(80) NOT NULL,
pub_id VARCHAR(4) NULL,
PRIMARY KEY (title_id)
)
DISTRIBUTED BY DUPLICATE(g1);
```

- 多级分片表

```
> DELIMITER //
CREATE TABLE titles(
title_id INT NOT NULL,
cust_id INT NOT NULL,
cust_date DATETIME NOT NULL,
PRIMARY KEY (title_id)
) DISTRIBUTED BY
CASE YEAR(cust_date)
WHEN 2019 THEN g1 ;
WHEN 2018 THEN g2 ;
WHEN 2017 THEN g3 ;
ELSE g4;
```

```
END CASE; //
DELIMITER ;
```

- 多级分片表

```
> DELIMITER //
CREATE TABLE titles(
title_id INT NOT NULL,
cust_id INT NOT NULL,
pub_id INT NOT NULL,
PRIMARY KEY (title_id)
) DISTRIBUTED BY
CASE pub_id
WHEN 1 THEN
CASE
WHEN cust_id<100 THEN SUBDISTRIBUTED BY HASH(title_id)(g1);
ELSE SUBDISTRIBUTED BY HASH(title_id)(g2);
END CASE;
WHEN 2 THEN SUBDISTRIBUTED BY HASH(title_id)(g3);
ELSE SUBDISTRIBUTED BY HASH(title_id)(g4);
END CASE; //
DELIMITER ;
```

- 多级分片表:

```
> DELIMITER //
CREATE TABLE titles1(
title_id INT NOT NULL,
cust_id INT NOT NULL,
pub_id INT NOT NULL,
PRIMARY KEY (title_id)
) DISTRIBUTED BY
case pub_id
when 1 then
case
when cust_id<100 then SUBDISTRIBUTED BY RANGE (title_id)(g1 [
100,200));
else SUBDISTRIBUTED BY DUPLICATE(g2);
end case;
else SUBDISTRIBUTED BY LIST(title_id)(g3 values in (10,20,30));
END CASE; //
delimiter ;
```

- 多级分片表:

```
> DELIMITER //
CREATE TABLE titles2(
title_id INT NOT NULL,
cust_id INT NOT NULL,
pub_id INT NOT NULL,
```

```
PRIMARY KEY (title_id)
) DISTRIBUTED BY
case pub_id
when 1 then
case
when cust_id<100 then SUBDISTRIBUTED BY RANGE (title_id)(g1
[100,200) [200,300));
else SUBDISTRIBUTED BY HASH(title_id)(g2);
end case;
else SUBDISTRIBUTED BY LIST(title_id)(g3 values in (10,20,30));
END CASE; //
DELIMITER ;
```

- 不支持 `RANGE` 区间形式和 `RANGE LESS THAN` 形式混用，例如以下用例报错：

```
> CREATE TABLE titles3(
title_id INT NOT NULL,
cust_id INT NOT NULL,
pub_id INT NOT NULL,
PRIMARY KEY (title_id)
) DISTRIBUTED BY RANGE (title_id)(
g1 [100,200),
g2 values less than(5000));
> DELIMITER //
CREATE TABLE titles3(
title_id INT NOT NULL,
cust_id INT NOT NULL,
pub_id INT NOT NULL,
PRIMARY KEY (title_id)
) DISTRIBUTED BY
case pub_id
when 1 then
case
when cust_id<100 then SUBDISTRIBUTED BY RANGE (title_id)(g1
[100,200));
else SUBDISTRIBUTED BY RANGE (title_id)(g2 values less than(5000))
;
end case;
else SUBDISTRIBUTED BY LIST(title_id)(g3 values in (10,20,30));
end case; //
DELIMITER ;
```

- `CREATE TABLE LIKE`：

  CREATE TABLE [IF NOT EXISTS] table_name LIKE schema.table_name_1

```
> CREATE TABLE t1 LIKE t2;
```

- `CREATE TABLE AS SELECT`：

CREATE TABLE [IF NOT EXISTS] table_name AS SELECT select_statement

```
> CREATE TABLE t1 AS SELECT col1 FROM t2;
```

使用说明

主键的限制

- 表必须定义主键。
- 主键不允许使用 BLOB、TEXT、FLOAT、DOUBLE 类型。
- 无主键表参见 无主键表。

分发键限制

- `HASH`：分发键支持全部类型（除 TEXT、BLOB、ENUM、SET、BINARY、VARBINARY、JSON），不支持表达式和函数。
- `RANGE LIST`：分发键表达式的结果支持整数、时间函数和字符串类型。具体介绍如下：
  - 整形类型：支持整形和加减乘除简单表达式。
  - 字符串型：支持用字符的码值比较，支持 `SUBSTRING` 函数，不支持表达式和其它函数。建议使用有实际意义的字符串进行比较。
- 时间类型：支持时间函数使用时间函数包括 DATE/DAY/MONTH/YEAR/HOUR/MINUTE/TIME，时间函数中的字段类型支持 TIME/DATE/DATETIME/TIMESTAMP 类型。

  (注意：不建议BIGINT UNSIGNED类型作为分发键使用)
- 支持 DIV、MOD 运算，表达式的结果取整进行分发处理，推荐在使用 DIV、MOD 运算时使用整形类型进行处理。
- 若用户不指定分发信息，优先依据分发策略配置文件中配置的分发策略进行分发；分发策略配置文件未配置、不生效或不支持场景下，依据配置项 `default_distribute_method` 指定的分发策略进行分发。
- 单表/多表 `WHERE` 多分发键过滤功能说明
  - 对于 RANGE 表多分发键不支持范围交集过滤，比如"k1>20 AND k2 >10 > AND k1<10 AND k2<20"不支持精确过滤。
  - 分布表达式的函数值必须是单调的，比如：x*x, x/y, x*y + > 1等，这些表达式都是非单调表达式，目前无法正确过滤。
  - 不支持表达式过滤，比如WHERE t.k1+t.k2 > 100。
- 多级分片表限制
  - 支持创建多级分片表 `RANGE` 带区间形式，仅支持左闭右开。

  - 创建多级分片表 `RANGE` 带区间形式，区间范围不能交叉。
  - 创建多级分片表 `RANGE` 带区间形式不支持与 `RANGE LESS THAN` 形式混用。
  - 创建多级分片表 `RANGE` 带区间形式不同层不允许出现相同的分片节点，同一层可以有相同的分片节点。

- `HASH` 分发以外的分发键不能为空，当配置文件开启功能后，`HASH` 分发支持分发键设置为 NULL，目前此功能支持 char、varchar、varchar2 类型的分发键。

- 当分发键指定默认值时，支持 `INSERT` 时不指定分发键，目前此功能支持int、smallint、bigint、tinyint、mediumint、integer、float、double、decimal、numeric、char、varchar、number、dec、date、timestamp、datatime、time、year、varchar2 类型的分发键。

例如：

```
> CREATE TABLE t1(id INT KEY, b VARCHAR(10) DEFAULT 'a')
DISTRIBUTED BY HASH(b)(g1,g2);
> INSERT INTO t1(id,b) values(1,'a'); -->成功
> INSERT INTO t1(id) values(1); -->成功
```

表限制

- 表定义不能有 GTID 为前缀的列。
- `CREATE ... SELECT` 语句的限制。
  - `SELECT` 语句目前支持单表，该表的分发类型任意。
  - 新建的表必须显式指定一个主键字段。
  - 新建的表使用 `SELECT` 表的分发属性作为新表的分发属性，暂不支持显式指定 `CREATE TABLE ... SELECT` 的分发属性。
  - 新建表中不支持指定全局索引、自增列，不支持添加新的列。
  - 新建表为多级分片表时，不支持修改新表分发键名。
  - 虚拟列对应表达式的长度不能超过 252 字节。
  - SELECT语句中的字段必须包含 `SELECT` 语句中单表的分发键，如：

```
> CREATE TABLE abczyy_cs.tb1(a INT KEY, b INT,c VARCHAR(30))DISTRI
BUTED BY HASH(a)(g1,g2,g3);
> CREATE TABLE abczyy_cs.tb2(d INT KEY default 0);
> SELECT a,b,c FROM abczyy_cs.tb1;
```

数据类型限制

所有的 DDL 操作中避免使用中文的库名、表名字段名。

自增列相关限制

- 支持创建自增列表，不支持建表后，再通过 `ALTER TABLE` 增加自增列。

- 不支持通过 `ALTER TABLE MODIFY` 增加自增列，修改列时，保留 `auto_increament` 属性。
- 自增列字段支持整形字段类型。
- 临时表不支持自增列。

字段属性限制

- 不支持字段列表中字段属性为 `SIGNED`。

  如：

```
> CREATE TABLE IF NOT EXISTS tests123.tb1(a INT KEY, b INT, c INT,
 d INT, e INT SIGNED) DISTRIBUTED BY DUPLICATE(g1,g2,g3);
```

- 字段默认值使用函数时需使用括号转义的格式。

  如使用 `to_number` 函数：`pid decimal(1) (default to_number(substr(to_char(sysdate,'ss'),-1,1)))`。错误示例：`pid decimal(1) default to_number(substr(to_char(sysdate,'ss'),-1,1))`。

自增列有关配置项

对于表中的自增列，会创建一个 `SEQUENCE` 用于全局控制该自增列的特性，存在配置项可以控制自增列是否使用 `SEQUENCE` 流程。

[GENERAL]

`proxy_ignore_auto_inc = 0`

● 0 使用 `SEQUENCE` 流程

● 1 不适用 `SEQUENCE` 流程，在 CN 层面不对自增值进行控制，各 DN 上使用自己的自增特性。

自增列不走 `SEQUENCE` 流程，使用 DN 节点自增列功能影响评估如下：

1. 复制表可能会数据不一致。

原因：并发场景下 `INSERT` 顺序会不一致。

2. `HASH`、`RANGE`、`LIST` 表不同节点自增列会有相同值，不能保证全局唯一。

3. 自增列字段不能做分发键。

原因：作为分发键在 `INSERT` 中不显示指定值，会计算不出具体 group 节点会插入失败。

4. 自增列不能做全局索引。

原因：自增列的值 CN 获取不到，无法插入全局索引表中。

5. 自增列不能做主键。

原因：在 `UPDATE` 分发键时会导致主键冲突或者当前值跳值，当前值跳值会导致当前DN自增值发生变化，跳值不符合步长，可能会导致复制表数据不一致或多group场景间隔自增性被破坏。

6. 重分布可能会有主键冲突。

7. 导入功能可能有主键冲突。

工行：支持特殊的广播查询用法

新增一种 hash，无需用户指定分片字段。

```
CREATE TABLE TEST
(a INT KEY,b INT)DISTRIBUTED BY HASH()(g1,g2);
```

走 CN 透传模式（JDBC 的 URL 中指定）时，DML/DQL 保持原有实现，直接透传到指定分片。 走 CN 非透传模式时，SELECT 一律广播，下所有分片，并根据 SQL 要求对最终结果进行计算汇总。走 CN 非透传模式进行 DML 操作时，UPDATE/DELETE 语句带 STORAGEDB 场景正常执行， 不带 STORAGEDB 报错， INSERT 语句一律报错。

最佳实践

表级别定义原则

- 小表：数据量级别在 10 万以下。
- 大表：数据量级别在百万以上。

表的设计原则

- 单节点存储的表

  适用于下面几种场景：

- 非大表建议采用单节点存储。
- 需要进行 `JOIN` 的表要考虑放到同一个数据节点上。
- 建表时要考虑各个数据节点的数据均衡。

- 多节点分布存储的表

  适用于下面几种场景：

- 大表考虑采用多节点分布存储。

- 需要进行 `JOIN` 的大表要考虑采用相同的分发字段和分发策略。

  例如：在交易系统中，分布字段首先考虑要满足对联机交易的性能优化支持，比如账户表可以采用用户卡号作为分发字段。

- 复制表

  适用于下面几种场景：

- 小表且更新不频繁。
- 在 `JOIN` 或者子查询中使用的表。

- 去范式化原则

  在分布式系统中，通常去范式化以空间换时间来提升系统性能，即在"不影响数据唯一化"原则下，去范式化后的冗余数据字段在数据库中只能唯一属于一张表，否则无法确保数据一致性。

  当查询语句中出现表的 `JOIN` 操作时，尤其是针对分布式表间出现关联查询时，建议首先考虑采用去范式化设计来规避 `JOIN` 操作，达到提高查询效率的目的。

  去范式化原则特别适合于需要冗余的字段很少更新的业务场景。

1.10.7.5. DROP TABLE

描述

删除数据库中的表

语法

```
DROP TABLE [IF EXISTS] tbl_name
```

参数解释

参数	描述
IF EXISTS	这是一个可选的部分。如果包含了 IF EXISTS，那么当指定的表 tbl_name 不存在时，数据库系统不会报错，而是仅仅给出一个提示，表明该表不存在，删除操作未执行。
tbl_name	这是要删除的表的具体名称。

示例

闪回开关关闭时，drop table直接删除表。

```
DROP TABLE IF EXISTS db1.t1;
```

闪回开关开始时，drop table并不直接删除表，生成闪回备份。

```
DROP TABLE IF EXISTS db1.t1;
```

闪回开关开始时，drop table purge直接删除表。

```
DROP TABLE IF EXISTS db1.t1 purge;
```

闪回开关开始时，drop table no purge并不直接删除表，生成闪回备份。

```
DROP TABLE IF EXISTS db1.t1 no purge;
```

使用说明

- 不支持删除多个表的语法。
- 闪回开关开启时，`DROP TABLE` 与 `DROP TABLE no purge` 相同。
- 不支持删除全局索引表辅助表（全局索引表辅助表是在建立全局索引时，CN 根据索引键建立的表，由库名+表名+索引名根据 `HASH` 算法计算出，以 t 开头，只有在 DB 上可以看到，例如"t9180007576100144884000000000000001524583232367101482714 95"）。

1.10.7.6. ALTER TABLE

描述

修改已存在的表结构。

语法

```
ALTER TABLE tbl_name
    [alter_specification]
    [partition_options] [distributed_option
distributed_force_option]

alter_specification:
add_col_def [, add_col_def] ...
| ADD {INDEX | KEY} [index_name]
[index_type] (index_col_name,...)
| ADD [CONSTRAINT [symbol]]
UNIQUE [INDEX | KEY] [index_name]
| ADD GLOBAL [UNIQUE] INDEX [index_name]
[index_type] (index_col_name,...)
| ADD UNIQUE [INDEX | KEY] [index_name]
```

```
[index_type] (index_col_name,...)
| ALTER [COLUMN] col_name {SET DEFAULT literal DROP DEFAULT}
| MODIFY [COLUMN] col_name column_definition
| DROP [COLUMN] col_name
| DROP {INDEX | KEY} index_name
| DROP PRIMARY KEY, ADD PRIMARY KEY(key_name)
| ADD PARTITION (partition_definition)
| DROP PARTITION partition_names
| TRUNCATE PARTITION {partition_names ALL}
| REMOVE PARTITIONING
| REORGANIZE PARTITION partition_names INTO (partition_definitions
)
| COALESCE PARTITION number
| RENAME [TOAS] new_tbl_name
| CHANGE [COLUMN] old_col_name new_col_name column_definition
[FIRSTAFTER col_name]
| COMPRESSION = {'ZLIB' 'LZ4' 'NONE'}
| RENAME [INDEX | KEY] old_idx_name TO new_idx_name
| RENAME COLUMN old_col_name TO new_col_name
| DATA DIRECTORY = 'directory_name'
| ADD SUPPORT_HASH_NULL
| REMOVE SUPPORT_HASH_NULL

add_col_def:
ADD [COLUMN] col_name column_definition
[FIRST AFTER col_name ]

index_col_name:
col_name [(length)]

partition_options:
partition_option [partition_option] ...

partition_option:
ADD PARTITION (partition_definition)
| DROP PARTITION partition_names
| TRUNCATE PARTITION {partition_names ALL}
| COALESCE PARTITION number
|REORGANIZE PARTITION partition_names INTO (partition_definitions)
| EXCHANGE PARTITION partition_name WITH TABLE tbl_name
| ANALYZE PARTITION {partition_names ALL}
| CHECK PARTITION {partition_names ALL}
| OPTIMIZE PARTITION {partition_names ALL}
| REBUILD PARTITION {partition_names ALL}
| REPAIR PARTITION {partition_names ALL}
| REMOVE PARTITIONING
```

注意：针对多分片场景下PARTITION BY LIST COLUMNS且含有DEFAULT分区的情况，只支持ADD，DROP、REORGANIZE、TRUNCATE和REMOVE的分区操作。

```
distributed_option:
    DISTRIBUTED BY
| RANGE(expr)({groupno [value,value)},...)
| RANGE(expr)({groupno [value,value) [value,value)...},...)
```

```
| LIST(expr) ({groupno VALUES IN (value_list)},...)

distributed_force_option:
DISTRIBUTED BY distributed_option FORCE;

distributed_option:
    DISTRIBUTED BY
    HASH(column_list) ({groupno},...)
| RANGE(expr) ({groupno VALUES LESS THAN { value MAXVALUE}},...)
| RANGE(expr)(groupno [value,value),...)
| LIST(expr) ({groupno VALUES IN (value_list DEFAULT)},...)
| DUPLICATE ({groupno},...)
```

多级分片表（该功能仅企业版支持）：

```
ALTER TABLE tbl_name DISTRIBUTED by
CASE case_value
WHEN when_value THEN statement_list
[WHEN when_value THEN statement_list] ...
[ELSE statement_list]
END CASE
```

或者：

```
ALTER TABLE tbl_name DISTRIBUTED by
CASE
WHEN case_value > when_value THEN statement_list
[WHEN case_value < when_value THEN statement_list] ...
[ELSE statement_list]
END CASE
```

参数解释

参数	描述
ADD [COLUMN]	增加列。
ADD GLOBAL [INDEX]	增加全局索引。
ADD UNIQUE [INDEX]	增加唯一索引。
ADD [INDEX]	增加普通索引。

ADD PARTITION	增加分区。
ANALYZE PARTITION	分析表的分区。
ALTER [COLUMN]	修改列的默认值。
ALTER [INDEX]	修改索引属性。
ALTER [INDEX]	修改索引属性。
CHANGE [COLUMN]	修改列名和列属性。
CHECK PARTITION	检查指定分区的数据完整性和一致性。
COALESCE PARTITION	将相邻的分区合并成一个分区。
COMPRESSION	设置表的压缩算法。
MODIFY [COLUMN]	修改列属性。
DATA DIRECTORY	更改表的数据存储目录。
DROP [COLUMN]	删除列，不允许删除主键列或者包含索引的列。
DROP [INDEX]	删除索引。
DROP PARTITION	删除分区。
OPTIMIZE PARTITION	优化表的指定分区。
REBUILD PARTITION	重建表的指定分区。

REPAIR PARTITION	修复表的执行分区。
REMOVE PARTITIONING	删除分区。
REMOVE SUPPORT_HASH_NULL	取消允许表的分片键为空。
RENAME	重命名。
REORGANIZE PARTITION	分区重组。
TRUNCATE PARTITION	删除分区数据。
case_value	1) 字段名； 2) 简单表达式(+、-、*、/)； 3) time、date、hour、minute、day、month、year和substr (substring)函数。
when_value	value值。
statement_list	1) case分支，subdistributed by； 2) group值； 3) 多组分发策略； 4) SUBDISTRIBUTED by 支持range分发带区间形式(仅支持左闭右开)及less > than形式； 5) SUBDISTRIBUTED by 支持list分发增加新的group或新的group值; 6) SUBDISTRIBUTED by 支持hash、duplicate分发;

支持如下语法：

- 碎片整理语法：

1. `ALTER TABLE ENGINE` 是在线 DDL 操作，对 DML 操作影响不大。
2. 如果表存在全文索引，`ALTER TABLE ENGINE` 变成离线 DDL，可能会长时间阻塞 DML 操作。
3. DML 操作需要尽早提交，不能有大事务，否则会导致 `ALTER TABLE ENGINE` 在线 DDL 失败。

4. 配置项 `innodb_sort_buffer_size`、`innodb_online_alter_log_max_size` 与 `innodb_online_alter_log_max_size` 设置必须够大，`MySQL` 临时文件夹也必须够大，否则会失败。

```
> ALTER TABLE t2 ENGINE=innodb, ALGORITHM=INPLACE, LOCK=NONE;
```

- 压缩算法语法：

GoldenDB 支持数据压缩功能，可有效减少数据存储所需的空间。压缩率依据表结构和数据内容，与原数据库相比，至少能够节省 70% 的空间。

1. 可在 `CREATE TABLE` 或 `ALTER TABLE` 时指定 COMPRESSION 属性，支持的压缩算法包括 Zlib 和 LZ4。
2. 使用 `ALTER TABLE` 时，只有新数据才会使用压缩算法，如想对已有文件处理，需使用 `OPTIMIZE TABLE`。
3. 操作系统需支持： `sparse file and hole punching`（稀疏文件和文件打洞），每个表使用独立表空间。
4. 配置项 `innodb_compression_level`（1-9）用于动态调整 zlib 的压缩级别，默认为 6。越小压缩效果越差，但压缩速度越快。值越大，压缩效果越好且能降低压缩失败及 page split 的概率，但压缩速度变慢，产生的 CPU 开销也越大。

```
> ALTER TABLE t1 COMPRESSION="zlib";
> OPTIMIZE TABLE t1;
```

- 加密语法：

可在 `ALTER TABLE` 时新增或修改列加密属性，支持的加密方式包括 AES 和 SM4：

```
> ALTER TABLE t1 add column c varbinary(1000) ENCRYPTION= 'SM4';
> ALTER TABLE t1 modify c varbinary(1000) ENCRYPTION= 'AES';
```

示例

- 例子1：

```
> ALTER TABLE db2.t2 ADD COLUMN c1 int, ADD COLUMN c2 int;
```

- 例子2：

```
> CREATE TABLE IF NOT EXISTS t2(a INT PRIMARY KEY,b CHAR(10))DISTR
IBUTED
  BY RANGE(a)(g1[0,100));
  ALTER TABLE t2 DISTRIBUTED BY RANGE(a)(g1[0,100), g2[100,200));
```

- 例子3:

```
> ALTER TABLE db.t2 MODIFY a TINYINT NOT NULL;
```

- 例子4:

```
> ALTER TABLE db.t2 ADD INDEX (d);
```

- 例子5:

```
> ALTER TABLE t1 ADD COLUMN b VARCHAR(10) NOT NULL AFTER a;
```

- 例子6:

```
> ALTER TABLE t1 ALTER COLUMN c DROP DEFAULT;
```

- 例子7:

```
> ALTER TABLE t1 ALTER COLUMN a SET DEFAULT 10;
```

- 例子8:

```
> ALTER TABLE t1 MODIFY col1 BIGINT UNSIGNED DEFAULT 1 COMMENT 'my
  column';
```

- 例子9:

```
> ALTER TABLE t1 MODIFY COLUMN d INT DEFAULT 666;
```

- 例子10:

```
> ALTER TABLE t1 DROP COLUMN d;
```

- 例子11:

```
> ALTER TABLE t1 DROP INDEX idx;
```

- 例子12:

```
> ALTER TABLE t1 RENAME TO t1_new;
```

- 例子13:

```
> ALTER TABLE t1 TRUNCATE PARTITION p0 UPDATE GLOBAL INDEXES STORA
GEDB G1;
```

- 例子14:

```
> ALTER TABLE t1 TRUNCATE SUBPARTITION s0 UPDATE GLOBAL INDEXES ST
ORAGEDB G1;
```

- 例子15:

```
> ALTER TABLE t1 CHANGE b a INT NOT NULL;
```

- 例子16:

```
> ALTER TABLE t1 DISTRIBUTED BY RANGE(a)(g1 [100,200),g2 [400,500)
,g3
  [200,300));
```

- 例子17:

```
> ALTER TABLE t1 DISTRIBUTED BY LIST(a)(g1 VALUES IN (10,23,50),g2
 VALUES
  IN (7,21,60,77),g3 VALUES IN (80,82));
```

- 例子18:

```
> DELIMITER //
ALTER TABLE titles1 DISTRIBUTED BY
case pub_id
when 1 then
case
when cust_id<100 then SUBDISTRIBUTED BY range (title_id)(g1 [
100,200),g4[ 200,300));
else SUBDISTRIBUTED BY duplicate(g2);
end case;
else SUBDISTRIBUTED BY list(title_id)(g3 values in (10,20,30));
END CASE; //
delimiter ;
```

- 例子19:

```
> DELIMITER //
ALTER TABLE titles2 DISTRIBUTED BY
case pub_id
when 1 then
case
when cust_id<100 then SUBDISTRIBUTED BY range (title_id)(g1 [
100,200));
else SUBDISTRIBUTED BY hash(title_id)(g2);
end case;
else SUBDISTRIBUTED BY list(title_id)(g3 values in (10,20,30),g4
values in (40,50));
END CASE; //
delimiter ;
```

- 例子20:

```
> ALTER TABLE db.t2 ADD GLOBAL INDEX g_idx(d);
```

- 例子21:

```
ALTER TABLE db.t2 ADD GLOBAL UNIQUE INDEX g_u_idx(d);
```

- 例子22:

```
> ALTER TABLE db.tb RENAME INDEX func_idx TO new_func_idx;
```

- 例子23:

```
> ALTER TABLE db.tb RENAME COLUMN old_col_name TO new_col_name;
```

不同分片不同分区管理

可以通过 `alter table add/drop/coalesce/reorganize/remove partitioning storagedb` 进行不同分片不同分区表管理。

- 例子1:

```
> ALTER TABLE ps_test.range_x ADD PARTITION (PARTITION p4 VALUES L
ESS
  THAN(500)) STORAGEDB G1, G2;
```

- 例子2:

```
> ALTER TABLE ps_test.range_x DROP PARTITION p0 STORAGEDB G1;
```

- 例子3：

```
> ALTER TABLE ps_test.hash_x COALESCE PARTITION 2 STORAGEDB G1;
```

- 例子4：

```
> ALTER TABLE ps_test.range_x REORGANIZE PARTITION p0 INTO
(
PARTITION s0 VALUES LESS THAN(50),
PARTITION s1 VALUES LESS THAN(100)
) STORAGEDB G1, G2;
```

- 例子5：

```
> ALTER TABLE ps_test.range_x REMOVE PARTITIONING STORAGEDB G1, G2
;
```

使用说明

- 最后一列必须是 `GTID` 列，且不能变更 `GTID` 列的任何属性。
- 不支持增加/删除/修改自增列。
- 不支持 `ALTER RANGE` 与 `ALTER LIST` 间的切换。
- `ALTER RANGE` 不支持用开闭区间分发形式的 `ALTER` 语句去修改原有 `LESS THAN` 形式创建的表结构。
- `ALTER RANGE` 各分片的区间范围不能有交叉、不能修改或删除原有分片范围，不能修改分发键，且区间段数量不超过分片数量最大值限制。
- `ALTER LIST` 各分片的区间范围不能有交叉、不能修改或删除原有分片范围，不能修改分发键。
- `ALTER LIST` 对原有分片增加 `LIST` 值，增加的值不在原有分片内的情况下，不改变原有数据分布。
- 对于 `ALTER` 多级分片表的限制说明。

    1. `RANGE` 分发：

        支持 `ALTER RANGE` 带区间形式（左闭右开）增加分片，但分片的区间范围不能有

交叉、不能修改或删除原有分片范围、不能修改分发键、区间段数量不超过分片数量最大值限制，否则报错。

不支持修改含 `LESS THAN` 语法的 `RANGE` 分发，包括 `ALTER` 为 `RANGE` 区间形式或修改原 `LESS THAN` 范围，否则报错；

2. `LIST` 分发：

   支持修改 `LIST` 分发策略增加分片或 list 值，但不能删除原有分片信息，且新增 list 值不能重复，并且不能修改分发键，否则报错。

3. `HASH/DUPLICATE` 分发：

   不支持修改原表 `HASH` 或 `DUPLICATE` 分发策略。

- `ALTER RANGE` 修改表分发信息支持以下三种操作：

  1. 增加分片；
  2. 已有分片上增加区间段；
  3. 已有区间段取值范围扩充。

  其中，1 和 2 可以同时进行(一条 alter 语句同时增加分片和增加已有分片上的区间段)，但 3 须独立操作，不能和 1 或者 2 同时进行。

- 对于同一条语句进行多个操作的限制：

  仅支持同一条语句进行多个 `add column` 操作，不支持同时进行其他操作。

  当进行多列操作时，仅有所有的 `add` 操作都不指定位置信息的场景为在线操作，不影响业务。

- 对于 `ALTER TABLE DISTRIBUTED BY ...FORCE` 语句，强制更新表元数据，业务的正确性由业务人员自己保证。

- 对于 `ALTER` 创建全局[唯一]索引的限制说明。

  - 支持表创建好之后新增全局索引、全局唯一索引。
  - 支持表创建好之后且表中有数据的情况下新增全局索引、全局唯一索引，数据会自动插入索引辅助表中。
  - 创建全局索引、全局唯一索引时需要给定索引名，不支持不带索引名的情况。
  - 索引名字已经存在，创建全局索引、全局唯一索引时报错。
  - 全局[唯一]索引字段不支持bigint unsigned, binary, blob, text, set, enum, varbinary。
  - 不支持在复制表上创建全局索引、全局唯一索引。
  - 删除全局索引、全局唯一索引，可使用 `ALTER TABLE DROP INDEX`，与普通索引

的删除方法相同。
    - 创建全局索引、全局唯一索引失败后，需使用 `drop remain index 全局索引名 on 表名或库名.表名` 来清理中间过程的残留（包括数据节点上的索引以及索引辅助表）。
    - 对于一个无分区并且带有全局索引的表，通过 `ALTER TABLE PARTITION BY` 语句增加分区，因为全局索引辅助表在添加分区前就以创建，不一定包含分区字段，CN 对于这种操作会自动进行识别处理，如果全局索引辅助表带有新加分区的分区字段，那给全局索引辅助表也增加分区信息，如果不包含分区字段，添加分区的操作只作用于主表，忽略全局索引辅助表。

- 对于 `ALTER TABLE RENAME COLUMN` 的使用说明。
    - 不支持对自增列、分区键的重命名。
    - 不支持界面下发语句块执行。

- 对于 `ALTER TABLE RENAME [INDEX|KEY]` 的使用说明。
    - 支持修改普通本地索引、本地唯一索引、普通全局索引、全局唯一索引、函数索引。
    - 不支持修改临时表的索引名。
    - 不支持界面下发语句块执行。

1.10.7.7. TRUNCATE TABLE

描述

用于清空指定表，但是保留表结构。

语法说明

```
TRUNCATE [TABLE] tbl_name [NO PURGE];
```

示例

```
> TRUNCATE TABLE db1.t1 NO PURGE;
```

使用说明

TRUNCATE TABLE 操作需在客户端链接上执行，且必须有 CREATE 和 DROP 权限。

1.10.7.8. DROP INDEX

描述

删除索引。

语法

```
DROP INDEX index_name ON tbl_name
```

参数解释

参数	描述
index_name	要删除的索引的名称。
tbl_name	指定要删除索引所在的表的名称。

示例

```
> DROP INDEX idx1 ON t1;
```

1.10.7.9. CREATE INDEX

描述

创建表的索引。

语法

```
CREATE [GLOBAL] [UNIQUE] INDEX index_name
ON tbl_name (index_col_name,...)
[index_type]
| CREATE GLOBAL INDEX index_name
ON tbl_name (index_col_name,...)
| CREATE GLOBAL UNIQUE INDEX index_name
ON tbl_name (index_col_name,...)

index_col_name:
col_name [(length)]

index_type:
USING {BTREE HASH}
```

参数解释

参数	描述
[GLOBAL]	可选参数，如果指定了 GLOBAL，表示创建的是全局索引。

[UNIQUE]	可选参数，如果指定了 UNIQUE，表示创建的是唯一索引。
index_name	要创建的索引的名称。
tbl_name	指定要为哪个表创建索引。
(index_col_name,...)	括号内列出了用于创建索引的列名，可以是一个或多个列。
[index_type]	可选参数，允许指定索引的类型，例如 B 树索引、哈希索引等。

示例

- 例子1：

```
> CREATE INDEX idx1 ON db1.t1(a);
```

- 例子2：

```
> CREATE UNIQUE INDEX idx2 ON db1.t2(b);
```

- 例子3：

```
> CREATE GLOBAL INDEX idx2 ON db1.t2(b);
```

- 例子4：

```
> CREATE GLOBAL UNIQUE INDEX idx2 ON db1.t2(b);
```

- 例子5：

```
> CREATE INDEX idx_hash_3 ON db1.t3(c) USING HASH;
```

使用说明

对于全局[唯一]索引

- 支持表创建好之后新增全局索引、全局唯一索引。
- 支持表创建好之后且表中有数据的情况下新增全局索引、全局唯一索引，数据会自动插入索引辅助表中。
- 索引名字已经存在，创建全局索引、全局唯一索引时报错。

- 全局[唯一]索引字段不支持bigint unsigned, binary, blob, text, set, enum, varbinary。
- 不支持在复制表上创建全局索引、全局唯一索引。
- 删除全局索引、全局唯一索引，可使用 `ALTER TABLE DROP INDEX`，与普通索引的删除方法相同。
- 创建全局索引、全局唯一索引失败后，需使用 `DROP REMAIN INDEX 全局索引名 ON 表名或库名.表名` 来清理中间过程的残留（包括数据节点上的索引以及索引辅助表）。

对于外键约束

- CN打开开关support_foreign_key后可以支持分片内的外键约束，打开该开关后在CN使用中不能关闭。

1.10.7.10. COPY TABLE

描述

拷贝表结构及数。

语法

实现从 tb1 拷贝数据到 tb2，全分片拷贝。

```
> COPY TABLE FROM tb1 TO tb2 STORAGEDB all;
```

实现从 tb1 拷贝数据到 tb2，只 copy 单个分片。

```
> COPY TABLE FROM tb1 TO tb2 STORAGEDB g1;
```

参数解释

参数	描述
STORAGEDB	指定分片。

使用说明

- 非分区表限制
  - 不支持异构表（存储引擎/索引/字段类型/字段个数/行格式不一致）。
  - 不支持全文索引/外键/非独立表空间/表空间加密/页面大小不一致。
  - 不支持系统表/多表/视图/临时表。
  - 数据节点的用户需要如下权限 `ALTER`, `DROP`, `RELOAD`, `LOCK TABLES`, > `SELECT`, `CREATE`。
- 分区表限制 > 非分区表的使用 `COPY TABLE` 的限制同样适用于分区表，除此之外还内包括如下限制：

- 不支持含有子分区的表。
- 不支持分区算法/名称/个数/类型/区间不同。
- 配置项说明 `support_copy_table`：
  - 动态生效，默认值是 0。
  - 需要 `COPY TABLE` 功能时，要在配置项[GENERAL]中添加 `support_copy_table=1`。

1.10.7.11. CHECK TABLE

描述

检查表是否存在错误。

语法

```
CHECK TABLE tbl_name [option];

option = {QUICK FAST MEDIUM EXTENDED CHANGED}
```

参数解释

参数	描述
QUICK	执行一个快速的检查，只检查表是否正确关闭。
FAST	检查未正确关闭的表，不扫描行，不检查错误的链接。
MEDIUM	扫描行来检查错误的链接。
EXTENDED	对表进行最彻底的检查，包括逐行检查所有字段。
CHANGED	只检查自上次检查以来更改过的表。

示例

- 例子1：

```
> CHECK TABLE db1.tb1;
```

- 例子2：

```
> CHECK TABLE tb1;
```

- 例子3：

```
> CHECK TABLE tb1 QUICK;
```

- 例子4：

```
> CHECK TABLE tb1 QUICK CHANGED;
```

使用说明

单条 `CHECK TABLE` 语句只支持 `CHECK` 单张表。

1.10.7.12. OUTLINE固定执行计划

描述

SQL 执行时，受上下文环境等影响，会有多种执行计划选择的可能性，不同的执行计划会带来性能较大的差异，因此固定一条稳定且有效的执行计划，可达到稳定生产效率的效果。

语法

```
CREATE [OR REPLACE] [public_option] OUTLINE outline_name
[create_outline_specification]

create_outline_specification:
[FOR CATEGORY category_name] ON [statement]
| FROM [public_option] source_outline

ALTER OUTLINE [public_option] outline [alter_outline_specification
]

alter_outline_specification:
REBUILD
| RENAME TO new_outline
| CHANGE CATEGORY TO new_category_name
| ENABLE
| DISABLE

DROP OUTLINE [IF EXISTS] outline

public_option = {PRIVATEPUBLIC}
```

参数解释

参数	描述

[public_option]	可选参数，用于指定大纲的公共选项。
outline_name	要创建的大纲的名称。
[FOR CATEGORY category_name]	指定大纲所属的类别名称。
ON [statement]	指定要应用大纲的语句。
FROM [public_option] source_outline	从其他公共选项或源大纲中获取创建大纲的信息。

示例

- 例子1：

```
> CREATE OUTLINE ot1 ON SELECT/*+ INDEX(t1 idx)*/  * FROM t1 WHERE id=114528;
```

- 例子2：

```
> CREATE OR REPLACE OUTLINE ot2 ON SELECT/*+ JOIN_PREFIX(t1)*/ * FROM t1 LEFT JOIN t2 on t1.a=t2.a;
```

使用说明

- 固定执行计划仅支持简单下压语句，请配合 `EXPLAIN` 使用。
- 需要在原始未经 CN 改写的语句上创建固定执行计划可以使用 `CREATE OUTLINE2` 语法，在一些场景下语句直接透传到 DN 节点时可以使用，使用时必须带库名。

1.10.7.13. 在线DDL

在线 DDL 不阻塞业务读写访问，对业务基本无影响，业务无需停机。

GoldenDB 支持的在线 DDL 语句包括：

- 创建索引：`CREATE INDEX`。
- 删除索引：`DROP INDEX`。
- 新增索引：`ALTER TABLE ADD INDEX`。
- 删除索引：`ALTER TABLE DROP INDEX`。

- 重命名索引：`ALTER TABLE RENAME INDEX`。
- 重命名列名：`ALTER TABLE RENAME COLUMN`。
- 删除列：`ALTER TABLE DROP COLUMN`。
- 分区操作 `ADD/DROP/COALESCE/TRUNCATE/REMOVE/REORGANIZE PARTITION`。
- 不指定位置信息(`FIRST/AFTER COLUMN`)的增加列操作。
- 增加多列，并且都未指定 `FIRST/AFTER` 时为在线 DDL。
- 修改字段 `COMMENT`。
- 扩展 VARCHAR 类型字段长度，只支持 0~255 字节或大于 256 个字节内部扩，不能从小于 256 个字节扩展到大于 256 个字节。
- 修改分片策略 `ALTER TABLE DISTRIBUTED BY ...` 带 `FORCE HINT` 信息。

1.10.7.14. HINT固定执行计划

描述

对匹配固定执行计划的 SQL 增加 HINT。

语法

```
CREATE [OR REPLACE] GDB_OULINE outline_name ON [sql_statement]
USING_HINT \[hint_statement\]

DROP OUTLINE \[IF EXISTS\] outline_name
```

参数解释

参数	描述
outline_name	要创建的大纲的名称。
sql_statement	指定要应用大纲的语句。
hint_statement	对匹配的语句要加的 hint

示例

- 例子1：

```
> CREATE GDB_OUTLINE ot1 ON select * from t1 where id=114528 USING
_HINT /*+ INDEX(t1 idx) */;
```

- 例子2：

```
> DROP GDB_OUTLINE IF EXISTS ot1;
```

使用说明

- `sql_statement` 和 `hint_statement` 中的 hint 均使用/+ /格式。
- 当 `sql_statement` 和 `hint_statement` 中的 hint 是同类型且冲突时，执行结果等价于 `hint_statement` 的hint插入 `sql_statement` 的 hint 位置之前。

1.10.8. DML

1.10.8.1. INSERT

描述

在表格中插入数据。

语法

```
INSERT INTO tbl_name
[PARTITION number]
[col_name,...]
{VALUES (col_values [,col_values]...),(...),...
select_statement}
[ ON DUPLICATE KEY UPDATE
col_name=expr [, col_name=expr] ... ]
[SAMEDB]
```

- 说明：

  select_statement 使用参考 `SELECT` 说明。

参数解释

参数	描述
tbl_name	指定要插入数据的表名。

PARTITION number	可选参数，用于指定要插入数据的分区编号。
VALUES (col_values [,col_values]...),(...),...	这是提供要插入的数据值。可以插入多行数据，每行数据用括号括起来，并用逗号分隔
select_statement	可以使用一个查询语句的结果来插入数据。
ON DUPLICATE KEY UPDATE col_name=expr [, col_name=expr]...	如果插入的数据导致主键或唯一键冲突，使用此部分来更新冲突的行。
SAMEDB	可选参数，复用前一条语句的下发节点。

示例

- 例子1：

```
> INSERT INTO db1.t1
(
int_1,
integer_1,
smallint_1,
tinyint_1,
mediumint_1,
bigint_1,
decimal_1,
numeric_1,
float_1,
double_1,
date_1,
timestamp_1,
datetime_1,
time_1,
varchar_1,
char_1,
char_2,
varchar_2,
text_1,
binary_1,
binary_2,
varbinary_1,
varbinary_2,
blob_1,
year_1,
```

```
tinyint_2,
enum_1,
set_1)
VALUES
(20150310,1073741824,16384,68,419430
4,4611686018427387903,12345678901234567890123456789012345.12345678
901
2345678901234567809,12345678901234567890123456789054321.1234567890
123
4567890987654321,1.123456789,1.123456789,'2014-10-13','2014-10-13
23:05:10','2014-10-13
23:05:10','23:05:10','aaaaaaaaaabbbbb','bbb
','aaa','xewer','""','hellohellohellohellohellohellohello',
'bbbb','aaaaa2','xxxxx','yyyyyyyy1111',2016,0,5,6);
```

- 例子2：

```
> INSERT INTO t1 (a,b,c) VALUES (1,2,3)
ON DUPLICATE KEY UPDATE c=c+1;
```

- 例子3：

```
> INSERT INTO t1 (a,b,c) VALUES (1,2,3),(4,5,6)
ON DUPLICATE KEY UPDATE c=VALUES(a)+VALUES(b);
```

- 例子4：

```
> INSERT INTO t1 (a, b)
SELECT * FROM
(SELECT c, d FROM t2
UNION
SELECT e, f FROM t3) AS dt
ON DUPLICATE KEY UPDATE b = b + c;
```

- 例子5：

```
> INSERT INTO t1 (a,b,c) VALUES (1,2,3) returning id;
```

使用说明

- `ON DUPLICATE KEY UPDATE` 的使用约束：
  - 表带有全局唯一索引时，不支持 `INSERT` 操作带有 `ON DUPLICATE KEY UPDATE` 子句。
  - 带有此子句的操作，不判断数据是否存在，存在的数据不判断活跃性，不加锁；由于计算节点和存储节点存在延时，且延时会实时变化，所以带有此子句的操作，不建议对复制表使用。

  - 在 `INSERT VALUES` 场景下，配置文件 `proxy.ini`，配置段 GENERAL，配置项 `switch_insert_value_on_duplicate_key_update_sw`，默认为 1，语句的写属性为 SW，否则报错；配置为 0 时，语句写属性可为 CW；配置项修改后可动态生效。
  - 在 `INSERT VALUES` 场景下，配置文件 `proxy.ini`，配置段 GENERAL，配置项 `insert_value_on_duplicate_key_update_usage_level`，默认值为0，要求主键和唯一索引必须包含分发键，相同的主键和唯一索引值会分发到同一个节点上，此功能可认为在整个分布式起作用，但是增加主键和唯一索引会影响此功能使用；配置为1时，不要求主键和唯一索引包含分分发键，只在各个分片上使用此功能；配置项修改后可动态生效。
  - 在 `INSERT VALUES` 语句中，语句写属性为 SW，否则报错处理： `INSERT /*M!gdb_isolation_grade=sw*/ INTO db.tb (id1,id2,id3) VALUES(2,2,2) ON DUPLICATE KEY UPDATE id2=2;`
  - 在 `INSERT VALUES` 语句中，计算节点会判断 `INSERT` 和 `UPDATE` 数据是否在同一数据节点，不在同数据节点或判断不出来时会报错；目前分发键字段对应的右值只能为常量值，本身字段或对本身字段引用的VALUES函数如： `INSERT /*M!gdb_isolation_grade=sw*/ INTO db.tb (id1,id2,id3) VALUES(2,2,2) ON DUPLICATE KEY UPDATE id2=2; INSERT /*M!gdb_isolation_grade=sw*/ INTO db.tb (id1,id2,id3) VALUES(2,2,2) ON DUPLICATE KEY UPDATE id2=id2; INSERT /*M!gdb_isolation_grade=sw*/ INTO db.tb (id1,id2,id3) VALUES(2,2,2) ON DUPLICATE KEY UPDATE id2=VALUES(id2);` 注：假设 `id2` 为 `db.tb` 表的分发键。
  - 同一分发键在子句左值多次出现时，最后一次的右值有效。
  - 在 `INSERT SELECT` 语句中，对带有分发键的表，`ON DUPLICATE KEY UPDATE` 子句的左值不能出现分发键字段；右值出现字段时，只能为 `INSERT` 表的字段，如果字段不属于 `INSERT` 表时，直接报错；如果字段同时出现在 `SELECT` 表中时，不做进一步校验，默认为insert表的字段；此处跟直连 DB 操作不同，直连 DB 操作时，报歧义错误（ERROR 1052 (23000): Column '***' in field list is ambiguous）。
  - 在 `INSERT SELECT` 语句中，唯一索引未包含分发字段存在一个显著的限制：不同数据节点间数据没有唯一性约束。即只能保证其所下压执行 group 上的功能正确性，如果和其它group上的数据有冲突或关联，则无法保证功能整体正确性。
- `INSERT VALUES` 带有同义词时，VALUES 中如果出现字段，不能用同义词修饰。
- `INSERT` 插入数据时，对于分发键列，需直接输入分发键值，避免使用函数取值，例如：

```
> CREATE TABLE t1(id INT KEY, f INT) DISTRIBUTED BY
HASH(id,f)(g1,g2,g3);
> INSERT INTO t1(id,f) VALUES(1,1+1); -->失败
> INSERT INTO t1(id,f) VALUES(1,2); -->成功
```

- 指定 group 下发 `INSERT` 插入数据时，必须指定字段名。 例如：INSERT INTO t1 (id1,id2,id3) VALUES (1,2,3),(4,5,6); 错误示例：INSERT INTO t1 VALUES (1,2,3),(4,5,6);
- `INSERT INTO` SQL 支持表带别名(`INSERT ALL` 除外)

- `INSERT INTO` SQL 不支持对全局自增列字段显示指定具体的值： 例如： CREATE TABLE `t1` (`a` int NOT NULL AUTO_INCREMENT, `b` int DEFAULT NULL, PRIMARY KEY (`a`)) DISTRIBUTED BY HASH(`a`)(g1,g2)/+*support_global_auto_inc/* 报错SQL：INSERT INTO t1 values(1,1); 不报错SQL：INSERT INTO t1(b) values(1);

最佳实践

`INSERT SELECT`

针对 `INSERT INTO SELECT` 语句，考虑到语句的执行效率，在表设计时建议优先考虑下面几种模型：

- 插入表和查询表都是单节点存储表且位于同一个数据节点上。
- 插入表是单节点存储，且查询表是复制表。

对需要建二级索引的表，当大批量数据加载操作，建议采用先加载数据后创建二级索引的模式来提升性能，如果已有索引时，在加载数据前需要先删除。

1.10.8.2. DELETE

描述

删除表中满足条件的数据。

语法

```
DELETE FROM tbl_name
[PARTITION number]
[WHERE where_condition]
[ORDER BY ...]
[LIMIT row_count]
[CONSISTENCY_OPTION]
[STORAGEDB groupno]
[SAMEDB]

CONSISTENCY_OPTION:
[SW CW]

where_condition:
expr

expr:
expr OR expr
| expr expr
| expr XOR expr
| expr AND expr
| expr && expr
| NOT expr
```

```
| ! expr
| boolean_primary IS [NOT] {TRUE FALSE UNKNOWN}
| boolean_primary

boolean_primary:
boolean_primary IS [NOT] NULL
| boolean_primary <=> predicate
| boolean_primary comparison_operator predicate
| predicate
comparison_operator: = >= > <= < <> !=

predicate:
bit_expr [NOT] IN (subquery)
| bit_expr [NOT] IN (expr [, expr] ...)
| bit_expr [NOT] BETWEEN bit_expr AND predicate
| bit_expr [NOT] LIKE simple_expr [ESCAPE simple_expr]
| bit_expr

bit_expr:
bit_expr + bit_expr
| bit_expr - bit_expr
| bit_expr * bit_expr
| bit_expr / bit_expr
| bit_expr DIV bit_expr
| bit_expr MOD bit_expr
| bit_expr % bit_expr
| simple_expr

simple_expr:
literal
| identifier
| function_call
| - simple_expr
| ! simple_expr
| (subquery)
| EXISTS (subquery)
```

参数解释

参数	描述
tbl_name	指定要插入数据的表名。
PARTITION number	可选参数，用于指定要插入数据的分区编号。
WHERE where_condition	可选参数， 用于指定删除满足特定条件的行。

ORDER BY	可选参数，用于指定删除行的排序顺序。
LIMIT row_count	可选参数，用于限制删除的行数。
CONSISTENCY_OPTION	一致性选项，可选 SW 或 CW。
STORAGEDB group no	可选参数，指定下发节点。
SAMEDB	可选参数，复用前一条语句的下发节点。

示例

- 例子1：

```
> DELETE FROM somelog WHERE user = 'jcole'
ORDER BY timestamp_column LIMIT 1;
```

- 例子2：

```
> DELETE t1 FROM t1 LEFT JOIN t2 ON t1.id=t2.id WHERE t2.id IS NUL
L;
```

- 例子3：

```
> DELETE FROM t1 WHERE t1.id=1 returning id;
```

使用说明

- `DELETE` 单表支持 `WHERE` 带子查询，支持程度同 `SELECT WHERE` 带子查询。
- 支持 `DELETE` 单表 `FROM` 多个表。

如：

```
> DELETE tests123.tb2 FROM tests123.tb2 INNER JOIN tests123.tb1 WH
ERE
  tb2.id=tb1.id;
```

- 非指定数据节点场景，`DELETE` 语句只支持 `WHERE` 带有 `SELECT` 子句。
- `DELETE` 并发执行 相关使用说明，详见 SQL性能调优 章节的 语句级并行。

限制说明

- 多表语法中，只能指定删除一张表的数据，即 `DELETE` 后只能跟一张表，FROM后可以跟多张表关联，但是要删除的表必须在 `FROM` 表中的第一个。
- 多表语法中，表未指定别名时必须先执行 `USE db` 语句（`MySQL` 限制）。

最佳实践

- 为了降低加锁范围，`DELETE` 语句需要使用索引。
- 由于存储节点数据节点的处理机制问题，当主键或索引为复合列时，`WHERE` 条件中请避免使用 `IN` 运算符，建议使用 `AND OR` 方式。

举例：

```
> DELETE FROM TABLE WHERE pk1=1 AND pk2=2 OR pk1=3 AND pk2=4;
```

1.10.8.3. UPDATE

描述

更新表中满足条件的数据。

语法

```
Single-table:
UPDATE table_reference
SET col_name1=expr [, col_name2=expr ...]
[WHERE where_condition]
[ORDER BY ...]
[LIMIT row_count]
[CONSISTENCY_OPTION]
[STORAGEDB groupno]
[SAMEDB]

multiple-table:
UPDATE table_references
SET col_name1=expr [, col_name2=expr ...]
[WHERE where_condition]
[CONSISTENCY_OPTION]
[STORAGEDB groupno]

CONSISTENCY_OPTION:
[SW CW]

where_condition:
expr
```

- 说明：

`WHERE` 条件限制遵循 `SELECT` 的限制条件。

支持语法

- `UPDATE 分发键=确定值。
- 支持UPDATE tb SET` 多字段功能。

```
> UPDATE tb SET (coi1,col2)=(SELECT id1,id2 FROM tb2 WHERE ...)
```

- 左值单列多列均可，右值为带聚合函数的关联子查询语句

```
> UPDATE t1_hash SET (a_code,b_code)
= (SELECT coalesce(SUM(a_code),0),MAX(b_code) FROM t1_hash WHERE
EXISTS (SELECT 1 FROM t1_range WHERE t1_hash.infom=t1_range.infom)
);
```

- 右值为带聚合函数关联子查询语句的计算表达式

```
> UPDATE t1 SET a_code = (a_code +(SELECT coalesce(MAX(d_code),0)
FROM t2
  WHERE t1.id=t2.id));
```

参数解释

参数	描述
table_reference	指定要更新的表。
SET col_name1=expr [, col_name2=expr...]	用于设置要更新的列及其新的值，可以同时更新多个列。
WHERE where_condition	可选参数， 用于指定更新满足特定条件的行。
ORDER BY	可选参数，用于指定更新行的排序顺序。
LIMIT row_count	可选参数，用于限制更新的行数。
CONSISTENCY_OPTION	一致性选项，可选 SW 或 CW。

STORAGEDB group no	可选参数，指定下发节点。
SAMEDB	可选参数，复用前一条语句的下发节点。

示例

- 例子1：

```
> UPDATE tb1 SET col1=1 WHERE id=1;
```

- 例子2：

```
> UPDATE tb1 SET(col1,col2)=(SELECT id1,id2 FROM tb2 WHERE id=1);
```

- 例子3：

```
> UPDATE tb1 SET col1=1 WHERE id=1 returning  col;
```

使用说明

- 多表语法中，只能更新第一张表中的列。
- `JOIN` 场景，遵循 `SELECT` 限制。
- 非指定数据节点场景，带有 `SELECT` 子句的 `UPDATE`，`SELECT` 子句遵循 `SELECT` 限制；对于 `SELECT` 语句有重数限制的，`UPDATE` 中的 `SELECT` 语句重数减 1。
- 不支持 `UPDATE SET` 主键/全局（唯一）索引=函数。
- `UPDATE` 的 `SET` 中不支持= AND逻辑。

  Eg：`UPDATE tb1 SET id = 1 AND id1=1 或者UPDATE tb1 SET id = 1 AND 1`。

- 不支持 `SET` 分发键=函数/表达式/自增列/`SEQUENCE`。
- 不支持分发键=值 `STORAGEDB/SAMEDB`。
- 不支持全局索引表修改分发键。
- 支持 `UPDATE tb SET` 多字段功能说明：

只支持单表更新语句，即 `UPDATE db.tb SET` 场景。

语句不支持 `ORDER BY` 和 `LIMIT` 子句。

只支持一个 `SELECT` 语句存在 `SET` 子句中，且此 `SELECT` 子句为对单表的查询语句，不支持 `UNION`、`JOIN`，和通配符*；更新表字段出现在`SELECT` 子句中的关联场景，出现聚合函数时支持单列及多列的查询场景。

目前不支持 `EXPLAIN` 和 `SEQUENCE` 功能。

目前全局索引表不支持更新分发键。

全局索引表不支持将待更新字段用作更新值(或更新值的部分)。

- `UPDATE` 带聚合函数的关联子查询语句只支持隔层关联，不支持跨层关联，且不支持带 `IN/NOT IN` 的嵌套子查询中带聚合函数。
- `UPDATE` SQL 不支持对全局自增列字段显示指定具体的值： 例如： CREATE TABLE `t1` (`a` int NOT NULL AUTO_INCREMENT, `b` int DEFAULT NULL, PRIMARY KEY (`a`)) DISTRIBUTED BY HASH(`a`)(g1,g2)/+*support_global_auto_inc/* 报错SQL： UPDATE t1 set a=1; 不报错SQL：UPDATE t1 set b=1;

并发更新

说明：

- 仅 `UPDATE`、 `DELETE` 语句支持并发更新。
- 目前支持两种并发更新，基于表分区的并发更新（默认）和基于表直方图的并发更新。
- 并发更新 HINT：`/*+ parallel(n [, mod])*/`。 其中，`n` 为期望并发数； `mod` 为并发模式，包含 `PARTITION` （默认）、 `HISTOGRAM` 两种，不写默认为 `PARTITION`。
- 实际的并发数 和 并发更新的执行效果 因受 待更新表的结构(包括表定义、分区数量或直方图定义等)、数据分布、系统资源 等客观因素影响，与并发更新 HINT 指定值可能存在一定偏差，属于正常现象。
- 推荐使用基于表分区的并发更新功能。

Eg：

```
update /*+parallel(10,HISTOGRAM)*/ t1 set id6 = 3;

delete /*+parallel(10,PARTITION)*/ from t1;
```

使用限制：

- 并发更新仅支持 `UPDATE/DELETE` 单表。
- 并发更新 HINT 需紧跟 `UPDATE/DELETE` 关键字。
- 必须开启半一致性读。
- 并发更新语句不含子查询、 `ORDER BY`、 `LIMIT` 等。
- 待更新表上未建有触发器。
- 使用并发更新功能，需开启自动提交，或单语句的事务（带并发更新 HINT 的一条语句只能单条执行，不能和其他 SQL 放在一个事务里）。
- 语句更新数据量过大，或并发数过多时，会长时间占用大量系统资源，这可能会导致诸如 DN 节点断链等异常。
- 基于表分区的并发更新，要求表上建有分区（不同分片不同分区表不支持并发更

新），且对于 `UPDATE` 语句，待更新列不能为分区键和分发键。

- 基于直方图信息的并发更新，要求表上建有直方图，且对于 `UPDATE` 语句，待更新列不能完全覆盖建有直方图的列。
- 基于直方图信息的并发更新，建有直方图的所有字段（其中包含数值类型字段时，此处仅指所有的建有直方图的数值类型字段），数据区分度较低，或数据分布非常不均匀的场景下，可能会引起锁冲突，导致执行失败，建议在数据区分度高，且分布均匀的字段上建直方图。

最佳实践

关联方式的多表优化

关联方式的多表操作语句如下：

```
UPDATE db.t1 SET aa=1 WHERE EXISTS(SELECT 1 FROM db.t2 WHERE
t1.aa=t2.aa)
```

考虑到语句的执行效率，在表设计时建议优先考虑下面几种模型：

- 更新表和从表都是单节点存储表的且位于同一个数据节点上。
- 更新表是单节点存储，且从表是复制表。

非关联方式的多表优化

非关联方式的多表操作语句如下：

```
> UPDATE db.t1 SET aa=1 WHERE EXISTS(SELECT 1 FROM db.t2)
```

考虑到语句的执行效率，在表设计时建议优先考虑下面几种模型：

- 更新表和子查询表都是单节点存储表，且位于同一个存储节点上。
- 子查询表是复制表。

其他优化

- 多表方式复杂度高，比较影响性能，建议尽量使用单表方式的 `UPDATE`。
- 为了降低加锁范围，`UPDATE` 语句需要使用索引。
- 由于存储节点数据节点的处理机制问题，当主键或索引为复合列时，`WHERE` 条件中请避免使用 IN 运算符，建议使用AND、OR方式。

举例：

```
> UPDATE table SET a=2 WHERE pk1=1 AND pk2=2 OR pk1=3 AND pk2=4;
```

1.10.8.4. REPLACE

描述

插入新记录或者更新已存在的记录。

语法

```
REPLACE [INTO] tbl_name
[(col_name,...)]
{VALUES | VALUE} ({expr DEFAULT},...),(...),...
```

参数解释

参数	描述
tbl_name	要执行替换操作的表名。
(col_name,...)	可选的列名列表。 如果没有指定列名，那么提供的值的顺序就需要和表最初定义的列的顺序保持一致。
expr DEFAULT	具体要插入或用于替换的值。每个值的集合都用括号括起来，多个值集合之间用逗号分隔, 值可以是表达式。

示例

```
> REPLACE INTO table (id,nu) VALUES(1,4);
```

使用说明

`REPLACE INTO` 支持自增列字段不显示指定 - `REPLACE INTO` SQL 不支持对全局自增列字段显示指定具体的值： 例如： CREATE TABLE `t1` (`a` int NOT NULL AUTO_INCREMENT, `b` int DEFAULT NULL, PRIMARY KEY (`a`)) DISTRIBUTED BY HASH(`a`)
(g1,g2)/+*support_global_auto_inc*/ 报错SQL： REPLACE INTO t1 values(1,1); 不报错SQL：
REPLACE INTO t1(b) values(1); #### MERGE INTO

描述

将源表中的数据语更新获插入的方式合并到目标表中。

语法

```
MERGE INTO [target-table] A
USING [source-table sql] B
ON([conditional expression] and [...]...)
```

```
WHEN MATCHED THEN
[UPDATE sql [WHERE...] ]
WHEN NOT MATCHED THEN
[INSERT sql [WHERE ...] ]
```

其中 `source-table` 支持为虚拟表

参数解释

参数	描述
MERGE INTO	这是执行合并操作的关键字。
[target-table] A	指定目标表，并为其赋予别名 A 。
USING [source-table sql] B	指定用于合并的源表或源表的查询语句，并赋予别名 B 。
ON([conditional expression] and [...]...)	定义合并的条件表达式，用于确定哪些行匹配或不匹配。
WHEN MATCHED THEN [UPDATE sql [WHERE...]]	当在目标表中找到与源表匹配的行时，执行指定的更新操作，并可以使用 WHERE 子句进一步限制更新的行。
WHEN NOT MATCHED THEN [INSERT sql [WHERE...]]	当在目标表中未找到与源表匹配的行时，执行指定的插入操作，同样可以使用 WHERE 子句进行条件限制。

示例

- 例子1：

```
> MERGE INTO t1 USING t2 ON(t1.b=t2.b) WHEN MATCHED then UPDATE SET
t1.c = t2.c WHERE t2.a>0; # 只含有MATCHED
> MERGE INTO t1 q USING (select a as a1,b as b1,c c1 FROM t2) p
ON(q.b=p.b1) WHEN MATCHED then UPDATE SET
q.c = p.c1 WHERE p.a1>0; # USING子查询
```

- 例子2：

```
> MERGE INTO t1 USING t2 ON(t1.b=t2.b) WHEN NOT MATCHED then INSER
T
(t1.a,t1.b,t1.c) values(t2.a,t2.b,t2.c) WHERE t2.a>0; # 只含有NOT M
ATCHED
> MERGE INTO t1 q USING (select a as a1,b as b1,c c1 FROM t2) p
ON(q.b=p.b1) WHEN NOT MATCHED then INSERT (q.a,q.b,q.c)
values(p.a1,p.b1,p.c1) WHERE p.a1>0; # USING子查询
```

- 例子3：

```
> MERGE INTO t1 USING t2 ON(t1.b=t2.b) WHEN MATCHED then UPDATE SE
T
t1.c = t2.c WHERE t2.a>0 WHEN NOT MATCHED then INSERT
(t1.a,t1.b,t1.c) values(t2.a,t2.b,t2.c) WHERE t2.b>0; # 同时含有MAT
CHED和NOT MATCHED
> MERGE INTO t1 q USING (select a as a1,b as b1,c c1 FROM t2) p
ON(q.b=p.b1) WHEN MATCHED then UPDATE SET
q.c = p.c1 WHERE p.a1>0 WHEN NOT MATCHED then INSERT
(q.a,q.b,q.c) values(p.a1,p.b1,p.c1) WHERE p.a1>0; # 同时含有MATCHE
D和NOT MATCHED
```

- 例子4：

```
> MERGE INTO t1 USING dual ON (b=63) WHEN MATCHED then UPDATE SET
c=777 WHERE d=123456789 WHEN NOT MATCHED then INSERT
 (b,c,d) values(7777,77,77);    # source-table为虚拟表
```

限制说明

- `MERGE INTO` 不支持视图。
- 不支持 `UPDATE + DELETE` 操作。
- 仅含有 `MATCHED` 的情况，限制同 `UPDATE`。
- 仅含有 `NOT MATCHED` 的情况，限制同 `INSERT`。
- 同时含有 `MATCHED` 和 `NOT MATCHED` 的情况，不支持操作全局索引表和修改分发键。
  ***

1.10.9. DQL

1.10.9.1. SELECT

描述

用于在数据库表中检索数据。

语法

```
SELECT
expr , ...
[FROM table_references]
[WHERE where_condition]
[PARTITION number]
[GROUP BY col_name]
[HAVING where_condition]
[ORDER BY col_name [ASC DESC], ...]
[LIMIT [offset,] row_count ]
[FOR UPDATE [OF ...] LOCK IN SHARE MODE]]
[CONSISTENCY_OPTION]
[STORAGEDB groupno]
[READ_STRATEGY]
[SAMEDB]

READ_STRATEGY:
[READMASTER READSLAVE READBALANCE]

table_references:
table_reference [, table_reference] ...

table_reference:
table_factor
| join_table

table_factor:
tbl_name [[AS] alias]
| table_subquery [AS] alias

join_table:
table_reference [INNER ] JOIN table_factor [join_condition]
| table_reference {LEFTRIGHTCROSS} JOIN table_reference
join_condition

join_condition:
ON conditional_expr

where_condition:
expr
```

同时 GoldenDB 兼容支持 `Oracle` 在 `WHERE` 子句中使用 `+` 的左右连接用法。

左连接示例如下：

```
> SELECT * FROM testjoina a, testjoinb b WHERE a.id=b.id1(+);
```

参数解释

参数	描述

expr,...	指定要选择的列或表达式。
FROM table_references	指定要从哪些表或视图中获取数据。
WHERE where_condition	用于筛选出满足特定条件的行。
PARTITION number	可选参数，指定要操作的分区编号。
GROUP BY col_name	按照指定的列对结果进行分组。
HAVING where_condition	在分组之后，用于筛选分组的条件。
ORDER BY col_name [ASC DESC],...	按照指定的列对结果进行排序，ASC 表示升序（默认），DESC 表示降序。
LIMIT [offset,] row_count	用于限制返回的行数，可以指定偏移量和行数。
FOR UPDATE [OF...] LOCK IN SHARE MODE	用于锁定行以进行更新或共享读取。
CONSISTENCY_OPTION	一致性选项。
STORAGEDB groupno	可选参数，指定下发节点。
READ_STRATEGY	读取策略。
SAMEDB	可选参数，复用前一条语句的下发节点。

示例

- 例子1：

```
> SELECT CONCAT(last_name,', ',first_name) AS full_name FROM mytab
le ORDER BY full_name;
```

- 例子2:

```
> SELECT t1.name, t2.salary FROM employee AS t1, info AS t2 WHERE
t1.name
  = t2.name;
```

- 例子3:

```
> SELECT college, region, seed FROM tournament ORDER BY region, se
ed;
```

- 例子4:

```
> SELECT a, COUNT(b) FROM test_table GROUP BY a ORDER BY NULL;
```

- 例子5:

```
> SELECT COUNT(col1) AS col2 FROM t GROUP BY col2 HAVING col2 = 2;
```

- 例子6:

```
> SELECT user, MAX(salary) FROM users GROUP BY user HAVING MAX(sal
ary) > 10;
```

- 例子7:

```
> SELECT * FROM tbl LIMIT 5,10;
```

- 例子8:

```
> SELECT 12 AS a, a FROM t GROUP BY a STORAGEDB g1;
```

- 例子9:

```
> SELECT college, region, seed FROM tournament ORDER BY region, se
ed FOR
  UPDATE;
```

使用说明

SELECT LIST

- 不支持关系表达式（AND/OR/&&/||之类）。
- 不支持 SOME/ANY/ALL 操作。
- `SELECT LIST` 中不支持自定义函数与其他函数或常量的嵌套组合。
- 查询字段最大数为 2048 个，列名称或别名最大长度为 256 字节。
- 涉及多表 SQL 语句中， `SELECT LIST` 中可以含有二义性字段，该字段自动拼接第一个匹配上的表的名字，不会报错。如下（`a` 是 `t1`、`t2` 同名字段）。

```
> SELECT a FROM t1 JOIN t2 ON t1.b1 = t2.b2;
```

FUNCTION

- `COUNT` 后面支持 `DISTINCT` 单个表达式列表的语法。

如：

```
> SELECT COUNT(DISTINCT a) FROM tests123.tb1;
```

- 聚合函数中含有表达式、表达式中含有聚合函数时精度和数据节点不一致，聚合函数中有 FLOAT、DOUBLE，只能保证有效精度范围之内的结果和数据节点一致。
- 汇聚函数中不能嵌套汇聚函数。
- `SUM`、`AVG` 需使用数值类型(整型/浮点/DECIMAL)。
- `GROUP_CONCAT` 目前带 `DISTINCT` 去重存在问题（示例：`SELECT GROUP_CONCAT(DISTINCT id) FROM t1`），但带有 `DISTINCT` 和 `ORDER BY` 的场景是支持的（示例：`SELECT GROUP_CONCAT(DISTINCT id ORDER BY id) FROM t1`）。

`GROUP_CONCAT` 列最大字段长度为 1k，`GROUP_CONCAT` 含有列个数最多支持 16 个。

WHERE

- `SELECT` 表达式中支持 `=SOME/=ANY/<>ALL` 非直接下压情况，`=SOME/=ANY/<>ALL` 限制同 `IN/NOT IN`，`SOME/ANY/ALL` 范围比较不支持。
- 不支持 `WHERE SOME/ANY/ALL` 子查询执行 `JOIN` 操作。
- `WHERE` 条件中支持对分片键使用 `substr` 函数，能够精准下发。

data type

BLOB、TEXT系列类型只能单独出现在最顶层 `SELECT LIST` 中。

多表

- 支持 `SELECT ... FROM t1,t2,t3... WHERE ...`，限制场景同 `JOIN` 外，还增加 `WHERE` 条件中不能有 `OR`。
- `UNION/UNION ALL` 语句的合并暂时支持最大只有一张非复制表场景。
- 多表查询中，如果多表有多个相同的字段名，字段被添加的 db、tb 会是他遇到的第一个库、表；因此这种情况，建议手动添加上 db、tb。
- 只支持 `FROM`、`SELECT LIST` 和 `WHERE` 子查询，且 `FROM` 子查询最多嵌套 10 层。
- `SELECT LIST` 中含有子查询时存在以下约束：
  - 在 CR 场景下目前只支持 `single row` 类型的子查询。
  - 子查询为非 `single row` 类型时，不支持主子查询不能合。
  - 不支持含有 `Oracle` 函数。
  - 不支持需要在计算节点层做 `WHERE` 和 `HAVING` 过滤的 `SELECT LIST` 子查询。
- 子查询支持 `EXISTS`/`NOT EXISTS`/`IN`/`NOT IN`/比较运算符。
- 支持等值条件的关联查询，即直接的等值操作，如 `db.t1.a=db.t2.a`，其它场景不支持。
- `IN`/`NOT IN` `WHERE`子查询不支持有 `LIMIT` 且不能合并场景。
- FLOAT，DOUBLE 类型字段出现 `WHERE` 子查询条件中且不能合并时，只能是 `EXISTS` 子查询。

SELECT ... readslave

- 不论是否开启读写分离，带 `readslave` 的 `SELECT` 语句，满足读写分离条件的情况下，优先选择备 DB 下发。

其它

- `ORDER BY`，`GROUP BY`，`HAVING` 支持 `SELECT LIST` 中的字段别名，`JOIN ON` 不支持 `SELECT LIST` 中的字段别名。

  如：

```
> SELECT t1.id AS ID,t2.id AS TB FROM t1 JOIN t2 on ID=TB;
```

- `GROUP BY` 后不支持聚合函数/常量。

- 不支持聚合函数别名与字段名相同且该字段或同名字段位于 `GROUP BY` 中的场景。

  如：

```
> SELECT SUM(t1.b) AS b FROM t1,t4 GROUP BY SUM(t1.b);
```

- 不支持 EXISTS/NOT EXISTS 中的 SELECT 查询字段为外部表字段

如：

```
> SELECT REF_1.ENTITY AS C0, REF_1.TYPE AS C1  FROM GD.EC_ORG_ROLE
 AS REF_1 WHERE NOT EXISTS ( SELECT REF_2.UPDATE_TIME AS C0, NULL
AS C2, REF_1.TYPE AS C3 FROM GD.POIN_FLOW_NODE_CMD AS REF_2);
```

- `ORDER BY` 后面的每个字段结果的长度不能超过 32。
- `ORDER BY` 需要计算节点重新排序(数据节点排好序后计算节点在此基础上再排序不包括在内)的最大列数不能超过 30 列(31 列开始不支持)。
- `ORDER BY/GROUP BY` 中字段必须出现在 `FROM` 子查询的 `SELECT LIST` 中。
- `HAVING` 后面不支持 BLOB 类型。
- `HAVING` 后的表字段如果不是聚合函数参数，则需要在查询字段中出现。
- 支持 `HAVING CASE WHEN`。
- 系统变量只支持变量本身查询，不支持四则运算与函数，不支持 `UPDATE`、`DELETE`、`INSERT`，变量不支持写操作，例如 `SELECT id INTO @a FROM t1`。
- 支持：

```
> SET@a=0; SET@b=1
> SELECT @a,@b FROM t1;
```

- 不支持：

```
> SELECT @a+1 FROM t1;
> SELECT @a+@b FROM t1;
> SELECT abs(@a) FROM t1;
```

- 不支持 `SELECT ...FROM` 子查询 `FOR UPDATE OF`
- 视图
  - 视图不支持 `SELECT *`。
  - 视图不支持 `SELECT NULL`。
  - 视图不支持子查询；
    - 分布式视图(`support_distribute_view = 1`)支持 DML 操作，要求视图的源表为单表，视图查询的字段包含全部的主键和分发键，相关字段 不能有函数处理。多级分片表视图不支持 DML 操作。当视图对应的源表发生元数据变更后，要重新刷新视图才可以进行 DML 操作。
- `DISTINCT`
  - `DISTINCT` 对字符串尾部空格不去除。
- 计算节点对行记录大小 1M 的使用说明
  - 当查询结果需要在计算节点层做二次处理的时候，比如跨分片 `JOIN`、`ORDER BY` 等场景下，为了防止结果集以及单行记录过大，计算节点计算耗时过长导致查询语句长

时间挂住，计算节点对结果集行记录进行了 1M 的大小限制：如果单行记录长度超过 1M，则计算节点报错。
    - 当 `SELECT` 语句仅涉及到单 group，比如复制表或者 `WHERE` > 分发键=具体值（能计算出具体分片），计算节点不需要对结果集进行二次处理，没有行记录大小 1M 的限制。
    - 1M是固定限制，计算节点无相关配置。
- `START WITH ... CONNECT BY ...`
    - `FROM` 的表是只能是单表。
    - 不支持参数：`nocycle`，`connect_by_iscycle`，`connect_by_isleaf`，`level`。
    - `CONNECT BY` 后面条件，多分片只支持 列=列，不支持 > 列=变量、表达式、 函数；单分片无此限制。
    - 语法顺序只支持：`START WITH ... CONNECT BY`，顺序调换不支持。
    - 不支持 `START WITH` 嵌入在关联子查询中。
    - 不支持多级分片。
    - 多分片不支持视图。
    - 结果顺序跟 `Oracle` 不一样，结果集一样。
- 批量查询`sequence`值
    - 提供语法`SELECT @SEQUENCE_NAME.NEXTVAL FROM DUAL CONNECT BY ROWNUM <= @num` 进行一次性获取多个sequence值
    - `@SEQUENCE_NAME` 为用户创建的普通sequence名，`@num`为大于0常量数字，期望获取到sequence个数
    - 获取`sequence`数量最小值为1，最大值为300000
    - 不支持实表关联查询或子查询,不支持join on
- 兼容 `Oracle` 闪回查询
    - 支持 `as of scn/timestamp expr` 语法。
- `SELECT ... ROWNUM\LIMIT` 限定条数的情况下,可使用 hint 从所有分片均匀取数据 > 如：

SELECT /*+uniform_select*/ * FROM t1 LIMIT 10000;
SELECT /*+uniform_select*/ * FROM t1 WHERE ROWNUM < 10000;

最佳实践

子查询的优化

- `FROM` 子查询

语句如下：

```
> SELECT sb1,sb2,sb3
FROM (SELECT s1 AS sb1, s2 AS sb2, s3*2 AS sb3 FROM db.t1) AS sb
WHERE sb1 > 1;
```

语句可以优化转换，去掉子查询，语句转换为：

```
> SELECT s1 AS sb1, s2 AS sb2, s3*2 AS sb3 FROM db.t1 WHERE s1 > 1
;
```

- `EXISTS` 子查询 > 语句如下：

```
> SELECT t1.id FROM t1
WHERE EXISTS (SELECT id FROM t2 WHERE t1.id = t2.id and  t2.id1 =2
);
```

语句可以加优化的 hint：

```
> SELECT t1.id FROM t1 /*+set_optimize_sqlmod=exists_to_in*/
WHERE EXISTS (SELECT id FROM t2 WHERE t1.id = t2.id and  t2.id1 =2
);
```

加 hint 后转换为：

```
> SELECT t1.id FROM t1
WHERE t1.id (SELECT id FROM t2 WHERE t2.id1 =2);
```

`EXIST` 转 `IN` 优化在 `UPDATE`、`DELETE` 语句中同样适用。

- `NOT EXIST` 子查询 > 语句如下:

```
> SELECT t1.id FROM t1
WHERE NOT EXISTS (SELECT id FROM t2
WHERE t1.id = t2.id and  t2.id1 =2);
```

语句可以加优化的 hint：

```
> SELECT t1.id FROM t1 /*+set_optimize_sqlmod=noexists_to_leftjoin
*/
WHERE NOT EXISTS (SELECT id FROM t2 WHERE t1.id = t2.id and t2.id1
 =2);
```

加 hint 后转换为：

```
> SELECT t1.id FROM t1 LEFT JOIN (SELECT id as __gtid_col1 FROM t2
)
_gtid_alias ON (t1.id = _gtid_alias.__gtid_col1 AND t2.id1 = 2)
WHERE _gtid_alias .__gtid_col1 IS NULL;
```

`NOT EXIST` 转 `LEFT JOIN` 优化在 `UPDATE`、`DELETE` 语句中同样适用。

- `IN` 子查询 > 语句如下:

```
> SELECT * FROM t1 WHERE a IN (SELECT a FROM t2);
```

语句可以加优化的 hint:

```
> /*+set_optimize_sqlmod=batch_convert*/ SELECT * FROM t1 WHERE a
IN (SELECT a FROM t2);
```

加 hint 后转换如下，其中 `n` 是 `proxy.ini` 中 `subwhere_result_line` 设置的值，表示把子查询结果集等分下压到 DN:

SELECT * FROM t1 WHERE a IN (常量1, 常量2 ... 常量n); SELECT * FROM t1 WHERE a IN (常量n+1, 常量n+2 ... 常量2n); SELECT * FROM t1 WHERE a IN (常量2n+1, 常量2n+2 ... 常量3n);

`IN` 批量下压优化在 `UPDATE`、`DELETE` 语句中同样适用。

- 关联子查询

语句如下:

```
> SELECT col1,col2,col3 FROM db.t1 WHERE col1 IN
(SELECT col1 FROM db. t2 WHERE db.t2.col2 = db.t1.col2);
> SELECT col1,col2,col3 FROM db.t1 WHERE EXISTS
(SELECT col1 FROM db.t2 WHERE db.t2.col2 = db.t1.col2)
```

考虑到语句的执行效率，在表设计时建议优先考虑下面几种模型:

- 主表和子查询表都是单节点存储表，且位于同一个 DN 节点上。
- 主表是单节点存储，且子查询表是复制表。
- 非关联子查询

语句如下:

```
> SELECT col1 FROM db.t1 WHERE EXISTS (SELECT col1 FROM db.t2);
> SELECT s1 FROM db.t1 WHERE s1 IN (SELECT s1 FROM db.t2);
```

考虑到语句的执行效率，在表设计时建议优先考虑下面几种模型:

- 主表和子查询表都是单节点存储表，且位于同一个 DN 节点上。
- 子查询表是复制表。
- 子查询含有 `DISTINCT`

语句如下：

```
SELECT col1,col2,col3 FROM db.t1 WHERE col2 IN (SELECT DISTINCT co
l2
  FROM db.t2);
```

考虑到语句的执行效率，在表设计时建议优先考虑下面几种模型：

- 主表和子查询表都是单节点存储表的，且位于同一个 DN 节点上。
- 子查询表是复制表。
- 子查询含有分组聚合。

语句如下：

```
> SELECT col2 FROM db.t1 WHERE col1 IN (SELECT SUM(col1) FROM db.t
2);
```

考虑到语句的执行效率，在表设计时建议优先考虑下面几种模型：

- 主表和子查询表都是单节点存储表的，且位于同一个 DN 节点上。
- 子查询表是复制表。
- 带聚合函数的关联子查询在 `SELECT WHERE` 中

语句如下：

```
> SELECT * FROM t1_hash WHERE a_code > (SELECT COUNT(c_code) FROM
  t1_range WHERE t1_hash.infom=t1_range.infom);
```

具体支持情况如下：

1) 只支持 `SUM`、`MAX`、`MIN`、`COUNT`，暂不支持其他聚合函数。

2) 关联条件支持使用 and 拼接多个条件，但只支持等值关联方式。

3) `SELECT WHERE` 带聚合函数的有关联条件的嵌套子查询支持关联多级分片类型的表，支持不同隔离级别 `sw/cw/ur/cr`，支持相同/不同分发类型的关联表。

4) 带聚合函数的有关联条件的嵌套子查询支持多层嵌套，但关联条件仅出现在相邻两层，不支持隔层表关联等值条件。

5) 不支持带 `IN/NOT IN` 的嵌套子查询中带聚合函数。

- 带聚合函数的关联子查询在计算表达式中

语句如下：

```
> SELECT a_code - (SELECT MIN(d_code) FROM t1 WHERE t1.id=t2.id) F
ROM t2;
```

具体支持情况如下：

1) 只支持 `SUM`、`MAX`、`MIN`、`COUNT`，暂不支持其他聚合函数。

2) 关联条件支持使用 and 拼接多个条件，但只支持等值关联方式。

3) 表达式支持 +、-、*、/、mod、div。

JOIN的优化

- 使用说明

  `JOIN` 的规避方法：

  一般情况下 `JOIN` 复杂度和执行效率上都比简单查询的代价要高，建议在表的模型设计时，采用去范式化手段把 `JOIN` 转换为简单查询。

- 多表关联：关联查询的表个数不能超过 61 个。
- `LEFT JOIN/RIGHT JOIN` 能合并的场景，第一张外表不能是复制表。
- 不支持 `natural JOIN` 语法。

  如：

```
> SELECT tests123.tb1.id FROM tests123.tb1 natural RIGHT JOIN
  tests123.tb2;
```

- 不支持 `ON` 条件里存在子查询。

  如：

```
> SELECT id FROM t1 JOIN t2 on t1.id=(SELECT 1 FROM t2);
> SELECT * FROM t1 LEFT JOIN t2 on t1.id=t2.id AND t1.id1=(SELECT
id FROM t3 LIMIT 1);
```

- 多表 `JOIN` 的用法：当前关联表名必须在其后的 `ON` 条件中出现。
- 大小表的判断标准：

  小表和大表的区分不是根据表的原始数据来判断的，而是根据条件过滤后的数据来判断。

- 小表和小表之间两表关联

  考虑到语句的执行效率，在表设计时建议优先考虑下面几种模型：

- 两个表都是单节点存储表的，且位于同一个 DN 节点上。
- 存在一个表是复制表。

- 小表和大表的关联

考虑到语句的执行效率，在表设计时建议优先考虑：大表采用多节点分布存储，小表采用复制表。

- 使用 hint 优化

已知大表、小表分别是哪个时，对于简单的两表 `JOIN` 语句，可以使用hint 进行优化，提高查询效率。该 hint 为：`/*+ leading(small_table_name) use_nl(big_table_name) */`，示例如下：

```
> SELECT /*+ leading(tb1) use_nl(tb2) */ tb1.colour, tb2.location
FROM
    tb1 LEFT JOIN tb2 ON tb1.id=tb2.id AND tb1.location=tb2.locati
on;
```

该 hint 支持两表做简单的 `INNER JOIN`、`CROSS JOIN`、逗号 `JOIN`、`LEFT JOIN`、`RIGHT JOIN` 或者 (+) 构成外联结的场景，使用时需要注意以下事项：

- 支持 `SELECT`、`INSERT SELECT` 语句，不支持其他类型的语句。
    - 支持 `FROM` 子查询，不支持其他类型的子查询。
    - 不支持语句中有跨层关联字段。
    - 如果语句采用了其他优化手段，可能不再走该优化。
    - 具体优化效果依表内数据和具体语句而定。
    - 根据表数据量优化。

在支持使用 hint 优化的基础上，支持根据表中数据的行数自动判断小表和大表，实现与使用 hint 相同的优化效果。该功能支持两表 `JOIN` 语句，也支持三表及三表以上的多表 `JOIN` 语句。

该功能对应有 CN 配置项 `small_table_rows_num`，用于配置小表数据量的上限值。配置项位于 `proxy.ini` 文件 [GENERAL] 段中，默认值为 0，表示不开启该优化。自动判断大小表需要用到表总行数，需要提前执行语句 `ANALYZE HISTOGRAM;` 用于获取当前时刻所有表的总行数；如果想使用表结果集的行数，则需要提前在关注的字段上额外建立直方图。

需要注意获取的表行数是近似值，并不与表中实际数据量完全一致。其他注意事项与使用 hint 优化相同。

分组聚合的优化

考虑到语句的执行效率，在表设计时建议优先考虑采用单节点存储方式，或复制表方式。

两个限制：

- `COUNT(DISTINCT)` 函数会导致网络间大量的网络传输，性能较差。
- 在非脏读隔离级别下，查询中带聚合函数时，如果数据存在读和写并发的情况，可能会出现聚合函数计算不准确的问题，即出现脏读，业务需要慎重考虑此场景的影响。

分页的优化

- 传统分页

```
> SELECT db.table.col2 FROM db.table LIMIT 10000,10;
```

说明：

偏移量越大则越慢。

- `LIMIT` 方式推荐分页

方式一：

```
> SELECT col1 FROM db.table WHERE id>=23423 LIMIT 10; # (每页10条)
```

说明：

只适合包含单增类型字段的场景。 如果存在 `ORDER BY` 时，排序字段中需要包含此单增类型字段。

方式二：

```
> SELECT id FROM table WHERE id>1 LIMIT 10000,10;
> SELECT col FROM table WHERE id IN ('123',456...) ;
```

说明：

id是指主键或者唯一索引。 如果存在 `ORDER BY` 时，排序字段中需要包含此字段。

- `LIMIT` 分页与缓存结合

先一次性查询多页记录数据缓存到内存，业务程序实现分页功能。

1.10.9.2. UNION

描述

用于合并两个或多个 `SELECT` 语句的结果集。

语法

```
SELECT ...
  UNION [ALL ] SELECT ...
  [UNION [ALL ] SELECT ...]
```

参数解释

参数	描述
UNION	用于将多个 SELECT 语句的结果集合并在一起。
ALL	如果使用了 UNION ALL，则会保留所有的结果，包括重复的行。如果只是使用 UNION，则会自动去除重复的行。

示例

- 例子1：

```
> (SELECT a FROM t1 WHERE a=10 AND b=1 ORDER BY a LIMIT 10)
  UNION
  (SELECT a FROM t2 WHERE a=11 AND b=2 ORDER BY a LIMIT 10);
```

- 例子2：

```
> SELECT column_name(s) FROM table_name1
  UNION ALL
  SELECT column_name(s) FROM table_name2
```

使用说明

- `UNION` 语句对于跨类型的连续多个 `UNION` 操作不保证结果准确。
- `UNION` 语句不支持 BLOB 和 TEXT 及相关的大字段类型。
- 对于应用在 `UNION` 语句前的应用 `ORDER BY` 的 `SELECT` 语句，应在添加括号后使用，如用例 1。

1.10.9.3. 通用表表达式

描述

通用表表达式（Common Table Expressions，CTE）是一个命名的临时结果集，作用范围是当前语句，不实际作为对象存储，仅在查询执行期间被使用；与派生表不同，CTE 可以是自引

用，也可以在同一查询中多次引用。

使用场景： 通过将复杂的子查询提取为 CTE，可以使整个查询结构更清晰，更容易理解。 定义一次 CTE 后，可以在同一个查询中的多个地方使用它，避免重复编写相同的子查询逻辑。 对于一些需要分步计算或处理复杂关系的查询，CTE 可以将这些步骤清晰地分开，使查询逻辑更有条理。

GoldenDB 支持非递归的 CTE。

语法

```
WITH cte_name [(column_name [, column_name] ...)] AS (subquery)
      [, cte_name [(column_name [, column_name] ...)] AS (subquery
)] ...
```

参数解释

参数	描述
cte_name	命名通用表表达式，可被包含 WITH 子句的的表引用。
column_name	选的列名列表，用于为 CTE 中的列指定别名。这使得在主查询中，可以使用更易读的列名。
AS(subquery)	CTE 子查询，用于生成 CTE 结果集。AS 后面必须带括号。

示例

```
> WITH cte1 AS (SELECT a, b FROM t1), cte2 AS (SELECT c, d FROM t2
)
SELECT b, d FROM cte1 JOIN cte2 WHERE cte1.a = cte2.c;
> WITH cte (col1, col2) AS( SELECT 1, 2 UNION ALL SELECT 3, 4) SEL
ECT col1, col2 FROM cte;
> WITH cte AS( SELECT 1 AS col1, 2 AS col2 UNION ALL SELECT 3, 4)S
ELECT col1, col2 FROM cte;
```

使用说明

不支持递归 CTE，即 CTE 中引用自身的表达式, 如：

```
WITH RECURSIVE cte (n) AS( SELECT 1 UNION ALL SELECT n + 1 FROM ct
e WHERE n < 5)SELECT * FROM cte;
```

1.10.10. VIEW

描述

视图的作用包括：

- 简化复杂的查询：可以将一个复杂的查询定义为视图，然后在后续的查询中直接使用视图，使查询更简洁和易于理解。
- 提供数据的特定视角：只展示用户关心的数据部分。
- 增加数据的安全性：可以限制用户对基础表的直接访问，只通过视图来获取特定的数据。

语法

CREATA VIEW

```
CREATE
[OR REPLACE]
[ALGORITHM = {UNDEFINED MERGE TEMPTABLE}]
[DEFINER = { user CURRENT_USER }]
[SQL SECURITY { DEFINER INVOKER }]
VIEW view_name [(column_list)]
AS select_statement
[WITH CHECK OPTION]
```

ALTER VIEW

```
ALTER
[ALGORITHM = {UNDEFINED MERGE TEMPTABLE}]
[DEFINER = { user CURRENT_USER }]
[SQL SECURITY { DEFINER INVOKER }]
VIEW view_name [(column_list)]
AS select_statement
[WITH CHECK OPTION]
```

DROP VIEW

```
DROP VIEW [IF EXISTS]
view_name [, view_name] ...
[RESTRICT CASCADE]
```

参数解释

参数	描述

ALGORITHM = {UNDEFINED MERGE TEMPTABLE}	指定视图的处理算法。
DEFINER = { user CURRENT_USER }	定义视图的创建者或执行视图时使用的用户。
SQL SECURITY { DEFINER INVOKER }	指定执行视图时的安全上下文，DEFINER 表示使用创建者的权限，INVOKER 表示使用调用者的权限。
VIEW view_name [(column_list)]	指定要创建的视图的名称和可选的列列表。
AS select_statement	定义视图的查询语句，即视图所基于的数据来源。
WITH CHECK OPTION	用于限制对视图的更新操作，确保更新后的数据仍然满足视图的定义条件。
RESTRICT	如果指定了 RESTRICT 选项，只有当没有其他对象依赖于该视图时，才能删除。
CASCADE	如果指定了 CASCADE 选项，会自动删除依赖于该视图的对象，例如基于该视图创建的其他视图或相关的权限等。

示例

- 例子1：

```
> CREATE VIEW v AS SELECT qty, price, qty*price AS value FROM t;
```

- 例子2

```
> ALTER VIEW view_students_info AS SELECT id,name,age FROM
  tb_students_info;
```

- 例子3：

```
> DROP VIEW view_students_info;
```

使用说明

- 计算节点支持不参与对结果集进行二次处理(简单合并可以)的视图。

  如：

```
> CREATE VIEW view_name AS SELECT c FROM tb1 ORDER BY tb1.a;
```

- 分布式视图开关打开时，下列视图无法创建简单视图:

  多表视图（`CREATE VIEW v1 AS SELECT * FROM (SELECT * FROM t1)`、`t1 join t1` 这种也算多表视图）。

  视图的创建 DDL 中 `SELECT LIST` 带聚合函数、排序、`rownum` 或 `row_number` 或 `level` 等需要在 CN 进行二次计算的情况。

  虚表视图（`FROM DUAL`）。

  视图定义中分发键的列别名和源表不一致（`CREATE VIEW v2 as SELECT c1 AS c11 FROM t1`; `c1` 是源表的分发键）。

最佳实践

视图冗余列优化:

须修改配置文件 [GENERAL] #是否支持分布式视图 #0：关闭 1：启用，默认为0 #unit: NA, range: 0-OFF,1-ON, default: 0, dynamic: no; `support_distribute_view = 1`

#是否支持分布式视图优化冗余列 #CREATE v1 AS SELECT a,b FROM t1; #SELECT a FROM v1 –> SELECT a FROM (SELECT a FROM t1); #0：关闭 1：启用，默认为0 #unit: NA, range: 0-OFF,1-ON, default: 0, dynamic: yes; `optimize_view_itemlist = 1`

使用限制： 1. 仅对视图是分布式视图有效。 2. 查询视图不能含有子查询。 3. 查询视图不能是 `UNION` 语句。 4. 定义视图中的 `SELECT` 列只能是字段或者*。 5. 定义视图中 selectsql explain 出来只能是单个 sqlnode。

CREATE v1 as SELECT a,b FROM t1;
SELECT a FROM v1; 优化成–> SELECT a FROM (SELECT a FROM t1);

1.10.11. TRIGGER

描述

TRIGGER（触发器）用于在特定的数据库操作（如插入、更新、删除等）发生时自动执行一些

指定的操作。

语法

CREATE TRIGGER

```
CREATE [OR REPLACE]
TRIGGER
trg_action_time
trg_event
ON {tab_name view_name}
[referencing_new_or_old]
[FOR EACH ROW]
[ENABLE | DISABLE]
[WHEN (expr)]
trigger_body

trg_action_time:
BEFORE AFTER INSTEAD OF

trg_event:
dml_trg_event [OF col_name , ...]
| trg_event OR_SYM dml_trg_event [OF col_name , ...]

dml_trg_event:
INSERT UPDATE [OF col_name , ...] DELETE

referencing_new_or_old:
REFERENCING OLD AS old_alias NEW AS new_alias
| REFERENCING NEW AS new_alias OLD_SYM AS old_alias
| REFERENCING OLD AS old_alias
| REFERENCING NEW AS new_alias
| REFERENCING
```

ALTER TRIGGER

```
ALTER TRIGGER trg_name
| {enable disable}
| RENAME TO new_trg_name
```

DROP TRIGGER

```
DROP TRIGGER [IF EXISTS] trg_name;
```

参数解释

参数	描述

trg_action_time	BEFORE：表示在触发事件（如插入、更新、删除）发生之前执行触发器中的操作。可以用于在数据修改之前进行一些检查、数据预处理或其他相关操作。 AFTER：表示在触发事件完成之后执行触发器中的操作。常用于记录操作日志、更新相关的统计信息等。 INSTEAD OF：用于视图上的触发器。
trg_event	指定触发事件，如 INSERT、UPDATE 或 DELETE 等。
ON {tab_name view_name}	指定触发器作用的表或视图的名称。
referencing_new_or_old	用于引用新行或旧行的数据，例如 NEW 表示新插入或更新后的行，OLD 表示更新或删除前的行。
FOR EACH ROW	表示这是行级触发器，即对表中的每一行操作都触发。
ENABLE/DISABLE	可选参数，用于设置触发器是启用还是禁用。
WHEN (expr)	可选的条件，只有当条件表达式为真时才触发。
trigger_body	触发器执行的具体操作，通常是一段 SQL 代码。

示例

- 例子1：

```
> DELIMITER //
CREATE OR REPLACE TRIGGER trg_t1
AFTER DELETE OR UPDATE OF size, colour ON t1
REFERENCING
FOR EACH ROW
ENABLE
WHEN(OLD.id<>1)
BEGIN
INSERT INTO t1_history (id, location, colour, shape, size) VALUES
(:OLD.id, :OLD.location, :OLD.colour, :OLD.shape, :OLD.size);
END; //
DELIMITER ;
```

- 例子2：

```
> DELIMITER //
```

```
CREATE OR REPLACE TRIGGER trg_vw1
INSTEAD OF INSERT OR UPDATE OR DELETE ON vw_1
FOR EACH ROW
BEGIN
IF inserting THEN
INSERT INTO info (id, NAME) VALUES (:NEW.id, :NEW.name);
END IF;
END; //
DELIMITER ;
```

- 例子3：

```
> DROP TRIGGER IF EXISTS trg_t1;
```

- 例子4：

```
> ALTER TRIGGER trg_t1 DISABLE;
```

- 例子5：

```
> ALTER TRIGGER trg_t1 RENAME TO trg_new;
```

使用说明

- 不支持利用 `DEFINER` 关键字自定义特定用户有权限执行语句。

如：

```
> CREATE DEFINER=CURRENT_USER
TRIGGER test_trigger1 BEFORE DELETE ON t1
FOR EACH ROW BEGIN
INSERT INTO t2 (id, name, info) VALUES (OLD.id, OLD.name, OLD.info
);
END;
```

- 视图触发器只支持 `INSTEAD OF`，不支持 `BEFORE` 或 `AFTER`；同时表触发器只支持 `BEFORE` 和 `AFTER`，不支持 `INSTEAD OF`。
- 创建触发器要求其针对的表或视图实际存在，不可以创建针对不存在对象的触发器。
- 创建语句级触发器（不添加 `FOR EACH ROW` 关键词的触发器）不支持 `WHEN` 条件。
- 创建的针对视图的 `INSTEAD OF` 触发器均为行级触发器，无论是否添加 `FOR EACH ROW` 关键词。
- 触发器的触发条件中只有存在 `UPDATE` 时，才允许添加 `of col_name`，...。
- 视图触发器与分布式视图（配置项 `supoort_distribute_view=1`）不兼容。

1.10.12. TRANSACTION AND LOCKING STATEMENTS

1.10.12.1. TRANSACTION

描述

在数据库中，`TRANSACTION`（事务）用于确保一系列数据库操作要么全部成功执行，要么全部不执行，以保持数据的一致性和完整性。

语法

```
START TRANSACTION [CONSISTENCY_OPTION][SAMEDB]
COMMIT
ROLLBACK
```

说明：

如果一致性级别设置在事务和 SQL 语句中互相冲突时，以 SQL 语句中一致性级别为准。

参数解释

参数	描述
START TRANSACTION	可以开始一项新的事务。
CONSISTENCY_OPTION	一致性选项。
SAMEDB	可选参数，复用前一条语句的下发节点。
COMMIT	可以提交当前事务，是变更成为永久变更。
ROLLBACK	可以回滚当前事务，取消其变更。

使用说明

不支持 `SET TRANSACTION` 语法。

如：

```
> SET TRANSACTION READ ONLY ;
```

最佳实践

数据一致性级别对性能影响

数据一致性级别要求越高时，对事务的性能影响越大。

1.10.12.2. SAVEPOINT

描述

SAVEPOINT（保存点）用于在事务中创建一个标记点，以便在需要时能够回滚到该点，而不是完全回滚整个事务。

语法

SAVEPOINT

```
SAVEPOINT identifier
```

ROLLBACK TO SAVEPOINT

```
ROLLBACK TO [SAVEPOINT] identifier
```

RELEASE SAVEPOINT

```
RELEASE SAVEPOINT identifier
```

参数解释

参数	描述
identifier	保存点指定的名称，用于标识和区分不同的保存点，以便在后续的回滚操作中能够准确地指定要回滚到的特定保存点。
ROLLBACK	执行回滚操作。
RELEASE	用于释放（删除）之前在事务中创建的指定保存点。

示例

```
> DECLARE
v_number number;
BEGIN
v_number := 1;
INSERT INTO t11 VALUES(T11_SEQ.nextval,v_number);
savepoint A;
INSERT INTO t11 VALUES(T11_SEQ.nextval,v_number+1);
```

```
savepoint B;
INSERT INTO t11 VALUES(T11_SEQ.nextval,v_number+2);
savepoint C;
ROLLBACK TO B;
END;
```

1.10.13. SYNONYM

描述

SYNONYM（同义词）用于为数据库对象（如表、视图、存储过程等）创建一个替代名称。

语法

CREATE SYNONYM

```
CREATE [ OR REPLACE ] [ PUBLIC ] SYNONYM [ schema. ] synonym_name
FOR [ schema. ] object
```

CREATE DBLINK SYNONYM

```
CREATE [ OR REPLACE ] [ PUBLIC ] SYNONYM [ schema. ] synonym_name
FOR [ schema. ] object@dblink_name
```

DROP SYNONYM

```
DROP [PUBLIC] SYNONYM [schema .] SYNONYM [FORCE]
```

参数解释

参数	描述
OR REPLACE	可选参数，如果同义词已经存在，则替换它。
PUBLIC	可选参数，如果指定，则创建的是公共同义词，否则创建的是私有同义词，只能当前 schema 访问。
schema	指定模式，如果不指定，则使用当前默认模式。
synonym_name	要创建的同义词的名称。

object	指定同义词所代表的对象，可以是表、视图、存储过程等。
object@dblink_name	表示这个同义词是为位于远程数据库（通过数据库链接 dblink_name 指定）中的对象创建的。
FORCE	强制删除同义词，即使存在依赖它的对象。使用此选项时需要谨慎，可能会导致相关依赖对象出现异常。

示例

- 例子1：

```
> CREATE [ OR REPLACE ] [PUBLIC ] SYNONYM syn_abc FOR db1.tb1;
```

- 例子2：

```
> CREATE [ OR REPLACE ] [PUBLIC ] SYNONYM syn_dblink FOR tb2@dblink1;
```

- 例子3：

```
> DROP [PUBLIC] SYNONYM syn_abc;
> DROP [PUBLIC] SYNONYM syn_dblink;
```

- 例子4：

```
> SELECT * FROM syn_abc;
> SELECT * FROM syn_dblink;
```

使用说明

- 支持跨库同义词。
- 多个跨库同义词不要同名。
- 支持 dblink 类型同义词的创建、删除及查询执行。
- 创建 dblink 同义词时，若对应 dblink 的 dbtype 为 GoldenDB，要求 MPP 侧创建的 dblink 必须指定库 DATABASE 参数。

1.10.14. 数据库管理语句

1.10.14.1. 账户管理

描述

在数据库中，账户管理是确保数据库安全性和访问控制的重要方面。

语法

- CREATE USER

```
CREATE USER [IF NOT EXISTS]
user auth_option
[password_option] ...

auth_option:
{ IDENTIFIED BY 'auth_string' }

password_option:
{ PASSWORD EXPIRE [DEFAULT NEVER INTERVAL N DAY]
| PASSWORD HISTORY {DEFAULT N}
| PASSWORD REUSE INTERVAL {DEFAULT N DAY} }
```

注：建议通过 Insight 页面创建用户并进行管理

user 名

用户名	是否支持
'user_name' @'10.10.10.10'	是
'user_name' @'10.10.10.10/8'	是
'user_name' @'10.10.10.10/255.255.255.0'	不支持，与 / 功能重复
'user_name' @'10.10.10.%'	是
'user_name' @'8888:FF:FF:FF:FF:FF:FF:1111'	不支持
'user_name' @'8888::1111'	不支持，ipv6不支持
'user_name' @'8888:0:0:0:0:0:%'	不支持，ipv6不支持
'user_name' @'%'	是
'user_name' @ 'localhost'	不支持

'user_name' @ '127.0.0.1'	是
'user_name' @ '::1'	不支持，ipv6不支持
'user_name' @ '%.mysql.com'	不支持
多个 user	否，暂时只支持一个user

password_option

选项	备注
PASSWORD EXPIRE [DEFAULT | NEVER | INTERVAL N DAY]	密码到期选项
PASSWORD HISTORY {DEFAULT | N}	密码重用选项，更改次数
PASSWORD REUSE INTERVAL {DEFAULT | N DAY}	密码重用选项，时间

- ALTER USER

```
ALTER USER [IF EXISTS]
user [auth_option]
[password_option] ...

auth_option:
{ IDENTIFIED BY 'auth_string'
| DISCARD OLD PASSWORD }

password_option:
{ { RETAIN CURRENT PASSWORD
| PASSWORD EXPIRE [DEFAULT NEVER INTERVAL N DAY]
| PASSWORD HISTORY {DEFAULT N}
| PASSWORD REUSE INTERVAL {DEFAULT N DAY} }
```

- DROP USER

```
DROP USER user
```

- RENAME USER

```
RENAME USER old_user TO new_user
```

注：只支持修改 ip，不支持修改用户名，不支持多个用户；RENAME后应用需要使用new_user重新创建与CN的连接。

- CREATE ROLE

```
  CREATE ROLE [ IF NOT EXISTS ] role [ , role ] ...
```

- DROP ROLE

```
  DROP ROLE [ IF EXISTS ] role [ , role ] ...
```

- GRANT

```
GRANT
priv_type
[, priv_type] ...
ON [object_type] priv_level
TO user_or_role
[WITH GRANT OPTION]

GRANT role [, role] ...
to user_or_role [, user_or_role] ...[WITH ADMIN OPTION]
}

object_type:
{
TABLE
| FUNCTION
| PROCEDURE
}

priv_level:
{
*
| *.*
| db_name.*
| db_name.tbl_name
| tbl_name
| db_name.routine_name
}
```

- REVOKE

```
REVOKE
priv_type
[, priv_type] ...
ON [object_type] priv_level
FROM user_or_role
REVOKE ALL [PRIVILEGES], GRANT OPTION
```

```
FROM user_or_role
REVOKE role FROM user_or_role
```

参数解释

参数	描述
CREATE USER	创建用户的操作指令。
auth_option	用于指定用户的认证方式，例如指定用户的身份验证插件、密码加密方式等。
password_option	用于设置用户的密码相关选项，比如直接设置密码、指定密码的生成规则等。
ALTER USER	修改用户相关设置的操作指令。
DROP USER	删除用户的操作指令。
RENAME USER	重命名用户的操作指令。
CREATE ROLE	创建角色的操作指令。角色在数据库中用于对用户进行权限的分组和管理，创建角色有助于更高效地分配和管理权限。
role	要创建的角色名称，可以同时创建多个角色，用逗号分隔。
DROP ROLE	删除角色的操作指令。
GRANT	授予权限的操作指令。
priv_type	要授予的权限类型，例如SELECT、INSERT、UPDATE、DELETE等，可以同时授予多个权限，用逗号分隔。
ON [object_type] priv_level	指定权限作用的对象类型（如表、视图等）和对象级别（整个表、特定列等）。
TO user_or_role	将权限授予给指定的用户或角色。

WITH GRANT OPTION	授予接收权限的用户或角色将所获得的权限再授予给其他用户或角色的能力。
GRANT role	要授予的角色。
to user_or_role	将角色授予给指定的用户或角色。
WITH ADMIN OPTION	授予接收角色的用户或角色管理所授予角色的能力。

示例

```
> CREATE USER 'jeffrey'@'%' IDENTIFIED BY 'MYPASS@test01';
> GRANT ALL ON db1.* TO 'jeffrey'@'%';
> REVOKE INSERT ON *.* FROM 'jeffrey'@'%';
> CREATE ROLE 'app_developer', 'app_read', 'app_write';
> GRANT ALL ON db1.* TO 'app_developer';
> GRANT SELECT ON db1.* TO 'app_read';
> GRANT INSERT, UPDATE, DELETE ON db1.* TO 'app_write';
> GRANT 'app_read', 'app_write' TO  'jeffrey'@'%';
> SHOW GRANTS FOR  'jeffrey'@'%';
> SHOW GRANTS FOR  'jeffrey'@'%' USING 'app_read';
> REVOKE SELECT ON db1.* FROM  'app_read';
> REVOKE 'app_read' FROM  'jeffrey'@'%';
> REVOKE 'app_write' FROM  'jeffrey'@'%';
> DROP ROLE  'app_developer' ;
> DROP ROLE  'app_read',  'app_write';
> DROP USER 'jeffrey'@'%';
> ALTER USER app_user@'%' identified by 'new_password' RETAIN CURR
ENT PASSWORD;
> ALTER USER app_user@'%' DISCARD OLD PASSWORD;
```

使用说明

双密码场景注意事项：当某一用户存在双密码，通过 CN 更改插件后，CN 侧副密码不会失效。 示例：当前默认插件为：`mysql_native_password`，通过第三条语句更改插件后，副密码 `old_password` 不会失效

```
> CREATE USER app_user@'%' identified by 'old_password'; //old_pas
sword可用
> ALTER USER app_user@'%' identified by 'new_password' RETAIN CURR
ENT PASSWORD;  //old_password和new_password可用
> ALTER USER app_user@'%' identified WITH caching_sha2_password by
 'newnew_password'; //old_password和newnew_password可用
```

1.10.14.2. SHOW

描述

`SHOW` 命令用于获取数据库的各种信息。

语法

- SHOW DATABASES

```
SHOW {DATABASES SCHEMAS} [LIKE 'pattern' ]
```

- SHOW CREATE DATABASE

```
SHOW CREATE {DATABASE SCHEMA} [IF NOT EXISTS] db_name
```

- SHOW TABLES

```
SHOW TABLES [{FROM IN} db_name]
```

- SHOW CREATE TABLE

```
SHOW CREATE TABLE tbl_name
```

说明：

输出表结构信息，同时携带计算节点扩展的表分布信息。

- SHOW CREATE PROCEDURE

```
SHOW CREATE PROCEDURE proc_name
```

- SHOW CREATE FUNCTION

```
SHOW CREATE FUNCTION func_name
```

- SHOW CREATE VIEW

```
SHOW CREATE VIEW view_name
```

- SHOW DISTRIBUTION FROM table_name

```
SHOW DISTRIBUTION FROM showdb.zte
```

- SHOW TABLE DISTRIBUTION

```
  SHOW TABLE DISTRIBUTION [{FROM | IN} db_name] [LIKE 'pattern']
[WHERE {HASH|RANGE|LIST|DUPLICATE[SINGLE|MULTI GROUP]|MULTIDISTRIB
UTION}] [DETAIL]
```

- SHOW VARIABLES

```
  SHOW [{SESSION | GLOBAL}] VARIABLES [LIKE xx]
```

- SHOW WARNINGS

```
SHOW WARNINGS;
```

- SHOW STATUS

```
  SHOW [{SESSION | GLOBAL | FUNCTION | PROCEDURE | TABLE}] [IN sho
wdb] STATUS [LIKE xx]
```

- SHOW _GDB_CN [FULL] PROCESSLIST

- 说明：

  功能：查询本 CN 与后端 DN 的链路关联信息, 集中式开启 FD 透传不支持该语法 限制：集中式开启 FD 透传查询请使用 dbtool 运维命令：dbtool -p[roxy] -[e]x[ec] -s[how]c[n]p[rocesslist] 拓展：由于show _gdb_cn processlist返回全量结果，不利于运维查看，多分片集群新增 `sys._gdb_cn_processlist` 内存表进行查询,请注意 show _gdb_cn processlist 字段名已与 sys._gdb_cn_processlist 内存表对齐, show _gdb_cn processlist 结果只包含内存表部分字段，同时该表只支持查询操作，不建议对该表进行高并发查询操作。

`sys._gdb_cn_processlist`表结构

```
CREATE TABLE `sys`.`_gdb_cn_processlist` (`dialogid` bigint unsi
gned NOT NULL,`linkon_time` datetime(6) DEFAULT NULL,`user_nam
e` varchar(32) DEFAULT NULL,`client_host` varchar(255) DEFAU
LT NULL,`exec_thread_id` int unsigned DEFAULT NULL,`_gdb_cn_po
rt` int unsigned DEFAULT NULL,`use_database` varchar(512) DEF
AULT NULL,`uuid` varchar(512) DEFAULT NULL,`tsn` varchar(512
) DEFAULT NULL,`digest` varchar(256) DEFAULT NULL,`origin_sq
l` longtext DEFAULT NULL,`begin_time` datetime(3) DEFAULT NULL
,`end_time` datetime(3) DEFAULT NULL,`time_ms` int unsigned DE
FAULT NULL,`state` varchar(10) DEFAULT NULL,`global_transacti
on_id` bigint unsigned NOT NULL DEFAULT 0,`backend_info` longt
ext DEFAULT  NULL,PRIMARY KEY (`dialogid`)) DEFAULT CHARSET=u
```

```
tf8mb4 COLLATE=utf8mb4_bin distributed by  duplicate(g1);
```

`sys._gdb_cn_processlist` 表支持聚合函数、 where 条件查询(等于、不等于、大于和小于、大于等于和小于等于、between、in、not in、is null、is not null、group by ... having)、order by、group by、limit、like 模糊匹配;暂不支持连接多表、子查询、嵌套查询。

- 元数据类

```
SHOW CHARACTER SET;
SHOW CHARACTER SET LIKE xx;

SHOW COLLATION;
SHOW COLLATION LIKE xx;

SHOW CREATE PROCEDURE proc_name;

SHOW CREATE FUNCTION func_name;

SHOW CREATE TRIGGER trigger_name;

SHOW DISTRIBUTION FROM showdb.zte;
SHOW DISTRIBUTION FROM showdb.zte WHERE id=xx;

SHOW ENGINES;

SHOW KEYS FROM showdb.zte;
SHOW KEYS FROM zte FROM showdb;

SHOW INDEX FROM showdb.zte;
SHOW INDEX IN zte FROM showdb;

SHOW COLUMNS FROM zte FROM showdb;
SHOW COLUMNS FROM showdb.zte;

SHOW FIELDS FROM ztet1 FROM showdb;
SHOW FIELDS FROM showdb.zte;
```

- SHOW GROUPS

```
SHOW GROUPS g1, g2;
SHOW master GROUPS g1, g2;
SHOW GROUPS ALL;
SHOW master GROUPS ALL;
```

- SHOW GRANTS

```
SHOW GRANTS [FOR user_or_role [USING role [, role] ...]]
```

- SHOW _GDB_CN [FULL] PROCESSLIST

```
SHOW _gdb_cn [full] processlist;
```

说明：

功能：查询本 CN 与后端 DN 的链路关联信息, 集中式开启 FD 透传不支持该语法。 限制：集中式开启 FD 透传查询请使用 dbtool 运维命令：`dbtool -p[roxy] -[e]x[ec] -s[how]c[n]p[rocesslist]`。

- SHOW _GDB_CN VARIABLES [like xx]

说明：

`dbtool -p[roxy] -show-variables [like xx]` 的SQL实现形式，结果集展示与 `SHOW VARIABLES` 类似，包含 `Value_name` 与 `Value` 两个字段，通配符只支持‘%’，不支持 ‘_’, like 后匹配内容不区分大小写。

参数解释

参数	描述
SHOW DATABASES	显示服务器中所有数据库名称。
SHOW CREATE DATABASE	显示创建的指定数据库。
SHOW TABLES	显示数据库中所有表名称。
SHOW CREATE TABLE	显示创建的指定表。
SHOW CREATE FUNCTION	显示创建的指定函数。
SHOW CREATE PROCEDURE	显示创建的指定存储过程。
SHOW CREATE VIEW	显示创建的指定视图。
SHOW DISTRIBUTION	显示表的分发类型和数据节点。
SHOW VARIABLES	显示数据库的各种变量及其当前值。

SHOW WARNINGS	显示执行语句中产生的告警信息。
SHOW STATUS	显示数据库的运行情况、性能指标。
SHOW _GDB_CN [FULL] PROCESSLIST	显示前后端链路关系。
SHOW CHARACTER SET	显示字符集设置。
SHOW COLLATION	显示字符序设置。
SHOW CREATE TRIGGER	显示创建的指定触发器。
SHOW ENGINES	显示数据库支持的存储引擎及其相关信息。
SHOW KEYS	显示表中的索引信息。
SHOW INDEX	显示表中的索引信息。
SHOW COLUMNS	显示表中的列信息。
SHOW FIELDS	显示表中的字段信息。
SHOW GROUPS	显示数据节点信息。
SHOW GRANTS	显示用户的权限信息。
SHOW _GDB_CN VARIABLES	显示 CN 的配置项及当前值。

示例

```
> SHOW DATABASES;
+-------------------------------------------------+
| Database                                        |
+-------------------------------------------------+
| testdb                                          |
| testdb_sj2                                      |
| zhl                                             |
| zz                                              |
+-------------------------------------------------+
4 rows in set (0.00 sec)
```

```
> SHOW CREATE DATABASE testdb;
+----------+-----------------------------------------------------
------------+
| DATABASE | CREATE DATABASE
            |
+----------+-----------------------------------------------------
------------+
| testdb   | CREATE DATABASE ` testdb ` /*!40100 DEFAULT CHARACTER
SET utf8 */ |
+----------+-----------------------------------------------------
------------+
1 row in set (0.01 sec)
> SHOW TABLES;
+------------------+
| Tables_in_test1  |
+------------------+
| blobtest         |
| detest           |
+------------------+
2 rows in set (0.00 sec)
> SHOW CREATE TABLE zhl.t1;
*************************** 1. row ***************************
Table: t1识别
CREATE Table: CREATE TABLE `zhl`.`t1` (
`col1` INT(11) DEFAULT NULL,
`col2` char(5) DEFAULT NULL,
`col3` DATE DEFAULT NULL,
`wp` INT(11) DEFAULT NULL
)
/*!50100 PARTITION BY HASH ( YEAR(col3)) PARTITIONS 4 */
DISTRIBUTED BY HASH(col1) (g1,g2);
1 row in set (0.02 sec)
> SHOW CREATE PROCEDURE simpleproc\G;
*************************** 1. row ***************************
Procedure: simpleproc
sql_mode: STRICT_TRANS_TABLES
Create Procedure: CREATE DEFINER=`root`@`zte-179` PROCEDURE
`simpleproc`( IN a INT,OUT param1 INT)
BEGIN
INSERT INTO t VALUES(@a);
SELECT COUNT(*) INTO param1 FROM t;
END
DISTRIBUTED BY DUPLICATE (g1,g2)
character_set_client: utf8
collation_connection: utf8_bin
Database Collation: utf8_bin
> SHOW CREATE function hello\G;
*************************** 1. row ***************************
Function: hello
sql_mode: STRICT_TRANS_TABLES
Create Function: CREATE DEFINER=`root`@`localhost` FUNCTION
`hello`(s CHAR(20)) RETURNS CHAR(50) CHARSET utf8 COLLATE utf8_bin
DETERMINISTIC
```

```
RETURN CONCAT('Hello, ',s,'!')
character_set_client: latin1
collation_connection: latin1_swedish_ci
Database Collation: utf8_bin
> SHOW groups g1,g2;
+--------+-------------+--------+----------+
| GroupID | DBIP DBPORT IsMaster               |
+--------+-------------+--------+----------+
| g1 10.229.31.54 5501 1                      |
| g2 10.229.31.54 5502 1                      |
+--------+-------------+--------+----------+
> SHOW master groups g1,g2;
+--------+-------------+--------+----------+
| GroupID DBIP DBPORT IsMaster                |
+--------+-------------+--------+----------+
| g1 10.229.31.54 5501 1                      |
| g2 10.229.31.54 5502 1                      |
+--------+-------------+--------+----------+
> SHOW groups all;
+--------+-------------+--------+----------+
| GroupID DBIP DBPORT IsMaster                |
+--------+-------------+--------+----------+
| g1 10.229.31.54 5501 1                      |
| g2 10.229.31.54 5502 1                      |
| g3 10.229.31.54 5503 1                      |
| g4 10.229.31.54 5504 1                      |
+--------+-------------+--------+----------+
> SHOW VARIABLES;
> SHOW VARIABLES LIKE '%max_connect%';
> SHOW SESSION VARIABLES LIKE '%max_connect%';
> SHOW GLOBAL VARIABLES LIKE '%max_connect%';
> SHOW STATUS;
> SHOW STATUS LIKE '%show%';
> SHOW SESSION STATUS LIKE '%show%';
> SHOW GLOBAL STATUS LIKE '%show%';
> SHOW PLUGINS;
> SHOW FUNCTION STATUS LIKE '%func%';
> SHOW PROCEDURE STATUS LIKE '%proc%';
> SHOW TABLE STATUS IN showdb;
> SHOW TABLE STATUS FROM showdb;
> SHOW TABLE STATUS IN showdb LIKE '%zte%';
> SHOW _gdb_cn VARIABLES;
> SHOW _gdb_cn VARIABLES LIKE 'log_level';
> SHOW _gdb_cn VARIABLES LIKE '%timeout%';
```

1.10.14.3. SET

描述

`SET` 命令通常用于设置或更改各种配置选项、变量或表中的数据。

语法

```
SET variable_assignment [, variable_assignment] ...

variable_assignment:
user_var_name = expr
```

参数解释

参数	描述
user_var_name	配置选项、变量或表中的数据。
expr	用于计算并为变量赋值，暂不支持函数或子查询。

示例

```
> SET @a=1;
> select @a;
```

1.10.14.4. DESC

描述

`DESC` 命令通常用于获取表结构的详细信息，以降序方式显示列的信息。

语法

```
{ DESCRIBE | DESC} tbl_name
```

参数解释

参数	描述
tbl_name	要获取结构信息的表名称。

示例

```
> DESC zhl.t1;
+-------+---------+------+-----+---------+-------+
| Field | Type    | Null | Key | Default | Extra |
+-------+---------+------+-----+---------+-------+
```

```
| col1   | INT(11)  | YES  |     | NULL    |       |
| col2   | CHAR(5)  | YES  |     | NULL    |       |
| col3   | DATE     | YES  |     | NULL    |       |
| wp     | INT(11)  | YES  |     | NULL    |       |
+-------+---------+------+-----+---------+-------+
```

1.10.14.5. 碎片整理

示例

```
> ALTER TABLE t2 ENGINE=innodb, ALGORITHM=INPLACE, LOCK=NONE;
```

当使用磁盘碎片整理功能时，可能会导致主备大量延时，影响故障切换。

1.10.14.6. 系统视图

支持系统视图查询功能，已支持的视图：DBA_TABLES、DBA_INDEXDBA_OBJECTS、DBA_SOURCE、DBA_TAB_COLUMNS、ALL_TABLES、ALL_INDEX、ALL_OBJECTS、ALL_SOURCE、ALL_TAB_COLUMNS、USER_TABLES、USER_INDEX、USER_OBJECTS、USER_SOURCE、USER_TAB_COLUMNS、DBA_VIEWS、DBA_SYNONYMS、DBA_PROCEDURES、DBA_SEQUENCES、DBA_TRIGGERS、ALL_VIEWS、ALL_SYNONYMS、ALL_PROCEDURES、ALL_SEQUENCES、ALL_TRIGGERS、USER_VIEWS、USER_SYNONYMS、USER_PROCEDURES、USER_SEQUENCES、USER_TRIGGERS、DBA_USERS。

示例

```
> SELECT * FROM USER_TABLES WHERE TABLE_NAME='T1';
```

1.10.15. 实用语法

1.10.15.1. EXPLAIN

描述

EXPLAIN 语句用于获取关于查询执行计划的信息。

语法

```
EXPLAIN [DBPROXYexplain_format] explainable_stmt [stroragedb]

explainable_stmt:
{
SELECT statement
| DELETE statement
```

```
| INSERT statement
| UPDATE statement
}

explain_format:
{
FORMAT = JSON
| FORMAT = Oracle
}
```

参数解释

参数	描述
explainable_stmt	表示要解释的 SQL 语句，可以是 SELECT、DELETE、INSERT 或 UPDATE 语句。
explain_format	指定了解释结果的格式，可选的值包括 JSON 和 Oracle 等。

示例

- 例子1：

```
> EXPLAIN SELECT * FROM t1 WHERE id=1 or id=11 ORDER BY id;
```

- 例子2：

```
> EXPLAIN PROXY SELECT * FROM t1 WHERE id=1 or id=11 ORDER BY id;
```

- 例子3：

```
> EXPLAIN DB SELECT * FROM t1 WHERE id=1 or id=11 ORDER BY id STOR
AGEDB
  g2;
```

- 例子4：

```
> EXPLAIN FORMAT = JSON SELECT * FROM t1 WHERE id=1 or id=11 ORDER
 BY
  id;
```

- 例子5：

```
> EXPLAIN FORMAT = JSON SELECT * FROM t1 WHERE id=1 or id=11 ORDER
```

```
BY
 id;
```

使用说明

- 支持仅输出计算节点的分布式执行计划。此时需使用 PROXY 选项，即 `EXPLAIN PROXY explainable_stmt`。
- 支持将分布式执行计划和 DB 执行计划结合输出。
- 支持仅输出 DB 的执行计划。

此时需使用 DB 选项，并指明要查看的存储节点。

示例：

```
> EXPLAIN DB SELECT * FROM t1 STORAGEDB g1;
```

- `EXPLAIN` 目前不支持的场景：

`UPDATE` 操作，`SET` 带有 `SELECT` 子句时，不支持 `EXPLAIN` 功能：

EXPLAIN UPDATE db.t1 SET t1.id2 = (SELECT t2.id2 FROM t2);

ERROR 11269 (HY000): ERR: the statement does not support the explain operation for one right value with subselect!

`INSERT`、`UPDATE` 和 `DELETE` 操作，语句中带有 `SEQUENCE` 时，不支持 `EXPLAIN` 功能:

EXPLAIN INSERT INTO seq.sing1 (a,b,c) values (seq.seq_t.NEXTVAL,'1',1);

ERROR 11251 (HY000): ERR: the statement with the sequence does not support the explain operation!

1.10.15.2. USE

描述

选择要使用的数据库。

语法

```
USE db_name
```

参数解释

参数	描述
db_name	将 db_name 变为后续执行 SQL 语句的默认库。

示例

```
> USE db1;
> SELECT COUNT(*) FROM mytable;
```

1.10.15.3. 外部流水号设置

语法

```
SET @transaction_serial_number="xxx";
```

参数解释

参数	描述
transaction_serial_number	为 GoldenDB 特有，请勿把它当作普通变量使用。

示例

```
> SET autocommit=0;
> SET @transaction_serial_number="xxx";
> UPDATE....
> INSERT...
> COMMIT;
```

1.10.15.4. IGNORE_SPACE

语法

计算节点 `parse mode` 设置为 `Oracle` 模式，数据节点的 `SQL Mode` 添加 `IGNORE_SPACE` 模式，将允许函数名和字符'('间带空格。

该模式下，内置函数名被视为保留字，在使用与内置函数同名的标识符时需要用反引号'`'。

示例

例如聚合函数 `COUNT()`，如果使用 `COUNT` 作为表名，建表时将报如下错误：

```
> CREATE TABLE COUNT (i INT) DISTRIBUTED BY DUPLICATE(g1,g2);
ERROR 1064 (42000): You have an error in your SQL syntax
```

表名必须使用反引号`:

```
> CREATE TABLE `COUNT` (i INT ,PRIMARY KEY(i)) DISTRIBUTED BY
DUPLICATE(g1,g2);
Query OK, 0 rows affected (0.00 sec)
```

不需要此功能时，不建议配置 `IGNORE_SPACE` 模式。

1.10.15.5. PURGE

语法

PURGE TABLE

```
PURGE TABLE table_name [FOR RECYCLEBIN_VERSION version];
```

PURGE RECYCLEBIN

```
PURGE RECYCLEBIN;
```

参数解释

参数	描述
FOR RECYCLEBIN_VERSION version	可选参数，用于指定回收站版本。

示例

`PURGE TABLE` 不带版本号，删除该表所有的闪回副本。

```
PURGE TABLE db.tb1;
```

`PURGE TABLE` 带版本号，删除该表对应版本的闪回副本。

```
PURGE TABLE db.tb1 FOR RECYCLEBIN_VERSION "version_info";
```

`PURGE RECYCLEBIN` 清除所有表的所有闪回副本。

```
PURGE RECYCLEBIN;
```

版本号查询。

```
SELECT ORIG_DB_NAME,ORIG_TABLE_NAME,VERSION_ID,OPERATION from information_schema.recyclebin;
```

版本号查询（新版本新增，可查询操作时间）

```
SELECT ORIG_DB_NAME,ORIG_TABLE_NAME,VERSION_ID,OPERATION,OPERATION_TIME from information_schema.recyclebin;
```

1.10.15.6. FLASHBACK

描述

FLASHBACK 用于实现数据的回滚或恢复到过去的某个时间点或事务状态。

语法

FLASHBACK TABLE

```
FLASHBACK TABLE table_name [FOR RECYCLEBIN_VERSION version] TO BEFORE DROP;
```

示例

`FLASHBACK TABLE` 不带版本号，闪回到最近的一个闪回副本。

```
> FLASHBACK TABLE db.tb1 TO BEFORE DROP;
```

`FLASHBACK TABLE` 带版本号，闪回到指定版本的闪回副本。

```
> FLASHBACK TABLE db.tb1 FOR RECYCLEBIN_VERSION "version_info" TO BEFORE DROP;
```

参数解释

参数	描述
FOR RECYCLEBIN_VERSION version	可选参数，用于指定回收站版本。

1.10.15.7. INITCAP

描述

INITCAP 函数用于将字符串中的每个单词的首字母转换为大写，其余字母转换为小写。

语法

INITCAP 函数只支持输入一个参数。当输入参数个数不为 1 时，就会报错。支持参数为常量、字段、函数等场景；支持增、删、改、查等场景，在 `WHERE` 条件、字段值、 `ORDER BY`、`GROUP BY` 等处使用 INITCAP。

当 `sql_mode` 设置为 `EMPTY_STRING_IS_NULL` 时，兼容 `Oracle` 将空串转换为 NULL 值，其他类似的 `Oracle` 兼容函数也类似。

示例

- 例子1：

```
> SELECT INITCAP (STRING) FROM TAB1 WHERE ID = 8;
```

- 例子2：

```
> SELECT INITCAP(A.STRING),INITCAP(B.STRING) FROM TAB1 A LEFT JOIN
 TAB2 B
  ON A.ID = B.ID;
```

- 例子3：

```
> SELECT STRING,INITCAP(STRING) AS RESULT FROM TAB1 GROUP BY STRIN
G HAVING COUNT(*) > 2;
```

限制说明

输入参数只能为 1 个

1.10.15.8. 语句加隔离级别

描述

在数据库中，事务的隔离级别可以通过一些特定的查询提示（hint）来改变，这些提示可以指导数据库如何处理特定的事务。

语法

```
/*M!gdb_isolation_grade=ur*/
/*!gdb_isolation_grade=ur*/
/*+gdb_isolation_grade=ur*/
/*M!gdb_isolation_grade=cr*/
/*!gdb_isolation_grade=cr*/
/*+gdb_isolation_grade=cr*/
/*M!gdb_isolation_grade=sw*/
/*!gdb_isolation_grade=sw*/
/*+gdb_isolation_grade=sw*/
/*M!gdb_isolation_grade=cw*/
/*!gdb_isolation_grade=cw*/
/*+gdb_isolation_grade=cw*/
```

参数解释

参数	描述
UR	脏读, 查询时读取的数据中允许包含脏数据，不检验来自各存储节点的数据是否活跃，即不检验来自各存储节点的数据是否来自同一版本或同一时刻的副本数据。
CR	强一致性读，查询时读取的数据中不允许包含脏数据，检验来自各存储节点的数据不能为活跃状态，即检验来自各存储节点的数据必须为同一版本或同一时刻的副本数据。
SW	单节点写，不考虑分布式事务下的对部分已提交事务的数据的修改，即不检验修改的数据是否活跃。
CW	强一致性写，需要考虑分布式事务下的对部分已提交事务的数据的修改，需要检验修改的数据是否处于活跃状态，活跃状态的数据不能修改，只能修改已经全局提交成功的数据。

示例

```
> UPDATE  /*M!gdb_isolation_grade=sw*/ t1 SET id1=1 WHERE b >5;
> UPDATE  /*!gdb_isolation_grade=sw*/  t1 SET id1=1 WHERE b >5;
> UPDATE  /*M!gdb_isolation_grade=cw*/ t1 SET id1=1 WHERE b >5;
> UPDATE  /*!gdb_isolation_grade=cw*/  t1 SET id1=1 WHERE b >5;
> SELECT /*M!gdb_isolation_grade=cr*/ * FROM t1 LEFT JOIN t2 on t1
.id1=t2.id1;
> SELECT /*!gdb_isolation_grade=cr*/ * FROM t1 LEFT JOIN t2 on t1.
id1=t2.id1;
> SELECT /*M!gdb_isolation_grade=ur*/ * FROM t1 LEFT JOIN t2 on t1
.id1=t2.id1;
> SELECT /*!gdb_isolation_grade=ur*/ * FROM t1 LEFT JOIN t2 on t1.
id1=t2.id1;
```

1.10.15.9. COMMENT ON 语法

描述

为数据库对象（如表、列、视图、存储过程等）添加注释。

语法

```
COMMENT ON COMMENT_OPTION IS 'text' ;

  COMMENT_OPTION:
  TABLE [ db. ] table
  | COLUMN [ db. ] table.column
```

参数解释

参数	描述
TABLE [db.] table	表示为表添加注释。其中 db 是数据库名称（可选），table 是表名。
COLUMN [db.] table.column	表示为表中的列添加注释。同样，db 是数据库名称（可选），table 是表名，column 是列名。

示例

```
> COMMENT ON column db.t1.id is 'aaa';
> COMMENT ON column t1.decimal1 is 'aaa';
> COMMENT ON table db1.t1 is 'aaa';
> COMMENT ON table t1 is 'aaa';
```

1.10.16. USER-DEFINED FUNCTION

描述

用户自定义函数（User-Defined Function，简称 UDF）允许根据特定的业务逻辑创建自己的函数来处理数据。

语法

CREATE FUNCTION

```
CREATE [ OR REPLACE ]
[DEFINER = { user CURRENT_USER }]
```

```
FUNCTION sp_name ([func_parameter[,...]])
RETURNS RETURN type   [AS]
param_name type
[characteristic ...]
BEGIN
routine_body
END

func_parameter:
param_name [ IN OUT ] type

type:
Any valid MySQL data type

characteristic:
COMMENT 'string'
| LANGUAGE SQL
| [NOT] DETERMINISTIC
| { CONTAINS SQL NO SQL READS SQL DATA MODIFIES SQL DATA }
| SQL SECURITY { DEFINER INVOKER }

routine_body:
Valid SQL routine statement
```

DROP FUNCTION

```
DROP { FUNCTION} [IF EXISTS] sp_name
```

参数解释

参数	描述
[DEFINER = {user CURRENT_USER}]	定义函数的创建者或执行权限。user 指定特定用户，CURRENT_USER 表示当前用户。
FUNCTION sp_name	sp_name 是要创建的函数名称。
([func_parameter[,...]])	函数的参数列表，每个参数包括参数名、输入输出方向（IN、OUT 或 INOUT）和数据类型。
RETURNS RETURN type	指定函数返回的数据类型。

[AS] param_name type	定义函数内部使用的变量及其数据类型。
COMMENT 'string'	为函数添加注释。
LANGUAGE SQL	指定函数使用的语言为 SQL。
[NOT] DETERMINISTIC	表示函数的结果是否是确定的。
{ CONTAINS SQL NO SQL READS SQL DATA MODIFIES SQL DATA }	描述函数对数据的操作类型。
SQL SECURITY { DEFINER INVOKER }	指定函数的执行权限是定义者还是调用者。
routine_body	包含函数的实际执行逻辑，由有效的 SQL 例程语句组成。

示例

CREATE FUNCTION: old function

```
> CREATE FUNCTION func1 (s CHAR(20))
  RETURNS CHAR(50) DETERMINISTIC
  RETURN CONCAT('Hello, ',s,'!');
> SELECT hello('world');
```

CREATE FUNCTION: new function

```
> DELIMITER //
CREATE OR REPLACE FUNCTION func2(num int) RETURNS TINYINT(1)
AS
base int;
BEGIN
   base:=1;
   if num > base then
       return true;
   end if;
       return false;
END;//
DELIMITER ;
```

CREATE FUNCTION: new function with in/out params

```
> DELIMITER //
create function func3(a in int, b out int) returns int
begin
       set b=a;
       return a;
END;//
DELIMITER ;
```

DROP FUNCTION: new or old

```
> DROP function IF EXISTS func1;
```

1.10.17. AP 节点(MPP)的使用

MPP 是分布式 SQL 查询引擎，适用于交互式分析查询，允许你查询存储于 GoldenDB 中的 GB 到 PB 量级的数据，并可实现联邦查询。MPP 利用多个 Worker 线程可以轻松实现并行查询，并可以通过集群进行横向扩展，增加计算节点增强查询效率。

1.10.17.1. hint 注释控制是否走 MPP

```
/*!parallel-processing=on*/ 或 /*M!parallel-processing=on*/
(注意：M一定要大写，否则不生效)
/*!parallel-processing=off*/ 或 /*M!parallel-processing=off*/
/*!parallel-processing=safe*/ 或 /*M!parallel-processing=safe*/
```

推荐使用 /*M! 这种注释，即使语句下DB也会当成注释处理，不会出现解析错误的问题

可添加在语句任意位置，例句：

```
> EXPLAIN /*!parallel-processing=on*/SELECT * FROM t1 WHERE b IN
(SELECT b FROM t2);
> EXPLAIN /*!parallel-processing=off*/SELECT * FROM t1 WHERE b IN
(SELECT b FROM t2);
```

开关：

```
#此 proxy 的服务端口中，专门对接 AP 类业务的端口号及其默认并行计算模式
#最多可配置 20 个，格式为，端口号:计算模式。不同端口间使用,分隔
#例如:8880:1,8881:2  即 8880 端口的业务计算模式为 1，8881 的计算模式为 2
#未写在此配置里的已有服务端口，其计算模式取默认值 0
#默认并行计算模式，0:关闭，1:打开，2:安全（智能识别）
conn_instance_for_mpp=
```

注：该配置项在 `proxy.ini` 配置文件。

1.10.17.2. 代价估算自适应配置

CN 为了自适应 MPP, 部分流程会对 SQL 语句进行代价估算，设定一个阀值，超过此阀值则由 MPP 处理，反之由 CN 层自行处理。

CN 层代价估算计算数阀值，超过则由 MPP 处理

```
unit64: B, range: 0-18446744073709551615 , default:  10000*10000*1
00, dynamic: yes;
sendtompp_cost_rows = 10000000000
```

1.10.17.3. MPP的限制

1. 目前只支持 UR 级别（读未提交）的事务隔离级别。CR 隔离级别暂时不支持。UR 由配置生效，不能以 hint 形式加在语句中。
2. MPP 只能查询静态的数据。当前事务中的 DML 操作的数据，对于 MPP 都是不可见的。
3. MPP 目前只支持 DQL。其中 `INSERT`、`UPDATE` 派生的 DQL 也支持。

1.10.18. 存储过程

1.10.18.1. 单机存储过程

描述

存储过程是一组为了完成特定任务的 SQL 语句集合，单机存储过程中计算节点直接将存储过程定义下发至数据节点执行。

语法

CREATE PROCEDURE

```
CREATE
[DEFINER = { user CURRENT_USER }]
PROCEDURE sp_name ([proc_parameter[,...]])
[characteristic ...] routine_body
[distributed_option]


proc_parameter:
[ IN OUT ] param_name type

type:
Any valid MySQL data type
```

```
characteristic:
COMMENT 'string'
| LANGUAGE SQL
| [NOT] DETERMINISTIC
| { CONTAINS SQL NO SQL READS SQL DATA MODIFIES SQL DATA }
| SQL SECURITY { DEFINER INVOKER }

routine_body:
Valid SQL routine statement

distributed_option:
DISTRIBUTED BY
HASH(column_list) ({groupno},...)
| RANGE(expr) ({groupno VALUES LESS THAN { value MAXVALUE}},...)
| LIST(expr) ({groupno VALUES IN (value_list)},...)
| DUPLICATE ({groupno},...)
```

DROP PROCEDURE

```
DROP {PROCEDURE FUNCTION} [IF EXISTS] sp_name
```

CALL PROCEDURE

```
CALL sp_name([parameter[,...]])

CALL sp_name[()]
```

参数解释

参数	描述
[DEFINER = {user CURRENT_USER}]	定义存储过程的创建者或执行权限。user 指定特定用户，CURRENT_USER 表示当前用户。
PROCEDURE sp_name	指定存储过程的名称。
([proc_parameter[,...]])	存储过程的参数列表，每个参数包括参数名、输入输出方向（IN、OUT 或 IN OUT）和数据类型。
[characteristic...]	存储过程的特性，如注释、语言、确定性等。
routine_body	包含存储过程的实际执行逻辑，由有效的 SQL 例程语句组成。

[distributed_option]	分布式选项，用于指定存储过程的分布式执行方式。
COMMENT 'string'	为存储过程添加注释。
LANGUAGE SQL	指定存储过程使用的语言为 SQL。
[NOT] DETERMINISTIC	表示存储过程的结果是否是确定的。
{ CONTAINS SQL NO SQL READS SQL DATA MODIFIES SQL DATA }	描述存储过程对数据的操作类型。
SQL SECURITY { DEFINER INVOKER }	指定存储过程的执行权限是定义者还是调用者。

示例

```
> DELIMITER //
CREATE PROCEDURE simpleproc (IN a INT, OUT param1 INT)
BEGIN
INSERT INTO t() VALUES(@a);
SELECT COUNT(*) INTO param1 FROM t;
END
DISTRIBUTED BY DUPLICATE (g1)
//
DELIMITER ;
CALL simpleproc(10,@b);
```

使用说明

- 计算节点直接将存储过程定义下发至数据节点执行，因此存储过程定义内容（除分发信息）必须完全兼容数据节点语法。
- 不支持分布式事务。
- 不支持 INOUT 类型参数。
- 支持单点下压的存储过程。根据入参判断出存储过程可直接下压 DN。
- 支持下发全部数据节点的存储过程，根据分发策略规则，将存储过程下发至所有数据节点进行执行，并对执行结果汇总后返回给客户端。

此类存储过程只能存在 IN 类型的参数。

- 创建存储过程必须指定分发策略。

- 如果带有 OUT 参数，必须是单节点表。
- 涉及 PROCEDURE 的语句中，若包含 `Oracle` 函数，是否设置 `parse_mode =2` 模式。

1.10.18.2. 分布式存储过程

描述

分布式存储过程是指可以在分布式数据库环境中执行的存储过程。

语法

存储过程创建

```
CREATE
[DEFINER = { user CURRENT_USER }]
PROCEDURE sp_name ([proc_parameter[,...]])
[characteristic ...] routine_body

proc_parameter:
[ IN OUT INOUT ] param_name type

characteristic:
COMMENT 'string'
| LANGUAGE SQL
| [NOT] DETERMINISTIC
| { CONTAINS SQL NO SQL READS SQL DATA MODIFIES SQL DATA }
| SQL SECURITY { DEFINER INVOKER }

routine_body:
Valid SQL routine statement
```

存储过程执行

```
CALL sp_name([parameter[,...]])

CALL sp_name[()]
```

存储过程修改

只支持同名存储过程整体替换，不支持仅修改指定字段。

```
CREATE OR REPLACE
[DEFINER = { user CURRENT_USER }]
PROCEDURE sp_name ([proc_parameter[,...]])
[characteristic ...] routine_body

proc_parameter:
[ IN OUT INOUT ] param_name type
```

```
characteristic:
COMMENT 'string'
| LANGUAGE SQL
| [NOT] DETERMINISTIC
| { CONTAINS SQL NO SQL READS SQL DATA MODIFIES SQL DATA }
| SQL SECURITY { DEFINER INVOKER }

routine_body:
Valid SQL routine statement
CREATE OR REPLACE PROCEDURE proc_name [characteristic ...]

characteristic:
COMMENT 'string'
| LANGUAGE SQL
| { CONTAINS SQL NO SQL READS SQL DATA MODIFIES SQL DATA }
| SQL SECURITY { DEFINER INVOKER }
```

存储过程删除

```
DROP PROCEDURE [IF EXISTS] sp_name
```

存储过程查看

查看单个存储过程：

```
SHOW CREATE PROCEDURE sp_name
```

查看所有存储过程状态信息：

```
SHOW PROCEDURE STATUS
```

存储过程中支持的 SQL 列表

```
START TRANSACTION
COMMIT
SAVEPOINT identifier
ROLLBACK
ROLLBACK TO identifier
RELEASE SAVEPOINT identifier
SELECT
INSERT
INSERT ... SELECT
REPLACE
REPLACE ... SELECT
DELETE
UPDATE
COPY TABLE
```

```
TRUNCATE TABLE
SET
CREATE DB
DROP DB
CREATE TABLE
需要按GoldenDB的语法指定分发选项
CREATE TABLE ... LIKE 语法在GoldenDB中不支持。
CREATE TABLE时FULLTEXT不支持，因为在GoldenDB中不支持。
DROP TABLE
RENAME TABLE
ALTER TABLE
CREATE INDEX
DROP INDEX
CREATE SEQUENCE
DROP SEQUENCE
ALTER SEQUENCE
CREATE SYNONYM
DROP SYNONYM
CREATE VIEW
DROP VIEW
CREATE PROCEDURE
DROP PROCEDURE
CALL PROCEDURE
SHOW VARIABLES
SHOW COLLATION
SHOW CREATE TABLE
SHOW CREATE TABLE后加WHERE不支持。
SHOW DATABASES
SHOW TABLES
SHOW CREATE DATABASE
SHOW SEQUENCES
SHOW CREATE SEQUENCE
SHOW TRIGGERS
SHOW CREATE TRIGGER
SHOW CREATE PROCEDURE
SHOW CREATE FUNCTION
SHOW FIELDS：例如SHOW FIELDS FROM tbl_1
SHOW CHARSETS
SHOW WARNS
SHOW STATUS
SHOW WARNINGS
SHOW ENGINES
SHOW TABLE STATUS
SHOW PROCEDURE STATUS
SHOW FUNCTION STATUS
SHOW PLUGINS
SHOW KEYS
SHOW EVENTS
SHOW PRIVILEGES
SHOW GRANTS
SIGNAL
RESIGNAL
GET DIAGNOSTICS
```

```
存储过程中的语句：
DECLARE语句
CURSOR相关语句：
OPEN CURSOR,
FETCH INTO
CLOSE CURSOR
```

存储过程中明确不支持的 SQL 列表

- 不支持 `ALTER PROCEDURE`，`CREATE/DROP/ALTER TRIGGER`。
- 不支持 `USE/SHOW ERRORS`。
- 不支持 `PREPARE`、`DEALLOCATE PREPARE`、`EXECUTE`。

参数解释

参数	描述
[DEFINER = {user CURRENT_USER}]	定义存储过程的创建者或执行权限。user 指定特定用户，CURRENT_USER 表示当前用户。
PROCEDURE sp_name	指定存储过程的名称。
([proc_parameter[,...]])	存储过程的参数列表，每个参数包括参数名、输入输出方向（IN、OUT 或 IN OUT）和数据类型。
[characteristic...]	存储过程的特性，如注释、语言、确定性等。
routine_body	包含存储过程的实际执行逻辑，由有效的 SQL 例程语句组成。
COMMENT 'string'	为存储过程添加注释。
LANGUAGE SQL	指定存储过程使用的语言为 SQL。
[NOT] DETERMINISTIC	表示存储过程的结果是否是确定的。
{ CONTAINS SQL NO SQL READS SQL DATA MODIFIES SQL DATA }	描述存储过程对数据的操作类型。

SQL SECURITY { DEFINER INVOKER }	指定存储过程的执行权限是定义者还是调用者。

使用说明

- 不支持自定义函数。
- 存储过程中不支持 @ 变量，和 @@ 变量。
- 分布式存储过程的出入参支持基本的数据类型(各种 INT/INT/UNSIGNED/CHAR/VARCHAR/FLOAT/DOUBLE/DECIMAL/DATE/TIME/DATETIME 等)。
- 支持的时间类型：支持 `yyyy-mm-dd hh:mm:ss` 标准格式，其它格式是未定义的。
- 建议尽可能地减少嵌套调用，或者适当调大 `SPENGINE` 的配置项 `max_sp_engine_stack_size`，并使其生效。
- 不支持隐式传参。
- 存储过程中，不支持 `IN/ALL/ANY/SOME subquery` 定义的 `SET` 语句。

1.10.18.3. 存储过程内常用语法

1. Loop 的使用

语法

```
[begin_label:] LOOP
    statement_list
END LOOP [end_label]
```

参数解释

参数	描述
begin_label	可选，循环开始的标签，可以用于在嵌套循环中标识特定的循环。
LOOP	表示循环的开始。
statement_list	在循环中要执行的一系列语句，可以是任何有效的语句。
END LOOP	表示循环的结束。
end_label	可选，循环结束的标签，与开始标签配合使用，用于在嵌套循环中更清晰地标识和控制流程。

示例

```
> CREATE PROCEDURE doiterate(p1 INT)
BEGIN
x int;
  label1: LOOP
    SET p1 := p1 + 1;
    IF p1 < 10 THEN
     LEAVE label1;
    END IF;
    LEAVE label1;
  END LOOP label1;
  SET x := p1;
END;
```

2. For Loop 的使用

语法

```
FOR var_name IN ..  LOOP
    statement_list
END LOOP
```

参数解释

参数	描述
var_name	变量名，用于在循环中进行迭代。
IN ..	这部分通常是一个值的范围或者一个集合。例如，可以是一个数字范围（如 IN 1..10），或者是一个查询返回的结果集。
LOOP	标志着循环的开始。
statement_list	在循环中要执行的一系列语句，可以是任何有效的语句。
END LOOP	表示循环的结束。

示例

```
> CREATE PROCEDURE forloop() is
v1 INT ;
TYPE REC_MEMS IS RECORD(
id  int,
name VARCHAR(50),
id_1 int);
```

```
TYPE V_MEMS_1 IS TABLE OF REC_MEMS INDEX BY BINARY_INTEGER;
V_MEMS V_MEMS_1;
BEGIN
  Select * from t1 bulk collect into V_MEMS;
  FOR I IN 1 .. V_MEMS.COUNT LOOP
    Insert into t2 (id,name) values( V_MEMS(I).id_1,V_MEMS(I).name
);
  end loop;
END;
```

3. ForALL 的使用

语法

```
forall ident in expr_start .. expr_end
        parallel_sql

forall ident in indices of expr
        parallel_sql

forall in values of expr
        parallel_sql
```

参数解释

参数	描述
ident	表示循环中的标识符，它在每次循环迭代中代表一个特定的值。
expr_start .. expr_end	这两个表达式定义了循环的范围。它们可以是数值、日期、字符串等数据类型。
parallel_sql	这是在每次循环迭代中要执行的 SQL 语句或语句块。
indices of expr	在 expr 的索引范围内，对每个 ident 执行 parallel_sql 操作。
values of expr	在 expr 的值范围内，对每个 ident 执行 parallel_sql 操作。

示例

```
> DECLARE
BEGIN
    FORALL i IN 1 .. 10
        INSERT INTO t1 VALUES (1,1);
    COMMIT;
```

```
END;
```

4. 游标的使用

- 普通游标

语法

```
CURSOR cursor_name IS select_statement
OPEN cursor_name
Fetch cursor_name [BULK COLLECT] INTO var_name
CLOSE cursor_name
```

参数解释

参数	描述
cursor_name	是定义的游标名称，用于标识这个特定的游标。
select_statement	是一个查询语句，用于确定游标要处理的数据范围和内容。
OPEN cursor_name	打开指定名称的游标，使其准备好进行数据提取操作。
BULK COLLECT	表示批量提取数据，提高数据处理效率。
var_name	用于存储从游标中提取的数据的变量。
CLOSE cursor_name	关闭指定的游标，释放相关资源。

示例

```
> CREATE OR REPLACE PROCEDURE sp5() IS
v_name INT;
v1 INT DEFAULT 5;
cursor c_name IS SELECT id_1 FROM t1;
BEGIN
  open c_name ;
  LOOP
    fetch c_name INTO v_name;
    exit WHEN c_name%notfound;
    V1 := v1+1;
  END LOOP;
```

```
END;
```

- Ref 游标

语法

```
Type cur_ref IS ref cursor;
Cur_name cur_ref;
Open cur_name for select_statement
Fetch cursor_name [BULK COLLECT] into var_name
CLOSE cursor_name
```

参数解释

参数	描述
Type cur_ref IS ref cursor	定义了一个游标类型 cur_ref，它是一个引用游标类型。
Cur_name cur_ref	声明了一个名为 Cur_name 的变量，其类型为之前定义的引用游标类型 cur_ref。
Open cur_name for select_statement	从打开的游标 cursor_name 中提取数据。BULK COLLECT（可选）表示批量提取数据。将提取的数据存储到变量 var_name 中。
Fetch cursor_name [BULK COLLECT] into var_name	从打开的游标 cursor_name 中提取数据。BULK COLLECT（可选）表示批量提取数据。将提取的数据存储到变量 var_name 中。
CLOSE cursor_name	关闭游标 cur_name，释放相关资源。

示例

```
> CREATE PROCEDURE sp3() IS
v_name INT;
va VARCHAR(50);
type r_name IS ref cursor;
c_name r_name;
BEGIN
  Open c_name for SELECT id,name FROM t1;
  LOOP
    fetch c_name INTO v_name,va;
    exit when c_name%notfound;
```

```
  END LOOP;
END;
```

- SYS_REFCURSOR 游标

描述

`Oracle` 内部定义的一种 ref 游标

示例

```
> CREATE OR REPLACE FUNCTION F_GETEMPNO(PNAME IN VARCHAR2)
RETURN SYS_REFCURSOR
IS
EMPNO SYS_REFCURSOR;
BEGIN
  OPEN EMPNO FOR
  SELECT EMPNO,ENAME FROM EMP WHERE ENAME=PNAME;
  RETURN(EMPNO);
END F_GETEMPNO;
```

- FOR 游标

语法

```
FOR cursor_name IN (select_statement) LOOP
  SQL_statement
END LOOP;
```

参数解释

参数	描述
cursor_name	循环中用于迭代的游标名称。
select_statement	这是一个查询语句，用于确定游标要遍历的数据集合。
LOOP	标志着循环的开始。
SQL_statement	在每次循环迭代中要执行的 SQL 语句。
END LOOP	标志着循环的结束。

示例

```
> CREATE OR REPLACE PROCEDURE curloop() IS
BEGIN
  FOR cur IN (SELECT * FROM t1) LOOP
    INSERT INTO tb12 VALUES(cur.id,cur.NAME,cur.UID);
  End LOOP;
End;
```

1.10.19. XA 事务语法

描述

在数据库中，`XA` 事务是一种分布式事务处理的规范。

`XA` 事务的主要特点包括：

- 能够协调多个资源管理器（如数据库）参与到同一个事务中。
- 保证在分布式环境下事务的原子性、一致性、隔离性和持久性（`ACID`）特性。

`XA` 事务通常包含两个阶段：

- 准备阶段：事务管理器通知各个资源管理器准备提交事务，资源管理器进行必要的预处理，但不真正提交。
- 提交/回滚阶段：根据准备阶段的结果，如果所有资源管理器都准备成功，则事务管理器通知它们提交事务；否则，通知回滚事务。

语法

XA START/BEGIN

```
XA {START/BEGIN} 'xid'
```

XA END

```
XA END 'xid'
```

XA PREPARE

```
XA PREPARE 'xid'
```

XA COMMIT

```
XA COMMIT 'xid' [ONE PHASE]
```

XA ROLLBACK

```
XA ROLLBACK 'xid'
```

XA RECOVER

```
XA RECOVER [CONVERT XID]
```

参数解释

参数	描述
XA {START/BEGIN}	表示开启 XA 事务的操作。
xid	是一个全局唯一的值，用于标识该事务。
XA END	表示结束一个 XA 事务的操作阶段。
XA PREPARE	通知参与 XA 事务的各个资源管理器（如数据库）准备提交事务。
XA COMMIT	表示提交 XA 事务。
ONE PHASE	如果指定了 ONE PHASE，则表示尝试以一阶段提交的方式完成事务提交。如果没有指定，则通常会按照两阶段提交的流程进行。
XA ROLLBACK	表示执行回滚操作。
XA RECOVER	查看处于 PREPARED 状态的 XA 事务。
CONVERT XID	如果指定了 CONVERT XID，则会将提供的 XID 进行转换，并列出与之相关的处于 PREPARED 状态的 XA 事务。

示例

- 例子 1：`XA` 事务预提交

```
> XA START 'aa';
> INSERT INTO testdb.t1 VALUES (1);
> XA END 'aa';
> XA PREPARE 'aa';
> XA COMMIT 'aa';
```

- 例子 2: `XA` 事务一阶段提交，释放 `xid`

```
> XA START 'aa';
> INSERT INTO testdb.t1 VALUES (1);
> XA END 'aa';
> XA COMMIT 'aa' ONE PHASE;
```

- 例子 3：查询已预提交的 `XA` 事务

```
> XA RECOVER;
```

使用说明

- 不能在普通事务中开启 `XA` 事务。
- 不能在 `XA` 事务中开启普通事务。
- 部分语句类型不能在 `XA` 事务中执行，如 DDL 语句。
- 不支持 `xid` 内容为空，如 `xa start ''`。
- GoldenDB 语法中，`data + formatID` 相同认为是同一个 `xid`。例如 `xid "aa","bb",7` 与 `"aab","b",7` 认为是同一个 `xid`。

1.10.20. Nogtid模式

此功能不使用 GoldenDB 内置的分布式事务（启用 `Nogtid` 模式），由应用层拆分交易实现每个事务只涉及单个分片。

1.10.20.1. 检测标准

- 语句类型：DML，DQL。
- 基准分片选择：以第一条非分布式的 DML 或 DQL 语句为准，如果该条语句本身就跨分片了，则忽略，不以其涉及的分片为基准分片。

1.10.20.2. 报错场景

- `Nogtid` 模式下，不论是否已选定基准分片，CN 节点检测到分布式事务时，能够检测出并报错，拒绝执行，向客户端返回错误。若已选择基准分片，则后续DML 和 DQL 语句只能涉及基准分片的语句，涉及其他分片语句，CN 节点报错。
- CN 节点检测到分布式事务，拒绝执行该事务，向客户端返回报错信息，并且将报错信息写入 CN 节点的日志中。
- 新增一个错误码，`xxxxx-nogtid` 模式下不允许执行分布式事务。`ERROR 10981 (HY000): ERR:Distributed transactions are not allowed in nogtid mode!`。

1.10.20.3. 新增控制开关

CN 新增一个配置项，`dist_check_in_nogtid`，为 1 表示对事务中的读写进行检测，为 2 表示检测时只检测事务中写语句忽略读语句，为 0 则不检测。默认为 0。当且仅当 `Nogtid` 模式打开时，该新增配置项才有效。

1.10.21. 保留字

本版本涉及的 CN `MySQL` 模式保留字如下：

```
A
ACCESSIBLE, ADD, ALL, ALTER, ANALYZE, AND, ARRAY, AS, ASC, ASENSITIVE
, AUDIT, AUTONOMOUS_TRANSACTION
B
BEFORE, BETWEEN, BIGINT, BINARY, BLOB, BOTH, BREAKPOINT, BY
C
CASE, CHANGE, CHECK, CHGDISKEY, CLOB, COLLATE, COLUMN, COLUMN_VALUE,
CONDITION, CONNECT, CONNECT_BY_ISLEAF, CONNECT_BY_ROOT, CONSTRAINT,
CONSTRUCTOR, CONTINUE, CONVERT, COUNT, CREATE, CROSS, CURRENT_DATE,
CURRENT_ROLE, CURRENT_TIME, CURRENT_TIMESTAMP, CURRENT_USER, CURRVA
L, CURSOR
D
DATABASES, DAY_HOUR, DAY_MICROSECOND, DAY_MINUTE, DAY_SECOND, DBTIME
ZONE, DEADLOCK, DEBUG, DEBUG_BACKSTRACE, DEBUG_CONTINUE, DEBUG_DELET
E, DEBUG_FINISH, DEBUG_INFO, DEBUG_NEXT, DEBUG_PRINT, DEBUG_STEP, DE
C, DECIMAL, DECLARE, DECODE, DEFAULT, DELAYED, DELETE, DELETING, DETE
RMINISTIC, DISTINCT, DISTINCTROW, DISTRIBUTED, DIV, DOUBLE, DROP, DU
AL
E
EACH, EDITIONABLE, ELSE, ELSEIF, ELSIF, EMPTY, ENCLOSED, ESCAPED, EXC
EPT, EXCEPTION, EXEC, EXISTS, EXIT, EXPIRE, EXTEND
F
FALSE, FETCH, FILELINE, FINAL, FIRST, FLASHBACK, FLOAT, FLOAT4, FLOAT
8, FOR, FORALL, FORCE, FOREIGN, FROM, FROM_TZ, FULL, FULLTEXT
G
GDB_NOLOCK, GDB_OUTLINE, GET, GOTO, GRANT, GROUP, GROUPS, GTIDSET
H
HAVING, HIGH_PRIORITY, HOUR_MICROSECOND, HOUR_MINUTE, HOUR_SECOND
I
IF, IGNORE, IN, INCREMENT, INDEX, INDICES, INNER, INSENSITIVE, INSERT
, INSERTING, INSTANTIABLE, INSTEAD, INT, INT1, INT2, INT3, INT4, INT8
, INTEGER, INTERSECT, INTERVAL, INTO, IS, ITERATE
J
JOIN, JSON
K
KEEP, KEEPALIVE, KEY, KILL
L
LAST, LATERAL, LEADING, LEAVE, LEFT, LEVEL, LIKE, LIMIT, LINEAR, LOAD
, LOCALS, LOCALTIME, LOCALTIMESTAMP, LOCK, LONG, LONGBLOB, LONGTEXT,
LOW_PRIORITY
M
MASTER_SSL_VERIFY_SERVER_CERT, MATCH, MATERIALIZED_VIEW_TMP_TABLE,
```

```
MAXVALUE, MEDIUMBLOB, MEDIUMINT, MEDIUMTEXT, MEMBER, MIDDLEINT, MINU
S, MINUTE_MICROSECOND, MINUTE_SECOND, MOD, MODIFIES, MULTI_STEP_QUER
Y, MVCC_PURGE
N
NATURAL, NEVER, NEXT, NEXTVAL, NOCACHE, NOCYCLE, NOGTID, NOLOGGING, N
OMAXVALUE, NOMINVALUE, NONEDITIONABLE, NOORDER, NOPARALLEL, NOT, NO_
WRITE_TO_BINLOG, NULL, NULLCOLS, NULLS, NUMERIC
O
OF, OFFSET, ON, OPTIMIZE, OPTIONALLY, OR, ORDER, OTHERS, OUT, OUTFILE
, OUTLINE, OUTSOCKET, OVER, OVERRIDING
P
PAGE_CHECKSUM, PARALLEL, PARALLEL_ENABLE, PARSE_VCOL_EXPR, PARTITIO
N, PARTITIONSTORAGEDB, PERSIST, PERSISTABLE, PIVOT, PLS_INTEGER, PRA
GMA, PRIMARY, PRIOR, PRIVATE, PROCEDURE, PROCESS, PUBLIC, PURGE
R
RAISE, RAISE_APPLICATION_ERROR, RAW, READ, READBALANCE, READMASTER,
READS, READSLAVE, READ_WRITE, REAL, RECURSIVE, RECYCLEBIN, RECYCLEBI
N_VERSION, REDISTRIBUTING, REF, REFERENCES, REFERENCING, REGEXP, REM
AIN, RENAME, REPLACE, REQUIRE, RESIGNAL, RETURNING, REVOKE, RIGHT, RL
IKE, ROWNUM, ROWTYPE
S
SAMEDB, SAMPLE, SCHEMAS, SECOND_MICROSECOND, SELECT, SEMI, SEQUENCES
, SESSIONTIMEZONE, SESSION_VARIABLES_ADMIN, SET, SHOW, SIBLINGS, SIG
NAL, SKIP, SMALLINT, SNAPSHOTPOINT, SOUNDS, SPATIAL, SPECIFIC, SQL, S
QLCODE, SQLERRM, SQLEXCEPTION, SQLSTATE, SQLWARNING, SQL_BIG_RESULT
, SQL_CALC_FOUND_ROWS, SQL_SMALL_RESULT, SSL, START, STARTING, STATI
C, STATS_AUTO_RECALC, STATS_PERSISTENT, STATS_SAMPLE_PAGES, STORAGE
CLUSTER, STORAGEDB, STORED, STRAIGHT_JOIN, SUBDISTRIBUTED, SYNONYM,
SYSDATE, SYSTIMESTAMP, SYS_CONNECT_BY_PATH, SYS_REFCURSOR
T
TABLE, TEMPFILE, TERMINATED, THEN, TINYBLOB, TINYINT, TINYTEXT, TO, T
RAILING, TRANSACTION_TIMEOUT, TRIGGER, TRIM, TRUE
U
UNDO, UNION, UNIQUE, UNLOCK, UNPIVOT, UNSIGNED, UPDATE, UPDATING, USE
, USER, USING, USING_HINT, UTC_DATE, UTC_TIME, UTC_TIMESTAMP
V
VALUES, VARBINARY, VARCHAR, VARCHAR2, VARCHARACTER, VARRAY, VARYING
W
WHEN, WHERE, WHILE, WINDOW, WITH, WRITE, WRITE_SAMEDB
X
XOR
Z
ZEROFILL
其他
_GDB_CN
```

本版本涉及的 CN `Oracle` 模式保留字如下：

```
A
ACCESSIBLE, ADD, ALL, ALTER, ANALYZE, AND, ARRAY, AS, ASC, ASENSITIVE
, AUDIT, AUTONOMOUS_TRANSACTION
```

```
B
BETWEEN, BIGINT, BINARY, BLOB, BOTH, BREAKPOINT, BULK, BY
C
CASE, CHANGE, CHECK, CHGDISKEY, CLOB, COLLATE, COLUMN, COLUMN_VALUE,
CONDITION, CONNECT, CONNECT_BY_ISLEAF, CONNECT_BY_ROOT, CONSTANT,CO
NSTRAINT, CONSTRUCTOR, CONVERT, COUNT, CREATE, CROSS, CURRENT_DATE,
CURRENT_ROLE, CURRENT_TIME, CURRENT_TIMESTAMP, CURRENT_USER, CURRVA
L, CURSOR
D
DATABASE, DATABASES, DATAFILES, DAY_HOUR, DAY_MICROSECOND, DAY_MINUT
E, DAY_SECOND, DBTIMEZONE, DEADLOCK, DEBUG, DEBUG_BACKSTRACE, DEBUG_
CONTINUE, DEBUG_DELETE, DEBUG_FINISH, DEBUG_INFO, DEBUG_NEXT, DEBUG_
PRINT, DEBUG_STEP, DEC, DECIMAL, DECLARE, DECODE, DEFAULT, DELAYED, D
ELETE, DELETING, DETERMINISTIC, DISTINCT, DISTINCTROW, DISTRIBUTED,
DIV, DOUBLE, DROP, DUAL
E
EACH, EDITIONABLE, ELSE, ELSEIF, ELSIF, EMPTY, ENCLOSED, ESCAPED, EXC
EPT, EXEC, EXISTS, EXIT, EXPIRE, EXTEND
F
FALSE, FETCH, FILELINE, FINAL, FIRST, FLASHBACK, FLOAT, FLOAT4, FLOAT
8, FOR, FORALL, FORCE, FOREIGN, FROM, FROM_TZ, FULL, FULLTEXT
G
GDB_NOLOCK, GDB_OUTLINE, GET, GOTO, GRANT, GROUP, GROUPS, GTIDSET
H
HAVING, HIGH_PRIORITY, HOUR_MICROSECOND, HOUR_MINUTE, HOUR_SECOND
I
IF, IGNORE, IN, INCREMENT, INDEX, INDICES, INNER, INSENSITIVE, INSERT
, INSERTING, INSTANTIABLE, INSTEAD, INT, INT1, INT2, INT3, INT4, INT8
, INTEGER, INTERSECT, INTERVAL, INTO, IS, ITERATE
J
JOIN, JSON
K
KEEP, KEEPALIVE, KEY, KILL
L
LAST, LATERAL, LEADING, LEAVE, LEFT, LEVEL, LIKE, LIMIT, LINEAR, LOAD
, LOCALS, LOCALTIME, LOCALTIMESTAMP, LOCK, LONG, LONGBLOB, LONGTEXT,
LOW_PRIORITY
M
MASTER_SSL_VERIFY_SERVER_CERT, MATERIALIZED_VIEW_TMP_TABLE, MEDIUMB
LOB, MEDIUMINT, MEDIUMTEXT, MEMBER, MIDDLEINT, MINUS, MINUTE_MICROSE
COND, MINUTE_SECOND, MOD, MODIFIES, MULTI_STEP_QUERY, MVCC_PURGE
N
NATURAL, NEVER, NEXT, NEXTVAL, NOCACHE, NOCYCLE, NOGTID, NOLOGGING, N
OMAXVALUE, NOMINVALUE, NONEDITIONABLE, NOORDER, NOPARALLEL, NOT, NO_
WRITE_TO_BINLOG, NULL, NULLCOLS, NULLS, NUMERIC
O
OF, OFFSET, ON, OPTIMIZE, OPTIONALLY, OR, ORDER, OTHERS, OUT, OUTFILE
, OUTLINE, OUTSOCKET, OVER, OVERRIDING
P
PAGE_CHECKSUM, PARALLEL_ENABLE, PARSE_VCOL_EXPR, PARTITION, PARTITI
ONSTORAGEDB, PERSIST, PERSISTABLE, PIVOT, PLS_INTEGER, PRAGMA, PRIMA
RY, PRIOR, PRIVATE, PROCEDURE, PROCESS, PUBLIC, PURGE
R
```

```
RAISE, RAISE_APPLICATION_ERROR, RAW, READ, READBALANCE, READMASTER,
READS, READSLAVE, READ_WRITE, REAL, RECURSIVE, RECYCLEBIN, RECYCLEBI
N_VERSION, REDISTRIBUTING, REF, REFERENCES, REFERENCING, REMAIN, REN
AME, REPLACE, REQUIRE, RESIGNAL, RETURNING, REVOKE, RIGHT, RLIKE, ROW
NUM, ROWTYPE
S
SAMEDB, SAMPLE, SCHEMAS, SECOND_MICROSECOND, SELECT, SEMI, SEQUENCES
, SESSIONTIMEZONE, SESSION_VARIABLES_ADMIN, SET, SHOW, SIBLINGS, SIG
NAL, SKIP, SMALLINT, SNAPSHOTPOINT, SOUNDS, SPATIAL, SPECIFIC, SQL, S
QLCODE, SQLERRM, SQLEXCEPTION, SQLSTATE, SQLWARNING, SQL_BIG_RESULT
, SQL_CALC_FOUND_ROWS, SQL_SMALL_RESULT, SSL, START, STARTING, STATI
C, STATS_AUTO_RECALC, STATS_PERSISTENT, STATS_SAMPLE_PAGES, STORAGE
CLUSTER, STORAGEDB, STORED, STRAIGHT_JOIN, SUBDISTRIBUTED, SYNONYM,
SYSDATE, SYSTIMESTAMP, SYS_CONNECT_BY_PATH, SYS_REFCURSOR
T
TABLE, TEMPFILE, TERMINATED, THEN, TINYBLOB, TINYINT, TINYTEXT, TO, T
RAILING, TRANSACTION_TIMEOUT, TRIGGER, TRIM, TRUE
U
UNDO, UNION, UNIQUE, UNLOCK, UNPIVOT, UNSIGNED, UPDATE, UPDATING, USE
, USER, USING, USING_HINT, UTC_DATE, UTC_TIME, UTC_TIMESTAMP
V
VALUES, VARBINARY, VARCHAR, VARCHAR2, VARCHARACTER, VARRAY, VARYING
W
WHEN, WHERE, WHILE, WINDOW, WITH, WRITE, WRITE_SAMEDB
X
XOR
Z
ZEROFILL
其他
_GDB_CN
```

本版本涉及的 DN 保留关键字如下：

```
A
ACCESSIBLE, ADD, ALL, ALTER, ANALYZE, AND, AS, ASC, ASENSITIVE
B
BETWEEN, BIGINT, BINARY, BLOB, BOTH, BY
C
CASCADE, CHANGE, CHECK, COLLATE, COLUMN, CONSTRAINT, CONVERT, CREATE
, CROSS, CUBE, CUME_DIST, CURRENT_TIME, CURRENT_USER, CURSOR
D
DATABASES, DAY_HOUR, DAY_MICROSECOND, DAY_MINUTE, DAY_SECOND, DEC, D
ECIMAL, DECLARE, DEFAULT, DELAYED, DELETE, DETERMINISTIC, DISTINCT,
DISTINCTROW, DIV, DOUBLE, DROP
E
EACH, ELSE, ELSEIF, ELSIF, EMPTY, ENCLOSED, ENTITYESCAPING, ESCAPED,
EXCEPT, EXISTS, EXIT
F
FALSE, FIRST_VALUE, FLASHBACK, FLOAT, FLOAT4, FLOAT8, FOR, FOREIGN, F
ROM, FULLTEXT
G
GENERATED, GET, GRANT, GROUP, GROUPING
```

```
H
HAVING, HIGH_PRIORITY, HOUR_MICROSECOND, HOUR_MINUTE, HOUR_SECOND
I
IF, IGNORE, IN, INDEX, INSENSITIVE, INSERT, INT, INT1, INT2, INT3, IN
T4, INT8, INTEGER, INTERSECT, INTERVAL, INTO, IO_AFTER_GTIDS, IO_BEF
ORE_GTIDS, IS, ITERATE
J
JSON_TABLE
K
KEY, KILL
L
LAG, LAST_VALUE, LATERAL, LEAD, LEADING, LEAVE, LIKE, LINEAR, LINES,
LOAD, LOCK, LOGGING, LONG, LONGBLOB, LONGTEXT, LOW_PRIORITY
M
MASTER_BIND, MASTER_SSL_VERIFY_SERVER_CERT, MEDIUMBLOB, MEDIUMINT,
MEDIUMTEXT, MIDDLEINT, MINUS, MINUTE_MICROSECOND, MINUTE_SECOND, MOD
, MODIFIES
N
NATURAL, NOENTITYESCAPING, NOLOGGING, NOT, NO_WRITE_TO_BINLOG, NTH_V
ALUE, NTILE, NULL, NUMERIC
O
OF, ON, OPTIMIZE, OPTIMIZER_COSTS, OPTION, OPTIONALLY, OR, ORDER, OUT
, OUTFILE, OUTSOCKET
P
PERCENT_RANK, PLS_INTEGER, PRIMARY, PROCEDURE, PURGE
R
READ, READS, READ_WRITE, REAL, RECURSIVE, REFERENCES, RENAME, REPLAC
E, REQUIRE, RESIGNAL, REVOKE, RLIKE, ROW, ROW_NUMBER
S
SCHEMAS, SECOND_MICROSECOND, SELECT, SET, SHOW, SIGNAL, SMALLINT, SP
ATIAL, SPECIFIC, SQLEXCEPTION, SQLSTATE, SQLWARNING, SQL_BIG_RESULT
, SQL_CALC_FOUND_ROWS, SQL_SMALL_RESULT, SSL, STARTING, STORED, STRA
IGHT_JOIN
T
TABLE, TERMINATED, THEN, TINYBLOB, TINYINT, TINYTEXT, TO, TRAILING, T
RIGGER, TRUE
U
UNDO, UNION, UNIQUE, UNLOCK, UNSIGNED, UPDATE, USE, USING, UTC_DATE,
UTC_TIME, UTC_TIMESTAMP
V
VALUES, VARBINARY, VARCHAR, VARCHARACTER, VARYING, VIRTUAL
W
WHERE, WHILE, WINDOW, WITH, WRITE
X
XOR
Z
ZEROFILL
```

注意项

- CURRVAL、DELETE、EXISTS、LIMIT 和 NEXTVAL 关键字做表名时不支持带库名方式创建，例如：

db.DELETE(不支持) db.`DELETE`(支持) DELETE(不带库名不支持)

1.10.22. 函数限制

1.10.22.1. 随机值函数和时间函数

复制表

对于复制表，`INSERT` 语句和 `UPDATE` 语句会下发到表的所有分片上执行，在各分片插入相同的数据或进行相同的更新。但某些场景下，存在实际插入或更新到各分片上的数据可能不一致的风险，主要是随机值函数和时间函数的使用所导致的。下面列出存在分片间数据不一致风险的场景，在实际使用时需评估风险、谨慎考虑。

- 复制表进行 `INSERT` 或者 `UPDATE` 操作，使用数据库的下述函数生成数据
- 生成随机值函数 `rand`
- 时间函数（获取当前时刻）
- `now`
- `sysdate`
- `curtime`
- `current_time`
- `current_timestamp`
- `localtime`
- `localtimestamp`
- `utc_time`
- `utc_timestamp`
- `systimestamp`
- 生成时间戳函数
- `unix_timestamp`
- 建表 DDL 中时间类型字段的自动更新特性
- `TIMESTAMP(n) DEFAULT CURRENT_TIMESTAMP(n) ON UPDATECURRENT_TIMESTAMP(n)`（不带精度同样也存在风险）
- 字段类型
- `TIMESTAMP`
- `DATETIME`
- `DEFAULT` 值
- `CURRENT_TIMESTAMP`
- `LOCALTIME`
- `LOCALTIMESTAMP`
- `UPDATE` 值
- `CURRENT_TIMESTAMP`
- `LOCALTIME`
- `LOCALTIMESTAMP`
- 虚拟列表达式中包含自动更新特性的时间函数类型的列

- `GENERATED ALWAYS AS` 表达式中含上述自动更新特性的时间函数类型的列

分发表

对于分发表，`UPDATE` 语句使用数据库的下述时间函数生成 `SET` 的数据时（即 `UPDATE` 语句的 `SET` 表达式中带下述时间函数），存在同一批更新的数据最终实际值不一致的风险，尤其是在所要更新数据的数量较大的场景下。

- 时间函数（获取当前时刻）
- `now`
- `sysdate`
- `curtime`
- `current_time`
- `current_timestamp`
- `localtime`
- `localtimestamp`
- `utc_time`
- `utc_timestamp`
- `systimestamp`
- 生成时间戳函数
- `unix_timestamp`

1.10.23. PACKAGE

描述

`PACKAGE`（包）通常用于将相关的函数、过程、变量等逻辑单元组合在一起，以提高代码的组织性和可维护性。

- **包头的创建**

  语法

```
CREATE PACKAGE pkg_name
  [characteristic ...] is_or_as [package_specification_list] END [
pkg_name]

characteristic:
    default_collation_clause
    | invoker_rights_clause
    | accessible_by_clause

package_specification_list:
    type_definition
    cursor_declaration
    item_declaration
```

```
    package_function_declaration
    package_procedure_declaration
```

参数解释

参数	描述
CREATE PACKAGE pkg_name	这是创建包的起始语句，pkg_name 是您要创建的包的名称。
default_collation_clause	用于指定包中数据的默认排序规则。这有助于在处理数据时确定默认的排序方式。
invoker_rights_clause	用于定义调用这个包的用户所具有的权限，例如是拥有执行权限还是具有修改包内内容的权限等。
accessible_by_clause	指定哪些用户或角色能够访问这个包。这有助于控制对包的访问权限，确保只有授权的用户或角色可以使用包中的功能。
is_or_as	用于连接前面的特性和后面的包规范列表。
type_definition	定义新的数据类型。
cursor_declaration	声明游标。
item_declaration	声明变量或常量。
package_function_declaration	定义包中的函数。
package_procedure_declaration	定义包中的过程。

- **包体的创建**

  语法

```
CREATE OR REPLACE PACKAGE BODY pkg_name
  [characteristic ...]
  is_or_as declare_section [initialize_section] END [pkg_name]

initialize_section:
```

```
  BEGIN statement [EXCEPTION exception_handler]
```

参数解释

参数	描述
CREATE OR REPLACE PACKAGE BODY pkg_name	表示创建或替换名为 pkg_name 的包体。
characteristic	可能是一些特定的属性或特性设置，具体取决于您所使用的数据库系统和上下文。
is_or_as	用于连接前面的特性和后面的声明部分。
declare_section	这是声明部分，用于声明包体内部使用的变量、游标、类型等。
BEGIN	标志着初始化部分的开始。
statement	在初始化部分执行的语句，例如对变量进行初始化赋值等操作。
EXCEPTION exception_handler	用于处理在初始化部分可能出现的异常情况。

- **包的执行**

  语法

```
CALL pkg_name.sp_name([parameter[,...]])
CALL pkg_name.sp_name[()]
SELECT pkg_name.fc_name([parameter[,...]])
SELECT pkg_name.fc_name[()]
```

参数解释

参数	描述
pkg_name	包的名称。
sp_name	是包内存储过程的名称。

parameter[,...]	是传递给存储过程的参数，可以有零个或多个。
CALL pkg_name.sp_name[()]	这表示调用包中的存储过程，但不传递任何参数。
SELECT	表示执行一个查询操作。
fc_name	包内函数的名称。
parameter[,...]	传递给函数的参数。
SELECT pkg_name.fc_name[()]	表示调用包中的函数且不传递参数进行查询。

- **包头的修改**

只支持同名包头整体替换，不支持仅修改指定字段。

语法

```
CREATE OR REPLACE PACKAGE pkg_name
  [characteristic ...] is_or_as [package_specification_list] END [pkg_name]

characteristic:
    default_collation_clause
    | invoker_rights_clause
    | accessible_by_clause

package_specification_list:
    type_definition
    cursor_declaration
    item_declaration
    package_function_declaration
    package_procedure_declaration
```

参数解释

参数	描述
CREATE OR REPLACE PACKAGE pkg_name	创建或替换一个名为 pkg_name 的包。

default_collation_clause	用于指定包中数据的默认排序规则。这有助于在处理数据时确定默认的排序方式。
invoker_rights_clause	用于定义调用这个包的用户所具有的权限，例如是拥有执行权限还是具有修改包内内容的权限等。
accessible_by_clause	指定哪些用户或角色能够访问这个包。这有助于控制对包的访问权限，确保只有授权的用户或角色可以使用包中的功能。
is_or_as	用于连接前面的特性和后面的包规范列表。
type_definition	定义新的数据类型。
cursor_declaration	声明游标。
item_declaration	声明变量或常量。
package_function_declaration	定义包中的函数。
package_procedure_declaration	定义包中的过程。

- **包体的修改**

只支持同名包头整体替换，不支持仅修改指定字段。

语法

```
CREATE OR REPLACE
  PACKAGE BODY pkg_name
  [characteristic ...]
  is_or_as declare_section [initialize_section] END [pkg_name]

initialize_section:
   BEGIN statement [EXCEPTION exception_handler]
```

参数解释

参数	描述

CREATE OR REPLACE PACKAGE BODY pkg_name	表示创建或替换名为 pkg_name 的包体。
characteristic	可能是一些特定的属性或特性设置，具体取决于您所使用的数据库系统和上下文。
is_or_as	用于连接前面的特性和后面的声明部分。
declare_section	这是声明部分，用于声明包体内部使用的变量、游标、类型等。
BEGIN	标志着初始化部分的开始。
statement	在初始化部分执行的语句，例如对变量进行初始化赋值等操作。
EXCEPTION exception_handler	用于处理在初始化部分可能出现的异常情况。

- **包头的删除**

  语法

```
DROP PACKAGE [IF EXISTS] pkg_name
```

参数解释

参数	描述
DROP PACKAGE	执行删除包的操作。
IF EXISTS	可选，如果指定了 IF EXISTS，当要删除的包不存在时，不会产生错误。
pkg_name	要删除的包的名称。

- **包体的删除**

  语法

```
DROP PACKAGE BODY [IF EXISTS] pkg_name
```

参数解释

参数	描述
DROP PACKAGE BODY	执行删除包体的操作。
IF EXISTS	可选，如果指定了 IF EXISTS，当要删除的包不存在时，不会产生错误。
pkg_name	要删除的包的名称。

- **包头内支持的 SQL 列表**

  语法

存储过程和自定义函数声明：

```
FUNCTION fc_name([list]) RETURN TYPE;
PROCEDURE sp_name([list]);
```

自定义类型声明定义：

```
定义一个数据类型
TYPE var_name IS TYPE  [DEFAULT value];
定义一个记录类型
TYPE var_name IS RECORD  '(' var_list ')';
定义一个表类型
TYPE var_name IS TABLE OF TYPE (INDEX BY TYPE );
定义一个可变长度数据类型
TYPE var_name IS (VARYING) VARRAY OF(num) of TYPE;
```

参数解释

参数	描述
FUNCTION	定义一个函数。
fc_name	函数名称。
list	函数的参数列表，可以为空，也可以包含一个或多个参数，每个参数需要指定数据类型和名称。

RETURN TYPE	指定函数返回值的数据类型。
PROCEDURE	定义一个存储过程。
sp_name	存储过程名称。
DEFAULT value	可选部分，为变量指定默认值。
var_list	包含在记录中的字段列表，每个字段都有其名称和数据类型。
VARYING	可选，表示数据长度可变。
num	指定数据的最大长度。

- **包体内支持的 SQL 列表**

  语法 存储过程和自定义函数定义：

```
FUNCTION fc_name([list]) RETURN TYPE IS function_body
PROCEDURE sp_name([list]) IS sp_body;
```

自定义类型声明定义：

```
定义一个数据类型
Type var_name IS TYPE  [DEFAULT value]
定义一个记录类型
TYPE var_name IS RECORD  '(' var_list ')';
定义一个表类型
TYPE var_name IS TABLE OF TYPE (INDEX BY TYPE )
定义一个可变长度数据类型
TYPE var_name IS (VARYING) VARRAY OF(num) of TYPE
```

参数解释

参数	描述
FUNCTION	定义一个函数。
fc_name	函数名称。

list	函数的参数列表，可以为空，也可以包含一个或多个参数，每个参数需要指定数据类型和名称。
RETURN TYPE	指定函数返回值的数据类型。
function_body	函数的主体部分，包含实现函数功能的具体代码和逻辑。
PROCEDURE	定义一个存储过程。
sp_name	存储过程名称。
sp_body	存储过程的主体部分，包含执行过程操作的具体代码和逻辑。
DEFAULT value	可选部分，为变量指定默认值。
var_list	包含在记录中的字段列表，每个字段都有其名称和数据类型。
VARYING	可选，表示数据长度可变。
num	指定数据的最大长度。

- **DBMS 系统包**

支持 `Oracle` 的 `DBMS` 系统包，需要在 `Oracle` 模式下，设置 `sql_mode_proxy=ORA_COMPATIBLE_MODE`。

使用说明

不支持包内定义游标。

1.10.24. 自定义类型

1.10.24.1. 全局自定义类型

描述

全局自定义类型是一种由用户根据特定需求创建的、可在整个数据库范围内使用的数据类型。

语法

TYPE 的创建

```
CREATE TYPE type_name
 is_or_as [type_def]

type_def:
    object_base_type_def
```

TYPE 的调用

```
TYPE_name.type_var
```

TYPE 的修改

```
CREATE OR REPLACE
TYPE type_name
is_or_as [type_def]

type_def:
    object_base_type_def
```

TYPE 的删除

```
DROP TYPE type_name
```

支持类型列表

```
CREATE OR REPLACE TYPE type_name  IS TABLE  OF basic_type;
CREATE OR REPLACE TYPE type_name  IS VARRAY (num)  OF basic_type;
CREATE OR REPLACE TYPE type_name  as object  (basic_type);
```

参数解释

参数	描述
CREATE TYPE type_name	表示创建一个新的数据类型，type_name 是要创建的类型的名称。
is_or_as	用于连接前面的类型名称和后面的类型定义。
object_base_type_def	这是一种对象基础类型的定义，可能包括对对象属性、方法等的描述。
IS TABLE OF	表示定义的类型是一个表类型。

IS VARRAY (num)	表示定义的类型是一个可变长度数组类型，num 表示数组的最大长度。
as object	表示定义的类型是一个对象类型。
basic_type	数据类型。

使用说明

目前只支持上述类型。

1.10.24.2. 存储过程中的自定义类型

描述

在数据库存储过程中使用自定义类型，可以使代码更具可读性、可维护性，并增强数据的一致性和准确性。

语法

TYPE的创建

```
Type var_name IS [type_def]  [DEFAULT value]
  SUBTYPE var_name IS base_type

type_def:
  TABLE OF data_type (INDEX BY type )
  [VARYING] VARRAY OF(num) of data_type
  RECORD  '(' var_list ')';
  REF CURSOR cursor_list

data_type:
  base_type
|  [ref] var_name
|  var_name % rowtype
|  var_name %type
```

示例

```
create or replace procedure p_cursor1() is
type t_fetch_record is record (
aa int,
cc int);
TYPE V_MEMS_1 IS TABLE OF t_fetch_record;
V_MEMS V_MEMS_1;
cursor cur_cursor is select id, b from t1 limit 3;
begin
```

```
  open cur_cursor;
  fetch cur_cursor BULK COLLECT into V_MEMS;
  select V_MEMS.count from dual;
  close cur_cursor;
end
```

1.10.25. 正则表达式

1.10.25.1. 正则表达式函数语法

- REGEXP_LIKE 函数

描述

用于检查字符串是否与指定的正则表达式模式匹配，返回 1 或 0。

语法

```
REGEXP_LIKE(source_char, pattern[, match_param])
```

参数解释

参数	描述
source_char	用作搜索值的字符表达式，可以是列名或者字符串常量、变量。
pattern	正则表达式。
match_param	匹配选项。取值范围： i：大小写不敏感； c：大小写敏感；n：允许 '.'与换行符匹配；m：多行模式。

- REGEXP_INSTR 函数

描述

返回字符串中与指定正则表达式模式匹配的子串的位置。

语法

```
REGEXP_INSTR(source_char, pattern[, position[, occurrence[, return_opt[, match_param]]]])
```

参数解释

参数	描述
source_char	用作搜索值的字符表达式，可以是列名或者字符串常量、变量。
pattern	正则表达式。
position	正则表达式匹配的起始字符位置。如果省略，默认为 1，表示第一个位置的字符串。
occurrence	正则表达式匹配的第几个匹配项。即 pattern 在 source_char 中的第 n 个匹配位置。如果省略，默认为 1。
return_opt	返回值选项，默认值 0 则返回出现的第一个字符的位置，默认值 1 则返回出现后的字符的位置。
match_param	匹配选项。取值范围： i：大小写不敏感； c：大小写敏感；n：允许 '.'与换行符匹配；m：多行模式。

- REGEXP_REPLACE 函数

描述

将字符串中与指定正则表达式模式匹配的部分替换为指定的字符串。

语法

```
REGEXP_REPLACE(source_char, pattern[, replace_string[, position[, occurrence[, match_param]]]])
```

参数解释

参数	描述
source_char	用作搜索值的字符表达式，可以是列名或者字符串常量、变量。
pattern	正则表达式。
replace_string	替换的字符串。

position	正则表达式匹配的起始字符位置。如果省略，默认为 1，表示第一个位置的字符串。
occurrence	正则表达式匹配的第几个匹配项。即 pattern 在 source_char 中的第 n 个匹配位置。如果省略，默认为 1。
match_param	匹配选项。取值范围： i：大小写不敏感； c：大小写敏感；n：允许 '.'与换行符匹配；m：多行模式。

- REGEXP_SUBSTR 函数

描述

返回字符串中与指定正则表达式模式匹配的子字符串。

语法

```
REGEXP_SUBSTR(source_char, pattern[, position[, occurrence[, match_param[, subexpr]]]])
```

参数解释

参数	描述
source_char	用作搜索值的字符表达式，可以是列名或者字符串常量、变量。
pattern	正则表达式。
position	正则表达式匹配的起始字符位置。如果省略，默认为 1，表示第一个位置的字符串。
occurrence	正则表达式匹配的第几个匹配项。即 pattern 在 source_char 中的第 n 个匹配位置。如果省略，默认为 1。
match_param	匹配选项。取值范围： i：大小写不敏感； c：大小写敏感；n：允许 '.'与换行符匹配；m：多行模式。
subexpr	是一个介于 0 到 9 之间的整数， 具有子表达式的 pattern，表示模式中的第几个子表达式是函数的匹配目标。

- REGEXP_COUNT 函数

描述

返回字符串中与指定正则表达式模式匹配的次数。

语法

```
REGEXP_COUNT(source_char, pattern[, position[, match_param]])
```

参数解释

参数	描述
source_char	用作搜索值的字符表达式，可以是列名或者字符串常量、变量。
pattern	正则表达式。
position	正则表达式匹配的起始字符位置。如果省略，默认为 1，表示第一个位置的字符串。
match_param	匹配选项。取值范围： i：大小写不敏感； c：大小写敏感；n：允许 '.'与换行符匹配；m：多行模式。

1.10.26. 透明加密

透明加密，透明加密顾名思义，对用户来说加密信息是不感知的，但底层的数据已经被加密，若被盗取或者丢失，需要密钥才能解密。 虽然底层数据是加密的，但用户在使用时，数据库反馈给用户的是已经解密的数据，所以对用户是透明的。

1.10.26.1. 表加密

1. 创建加密表 语法：参考表的创建语法，加密是其中表的可选属性。

示例

```
> CREATE TABLE t1 (c1 INT) ENCRYPTION = 'Y';
```

2. 修改加密表属性 语法：参考表的修改语法，加密是其中表的可选修改属性。

示例

```
> ALTER TABLE t1 ENCRYPTION = 'Y';
```

3. 更新密钥

语法

```
ALTER INSTANCE ROTATE 引擎名 MASTER KEY;
```

1.10.26.2. 列加密

1. 创建加密列表 语法：参考表的创建语法，加密列是其中列的可选属性。

示例

```
> CREATE TABLE t1 (a VARCHAR(300) ENCRYPTION ='AES');
```

2. 修改加密列属性 语法：参考表的修改语法，加密列是其中列的可选修改属性。

示例

```
> ALTER TABLE t1 MODIFY c VARBINARY(1000)  ENCRYPTION ='AES';
```

1.11. 批量协议功能

本章节旨在向开发人员介绍 GoldenDB 数据库的批量协议功能。

1.11.1. 批量协议功能总体介绍

GoldenDB 数据库的批量协议是一种 `prepare` 模式下优化大数据量处理性能的通讯协议。该协议需要 GoldenDB 数据库和前端驱动的同时支持，具体的交互流程为：

1. 前端驱动向数据库发送 `batch-prepare` 请求；
2. GoldenDB 数据库判断是否满足走批量协议的条件；
3. 若不满足，前端驱动会退化成普通协议继续执行；
4. 若满足走批量的条件，驱动继续下发 `batch-execute` 请求，数据库处理完后给驱动返回响应，流程结束。

使用建议:

1. 批量协议对应执行语句必须为 `prepare` 语句；
2. 一个批次里每个分片在 `500-1000` 条数据左右为佳，一个批次最大不超过 1 万条数据，需要综合考虑每次更新数据大小*每批数量，具体以实际测试为准；
3. 批量协议适用于大批量简单 `DML` 的场景，如批量插入十万条记录，批量更新十万条以上记录，批量删除十万条以上记录，部分复杂的 `DML` 场景不适用于批量协议。

1.11.2. 数据库支持批量协议说明

数据库层面对于批量协议的支持范围是简单的 `insert`，`delete` 和 `update` 语句。比如：

```
> insert into t1(f1,f2,f3) values(?,?,?);
> delete from t1 where f1 = ? and f2 = ?;
> update t1 set f1 = ? where f2 = ? and f3 = ?;
```

1.11.2.1. 批量 insert 不支持的场景

1. 不支持多级分片表、多节点复制表场景下，`insert` 批量带 `sequence`；
2. 不支持带全局索引场景下，`insert` 批量带 `sequence` ；
3. 不支持在 `insert` 批量带 `sequence` 场景下，`sequence` 所在的列做自增列；
4. 不支持并行功能和批量功能一起使用；
5. 不支持 `merge into`、`replace into` 功能。

1.11.2.2. 批量 delete 不支持的场景

1. 不支持带全局索引的表；
2. 不支持需要跨分片计算的 `where` 条件；
3. 不支持在 `prepare` 语句中带 `sequence`。

1.11.2.3. 批量 update 不支持场景

1. 不支持带全局索引的表；
2. 不支持需要跨分片计算的 `where` 条件；
3. 不支持更新分发键；
4. 不支持 `cw` 隔离级别。

1.11.2.4. 批量插入和 ORACLE 行为差异补充说明

GoldenDB 的批量插入一个批次具有原子性，即一个批次里的所有记录要么全部插入成功要么全部插入失败，`ORACLE` 数据库则没有这个特性。如果一个批次中间有一条插入失败，GoldenDB 的表现是整个批次全部插入失败，`ORACLE` 的表现是失败记录之前的记录插入成功，失败记录及之后的记录没有被插入。 两种表现是两种数据库的正常特性，无关优劣。为了兼容原 `ORACLE` 用户的使用习惯，GoldenDB 通过增加 `hint: /*+batchstatement=seria*/`，在下述场景下批量插入特性可以对齐 `ORACLE` 的特性：

1. 非自动提交；
2. 非全局唯一索引表（全局索引表）；
3. 非多级分片表。

注意事项：

1. 用户通过是否加 `hint:batchstatement=serial` 来控制是否走这一特性；
2. 在多分片复制表场景下，如果各个分片返回的错误信息里的执行结果不一致(返回的`affect rows`不一致)，最终返回的错误信息里 `affect rows` 是 0，最终事务的提交受开关 `commit_to_rollback` 控制；
3. 使用此特性时，DN 上的 `batch_insert_ignore_duplicate_key` 配置不能开启。

1.11.3. 驱动支持批量协议说明

目前，支持用户通过 `JDBC` 驱动，`ODBC` 驱动，`OCI` 驱动与 GoldenDB 数据库进行批量协议的交互。

1.11.3.1. OCI 驱动与 GoldenDB 数据库批量协议交互

OCI驱动支持 `insert`,`delete`,`update` 语句走批量协议，通过在 `prepare` 语句中添加 `hint : /*+batchstatement=true*/` 控制是否走批量协议。

1. 批量插入示例（一个批次保持原子性）

```
  //发起insert批量处理prepare
  static text* insert = (text*) "insert /*+batchstatement=true*/ i
nto test_batchinsert1 (f1,f2) values(?,?)";
  iRet = OCIStmtPrepare(stmthp, errhp, insert, strlen((char*)inser
t),  OCI_NTV_SYNTAX, OCI_DEFAULT);
  ub4 batchSize = 1000;
  ub4 bufLen = 64;
  ub1 *f1Value = (ub1*)malloc(batchSize*bufLen*sizeof(ub1));
  memset(f1Value,0,batchSize*bufLen);
  ub1 *f2Value = (ub1*)malloc(batchSize*bufLen*sizeof(ub1));
  memset(f2Value,0,batchSize*bufLen);
  ub1* pos1 = f1Value;
  ub1* pos2 = f2Value;
  for(int i=0;i<batchSize;i++)
  {
      snprintf((char*)pos1,bufLen,"f1Value%d",i);
      snprintf((char*)pos2,bufLen,"f2Value%d",i);
      pos1 += bufLen;
      pos2 += bufLen;
  }
  iRet = OCIBindByPos(stmthp, NULL, errhp, 1, f1Value, bufLen, SQL
T_STR, NULL, NULL,   NULL, 0, NULL, 0);
  iRet = OCIBindByPos(stmthp, NULL, errhp, 2, f2Value, bufLen, SQL
T_STR, NULL, NULL,   NULL, 0, NULL, 0);
  //执行insert批量处理
  iRet = OCIStmtExecute(svchp, stmthp, errhp, batchSize, 0, NULL,
NULL, OCI_DEFAULT);
  //提交改动
  iRet = OCITransCommit(svchp, errhp, OCI_DEFAULT);
```

2. 批量插入示例（允许失败前的插入不回滚）

```
  static text* insert = (text*) "insert /*+batchstatement=seria
l*/ into t1 (f1) values (?)";
  OCIStmtPrepare(stmthp, errhp, insert, strlen((char*)insert),  OC
I_NTV_SYNTAX, OCI_DEFAULT);
  ub4 batchSize = 1000;
  ub4 bufLen = 64;
  ub1 *f1Value = (ub1*)malloc(batchSize*bufLen*sizeof(ub1));
  memset(f1Value,0,batchSize*bufLen);
  ub1* pos1 = f1Value;
  for(int i=0;i<batchSize;i++)
  {
      snprintf((char*)pos1,bufLen,"f1Value%d",i);
      pos1 += bufLen;
  }
  OCIBindByPos(stmthp, NULL, errhp, 1, f1Value, bufLen, SQLT_STR,
NULL, NULL,   NULL, 0, NULL, 0);
  OCIStmtExecute(svchp, stmthp, errhp, batchSize, 0, NULL, NULL,
 OCI_DEFAULT);
  //假设第200行执行失败，因此第一次执行得到的影响行数是199，可以跳过第200行，
继续插入后面的800行
  OCIStmtExecute(svchp, stmthp, errhp, batchSize, 200, NULL, NU
LL,  OCI_DEFAULT);
```

3. 批量更新示例

基于 `OCI` 实现 `update` 走批量协议时，需要在 `hint: /*+batchstatement=true*/` 后添加 `/*M!gdb_isolation_grade=sw*/`。

```
  //发起update批量处理prepare
  static text* update1 = (text*) "update /*+batchstatement=true*/
/*M!gdb_isolation_grade=sw*/ test_batchinsert1 set f2 = ? where f1
 = ?";
  iRet = OCIStmtPrepare(stmthp, errhp, update1, strlen((char*)upda
te1),  OCI_NTV_SYNTAX, OCI_DEFAULT);
  iRet = OCIBindByPos(stmthp, NULL, errhp, 1, f2Value, bufLen, SQL
T_STR, NULL, NULL,   NULL, 0, NULL, 0);
  iRet = OCIBindByPos(stmthp, NULL, errhp, 2, f1Value, bufLen, SQL
T_STR, NULL, NULL,   NULL, 0, NULL, 0);
  pos1 = f1Value;
  pos2 = f2Value;
  for(int i=0;i<batchSize;i++)
  {
      snprintf((char*)pos1,bufLen,"f1Value%d",i);
      snprintf((char*)pos2,bufLen,"f2newValue%d",i);
      pos1 += bufLen;
      pos2 += bufLen;
  }
  //执行update批量处理
```

```
  iRet = OCIStmtExecute(svchp, stmthp, errhp, batchSize, 0, NULL,
NULL, OCI_DEFAULT);
  //提交改动
  iRet = OCITransCommit(svchp, errhp, OCI_DEFAULT);
```

4. 批量删除示例

```
  //发起delete批量处理prepare
  static text* delete1 = (text*) "delete /*+batchstatement=true*/
from test_batchinsert1 where f1 = ?";
  iRet = OCIStmtPrepare(stmthp, errhp, delete1, strlen((char*)dele
te1),  OCI_NTV_SYNTAX, OCI_DEFAULT);
  iRet = OCIBindByPos(stmthp, NULL, errhp, 1, f1Value, bufLen, SQL
T_STR, NULL, NULL,   NULL, 0, NULL, 0);
  printf("Bind finished !\n");
  pos1 = f1Value;
  for(int i=0;i<batchSize;i++)
  {
      snprintf((char*)pos1,bufLen,"f1Value%d",i);
      pos1 += bufLen;
  }
  //执行delete批量处理
  iRet = OCIStmtExecute(svchp, stmthp, errhp, batchSize/2, 0, NULL
, NULL,   OCI_DEFAULT);
  //提交改动
  iRet = OCITransCommit(svchp, errhp, OCI_DEFAULT);
```

1.11.3.2. ODBC 驱动与 GoldenDB 数据库批量协议交互

`ODBC` 驱动仅支持 `insert`，`delete` 语句走批量协议，`delete` 语句通过是否加 `hint` `/*+batchstatement=true*/` 控制是否走批量协议，`hint` 见示例。

1. 批量插入示例

```
  //执行insert prepare语句
  char szSQLPrepareInsert[256] = "insert into t1 values(?,?,?)";
  printf("prepare sql: %s\n\n", szSQLPrepareInsert);
  sret = SQLPrepare (hsmt,(SQLCHAR *)szSQLPrepareInsert,256);

  int param1[9] = {1,2,3,4,5,6,7,8,9};
  char param2[9][50] = {"name1","name2","name3","name4","name5","n
ame6","name7","name8","name9"};
  int param3[9] = {11,12,13,14,15,16,17,18,19};

  sret = SQLBindParameter(hsmt,1,SQL_PARAM_INPUT,SQL_C_SLONG,SQL_I
NTEGER,0,0,param1,0,NULL);
  sret = SQLBindParameter(hsmt,2,SQL_PARAM_INPUT,SQL_C_CHAR,SQL_CH
AR,5,0,param2,50,NULL);
  sret = SQLBindParameter(hsmt,3,SQL_PARAM_INPUT,SQL_C_SLONG,SQL_I
```

```
NTEGER,0,0,param3,0,NULL);
  sret = SQLSetStmtAttr(hsmt, SQL_ATTR_PARAMSET_SIZE,(SQLPOINTER)b
atchSize, 0);

  sret = SQLExecute(hsmt);
  //提交连接的事务
  SQLEndTran(SQL_HANDLE_DBC,hdbc,SQL_COMMIT);
```

2. 批量删除示例

`ODBC` 在走 `delete` 批量协议时，`prepare` 语句中必须添加 `/*+batchstatement=true*/` 来进行标识。

```
  //执行delete prepare语句
  char szSQLPrepareDelete[256] = "delete /*+batchstatement=true*/
from t1 where id = ? and name = ? and age = ?";
  printf("prepare sql: %s\n\n", szSQLPrepareDelete);
  sret = SQLPrepare (hsmt,(SQLCHAR *)szSQLPrepareDelete,256);

  size_t batchSize = 10*100;
  int param1[9] = {1,2,3,4,5,6,7,8,9};
  char param2[9][50] = {"name1","name2","name3","name4","name5","n
ame6","name7","name8","name9"};
  int param3[9] = {11,12,13,14,15,16,17,18,19};

  sret = SQLBindParameter(hsmt,1,SQL_PARAM_INPUT,SQL_C_SLONG,SQL_I
NTEGER,0,0,param1,0,NULL);
  sret = SQLBindParameter(hsmt,2,SQL_PARAM_INPUT,SQL_C_CHAR,SQL_CH
AR,5,0,param2,50,NULL);
  sret = SQLBindParameter(hsmt,3,SQL_PARAM_INPUT,SQL_C_SLONG,SQL_I
NTEGER,0,0,param3,0,NULL);
  sret = SQLSetStmtAttr(hsmt, SQL_ATTR_PARAMSET_SIZE,(SQLPOINTER)b
atchSize, 0);

  sret = SQLExecute(hsmt);
  //提交连接的事务
  SQLEndTran(SQL_HANDLE_DBC,hdbc,SQL_COMMIT);
```

1.11.3.3. JDBC 驱动与 GoldenDB 数据库批量协议交互

`JDBC` 驱动支持 `insert`,`delete`,`update` 语句走批量协议，需要开启批量协议开关，详情请查阅 `jdbc` 使用说明。

1. 批量插入示例

```
  private static void batchInsert() throws SQLException {
    String url = "jdbc:goldendb://ip:port/wmm?comStmtBatchFlag=tru
e&rewriteBatchedStatements=true&allowMultiQueries=true";
```

```
    String pass = "xxxxxx";
    String username = "xxxxxx";
    Connection con = DriverManager.getConnection(url, username, pa
ss);
    con.setAutoCommit(true);
    long start = System.currentTimeMillis();
    {
      PreparedStatement ps = con.prepareStatement("Insert into goo
d (id, name,price, create_time) values (?, ?, ?, ?)");
      System.out.println(ps);
      for (int i = 1; i < 100000; i++) {
        ps.setInt(1, i);
        ps.setString(2, "name");
        ps.setDouble(3, 12.12);
        ps.setString(4, "2022-02-02 12:12:12");
        ps.addBatch();
        if (i % 1000 == 0) {
            ps.executeBatch();
        }
      }
      ps.executeBatch();
      ps.close();
    }
    con.close();
    System.out.println(start - System.currentTimeMillis());
  }
```

2. 批量更新示例

```
  private static void batchUpdate() throws SQLException {
    String url = "jdbc:goldendb://ip:port/wmm?comStmtBatchFlag=tru
e&rewriteBatchedStatements=true&allowMultiQueries=true";
    String pass = "xxxxxx";
    String username = "xxxxxx";
    Connection con = DriverManager.getConnection(url, username, pa
ss);
    long start = System.currentTimeMillis();
    PreparedStatement ps = con.prepareStatement("update good set N
AME = 'sasd' where id = ?");
    System.out.println(ps);
    for (int i = 1; i < 11; i++) {
        ps.setInt(1, i);
        ps.addBatch();
    }
    ps.execute();
    System.out.println(start - System.currentTimeMillis());
  }
```

3. 批量删除示例

```
  private static void batchDelete() throws SQLException {
    String url = "jdbc:goldendb://ip:port/wmm?comStmtBatchFlag=tru
e&rewriteBatchedStatements=true&allowMultiQueries=true";

    String pass = "xxxxxx";
    String username = "xxxxxx";
    long start = System.currentTimeMillis();
    try (
        Connection con = DriverManager.getConnection(url, username
, pass);
        PreparedStatement ps = con.prepareStatement("delete from g
ood where id = ?");
        ){
      System.out.println(ps);
      for (int i = 1; i < 11; i++) {
        ps.setInt(1, i);
        ps.addBatch();
      }
      System.out.println("end");
      ps.executeBatch();
    }
    System.out.println(start - System.currentTimeMillis());
  }
```

1.11.3.4. 支持版本说明

以上说明仅针对最新版本 GoldenDB613.07SP6 及后续补丁版本，GoldenDB613.09 版本及后续版本，其他版本支持功能不全，需要结合实际测试结果确认。